

01 - 什么是 AI Agent

本章目标：理解 AI Agent 的核心概念、与 LLM 的本质区别、核心三要素、ReAct 框架，以及主流 Agent 框架对比。这是面试中最基础也最重要的一章。

目录

- 1.1 AI Agent 的定义
- 1.2 LLM vs AI Agent：本质区别
- 1.3 Agent 核心三要素
- 1.4 ReAct 框架详解
- 1.5 Agent 工作流程
- 1.6 主流 Agent 框架对比
- 1.7 Agent 的发展历史
- 1.8 面试话术
- 1.9 面试常考点总结
- 1.10 练习题与思考题

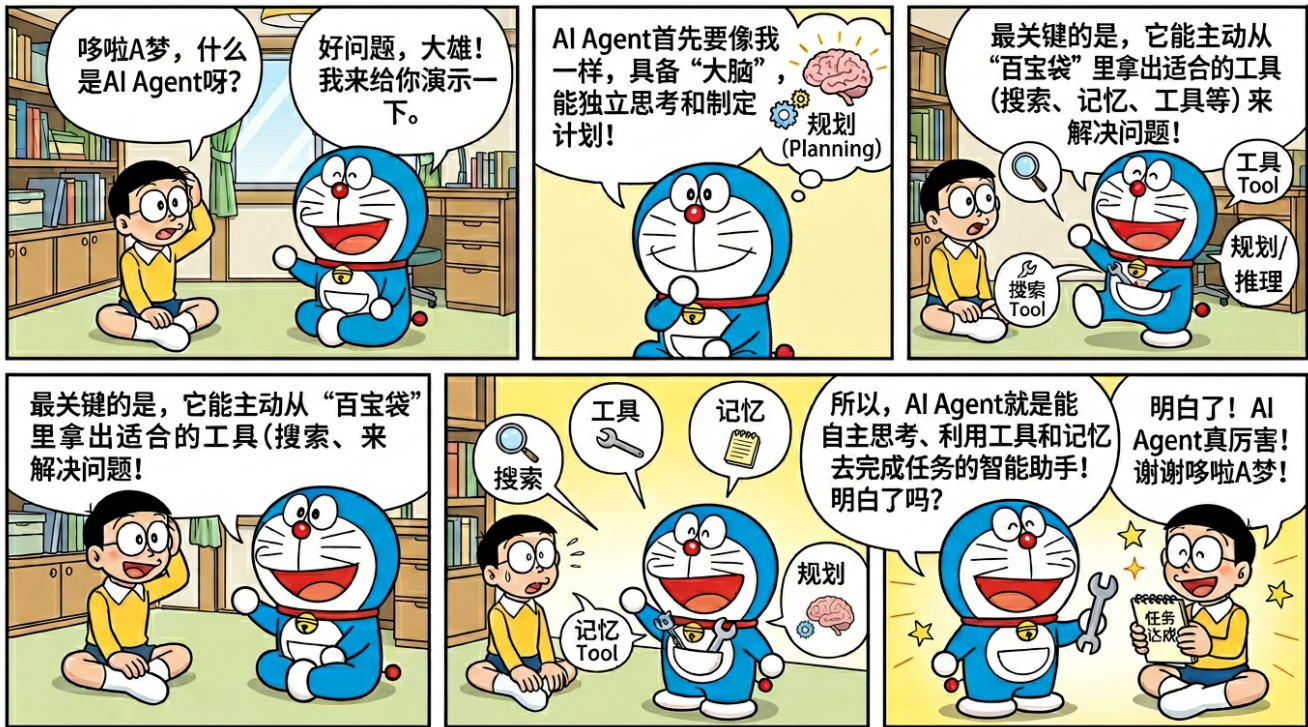


Figure 1: AI Agent 概念漫画

哆啦 A 梦带你理解 AI Agent：能自主思考、使用工具的智能助手

1.1 AI Agent 的定义

什么是 AI Agent?

AI Agent（智能体/智能代理）是一个以大语言模型（LLM）为“大脑”，能够自主感知环境、制定计划、执行行动并根据反馈迭代调整的智能系统。

用一句话概括：

AI Agent = LLM（大脑） + Planning（规划） + Memory（记忆） + Tool Use（工具使用）

形象类比

想象一个优秀的私人助理：

角色	AI Agent 中的对应
大脑/智商	LLM（如 GPT-4、Claude、DeepSeek）
思考和计划能力	Planning（规划能力）
记事本/记忆	Memory（短期和长期记忆）
手和工具	Tool Use（调用 API、搜索、写代码等）
眼睛和耳朵	感知层（接收用户输入和环境信息）

学术定义

从学术角度，AI Agent 的概念最早源于人工智能和认知科学领域：

- **Russell & Norvig (2020)**: Agent 是通过传感器（sensors）感知环境并通过执行器（actuators）作用于环境的实体。
- **Wooldridge & Jennings (1995)**: Agent 具有自治性（autonomy）、社会性（social ability）、反应性（reactivity）和主动性（pro-activeness）。
- **LLM-based Agent (2023-)**: 以大语言模型为核心推理引擎的新一代智能体，通过自然语言理解和生成来实现感知、规划和行动。

Agent 的四大特征

AI Agent 四大特征
1. 自主性 (Autonomy)
→ 不需要人类逐步指令，自主决策执行
2. 反应性 (Reactivity)
→ 能感知环境变化并实时响应
3. 主动性 (Pro-activeness)
→ 能主动采取行动以达成目标
4. 社会性 (Social Ability)
→ 能与其他 Agent 或人类交互协作

不同维度理解 Agent

从产品角度：AI Agent 是一种能完成复杂任务的 AI 应用，用户给出目标后，Agent 自动分解任务、调用工具、生成结果。

从技术角度：AI Agent 是一个围绕 LLM 构建的系统，增加了记忆管理、工具调用、任务规划等模块，使 LLM 从“回答问题”升级为“解决问题”。

从面试角度：理解 Agent 不是一个单一模型，而是一个系统架构。这个架构以 LLM 为核心，通过模块化设计实现自主决策和行动。

1.2 LLM vs AI Agent：本质区别

这是面试中最常问的问题之一。很多面试官会直接问：“LLM 和 Agent 有什么区别？”

核心区别一览表

维度	LLM（大语言模型）	AI Agent（智能体）
本质	语言模型，文本生成器	自主决策系统
能力	理解和生成文本	感知、规划、执行、反思
交互方式	单轮问答或多轮对话	自主循环，多步骤任务
是否有记忆	仅上下文窗口内	有短期记忆 + 长期记忆
能否使用工具	原生不能（需 Function Calling）	可以调用各种外部工具
能否行动	只能生成文本	可以执行操作（发邮件、写文件等）
任务复杂度	单步或简单多步	复杂多步骤任务
错误处理	无自我纠错机制	可根据反馈自我修正
主动性	被动响应	可主动发起行动
类比	一个知识渊博的大脑	一个有手有脚有记忆的人

形象对比

LLM（纯模型）：

用户：“帮我查一下北京今天天气”

LLM：“我无法访问实时天气数据，但你可以通过...” □ 只能生成文本

AI Agent：

用户：“帮我查一下北京今天天气”

Agent 思考：我需要查询天气 → 调用天气 API → 获取结果 → 整理返回

Agent：“北京今天晴，气温 22℃，东北风 3-4 级，适合外出。” □ 实际行动了

关键区分点

1. Agent 有”行动力”

LLM 只是一个”大脑”，它只能思考和说话。而 Agent 有”手”——它可以调用工具、访问数据库、发送消息、操作文件系统。

2. Agent 有”记忆”

LLM 的”记忆”仅限于上下文窗口（如 128K tokens），窗口之外的信息就丢失了。Agent 有专门的记忆模块：- 短期记忆：当前对话的上下文 - 长期记忆：跨会话持久化的信息（如 Nanobot 的 MEMORY.md）

3. Agent 能”自主决策循环”

LLM 是一问一答的模式。Agent 则进入一个自主循环：

观察(Observe) → 思考(Think) → 行动(Act) → 观察结果 → 继续思考 → ...

这个循环会一直持续，直到任务完成或达到最大迭代次数。

4. Agent 能”分解任务”

给 LLM 一个复杂任务，它可能一次性给出一个粗糙的答案。而 Agent 会把复杂任务拆解为子任务，逐步执行：

任务：帮我写一份项目调研报告

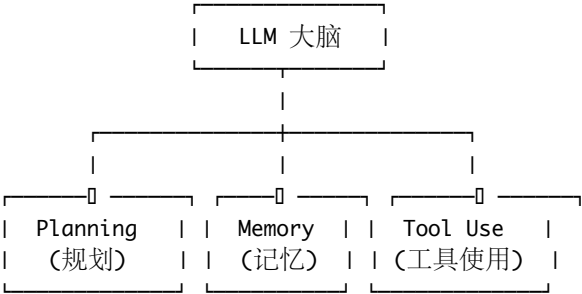
Agent 的处理方式：

1. 分析任务需求 → 确定报告结构
2. 搜索相关资料 → 调用搜索工具
3. 阅读和整理信息 → 总结关键点
4. 撰写报告初稿 → 生成文档
5. 检查和修改 → 自我审查
6. 返回最终报告

“LLM 本质上是一个文本生成模型，它只能根据输入生成输出文本。而 AI Agent 是以 LLM 为核心构建的自主决策系统。Agent 在 LLM 之上增加了三个关键能力：第一是 Planning，能自主分解和规划任务；第二是 Memory，有独立的记忆系统来持久化信息；第三是 Tool Use，能调用外部工具来执行真实操作。简单来说，LLM 是大脑，Agent 是一个有大脑、有记忆、有手脚的完整系统。”

1.3 Agent 核心三要素

AI Agent 的核心能力可以概括为三大要素。这个框架最早由 Lilian Weng 在其 2023 年的经典博文 “LLM Powered Autonomous Agents”中系统总结。



1.3.1 Planning（规划）

定义：Agent 将复杂任务分解为可管理的子任务，并制定执行计划的能力。

两种核心规划策略：

(a) 任务分解（Task Decomposition） 将一个大任务分解为多个小任务，有以下常见策略：

策略	描述	示例
Chain of Thought (CoT)	逐步推理	“让我一步步思考…”
Tree of Thoughts (ToT)	探索多条推理路径	生成多个方案，选最优
Plan and Solve	先制定整体计划再执行	列出步骤 1-5，然后逐步执行
ReAct	推理和行动交替	思考 → 行动 → 观察 → 思考 → …

具体示例：

用户任务：“帮我分析竞品公司的最新财报并生成对比报告”

Agent 的任务分解：

- ├─ 子任务1：搜索目标公司的最新财报
 - ├─ 1.1 确定搜索关键词
 - ├─ 1.2 调用搜索工具
 - └─ 1.3 下载财报PDF
- ├─ 子任务2：解析财报数据
 - ├─ 2.1 提取收入数据
 - ├─ 2.2 提取利润数据
 - └─ 2.3 提取关键指标
- ├─ 子任务3：生成对比分析
 - ├─ 3.1 计算同比变化
 - ├─ 3.2 对比各公司数据
 - └─ 3.3 生成图表描述
- └─ 子任务4：撰写报告
 - └─ 4.1 编写摘要

- └─ 4.2 编写正文
- └─ 4.3 编写结论

(b) 反思与改进 (**Reflection & Refinement**) Agent 能对自己的行动结果进行评估和反思:

- **Self-Reflection**: LLM 评估自己的输出质量
- **Reflexion**: 从失败中学习, 将错误经验存入记忆
- **Chain of Hindsight**: 回顾过去的决策, 优化未来策略

执行结果不理想

↓

反思: 哪一步出了问题?

↓

调整计划: 修改搜索策略/换一个工具

↓

重新执行

↓

检查结果是否满意

1.3.2 Memory (记忆)

定义: Agent 存储和检索信息的能力, 使其能在交互中保持上下文, 并从历史经验中学习。

记忆类型对比

记忆类型	描述	特点	实现方式
感觉记忆 短期记忆/工作记忆	原始输入的短暂保持 当前任务的上下文	极短暂, 几秒 有限容量, 对应上下文窗口	用户输入的 raw text LLM 的上下文窗口
长期记忆	跨会话持久化信息	理论上无限	外部存储 (文件/向量库/数据库)

长期记忆的实现方式

长期记忆实现方案
1. 文件存储 (Nanobot 方案)
MEMORY.md → 结构化的长期记忆
HISTORY.md → 完整的交互历史
2. 向量数据库
存储 embedding, 语义检索
如 Pinecone、Milvus、Chroma
3. 知识图谱
实体-关系存储
结构化知识表示
4. 关系型数据库
适合结构化数据存储
如 PostgreSQL、SQLite

Nanobot 的记忆系统（预览） **Nanobot** 采用了简洁而优雅的双文件记忆方案：

```
# MEMORY.md (长期记忆)
- 用户偏好：喜欢简洁回答，常用 Python
- 项目信息：正在开发电商后台系统
- 重要决策：数据库选择 PostgreSQL
```

```
# HISTORY.md (交互历史)
## 2026-04-01 10:30
用户请求分析代码性能问题...
Agent 使用 profiling 工具定位到瓶颈...
```

这种设计的优雅之处在于：- 完全透明，用户可以直接查看和编辑记忆文件 - 无需额外的向量数据库等基础设施 - 以 Workspace 为中心，自然隔离不同项目的记忆

1.3.3 Tool Use（工具使用）

定义：Agent 调用外部工具和 API 来扩展自身能力的机制。

这是 Agent 和纯 LLM 的最关键区别——LLM 的知识有截止日期，不能上网搜索，不能操作文件，不能执行代码。而 Agent 通过工具调用获得了“行动力”。

常见工具类型

工具类型	描述	示例
信息检索	获取外部信息	搜索引擎、数据库查询、API 调用
代码执行	运行代码	Python 解释器、Shell 命令
文件操作	读写文件	读取 PDF、写入 CSV、创建文档
通信工具	发送消息	邮件、Slack、飞书消息
专业工具	领域特定	图像生成、数据分析、翻译

工具调用流程

1. LLM 理解用户意图
2. LLM 决定需要调用哪个工具
3. LLM 生成工具调用参数（JSON 格式）
4. 系统执行工具调用
5. 将工具返回结果提供给 LLM
6. LLM 根据结果继续推理或返回用户

工具调用的实现方式 **Function Calling**（原生 LLM 能力）：

```
{
  "name": "get_weather",
  "arguments": {
    "city": "北京",
    "date": "2026-04-02"
  }
}
```

MCP 协议（标准化方案）：

MCP（Model Context Protocol）是 Anthropic 提出的工具调用标准化协议，让工具可以跨不同 Agent 框架复用。**Nanobot** 原生支持 MCP。（详见第 5 章）

面试中关于工具使用的常见问题 **Q: Agent 怎么知道该调用哪个工具？**

A: 在 Agent 系统中,所有可用工具的描述(名称、功能、参数)会作为 system prompt 的一部分提供给 LLM。LLM 根据用户意图和工具描述,决定调用哪个工具并生成相应参数。这本质上是一个“函数选择”问题,LLM 通过理解自然语言描述来匹配最合适的工具。

Q: 如果 Agent 调用工具出错了怎么办?

A: 好的 Agent 框架有错误处理机制。工具调用失败后,错误信息会返回给 LLM,LLM 会分析错误原因(如参数错误、权限不足),然后决定是修改参数重试、换一个工具,还是向用户报告错误。在 Nanobot 中,工具执行结果(包括错误)都会返回给 AgentRunner,由 LLM 决定下一步行动。

1.4 ReAct 框架详解

什么是 ReAct?

ReAct = Reasoning + Acting

ReAct 框架是 2022 年由 Yao et al. 在论文 “*ReAct: Synergizing Reasoning and Acting in Language Models*” 中提出的,它将推理 (Reasoning) 和行动 (Acting) 交替进行,是目前大多数 Agent 框架采用的核心范式。

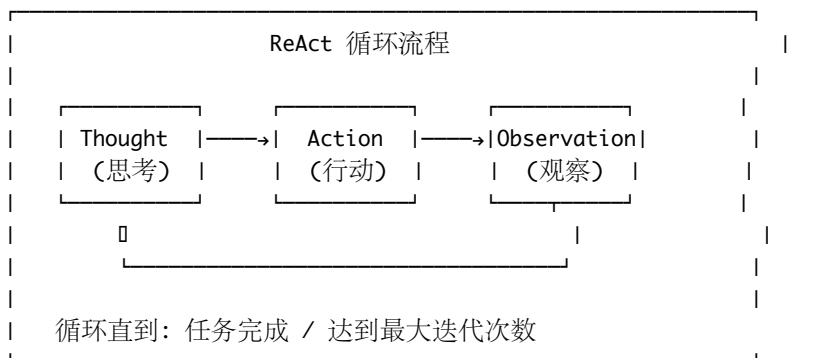
为什么 ReAct 重要?

在 ReAct 之前,有两种主流方法:

1. 纯推理 (**Reasoning Only**): 如 Chain-of-Thought, LLM 只是在内部推理,不与外部世界交互
2. 纯行动 (**Acting Only**): 如简单的工具调用链,没有推理和反思

ReAct 将两者结合,让 Agent 在推理的同时能采取行动,并根据行动结果更新推理。

ReAct 循环流程



详细示例

用户请求: “帮我查一下 Apple 的最新股价, 并告诉我最近一个月的涨跌情况”

=== 第 1 轮 ===

Thought: 用户想知道 Apple 的股价信息。我需要先查询 Apple 的当前股价, 股票代码是 AAPL。我应该调用股票查询工具。

Action: `search_stock(symbol="AAPL")`

Observation: AAPL 当前股价 \$245.32, 今日涨幅 +1.2%

=== 第 2 轮 ===

Thought: 我已经获得了当前股价。接下来需要查询最近一个月的历史数据来分析涨跌情况。

Action: `get_stock_history(symbol="AAPL", period="1month")`

Observation: 一个月前价格 \$231.50, 最高 \$248.90, 最低 \$225.10

=== 第 3 轮 ===

Thought: 现在我有了所有需要的数据。让我计算并整理结果:

- 当前: \$245.32
 - 一个月前: \$231.50
 - 涨幅: $(245.32 - 231.50) / 231.50 = 5.97\%$
 - 月内最高: \$248.90
 - 月内最低: \$225.10
- 我可以给用户一个完整的回答了。

Action: [返回最终答案给用户]

Final Answer: "Apple (AAPL) 当前股价 \$245.32。
近一个月涨幅约 5.97% (从 \$231.50 上涨至 \$245.32)。
期间最高触及 \$248.90, 最低回落至 \$225.10。整体呈上涨趋势。"

ReAct 在 Nanobot 中的体现

Nanobot 的 AgentRunner 本质上实现了 ReAct 循环:

```
# 简化的 ReAct 循环 (伪代码)
for iteration in range(max_iterations): # 最多 40 次
    # Thought + Action: LLM 思考并决定行动
    response = provider.chat_with_retry(messages)

    if response.has_tool_calls:
        # Action: 执行工具调用
        results = tool_registry.execute(response.tool_calls)
        # Observation: 将结果加入对话历史
        messages.append(tool_results_to_message(results))
    else:
        # 没有工具调用, 说明 LLM 认为任务完成
        break
```

ReAct vs 其他范式对比

范式	描述	优点	缺点
CoT (Chain-of-Thought)	纯推理链	推理能力强	无法与外界交互
Act-Only	直接行动	简单直接	缺乏推理和反思
ReAct	推理 + 行动交替	兼具推理和行动	可能过度推理
Plan-then-Act	先整体规划再执行	全局视角	计划可能与实际脱节
Reflexion	ReAct + 自我反思	能从错误中学习	增加了延迟和成本

面试重点

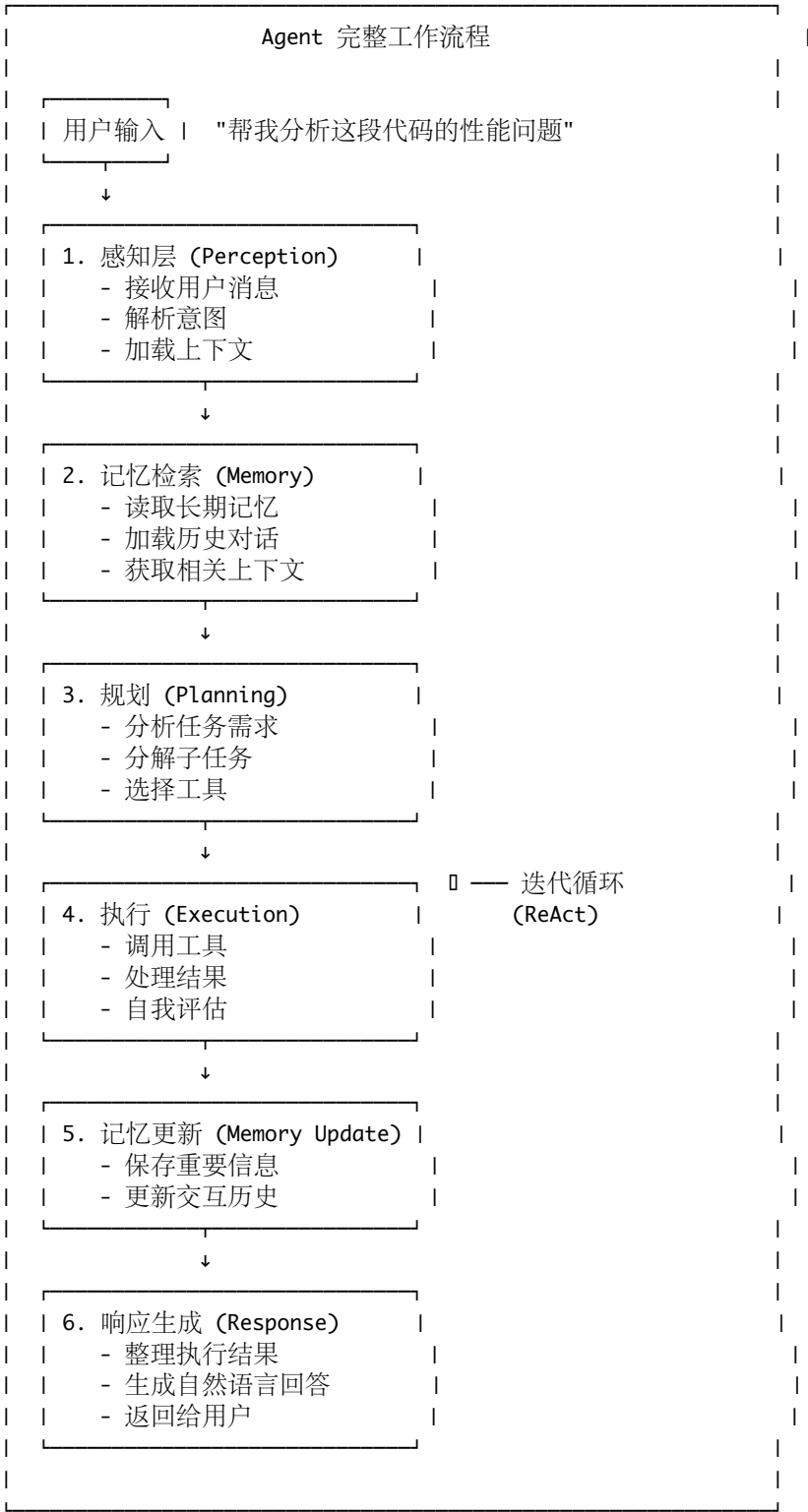
面试官问: “请解释 ReAct 框架”

回答: “ReAct 是 Reasoning + Acting 的缩写, 核心思想是让 AI Agent 在推理 (Thought) 和行动 (Action) 之间交替循环。每一轮, LLM 先根据当前状态进行推理 (Thought), 决定下一步该做什么; 然后执行具体行动 (Action), 比如调用工具; 接着观察行动结果 (Observation); 再进入下一轮推理。这个循环持续进行, 直到任务完成或达到最大迭代次数。Nanobot 的 AgentRunner 就是一个经典的 ReAct 循环实现, 默认最多迭代 40 次。”

1.5 Agent 工作流程

完整的 Agent 工作流程

下面描述一个典型 AI Agent 从接收用户输入到返回结果的完整流程：



在 **Nanobot** 中的具体实现

用户消息 → Channel(Telegram/Discord/...) → MessageBus(Inbound Queue)
→ AgentLoop.run() 消费消息
→ ContextBuilder 构建 system prompt + 历史消息
→ AgentRunner 进入 ReAct 循环
→ Provider(LLM) 生成响应
→ ToolRegistry 执行工具调用
→ 将结果加入消息列表
→ 继续循环或结束
→ MemoryStore 保存记忆
→ MessageBus(Outbound Queue) → Channel → 用户

1.6 主流 Agent 框架对比

框架对比总览

维度	Nanobot	LangChain	CrewAI	AutoGPT	OpenClaw
开发者	香港大学 HKUDS	Harrison Chase	João Moura	Toran Richards	清华系团队
语言	Python	Python/JS	Python	Python	Python
代码量	~4,000 行	~50 万行	~3 万行	~10 万行	~43 万行
GitHub Stars	37K+	100K+	25K+	170K+	15K+
核心定位	超轻量级个人 Agent	Agent 开发框架/工具链	多 Agent 协作	自主通用 AI	全栈 Agent 平台
MCP 支持	原生支持	通过扩展	有限	无原生支持	支持
记忆系统	文件系统 (MEMORY.md)	多种后端	内置	内置	多模态记忆
多平台支持	8+ 平台	需自行集成	无	Web UI	有限
学习曲线	低	高	中	中	高
适用场景	个人助理/小团队	复杂 AI 应用	多 Agent 协作	自任务务	企业级
首次发布	2026 年 2 月	2022 年 10 月	2023 年 12 月	2023 年 3 月	2025 年

详细分析

Nanobot (本项目重点) 优势: - 代码极简 (~4000 行), 一个周末就能读完全部源码 - 架构清晰, 非常适合学习 Agent 设计思想 - MCP 原生支持, 紧跟技术趋势 - 多平台支持 (微信、飞书、钉钉、Telegram 等), 接地气 - MIT 开源, 无商业限制

局限: - 单 Agent 为主, 多 Agent 协作能力有限 - 生态不如 LangChain 丰富 - 发布时间较短, 社区还在成长

LangChain 优势: - 生态最丰富, 几乎所有 LLM 和工具都有集成 - 文档详尽, 社区庞大 - LCEL (LangChain Expression Language) 灵活强大

局限: - 代码量大, 抽象层过多, 学习曲线陡峭 - “胶水代码”感重, 过度封装 - 性能开销大

CrewAI 优势: - 多 Agent 协作是其强项 - 角色定义直观 - 与 LangChain 集成好

局限: - 单 Agent 场景下过于复杂 - 依赖 LangChain 生态

AutoGPT 优势: - 先驱项目, 引领了 Agent 热潮 - 完全自主运行的理念

局限: - 实际可靠性有限 - Token 消耗大 - 容易陷入循环

面试推荐说法

“我学习过多个 Agent 框架，重点研究了 Nanobot。选择 Nanobot 的原因有三：第一，它只有 4000 行 Python 代码，便于深入理解 Agent 的核心设计思想，而不是被框架的复杂抽象所困扰；第二，它虽然轻量但五脏俱全——记忆系统、MCP 协议、多平台支持、子 Agent 机制一应俱全；第三，它是 2026 年的新项目，代表了 Agent 框架设计的最新趋势。”

1.7 Agent 的发展历史

AI Agent 发展时间线（2023-2026）

2023

- 03月 AutoGPT 发布，引爆 AI Agent 热潮
- 04月 BabyAGI 发布，简约 Agent 设计
- 06月 Lilian Weng 发表 "LLM Powered Autonomous Agents"
 - | → 系统定义了 Planning + Memory + Tool Use 三要素
- 07月 MetaGPT 发布，多 Agent 软件开发
- 10月 LangChain 发布 LangGraph
- 11月 OpenAI 发布 GPTs（自定义 Agent）
- 12月 CrewAI 发布，聚焦多 Agent 协作

2024

- 01月 OpenAI 发布 Assistants API（内置工具调用）
- 03月 Devin AI 发布（AI 软件工程师），引发 Agent 应用热潮
- 05月 Claude 3 发布，强推理能力推动 Agent 发展
- 06月 Function Calling 成为 LLM 标配
- 08月 Cursor / Windsurf 等 AI IDE 爆火
- 11月 Anthropic 发布 MCP 协议
 - | → 定义了 AI 工具调用的标准化协议
- 12月 Agent 框架百花齐放，生态日趋成熟

2025

- 01月 OpenAI 发布 Operator（Web Agent）
- 03月 MCP 协议被广泛采用
 - | → OpenAI、Google、Microsoft 纷纷支持
- 05月 Claude 3.5 的 Computer Use 推动 GUI Agent
- 06月 企业级 Agent 平台兴起
- 09月 多模态 Agent 成为趋势
- 12月 Agent 在企业中规模落地

2026

- 01月 Agent 成为 AI 应用主流范式
- 02月 HKUDS/nanobot 发布 □ 我们学习的项目
 - | → 超轻量级 Agent 框架，迅速获得 37K+ Stars
- 03月 MCP 生态全面成熟
 - | → 各类 MCP Server 生态丰富
- 04月 你正在学习本指南！

关键里程碑解读

2023：Agent 元年 - AutoGPT 的发布标志着 LLM-based Agent 的诞生 - 学术界和工业界开始系统研究 Agent 架构

2024：标准化之年 - MCP 协议的发布是里程碑事件，解决了工具调用标准化问题 - Function Calling 成为 LLM 标配，Agent 有了更可靠的工具调用基础

2025：落地之年 - Agent 从概念验证走向生产落地 - 企业开始大规模采用 Agent 解决方案

2026：成熟之年 - Agent 成为 AI 应用的主流范式 - 框架设计趋于收敛，Nanobot 代表了“极简而完整”的设计哲学

1.8 面试话术

话术一：30 秒解释什么是 AI Agent

“AI Agent 就是给大语言模型装上了‘手脚和记忆’的智能系统。普通的 LLM 只能根据输入生成文本，就像一个只有大脑的人。而 Agent 在 LLM 之上增加了三个核心能力：规划（Planning）——把复杂任务分解成一步步的计划；记忆（Memory）——能记住之前交互的重要信息；工具使用（Tool Use）——能调用搜索引擎、执行代码、访问数据库等外部工具。这三个能力让 Agent 能够自主完成复杂任务，而不仅仅是回答问题。”

话术二：为什么 Agent 是 AI 的未来方向

“单纯的 LLM 就像一个百科全书，能回答问题但不能帮你做事。Agent 则是能实际解决问题的 AI 系统。举个例子，你问 LLM ‘帮我订明天去上海的机票’，它只会告诉你怎么操作。但 Agent 可以真正地去搜索航班、比价、帮你下单。这种从‘问答’到‘行动’的转变，就是 Agent 的核心价值。从产业趋势看，各大公司都在布局 Agent，OpenAI 的 Operator、Google 的 Project Astra、还有我学习的 Nanobot 框架，都是这个方向的代表。”

话术三：解释你选择学习 Nanobot 的原因

“我选择深入学习 Nanobot 框架，主要有三个原因。第一是‘以小见大’，Nanobot 只有 4000 行 Python 代码，但它的架构设计涵盖了消息总线、AgentLoop、记忆系统、MCP 协议支持、多平台适配等所有核心模块，非常适合深入理解 Agent 的设计思想。第二是技术前沿性，它原生支持 MCP 协议，这是 2024 年 Anthropic 提出的工具调用标准化协议，代表了 Agent 技术的最新方向。第三是实用性，它支持微信、飞书、钉钉等国内平台，我可以直接用它搭建实际可用的 AI 助手。”

1.9 面试常考点总结

高频考点清单

编号	考点	考察频率	难度
1	LLM 和 Agent 的区别		低
2	Agent 核心三要素		低
3	ReAct 框架原理		中
4	Agent 的记忆系统设计		中
5	工具调用的实现机制		中
6	Agent 框架对比		中
7	Agent 的安全问题		高
8	多 Agent 协作		高
9	Agent 的评测方法		高
10	Agent 的商业应用场景		低

必须能回答的 5 个问题

Q1: 什么是 AI Agent? 和 LLM 有什么区别?

参考答案见 1.1 节 和 1.2 节

Q2: Agent 的核心三要素是什么? 分别是如何实现的?

参考答案见 1.3 节

Q3: 请解释 ReAct 框架的原理和工作流程。

参考答案见 1.4 节

Q4: 你了解哪些 **Agent** 框架？它们各有什么特点？

参考答案见 1.6 节

Q5: **Agent** 的记忆系统是怎么设计的？

核心回答思路：分为短期记忆（上下文窗口）和长期记忆（外部存储）。短期记忆利用 LLM 的上下文窗口，但容量有限；长期记忆需要外部存储方案，如 Nanobot 的文件系统方案（MEMORY.md），或者向量数据库方案。好的记忆系统需要解决信息存储、检索、压缩和更新等问题。

1.10 练习题与思考题

基础题（确认理解）

1. 填空题

AI Agent 的核心三要素是 _____、_____ 和 _____。

答案：Planning（规划）、Memory（记忆）、Tool Use（工具使用）

2. 判断题

“AI Agent 就是一个更强大的大语言模型。”这句话对吗？为什么？

答案：不对。Agent 不是一个模型，而是一个以 LLM 为核心的系统架构。它在 LLM 之上增加了规划、记忆和工具使用等模块。

3. 简答题

请用自己的话解释 ReAct 框架中 Thought、Action、Observation 三个步骤分别做什么。

进阶题（加深理解）

4. 对比分析

请对比 Nanobot 和 LangChain 的设计哲学差异。提示：从代码量、架构风格、目标用户三个维度分析。

思路：- Nanobot：极简（4000 行）、Workspace 为中心、个人开发者 - LangChain：全面（50 万行 +）、抽象层丰富、企业开发者 - 核心差异：Nanobot 追求“够用就好”，LangChain 追求“无所不能”

5. 设计题

如果你要设计一个 Agent 的记忆系统，你会如何设计？请考虑以下问题：- 如何决定哪些信息需要记住？- 如何处理记忆容量限制？- 如何在后续对话中检索相关记忆？

思路提示：可以参考 Nanobot 的方案（虚拟工具 save_memory + MEMORY.md），也可以设计向量数据库方案。关键是要考虑“存什么”、“怎么存”、“怎么取”三个问题。

6. 场景题

一个用户说：“帮我调研一下 2026 年最受欢迎的 5 个 Python Web 框架，写一份对比报告。”

请描述一个 Agent 如何用 ReAct 框架处理这个任务，列出每一轮的 Thought、Action 和 Observation。

思考题（开放讨论）

7. 你认为 AI Agent 目前面临的最大挑战是什么？（可靠性？安全性？成本？）

8. 为什么 Nanobot 能用 4000 行代码实现其他框架数万行甚至数十万行才能实现的功能？这说明了什么设计原则？

9. 如果 Agent 在执行任务的过程中做出了错误的决策（比如调用了错误的工具），Agent 系统应该如何处理？

10. MCP 协议的出现会对 Agent 生态产生什么影响？是否会让不同 Agent 框架趋于同质化？

下一章

准备好了吗? 让我们进入下一章, 深入了解 Nanobot 项目本身。

02 - Nanobot 项目概览

本章小结: AI Agent 是以 LLM 为核心、具备规划、记忆和工具使用能力的自主决策系统。ReAct 框架 (Reasoning + Acting) 是大多数 Agent 的核心运行范式。理解这些基础概念, 是学习任何 Agent 框架的前提。

02 - Nanobot 项目概览

本章目标：全面了解 HKUDS/nanobot 项目的背景、核心特性、设计理念，以及为什么它是面试学习的最佳选择。

目录

- 2.1 项目背景
- 2.2 为什么叫“超轻量级”
- 2.3 核心特性详解
- 2.4 项目数据与里程碑
- 2.5 为什么选 Nanobot 学习
- 2.6 与其他框架的差异化对比
- 2.7 面试话术
- 2.8 本章总结

2.1 项目背景

香港大学数据科学实验室（HKUDS）

HKUDS（The University of Hong Kong Data Science Lab）是香港大学的数据科学研究实验室，在大模型、推荐系统、图神经网络等方向有深厚的学术积累。

Nanobot 是 HKUDS 在 2026 年 2 月开源的项目，定位为一个超轻量级 **AI Agent** 框架。从发布到现在短短两个月内就获得了 37K+ Stars，增长速度惊人。

项目诞生的背景

Nanobot 诞生的时代背景	
问题 1: Agent 框架过于复杂	
└─ LangChain 50万行代码，学习曲线陡峭	
└─ AutoGPT 10万行代码，概念验证但不实用	
└─ 初学者和个人开发者难以入门	
问题 2: 缺少适合亚洲市场的 Agent 框架	
└─ 大多数框架不支持微信/飞书/钉钉	
└─ 国内 LLM（通义千问/DeepSeek）接入不便	
└─ 中文生态支持不足	
问题 3: MCP 协议刚刚成熟，需要原生支持	
└─ 2024年底 Anthropic 发布 MCP 协议	
└─ 2025年各大公司跟进支持	
└─ 需要一个从设计之初就原生支持 MCP 的框架	
Nanobot 的解答:	
→ 4000 行代码，极简但完整	
→ 原生支持微信/飞书/钉钉等 8+ 平台	
→ MCP 原生支持，从第一天就内置	
→ MIT 开源，对所有人免费	

项目定位

Nanobot 的定位非常清晰：

一个任何人都能在 5 分钟内启动、一个周末就能读完全部源码的 **AI Agent** 框架。

它不追求功能全面（那是 LangChain 的定位），也不追求自主通用（那是 AutoGPT 的定位），而是追求极简、实用、可理解。

2.2 为什么叫“超轻量级”

代码量对比

这是 Nanobot 最令人印象深刻的特点——仅用约 4000 行 Python 代码实现了完整的 Agent 框架功能。

v0.1-v0.2.1 更新：

核心代码 ~6,000 行，完整项目 64,496 行（173 个 Python 文件）。

框架	核心代码量	语言	依赖数量	安装大小
Nanobot	~4,000 行	Python	少量	~5MB
LangChain	~500,000 行	Python/JS	大量	~100MB+
AutoGPT	~100,000 行	Python	大量	~50MB+
CrewAI	~30,000 行	Python	中等	~30MB
OpenClaw	~430,000 行	Python	大量	~200MB+
MetaGPT	~50,000 行	Python	大量	~80MB+

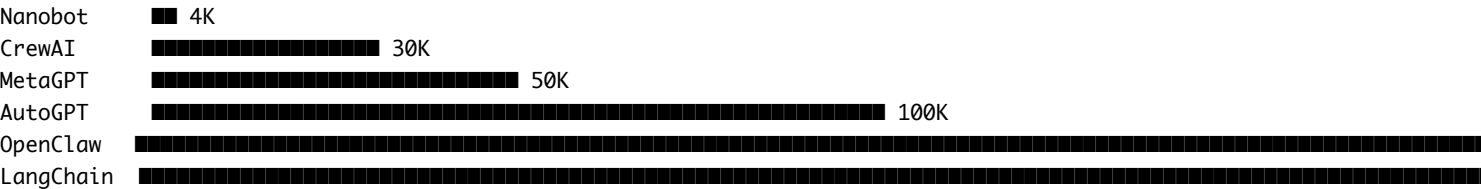
v0.1-v0.2.1 更新：

完整项目代码量对比：

框架	核心代码	完整项目
Nanobot	~6K 行	64K 行
LangChain	~500K 行	500K+ 行
OpenClaw	~430K 行	430K+ 行

可视化对比：

代码行数（K = 千行）



4000 行代码如何做到的？

Nanobot 的极简不是因为功能少，而是因为设计哲学：

1. 利用 **LLM** 的能力，而不是重新造轮子

很多框架花大量代码去实现复杂的规划器（Planner）、任务图（Task Graph）、状态机等。Nanobot 认为：LLM 本身就是最好的规划器，不需要额外的编排引擎。

```
# 其他框架的做法：复杂的任务编排
task_graph = TaskGraph()
task_graph.add_node("search", SearchTool())
task_graph.add_node("analyze", AnalyzeTool())
task_graph.add_edge("search", "analyze")
task_graph.compile().run()

# Nanobot 的做法：让 LLM 自己决定
# LLM 根据上下文自主决定调用什么工具、什么顺序
response = provider.chat_with_retry(messages, tools=available_tools)
```

2. 配置驱动而不是代码驱动

Nanobot 通过 YAML 配置文件来组装功能，而不是用代码硬编码：

```
# nanobot.yml - 一个配置文件定义完整 Agent
name: " 我的助手"
provider:
  model: "gpt-4"
  api_key: "${OPENAI_API_KEY}"
channels:
  - type: telegram
    token: "${TELEGRAM_TOKEN}"
mcp_servers:
  - name: filesystem
    command: "npx @anthropic/filesystem-server"
```

3. 文件系统代替数据库

记忆存储用 Markdown 文件（MEMORY.md、HISTORY.md），不需要向量数据库或关系型数据库。

4. 标准协议代替自定义抽象

使用 MCP 标准协议，不需要自己定义一套复杂的工具调用抽象层。

面试中如何解释“超轻量级”

“Nanobot 只有约 4000 行 Python 代码，是 LangChain 代码量的 1/125。它之所以能做到这么精简，核心在于三个设计决策：第一，充分利用 LLM 的自主规划能力，不需要额外的任务编排引擎；第二，采用配置驱动的架构，通过 YAML 文件组装功能；第三，使用 MCP 标准协议和文件系统来代替自定义抽象层和数据库。这种设计理念可以类比 Unix 哲学——‘做一件事，把它做好’。”

2.3 核心特性详解

特性一：11+ LLM 供应商支持

Nanobot 通过 Provider 抽象层，统一支持主流 LLM 供应商：

供应商	代表模型	Provider 类型
OpenAI	GPT-4o, GPT-4-turbo	openai
Anthropic	Claude 3.5, Claude 3	anthropic
Google	Gemini 2.0, Gemini 1.5	google
DeepSeek	DeepSeek-V3, DeepSeek-R1	deepseek
通义千问	Qwen-Max, Qwen-Plus	dashscope
Mistral	Mistral-Large	mistral
Groq	Llama 3, Mixtral	groq
Ollama	本地模型	ollama
Azure OpenAI	GPT-4 (Azure)	azure
OpenRouter	多模型路由	openrouter

供应商	代表模型	Provider 类型
自定义	任何兼容 OpenAI 格式	openai_compatible

v0.1-v0.2.1 更新:

实际支持 **39+** **Provider**，主要包括：

国际 **Provider**: OpenAI、Anthropic、Gemini、Mistral、Groq、OpenRouter、HuggingFace 国内 **Provider**: DeepSeek、DashScope (通义千问)、Zhipu (智谱)、Moonshot (Kimi)、MiniMax、StepFun (阶跃星辰)、Skywork 聚合 **Provider**: OpenRouter、AiHubMix、SiliconFlow、Novita 云平台 **Provider**: Azure OpenAI、AWS Bedrock、VolcEngine (火山引擎)、BytePlus **OAuth Provider**: OpenAI Codex、GitHub Copilot 本地 **Provider**: Ollama、vLLM、LM Studio 其他: LongCat、Atomic Chat、NVIDIA NIM 等

配置示例：

```
# 使用 DeepSeek (国产模型, 性价比高)
provider:
  type: deepseek
  model: deepseek-chat
  api_key: ${DEEPSEEK_API_KEY}
```

```
# 使用通义千问
provider:
  type: dashscope
  model: qwen-max
  api_key: ${DASHSCOPE_API_KEY}
```

```
# 使用本地 Ollama 模型 (完全免费)
provider:
  type: ollama
  model: llama3:70b
  base_url: http://localhost:11434
```

v0.1-v0.2.1 更新:

实际使用 JSON 格式配置 (~/.nanobot/config.json):

```
{
  "providers": {"deepseek": {"apiKey": "${DEEPSEEK_API_KEY}"},
  "modelPresets": {"primary": {"provider": "deepseek", "model": "deepseek-chat"}},
  "agents": {"defaults": {"modelPreset": "primary"}}
}
```

面试价值：展示你对 LLM 生态的广泛了解，以及对 Provider 模式的理解。

特性二：**8+** 聊天平台支持

这是 Nanobot 的差异化特色——它不只是一个开发框架，而是一个可以直接部署到各种聊天平台的 Agent 系统。

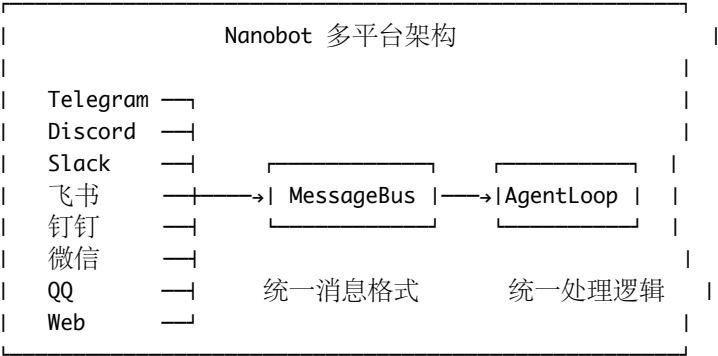
平台	国内/国际	Channel 类型	使用场景
Telegram	国际	telegram	个人/社群
Discord	国际	discord	社区/游戏
Slack	国际	slack	企业协作
飞书	国内	feishu	企业协作
钉钉	国内	dingtalk	企业协作
微信	国内	wechat	个人/企业

平台	国内/国际	Channel 类型	使用场景
QQ	国内	qq	个人/社群
Web	通用	web	通用

v0.1-v0.2.1 更新:

实际支持 15+ Channel:

国际平台: Telegram、Discord、Slack、WhatsApp、Signal、MSTeams、Matrix、Email 国内平台: Feishu (飞书)、DingTalk (钉钉)、Weixin (微信)、Wecom (企业微信)、QQ 其他平台: Napcat、MoChat 通用通道: WebSocket (WebUI)、CLI



设计亮点: 所有平台的消息通过 MessageBus 统一为 InboundMessage 格式, AgentLoop 完全不需要关心消息来自哪个平台。这是典型的适配器模式。

特性三: MCP 协议原生支持

Nanobot 从设计之初就内置了 MCP (Model Context Protocol) 支持。

```
# 配置 MCP Server
mcp_servers:
  - name: filesystem
    command: "npx @anthropic/mcp-server-filesystem /home/user"
  - name: github
    command: "npx @anthropic/mcp-server-github"
    env:
      GITHUB_TOKEN: "${GITHUB_TOKEN}"
  - name: database
    url: "http://localhost:3000/mcp"
```

v0.1-v0.2.1 更新:

实际使用 JSON 格式配置:

```
{
  "tools": {
    "mcpServers": {
      "filesystem": {"command": "npx", "args": ["-y", "@anthropic/mcp-server-filesystem", "/home/user"]},
      "github": {"command": "npx", "args": ["-y", "@anthropic/mcp-server-github"], "env": {"GITHUB_PERSONAL_ACCE
    }
  }
}
```

MCP 让 Nanobot 可以接入丰富的工具生态, 而不需要为每个工具写自定义代码。详见第 5 章。

特性四：双层记忆系统

Nanobot 的记忆系统简洁而有效：



v0.1-v0.2.1 更新：

记忆文件路径：

```
workspace/
├── memory/
│   ├── MEMORY.md      # 长期记忆
│   ├── history.jsonl   # 会话历史（JSONL格式）
│   ├── .cursor         # 读取游标
│   └── .dream_cursor   # Dream游标
├── SOUL.md            # 全局人格
└── USER.md            # 用户画像
```

新增 Dream 记忆系统（智能摘要）、GitStore 版本控制。

设计亮点： - 透明性：记忆文件就是普通 Markdown，用户可以随时查看和编辑 - 无依赖：不需要向量数据库等外部服务 - **Workspace** 隔离：每个项目有自己的记忆空间 - 虚拟工具：save_memory 不是真正的工具，而是直接在框架内部处理，减少不必要的 I/O

特性五：Skills 技能系统

Nanobot 的技能系统让 Agent 可以动态加载能力：

```
workspace/
└── .nanobot/
```

```
└─ skills/
  └─ code-review/
    └─ SKILL.md      # 代码审查技能
  └─ data-analysis/
    └─ SKILL.md      # 数据分析技能
  └─ writing/
    └─ SKILL.md      # 写作技能
```

每个 SKILL.md 文件定义了：- 技能描述（何时激活）- 详细指令（如何执行）- 约束条件

设计亮点：技能通过 Markdown 文件定义，无需写代码。用户可以创建自定义技能，按需加载到 Agent 的 system prompt 中。

特性六：子 Agent 与定时任务

子 Agent (SubAgent): - 主 Agent 可以使用 SpawnTool 创建后台子 Agent - 子 Agent 有独立的迭代限制（15 次 vs 主 Agent 的 40 次）- 子 Agent 的工具集受限（不能再 Spawn 子 Agent，防止递归）- 适用于后台长时间运行的任务

定时任务 (Cron Jobs):

```
cron:
- schedule: "0 9 * * *"      # 每天早上 9 点
  message: " 总结昨天的工作进展"
- schedule: "0 */6 * * *"    # 每 6 小时
  message: " 检查邮件并通知重要消息"
```

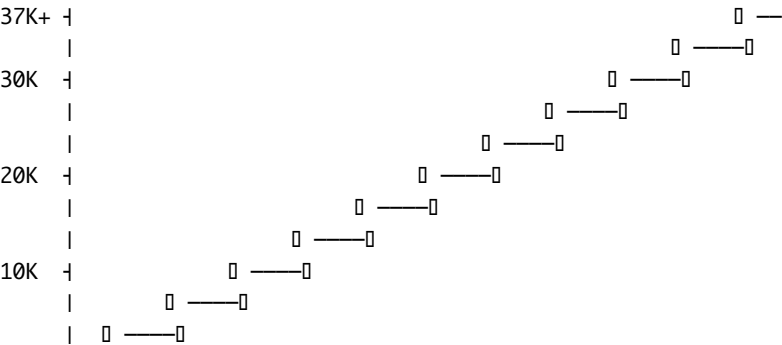
2.4 项目数据与里程碑

关键数据

指标	数据
GitHub Stars	37,000+
首次发布	2026 年 2 月
开源协议	MIT
核心代码量	~4,000 行
编程语言	Python
支持 LLM	11+ 供应商
支持平台	8+ 聊天平台
Contributors	50+
Forks	3,000+

增长轨迹

Stars 增长趋势:



2月初 2月中 2月底 3月初 3月中 3月底 4月
2026

为什么增长这么快？ 1. 时机好：Agent 概念已被市场充分教育，人们需要一个简单实用的框架 2. 定位精准：“超轻量级”这个标签极具吸引力 3. 开箱即用：支持多种国内外平台，降低使用门槛 4. 学术背景：HKUDS 的学术声誉为项目背书 5. 社区驱动：MIT 协议让社区贡献积极

2.5 为什么选 Nanobot 学习

面试价值分析

维度	Nanobot 的优势	其他框架的问题
源码可读性	4000 行，一个周末读完	数万行甚至数十万行，根本读不完
架构理解	五层架构清晰明了	抽象层过多，难以把握全局
设计模式	10+ 经典设计模式可讲	模式混杂，难以提炼
技术热点	MCP 原生支持（面试热门话题）	后来追加，理解不深
项目热度	37K Stars，面试官大概率听过	需要额外解释项目背景
差异化	很少有人深入研究 Nanobot	人人都说学过 LangChain
实战性	可以快速搭建真实的 Agent	搭建过程复杂，Demo 效果一般

“以小见大”的学习策略

通过 Nanobot 这 4000 行代码，你可以理解：

Agent 核心概念

- └ AgentLoop → 理解 Agent 的推理循环
- └ Memory → 理解 Agent 的记忆管理
- └ Tools → 理解 Agent 的工具调用
- └ MCP → 理解工具标准化协议

软件工程思想

- └ 异步编程 → asyncio 实战
- └ 设计模式 → 适配器、注册表、生产者-消费者等
- └ 配置驱动 → YAML 组装系统
- └ 关注点分离 → 层次清晰的模块化设计

系统设计能力

- └ 消息队列 → MessageBus 双队列设计
- └ 并发控制 → 会话锁 + 并发闸门
- └ 插件系统 → Skill/MCP 热加载
- └ 多平台适配 → 适配器模式

与 LangChain 学习路径对比

阶段	学习 Nanobot	学习 LangChain
入门	1 天：跑通示例，理解配置	3 天：理解概念，跑通示例
架构	2 天：通读源码，理解架构	2 周：部分模块源码，理解抽象层
深入	3 天：掌握设计模式和关键实现	1 月：深入部分模块，仍有盲区
面试	1 周内可完成面试准备	需要数周，且难以讲清全局

结论：Nanobot 让你在一周内就能达到面试中“项目深度”的要求，而且因为读过全部源码，面试时任何关于架构设计的追问都能从容回答。

2.6 与其他框架的差异化对比

全维度对比表

维度	Nanobot	LangChain	CrewAI	AutoGPT	OpenClaw
设计哲学	极简、够用	全面、生态	协作导向	全自主	企业全栈
核心代码	~4K 行	~500K 行	~30K 行	~100K 行	~430K 行
学习曲线	1 天上手	1 周入门	3 天入门	3 天入门	1 周入门
Agent 模式	单 Agent+SubAgent	灵活组合	多 Agent 协作	单 Agent 循环	单 Agent/多 Agent
LLM 支持	11+ 供应商	50+ 供应商	依赖 LangChain	主要 OpenAI	多供应商
MCP 支持	原生内置	扩展支持	有限	无	支持
记忆系统	文件系统	多后端	内置	内置	多模态
平台集成	8+	需自行集成	无	Web UI	有限
定时任务	内置 Cron	无	无	无	有限
安全机制	workspace 沙箱	基本	基本	基本	完善
配置方式	YAML	代码	代码 + 装饰器	代码	代码 + 配置
部署难度	简单 (pip)	中等	中等	较复杂	复杂
适合人群	个人/小团队	企业开发者	多 Agent 场景	研究者	企业客户

核心差异解读

Nanobot vs LangChain

LangChain 的设计理念: "提供一切可能需要的工具和抽象"

- └─ 优势: 生态丰富, 几乎什么都能做
- └─ 劣势: 过度封装, 代码复杂, 更新频繁导致 API 不稳定
- └─ 类比: 一个装满工具的大工具箱

Nanobot 的设计理念: "提供恰好够用的核心能力"

- └─ 优势: 代码简洁, 架构清晰, 一目了然
- └─ 劣势: 功能较少, 生态不如 LangChain 丰富
- └─ 类比: 一把精心打造的瑞士军刀

面试说法: > "如果说 LangChain 是一个大型超市, 什么都有但找东西费劲; 那 Nanobot 就是一家精品店, 东西不多但每样都精心挑选。从学习角度, Nanobot 更适合理解 Agent 的本质设计, 就像学操作系统不应该从 Linux 内核开始, 而应该从 xv6 这样的教学操作系统开始。"

Nanobot vs AutoGPT

AutoGPT: 追求"完全自主"

- └─ 想法前卫, 但实际可靠性低
- └─ Token 消耗大, 容易陷入循环
- └─ 更像一个概念验证 (PoC)

Nanobot: 追求"实用可靠"

- └─ 明确的迭代限制 (40次/15次) 防止无限循环
- └─ workspace 沙箱保证安全
- └─ 更像一个生产就绪的工具

Nanobot vs CrewAI

CrewAI: 多 Agent 协作框架

- └─ 擅长多个 Agent 角色分工

- └ 但单 Agent 场景下过于复杂
- └ 依赖 LangChain 生态

Nanobot: 以单 Agent 为核心

- └ 通过 SubAgent 实现有限的多 Agent
- └ 单 Agent 场景下简洁高效
- └ 完全独立, 无外部框架依赖

2.7 面试话术

话术一: 介绍你学的 Nanobot 项目

“我深入学习了 HKUDS/nanobot 这个项目, 它是香港大学数据科学实验室开源的超轻量级 AI Agent 框架, GitHub 上有 37K+ Stars。这个项目最大的特点是只用了约 4000 行 Python 代码, 就实现了一个完整的 Agent 框架, 包括消息总线、AgentLoop 推理循环、MCP 协议支持、双层记忆系统、技能加载系统, 以及 8 个以上聊天平台的适配。

我通读了它的全部源码, 重点研究了三个方面: 一是 AgentLoop 的 ReAct 循环实现, 包括会话锁和并发控制; 二是 MCP 协议在框架中的集成方式, 理解了 MCPToolWrapper 如何将远程 MCP 工具包装为本地工具; 三是双层记忆系统的设计, 特别是 save_memory 虚拟工具和 MemoryConsolidator 的压缩机制。”

话术二: 为什么选这个项目而不是 LangChain

“我选择 Nanobot 而不是 LangChain, 主要考虑了三点。第一, 深度胜过广度——Nanobot 只有 4000 行代码, 我可以读完全部源码并理解每一个设计决策, 这在面试中意味着任何追问我都能回答; 第二, Nanobot 的架构更纯粹, 它把 Agent 的核心概念——循环推理、记忆管理、工具调用——用最简洁的方式实现了, 没有 LangChain 那种过度封装的问题; 第三, 它原生支持 MCP 协议, 这是目前 AI 工具调用的标准化方向, 面试中可以展示我对技术趋势的把握。”

话术三: Nanobot 最大的技术亮点是什么

“我认为 Nanobot 最大的技术亮点是它的‘配置驱动 + LLM 自主规划’的设计哲学。很多 Agent 框架花大量代码去实现复杂的任务编排引擎、状态机、DAG 执行器。但 Nanobot 的核心洞察是: LLM 本身就是最好的规划器。所以 Nanobot 只需要实现一个简洁的 ReAct 循环 (AgentRunner), 让 LLM 自己决定调用什么工具、什么顺序、什么时候结束。这种设计让 4000 行代码就能实现其他框架数十万行才能做到的功能。”

2.8 本章总结

核心知识点回顾

本章核心要点	
1. Nanobot 是 HKUDS 开源的超轻量级 Agent 框架	
→ 4000行代码 37K+ Stars MIT 协议	
2. 六大核心特性	
→ 11+ LLM 供应商	
→ 8+ 聊天平台	
→ MCP 协议原生支持	
→ 双层记忆系统	
→ Skills 技能系统	
→ 子Agent + 定时任务	
3. "超轻量"的设计哲学	
→ 利用 LLM 规划能力	

	→ 配置驱动代替代码驱动	
	→ 文件系统代替数据库	
	→ 标准协议代替自定义抽象	
	4. 面试价值	
	→ 源码可读 架构清晰 差异化 热度高	

面试 Checklist

- ☐ 能用 30 秒介绍 Nanobot 项目
- ☐ 能解释“超轻量级”的含义和设计决策
- ☐ 能列举 Nanobot 的 6 大核心特性
- ☐ 能对比 Nanobot 与 LangChain/AutoGPT 的差异
- ☐ 能说出选择 Nanobot 学习的理由

下一章

接下来，我们将深入 Nanobot 的架构设计，理解它的五层架构和四大核心模块。

03 - 架构深入解析

本章小结：Nanobot 是一个“极简但完整”的 AI Agent 框架。它用 4000 行代码证明了：好的架构设计不在于代码多少，而在于是否抓住了问题的本质。对面试者而言，Nanobot 是一个“以小见大”的完美学习素材。

03 - 架构深入解析

本章目标：深入理解 Nanobot 的五层架构、四大核心模块、数据流以及 10 个关键设计模式。这一章是面试中展现“技术深度”的核心素材。

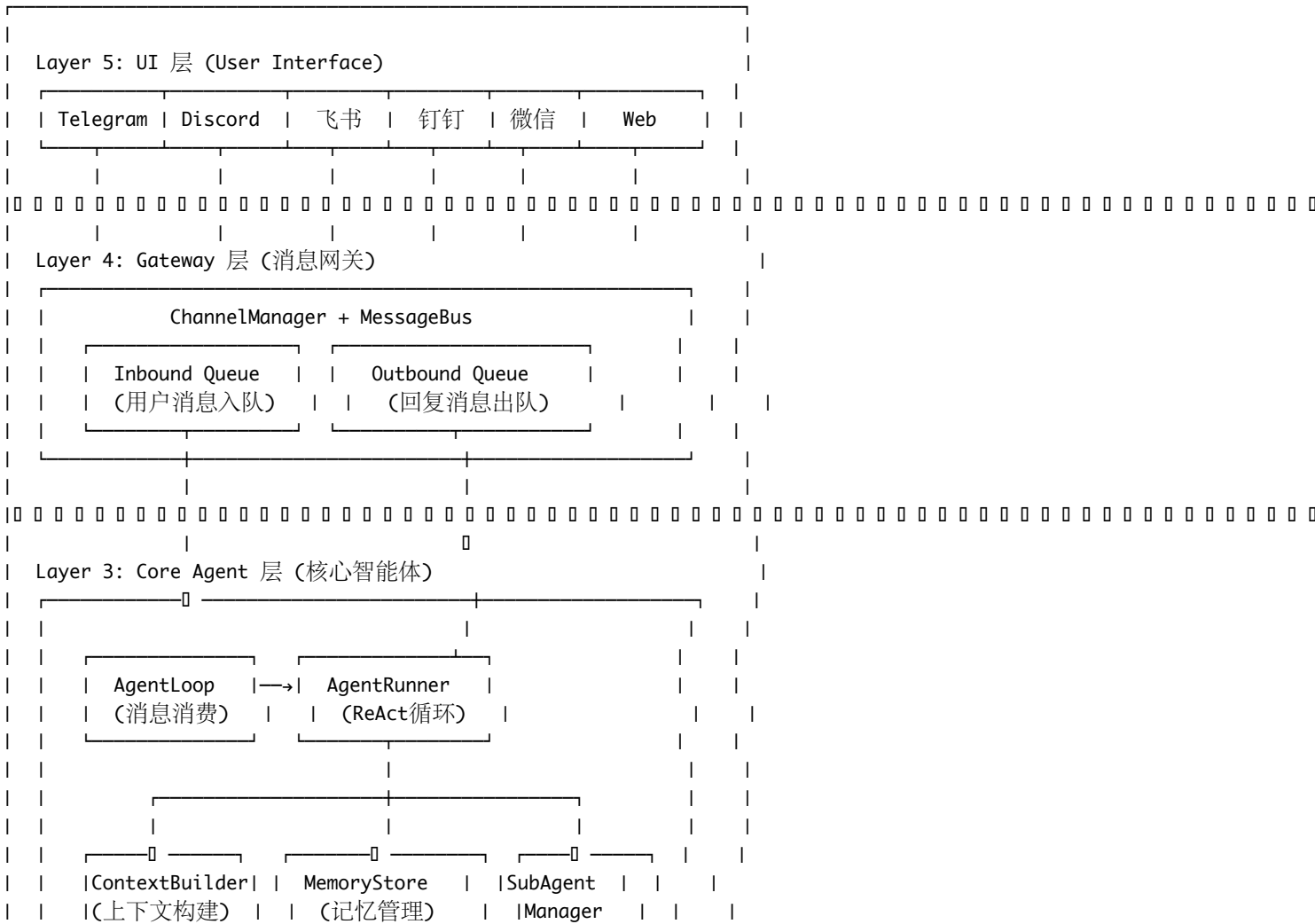
目录

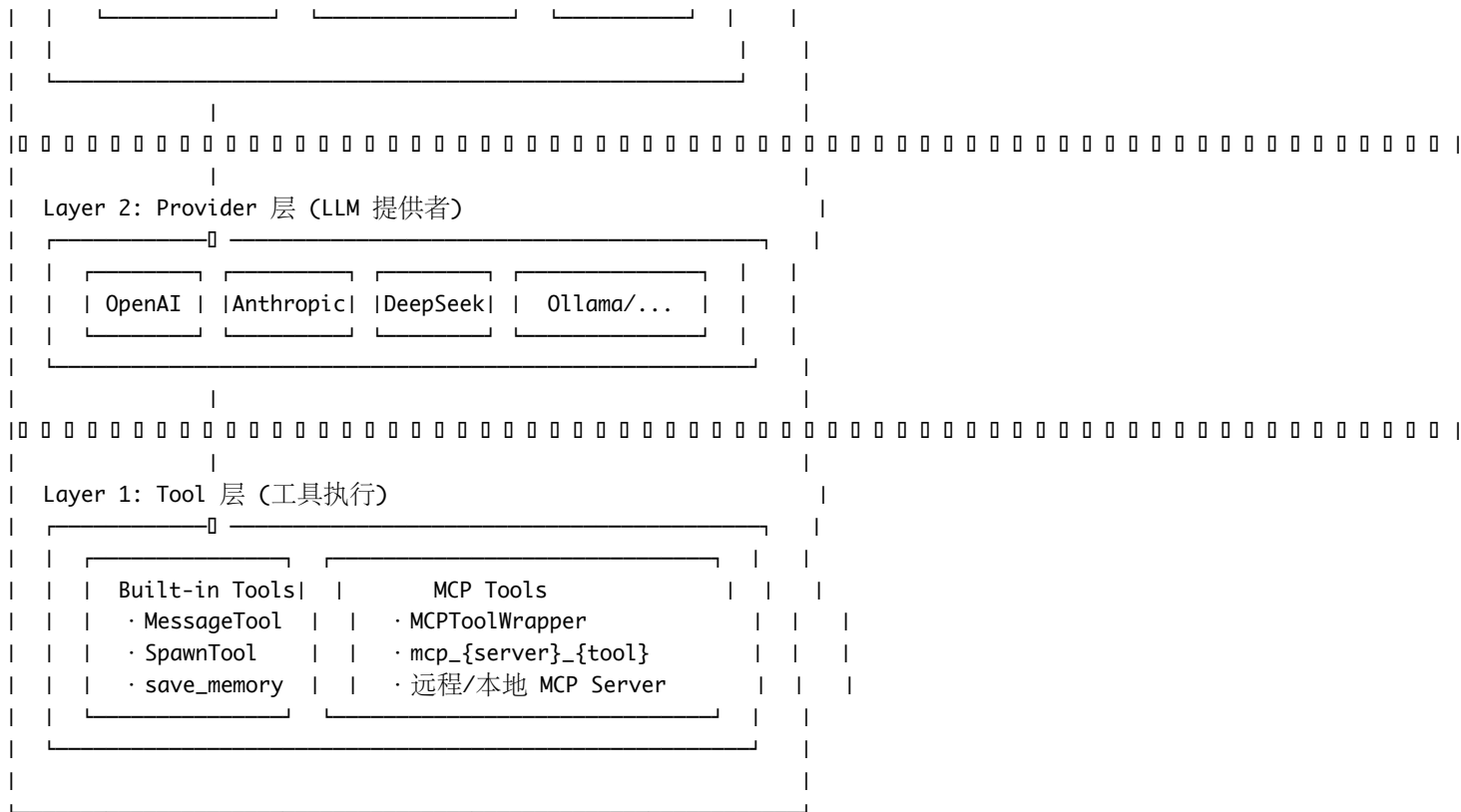
- 3.1 整体架构概览
- 3.2 四大核心模块详解
- 3.3 数据流图
- 3.4 模块间的协作关系
- 3.5 关键设计模式
- 3.6 架构对比
- 3.7 面试高频题
- 3.8 本章总结

3.1 整体架构概览

五层架构

Nanobot 的架构可以清晰地划分为五个层次，从上到下分别是：





各层职责说明

层次	名称	核心职责	关键组件
Layer 5	UI 层	面向用户的交互界面	Telegram/Discord/飞书/钉钉/微信等 Channel 适配器
Layer 4	Gateway 层	消息的统一收发	MessageBus（双队列）、ChannelManager
Layer 3	Core Agent 层	核心推理和决策	AgentLoop、AgentRunner、ContextBuilder、MemoryStore、SubagentManager
Layer 2	Provider 层	LLM 调用封装	OpenAI/Anthropic/DeepSeek 等 Provider
Layer 1	Tool 层	工具注册与执行	ToolRegistry、MCPToolWrapper、内置工具

架构设计原则

Nanobot 的架构遵循以下核心原则：

1. 关注点分离（Separation of Concerns）

每一层只负责自己的职责，层与层之间通过明确的接口交互。Channel 不需要知道 AgentLoop 的实现细节，AgentLoop 不需要知道消息来自哪个平台。

2. 依赖倒置（Dependency Inversion）

高层模块不依赖低层模块的具体实现，而是依赖抽象接口。例如 AgentRunner 不直接依赖 OpenAI SDK，而是通过 Provider 抽象层调用 LLM。

3. 单一职责（Single Responsibility）

每个类只负责一件事：AgentLoop 负责消息消费和会话管理，AgentRunner 负责 ReAct 循环，ContextBuilder 负责构建 prompt，MemoryStore 负责记忆读写。

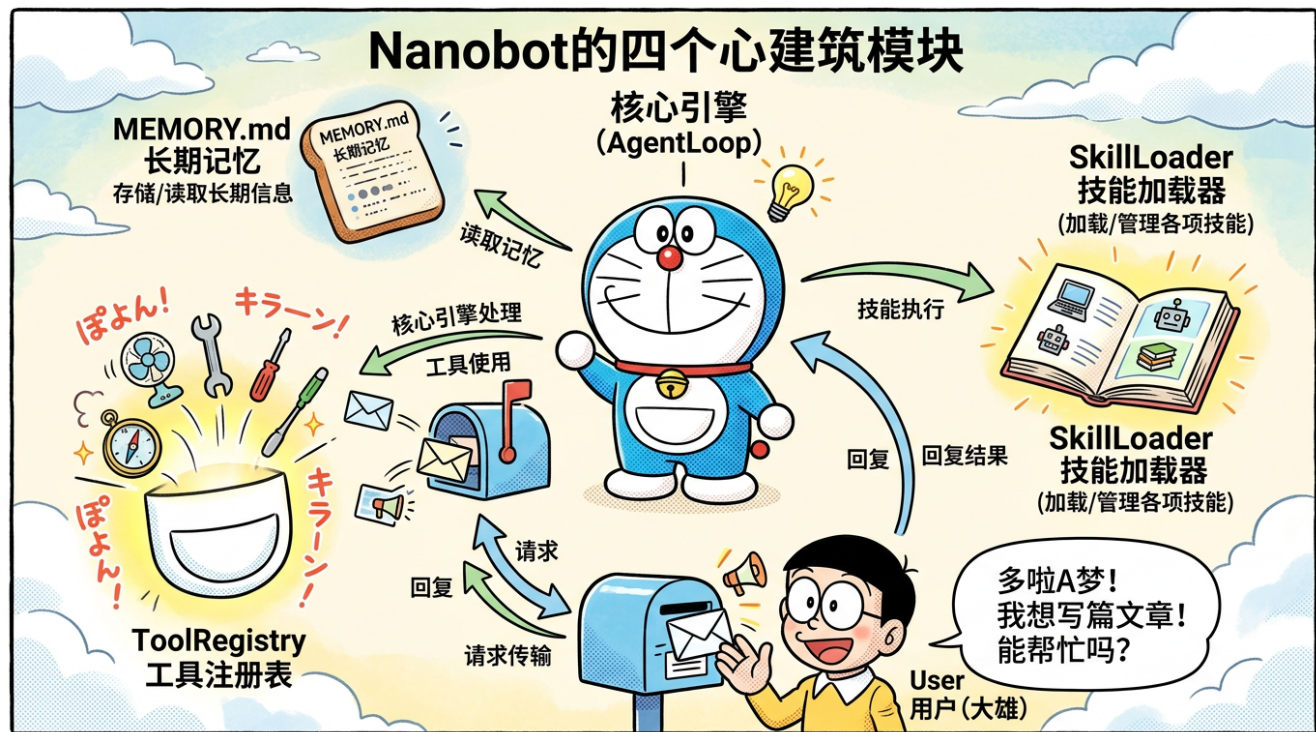


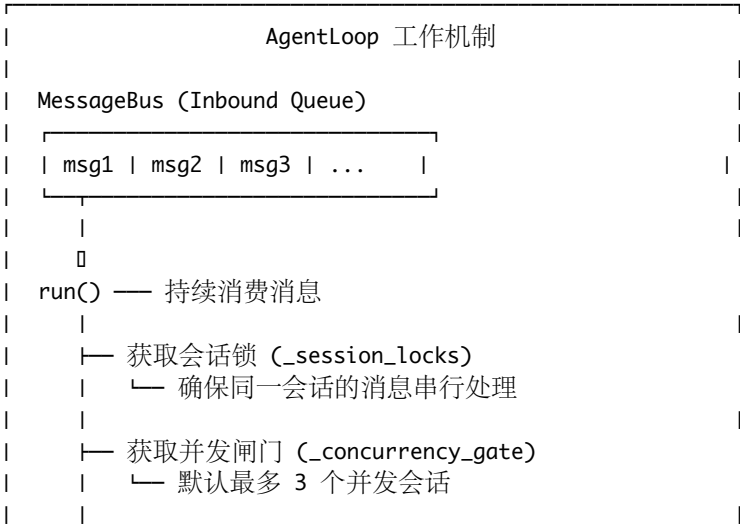
Figure 1: Nanobot 架构漫画

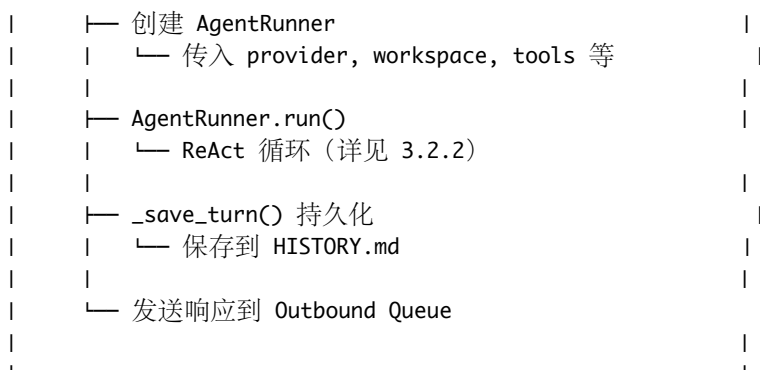
Nanobot 四大核心模块：AgentLoop（核心引擎）、ToolRegistry（工具注册表）、MEMORY.md（长期记忆）、SkillLoader（技能加载器）

3.2 四大核心模块详解

3.2.1 AgentLoop（智能体循环）——核心推理引擎

AgentLoop 是 Nanobot 的”心脏”，它负责： - 从 MessageBus 的 Inbound Queue 消费消息 - 管理会话级别的串行执行（会话锁） - 控制全局并发数（并发闸门） - 创建 AgentRunner 来处理每个消息 - 保存每轮对话（_save_turn）



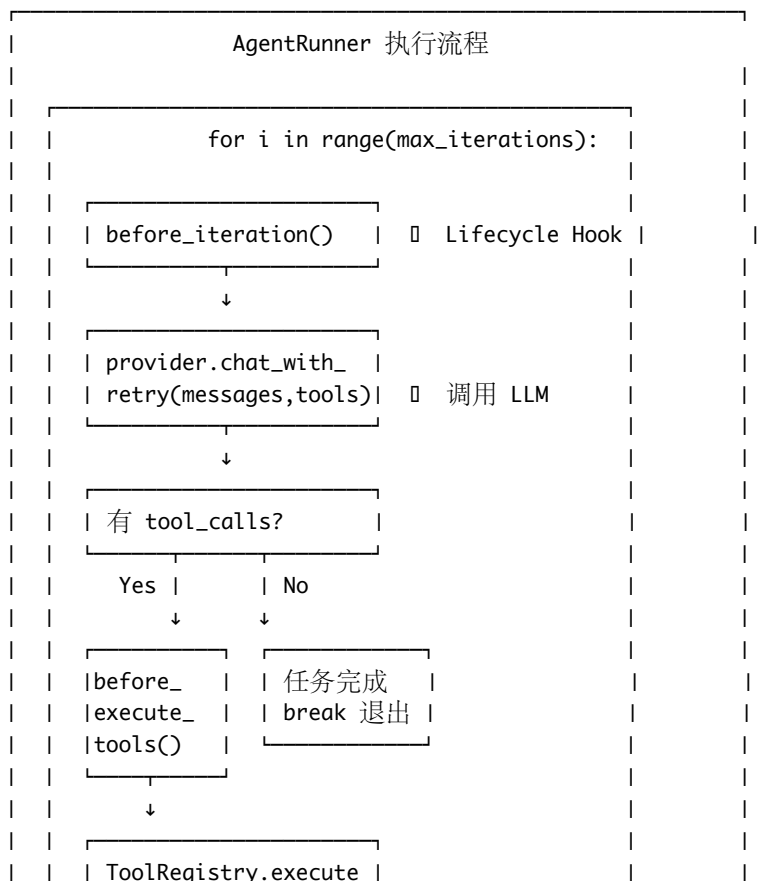


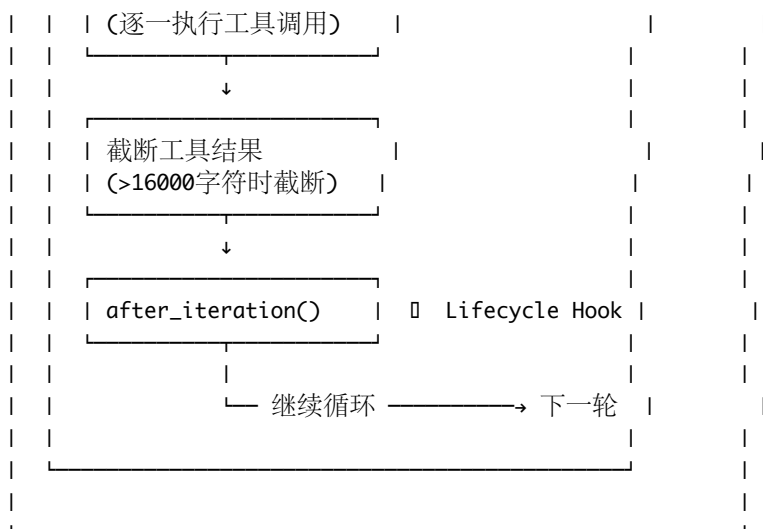
关键设计决策：

决策	设计	原因
会话锁	<code>_session_locks[session_id]</code>	防止同一用户的多条消息并发处理导致上下文混乱
并发闸门	<code>asyncio.Semaphore(3)</code>	控制全局并发，防止过多请求打垮 LLM API
最大迭代	主 Agent 40 次，子 Agent 15 次	防止无限循环，同时给予足够的迭代空间
工具注册	<code>_register_default_tools()</code>	启动时注册内置工具，支持运行时动态添加

3.2.2 AgentRunner (ReAct 循环) —— 迭代执行器

AgentRunner 实现了完整的 ReAct 循环，它是 Agent “思考-行动-观察”的执行者。





Lifecycle Hooks 详解:

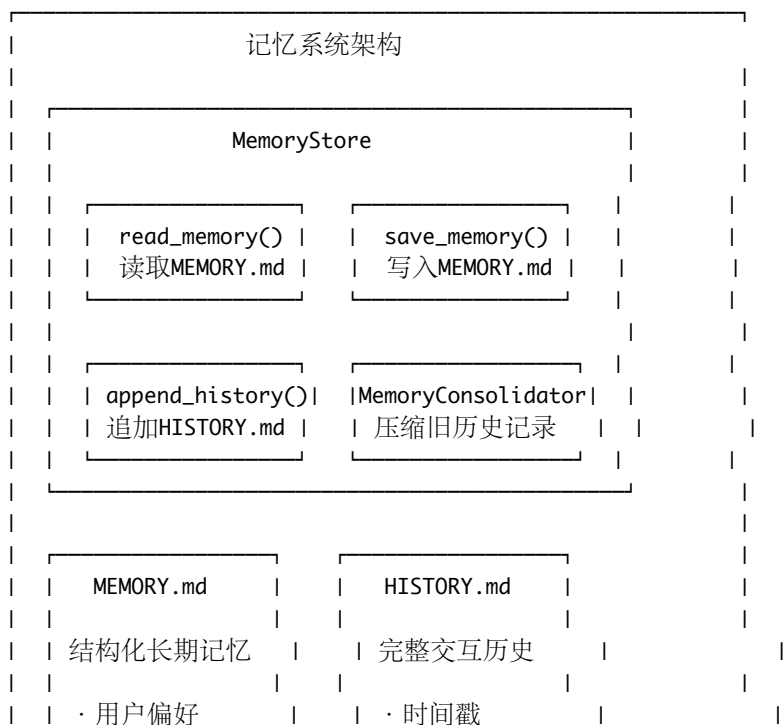
Hook	触发时机	用途
before_iteration	每轮迭代开始前	可用于日志记录、状态检查
before_execute_tools	工具执行前	可用于工具调用审查、安全检查
after_iteration	每轮迭代结束后	可用于结果记录、指标统计

工具结果截断:

AgentRunner 有一个重要的优化——`_TOOL_RESULT_MAX_CHARS = 16000`。当工具返回的结果超过 16000 个字符时, 会被截断。这是因为: - 过长的工具结果会占用大量上下文窗口 - LLM 处理超长文本的效果会下降 - 可以节省 Token 成本

3.2.3 MemoryStore (记忆系统) ——MEMORY.md + HISTORY.md

MemoryStore 管理 Nanobot 的双层记忆系统。



· 项目信息	· 用户消息摘要
· 重要决策	· Agent 回复摘要
· 学到的教训	· 工具调用记录
由 <code>save_memory</code>	自动追加
虚拟工具触发写入	定期压缩

`save_memory` 虚拟工具的巧妙设计：

`save_memory` 不是一个真正的外部工具——它不调用任何 API 或启动任何进程。当 LLM 决定调用 `save_memory` 时，`AgentRunner` 直接在内部拦截这个调用，将内容写入 `MEMORY.md` 文件。

这种“虚拟工具”设计的好处：- 对 LLM 来说，它和其他工具没有区别（统一的调用接口）- 执行效率高（无网络开销）- 实现简单（直接文件 I/O）

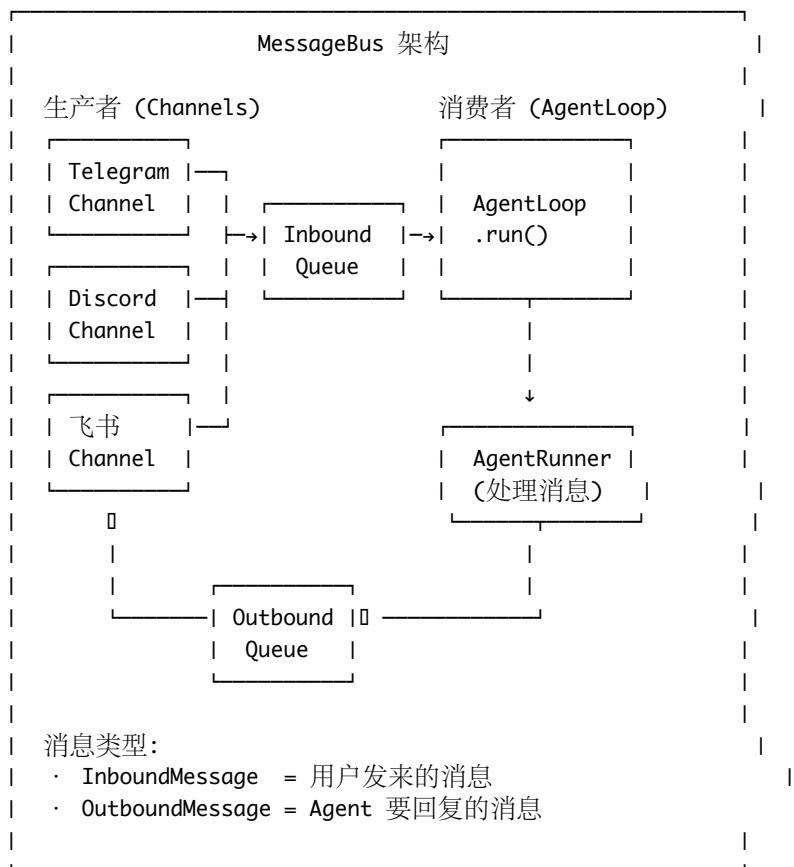
MemoryConsolidator 压缩机制：

当 `HISTORY.md` 增长到一定大小时，`MemoryConsolidator` 会：1. 读取较旧的历史记录 2. 使用 LLM 生成压缩摘要 3. 用摘要替换详细记录 4. 保留近期的详细记录

这确保了记忆文件不会无限增长，同时保留了重要的历史信息。

3.2.4 MessageBus（消息总线）——双队列架构

`MessageBus` 是 `Nanobot` 的消息中枢，使用经典的生产者-消费者模式。



关键设计：

设计	实现	作用
双队列	Inbound + Outbound	解耦消息收发，Channel 和 Agent 互不阻塞
异步队列 统一消息格式	<code>asyncio.Queue</code> <code>InboundMessage</code> / <code>OutboundMessage</code>	非阻塞的消息传递，高并发支持 屏蔽平台差异，Agent 不关心消息来源
会话标识	<code>session_id</code>	标识消息所属会话，用于会话锁定

3.3 数据流图

完整消息处理流程

下面描述一条用户消息从发出到收到回复的完整数据流：

步骤 1：用户发送消息

```
| 用户在 Telegram 发送: "帮我查一下北京天气" |
```



步骤 2：Channel 接收并转换

```
| TelegramChannel.on_message() |
| → 将 Telegram 消息转换为 InboundMessage |
| → InboundMessage { |
|   session_id: "tg_12345", |
|   user_id: "user_001", |
|   text: "帮我查一下北京天气", |
|   channel: "telegram", |
|   timestamp: 1711958400 |
| } |
```



步骤 3：进入 Inbound Queue

```
| MessageBus.inbound.put(inbound_message) |
```



步骤 4：AgentLoop 消费消息

```
| AgentLoop.run(): |
| → msg = await bus.inbound.get() |
| → 获取会话锁: _session_locks["tg_12345"] |
| → 获取并发闸门: _concurrency_gate.acquire() |
| → 加载 workspace 上下文 |
```



步骤 5：ContextBuilder 构建 Prompt

```
| ContextBuilder.build_system_prompt(): |
| → Identity: "你是一个 AI 助手..." |
| → Bootstrap: 基础行为规范 |
| → Memory: 读取 MEMORY.md 的内容 |
| → Skills: 加载匹配的 SKILL.md |
| |
| ContextBuilder.build_messages(): |
```

```
| → [system_prompt] + [history] + [user_message] |
```

↓

步骤 6: AgentRunner ReAct 循环

```
| 第 1 轮迭代: |
| → LLM: "用户想查北京天气, 我需要调用天气工具" |
| → tool_call: weather_api(city="北京") |
| → 工具返回: {"temp": 22, "weather": "晴"} |
| |
| 第 2 轮迭代: |
| → LLM: "我已经得到天气数据, 可以回复用户了" |
| → text: "北京今天天气晴朗, 气温 22°C, 适合外出。" |
| → 没有 tool_calls, 循环结束 |
```

↓

步骤 7: 保存记忆和历史

```
| MemoryStore.append_history(): |
| → 将本轮对话摘要追加到 HISTORY.md |
| |
| AgentLoop._save_turn(): |
| → 持久化本轮对话数据 |
```

↓

步骤 8: 发送响应

```
| MessageBus.outbound.put(outbound_message) |
| → OutboundMessage { |
|   session_id: "tg_12345", |
|   text: "北京今天天气晴朗, 气温 22°C, 适合外出。", |
|   channel: "telegram" |
| } |
```

↓

步骤 9: Channel 发送回复

```
| TelegramChannel.send(): |
| → 将 OutboundMessage 转换为 Telegram API 请求 |
| → 发送到 Telegram 服务器 |
| → 用户在 Telegram 收到回复 |
```

3.4 模块间的协作关系

依赖关系图

```
| 模块依赖关系 |
| |
| AgentLoop |
| └─ 依赖 → MessageBus (消费/推送消息) |
| └─ 依赖 → AgentRunner (委托执行) |
| └─ 依赖 → MemoryStore (保存历史) |
| └─ 依赖 → ToolRegistry (注册工具) |
```

	AgentRunner	
	├─ 依赖 → Provider (调用 LLM)	
	├─ 依赖 → ToolRegistry (执行工具)	
	├─ 依赖 → ContextBuilder (构建 prompt)	
	└─ 可选 → SubagentManager (创建子Agent)	
	ContextBuilder	
	├─ 依赖 → MemoryStore (读取记忆)	
	├─ 依赖 → SkillLoader (加载技能)	
	└─ 依赖 → Config (读取配置)	
	ToolRegistry	
	├─ 管理 → Built-in Tools (内置工具)	
	└─ 管理 → MCPToolWrapper (MCP 工具)	
	ChannelManager	
	├─ 管理 → 各 Channel 适配器	
	└─ 依赖 → MessageBus (消息收发)	

创建顺序

Nanobot 启动时的组件创建顺序:

1. 解析配置文件 (nanobot.yml)
2. 创建 MessageBus
3. 创建 Provider (LLM 客户端)
4. 创建 ToolRegistry
5. 注册内置工具
6. 连接 MCP Servers → 注册 MCP 工具
7. 创建 MemoryStore
8. 创建 ContextBuilder
9. 创建 AgentLoop
10. 创建 ChannelManager → 注册各 Channel
11. 启动 AgentLoop (开始消费消息)
12. 启动各 Channel (开始接收消息)

3.5 关键设计模式

Nanobot 在 4000 行代码中应用了多种经典设计模式。这是面试中展现软件工程素养的好素材。

模式 1: 异步消息总线 / 生产者-消费者模式

应用场景: MessageBus 的双队列设计

```
# 生产者 (Channel)
async def on_message(self, raw_msg):
    inbound = InboundMessage(...)
    await self.bus.inbound.put(inbound) # 生产

# 消费者 (AgentLoop)
async def run(self):
    while True:
        msg = await self.bus.inbound.get() # 消费
        await self._process(msg)
```

设计价值: - 解耦了消息的生产 (Channel) 和消费 (AgentLoop) - 支持多个生产者 (多 Channel) 同时工作 - 天然支持异步和并发 - 可以轻松增加消费者 (水平扩展)

模式 2: 注册表模式 (Registry Pattern)

应用场景: Provider 注册和 ToolRegistry

```
# Provider 注册表
PROVIDERS = {
    "openai": OpenAIProvider,
    "anthropic": AnthropicProvider,
    "deepseek": DeepSeekProvider,
    "ollama": OllamaProvider,
    # ...
}

# 使用时通过名称查找
provider_cls = PROVIDERS[config.provider.type]
provider = provider_cls(config.provider)

# 工具注册表
class ToolRegistry:
    def __init__(self):
        self._tools = {}

    def register(self, name, tool):
        self._tools[name] = tool

    def execute(self, name, args):
        return self._tools[name].run(args)
```

设计价值: - 新增 Provider 只需注册, 无需修改现有代码 (开闭原则) - 统一的查找和管理接口 - 支持运行时动态注册

模式 3: 策略/适配器模式 (Strategy/Adapter Pattern)

应用场景: Channel 适配器

```
# 抽象接口
class BaseChannel:
    async def receive(self) -> InboundMessage: ...
    async def send(self, msg: OutboundMessage): ...

# 具体适配器
class TelegramChannel(BaseChannel):
    async def receive(self):
        raw = await self.telegram_api.get_update()
        return InboundMessage(text=raw.text, ...)

    async def send(self, msg):
        await self.telegram_api.send_message(msg.text)

class FeishuChannel(BaseChannel):
    async def receive(self):
        raw = await self.feishu_api.get_event()
        return InboundMessage(text=raw.content, ...)

    async def send(self, msg):
        await self.feishu_api.send_card(msg.text)
```


设计价值: - AgentLoop 只面向 BaseChannel 接口编程, 不关心具体平台 - 新增平台只需实现 BaseChannel 接口 - 统一了不同平台的消息格式差异

模式 4: 包装器模式 (Wrapper/Decorator Pattern)

应用场景: MCPToolWrapper

```
class MCPToolWrapper:
    """ 将远程 MCP 工具包装为本地工具接口 """

    def __init__(self, server_name, tool_spec):
        self.name = f"mcp_{server_name}_{tool_spec.name}"
        self.description = tool_spec.description
        self.schema = self._normalize_schema(tool_spec.input_schema)

    async def run(self, args):
        # 将本地调用转换为 MCP 协议的远程调用
        result = await self.mcp_client.call_tool(
            self.original_name, args
        )
        return result

    def _normalize_schema(self, schema):
        """ 将 MCP schema 转换为 OpenAI 兼容格式 """
        # 处理 schema 差异
        return normalized_schema
```

设计价值: - 外部 MCP 工具对 AgentRunner 来说和内置工具没有区别 - 封装了 MCP 协议的通信细节 - 处理了 schema 格式差异 (MCP 格式 → OpenAI 格式)

模式 5: 配置驱动组装 (Configuration-Driven Assembly)

应用场景: nanobot.yml 驱动整个系统的构建

```
# nanobot.yml
name: " 我的 AI 助手"
provider:
    type: deepseek
    model: deepseek-chat
channels:
    - type: telegram
      token: ${TELEGRAM_TOKEN}
    - type: feishu
      app_id: ${FEISHU_APP_ID}
mcp_servers:
    - name: filesystem
      command: "npx @anthropic/mcp-server-filesystem /data"
memory:
    consolidation_threshold: 50

# 框架根据配置文件自动组装
def build_from_config(config_path):
    config = load_yaml(config_path)
    bus = MessageBus()
    provider = PROVIDERS[config.provider.type](config.provider)
    tools = ToolRegistry()
    for mcp in config.mcp_servers:
        tools.register_mcp(mcp)
    for ch in config.channels:
```

```

    ChannelManager.add(CHANNELS[ch.type](ch, bus))
    return AgentLoop(bus, provider, tools)

```

设计价值: - 用户无需写代码, 通过配置文件定义 Agent - 系统组装逻辑与业务逻辑分离 - 支持环境变量插值 (`${VAR}`), 安全管理密钥

模式 6: **Workspace** 为中心

应用场景: 所有与项目相关的文件都在 workspace 目录中

```

workspace/
├── nanobot.yml          # Agent 配置
├── MEMORY.md            # 长期记忆
├── HISTORY.md           # 交互历史
├── .nanobot/
│   └── skills/          # 技能文件
│       ├── coding/SKILL.md
│       └── writing/SKILL.md
└── user_files/          # 用户的项目文件
    ├── src/
    └── docs/

```

设计价值: - 自然隔离不同项目的上下文和记忆 - 所有配置和状态集中管理 - 版本控制友好 (可以 git 管理整个 workspace) - `restrict_to_workspace` 选项提供安全沙箱

模式 7: 渐进披露 (**Progressive Disclosure**)

应用场景: 配置和功能的逐步展示

```

# 最简配置 (5 行)
name: " 助手"
provider:
  type: openai
  model: gpt-4
  api_key: ${OPENAI_API_KEY}

```

```

# 进阶配置 (按需添加)
channels:
  - type: telegram
    token: ${TOKEN}
mcp_servers:
  - name: tools
    command: "..."
memory:
  consolidation_threshold: 50
cron:
  - schedule: "0 9 * * *"
    message: " 早安! "

```

设计价值: - 新手 5 行配置即可启动 - 高级用户可以逐步添加高级功能 - 降低了入门门槛

模式 8: 虚拟工具 (**Virtual Tool**)

应用场景: `save_memory` 工具

```

# save_memory 不是真正的外部工具
# 当 LLM 决定调用 save_memory 时:
if tool_call.name == "save_memory":
    # 直接在框架内部处理, 不走 ToolRegistry
    memory_store.save(tool_call.args["content"])
    return "Memory saved successfully"

```

设计价值：- 对 LLM 来说，save_memory 和其他工具没有区别 - 减少了不必要的 I/O 和网络开销 - 保持了工具调用接口的统一性

模式 9：会话并发控制

应用场景：AgentLoop 的会话锁和并发闸门

```
class AgentLoop:
    def __init__(self):
        self._session_locks = {}          # 会话级别的锁
        self._concurrency_gate = asyncio.Semaphore(3) # 全局并发限制

    async def _process(self, msg):
        session_id = msg.session_id

        # 1. 会话锁：同一会话串行
        if session_id not in self._session_locks:
            self._session_locks[session_id] = asyncio.Lock()

        async with self._session_locks[session_id]:
            # 2. 并发闸门：全局最多 3 个并发
            async with self._concurrency_gate:
                await self._run_agent(msg)
```

设计价值：- 会话锁防止同一用户的消息并发处理（上下文一致性）- 并发闸门防止全局过载（API 限流保护）- 两层控制互不冲突，互补保护

模式 10：Prompt Cache 优化

应用场景：ContextBuilder 的 prompt 构建策略

```
class ContextBuilder:
    def build_messages(self, user_msg, history):
        messages = []

        # System prompt 放在最前面，利用 LLM 的 prompt cache
        # 这部分跨请求几乎不变，可以被缓存
        system = self.build_system_prompt() # Identity + Bootstrap + Memory + Skills
        messages.append({"role": "system", "content": system})

        # 历史消息
        for h in history:
            messages.append(h)

        # 当前用户消息放在最后
        messages.append({"role": "user", "content": user_msg})

        return messages
```

设计价值：- 将不变的部分（system prompt）放在前面，利用 LLM API 的 Prompt Cache 功能 - 减少重复 Token 的计费 - 降低延迟（缓存命中时不需要重新处理）- Claude、GPT-4 等都支持 Prompt Cache

3.6 架构对比

Nanobot vs LangChain 架构对比

维度	Nanobot	LangChain
核心抽象	MessageBus + AgentLoop	Chain + Agent + Memory + Tool
编排方式	LLM 自主决策（隐式）	LangGraph 显式编排 / AgentExecutor
消息传递	双队列 MessageBus	Callback 机制
工具注册	ToolRegistry + MCPToolWrapper	BaseTool + Toolkit
配置方式	YAML 文件	Python 代码
记忆实现	MEMORY.md + HISTORY.md	ConversationBufferMemory / VectorStoreMemory 等
抽象层级	2-3 层	5-7 层
源码可读性	高（4K 行，架构清晰）	低（50 万行 +，抽象层多）

关键差异：

LangChain 的做法：

用户请求 → Prompt Template → Chain → Agent → Tool → Output Parser → Memory
 □ 每一步都有抽象层

Nanobot 的做法：

用户消息 → MessageBus → AgentRunner(LLM + Tools 循环) → 响应
 □ 最小必要抽象

Nanobot vs CrewAI 架构对比

维度	Nanobot	CrewAI
Agent 模型	单 Agent + SubAgent	多 Agent 角色
任务分配	LLM 自主决定	用户预定义角色和任务
协作方式	SubAgent 后台并行	Agent 间消息传递
适用场景	个人助手	团队协作模拟

3.7 面试高频题

Q1: 请描述 Nanobot 的架构设计

标准回答：

“Nanobot 采用五层架构设计。最上层是 UI 层，支持 Telegram、飞书、钉钉等 8+ 聊天平台；第四层是 Gateway 层，核心是 MessageBus 双队列（Inbound/Outbound），负责统一消息收发；第三层是核心的 Agent 层，包括 AgentLoop（消息消费和会话管理）、AgentRunner（ReAct 循环）、ContextBuilder（prompt 构建）、MemoryStore（记忆管理）等；第二层是 Provider 层，封装了 11+ LLM 供应商的调用；最底层是 Tool 层，包括内置工具和 MCP 工具。

这个架构的核心设计思想是：通过 MessageBus 解耦消息收发，通过 AgentRunner 实现 ReAct 循环，通过 Provider 抽象层支持多种 LLM，通过 MCPToolWrapper 统一工具调用。整体只有 4000 行代码，但层次清晰、关注点分离，是非常优雅的架构设计。”

Q2: Nanobot 用了哪些设计模式？

标准回答：

“Nanobot 虽然代码精简，但运用了多种经典设计模式。主要包括：

1. 生产者-消费者模式：MessageBus 的双队列设计，Channel 是生产者，AgentLoop 是消费者；
2. 注册表模式：Provider 注册表和 ToolRegistry，支持运行时动态注册；
3. 适配器模式：Channel 适配器统一了不同平台的消息格式；
4. 包装器模式：MCPToolWrapper 将远程 MCP 工具包装为本地工具接口；

- 5. 配置驱动组装：通过 YAML 文件驱动系统构建；
- 6. 虚拟工具模式：save_memory 不是真正的外部工具，而是框架内部拦截处理。

这些模式的共同特点是降低耦合、提高扩展性。例如新增一个 LLM 供应商只需在 PROVIDERS 注册表中添加一项，不需要修改任何现有代码。”

Q3: MessageBus 为什么用双队列而不是单队列？

标准回答：

“双队列设计 (Inbound + Outbound) 实现了消息收发的完全解耦。Channel 只需要向 Inbound Queue 推送消息、从 Outbound Queue 消费消息，不需要直接和 AgentLoop 交互。这有几个好处：

- 1. Channel 发送消息后不需要等待 Agent 处理完毕（异步非阻塞）；
- 2. 多个 Channel 可以同时向 Inbound Queue 推送消息，不会互相阻塞；
- 3. Agent 的处理速度和 Channel 的消息速度可以独立变化；
- 4. 未来可以轻松增加多个 AgentLoop 消费者来水平扩展。

如果用单队列，Channel 需要等待 Agent 处理完毕才能收到响应，这会阻塞 Channel 接收新消息，降低整体吞吐量。”

Q4: 会话锁和并发闸门的区别是什么？

标准回答：

“两者解决的是不同层面的并发问题。

会话锁 (_session_locks) 是细粒度的，按 session_id 分别加锁。它确保同一个用户的消息按顺序串行处理，不会出现两条消息同时处理导致上下文混乱的问题。不同用户之间互不影响。

并发闸门 (_concurrency_gate) 是粗粒度的，是一个全局信号量（默认值 3）。它限制了同一时刻最多只有 3 个会话在并行处理，防止过多并发请求打垮 LLM API 或占用过多资源。

两者是互补关系：会话锁保证数据一致性，并发闸门保证系统稳定性。”

3.8 本章总结

本章核心要点

五层架构

- └─ UI 层 → Channel 适配器
- └─ Gateway 层 → MessageBus 双队列
- └─ Core Agent 层 → AgentLoop + AgentRunner
- └─ Provider 层 → LLM 供应商抽象
- └─ Tool 层 → ToolRegistry + MCPToolWrapper

四大核心模块

- └─ AgentLoop → 消息消费 + 会话管理
- └─ AgentRunner → ReAct 循环执行
- └─ MemoryStore → 双层记忆管理
- └─ MessageBus → 异步消息传递

10 个设计模式

- └─ 生产者-消费者、注册表、适配器
- └─ 包装器、配置驱动、Workspace 中心
- └─ 渐进披露、虚拟工具
- └─ 会话并发控制、Prompt Cache

下一章

理解了架构设计之后，接下来我们将深入源码，逐文件、逐函数地解读 Nanobot 的核心实现。

04 - 源码逐行解读

本章小结：Nanobot 的架构设计是”极简但不简单”。五层架构清晰分离了关注点，四大核心模块各司其职，10 个经典设计模式保证了代码的扩展性和可维护性。理解这些架构设计，你就掌握了面试中最有深度的技术素材。

04 - 源码逐行解读

本章目标：逐文件、逐函数地解读 Nanobot 的核心源码。看完本章，你将能在面试中自信地说“我读过全部源码”。

目录

- 4.1 源码目录结构总览
 - 4.2 AgentLoop - loop.py
 - 4.3 AgentRunner - runner.py
 - 4.4 ContextBuilder - context.py
 - 4.5 MemoryStore - memory.py
 - 4.6 SubagentManager - subagent.py
 - 4.7 工具系统 - tools/
 - 4.8 MessageBus - bus/
 - 4.9 Channel 适配层 - channels/
 - 4.10 配置系统 - config/schema.py
 - 4.11 关键代码片段解读
 - 4.12 面试高频题
 - 4.13 本章总结
-

4.1 源码目录结构总览

```
nanobot/
├── __init__.py
├── __main__.py          # 入口点 → 启动 CLI
├── cli.py               # 命令行接口
├──
├── agent/               # 核心 Agent 模块
│   ├── __init__.py
│   ├── loop.py          # □ AgentLoop - 消息消费和会话管理
│   ├── runner.py        # □ AgentRunner - ReAct 循环执行器
│   ├── context.py       # □ ContextBuilder - Prompt 构建
│   ├── memory.py        # □ MemoryStore - 记忆系统
│   ├── subagent.py      # SubagentManager - 子 Agent 管理
│   └── tools/           # 工具子模块
│       ├── __init__.py
│       ├── registry.py  # □ ToolRegistry - 工具注册表
│       ├── mcp.py       # □ MCPToolWrapper - MCP 工具包装器
│       ├── message.py   # MessageTool - 消息回复工具
│       └── spawn.py     # SpawnTool - 子 Agent 生成工具
├──
├── bus/                 # 消息总线
│   ├── __init__.py
│   └── message_bus.py   # □ MessageBus - 双队列消息总线
├──
├── channels/            # 通道适配器
│   ├── __init__.py
│   ├── base.py          # BaseChannel - 通道基类
│   ├── telegram.py      # Telegram 适配器
│   ├── discord.py       # Discord 适配器
│   ├── feishu.py        # 飞书适配器
│   ├── dingtalk.py      # 钉钉适配器
│   └── wechat.py        # 微信适配器
```

```

|   ├── qq.py                # QQ 适配器
|   ├── slack.py             # Slack 适配器
|   └── web.py                # Web 适配器
|
|── providers/                # LLM 供应商
|   ├── __init__.py
|   ├── base.py              # BaseProvider - 供应商基类
|   ├── openai.py            # OpenAI 适配
|   ├── anthropic.py         # Anthropic/Claude 适配
|   ├── deepseek.py          # DeepSeek 适配
|   ├── dashscope.py         # 通义千问适配
|   ├── ollama.py            # Ollama 本地模型
|   ├── groq.py              # Groq 适配
|   └── registry.py          # PROVIDERS 注册表
|
|── config/                   # 配置管理
|   ├── __init__.py
|   ├── schema.py            # 配置数据类定义
|   └── loader.py            # YAML 配置加载
|
|── utils/                    # 工具函数
|   ├── __init__.py
|   ├── logging.py           # 日志
|   └── helpers.py           # 通用辅助函数

```

标注 的文件是核心文件，面试必须掌握。

4.2 AgentLoop - loop.py

文件定位

nanobot/agent/loop.py 是 Nanobot 的”心脏”，负责消息消费和会话管理。

核心类：AgentLoop

```

class AgentLoop:
    """
    智能体主循环 - 从 MessageBus 消费消息，创建 AgentRunner 处理。
    管理会话锁和全局并发控制。
    """

    def __init__(
        self,
        bus: MessageBus,
        provider: BaseProvider,
        workspace: str,
        tool_registry: ToolRegistry,
        memory_store: MemoryStore,
        context_builder: ContextBuilder,
        max_iterations: int = 40,
        restrict_to_workspace: bool = True,
    ):
        self.bus = bus
        self.provider = provider
        self.workspace = workspace
        self.tool_registry = tool_registry

```



```

self.memory_store = memory_store
self.context_builder = context_builder
self.max_iterations = max_iterations
self.restrict_to_workspace = restrict_to_workspace

# 会话锁：确保同一会话的消息串行处理
self._session_locks: dict[str, asyncio.Lock] = {}

# 并发闸门：控制全局最多同时处理的会话数
self._concurrency_gate = asyncio.Semaphore(3)

```

构造函数参数详解

参数	类型	默认值	说明
bus	MessageBus	必需	消息总线，用于收发消息
provider	BaseProvider	必需	LLM 供应商，用于调用大模型
workspace	str	必需	工作空间路径
tool_registry	ToolRegistry	必需	工具注册表
memory_store	MemoryStore	必需	记忆存储
context_builder	ContextBuilder	必需	上下文构建器
max_iterations	int	40	主 Agent 最大迭代次数
restrict_to_workspace	bool	True	是否限制文件操作在 workspace 内

run() 方法：核心消费循环

```

async def run(self):
    """ 持续从 Inbound Queue 消费消息并处理。"""
    self._register_default_tools()

    while True:
        # 1. 从队列获取消息（阻塞等待）
        msg: InboundMessage = await self.bus.inbound.get()

        # 2. 异步处理，不阻塞主循环
        asyncio.create_task(self._handle_message(msg))

    async def _handle_message(self, msg: InboundMessage):
        """ 处理单条消息（含会话锁和并发控制）。"""
        session_id = msg.session_id

        # 确保会话锁存在
        if session_id not in self._session_locks:
            self._session_locks[session_id] = asyncio.Lock()

        # 会话锁：同一会话串行
        async with self._session_locks[session_id]:
            # 并发闸门：全局最多 3 个并发
            async with self._concurrency_gate:
                await self._process_message(msg)

```

关键点解读：

1. `asyncio.create_task`：每条消息都创建一个独立的异步任务，不阻塞主循环继续消费下一条消息。
2. 双层并发控制：会话锁保证同一用户串行，并发闸门保证全局不过载。
3. 无限循环：`while True` 确保 `AgentLoop` 持续运行，永远在等待新消息。

`__register_default_tools()` 方法

```
def __register_default_tools(self):
    """ 注册框架内置的默认工具。"""
    # MessageTool: 允许 Agent 发送消息给用户
    self.tool_registry.register(
        MessageTool(bus=self.bus)
    )

    # SpawnTool: 允许 Agent 创建后台子 Agent
    self.tool_registry.register(
        SpawnTool(
            bus=self.bus,
            provider=self.provider,
            workspace=self.workspace,
        )
    )

    # save_memory: 虚拟工具, 保存信息到长期记忆
    # 注意: 这个工具的执行逻辑在 AgentRunner 中被拦截
    self.tool_registry.register_virtual(
        name="save_memory",
        description="Save important information to long-term memory",
        schema={"content": {"type": "string", "description": "Info to remember"}},
    )
```

`__save_turn()` 方法

```
async def __save_turn(self, msg: InboundMessage, response: str, tool_calls: list):
    """ 持久化单轮对话到 HISTORY.md。"""
    turn_summary = {
        "timestamp": datetime.now().isoformat(),
        "user": msg.text[:200],      # 截取用户消息前 200 字符
        "assistant": response[:500], # 截取回复前 500 字符
        "tools_used": [tc.name for tc in tool_calls],
    }
    await self.memory_store.append_history(turn_summary)
```

面试要点

面试官：“请描述 AgentLoop 的职责和工作流程。”

要点: 1. AgentLoop 是消息消费者, 持续从 MessageBus 的 Inbound Queue 获取消息 2. 使用会话锁保证同一用户的消息串行处理 3. 使用并发闸门 (Semaphore=3) 控制全局并发 4. 每条消息创建一个 AgentRunner 来执行 ReAct 循环 5. 处理完毕后保存历史 (__save_turn) 并发送响应到 Outbound Queue

AgentLoop 工作流程: 接收消息 → LLM 推理 → 工具执行 → 返回结果

4.3 AgentRunner - runner.py

文件定位

nanobot/agent/runner.py 实现了 Agent 的核心 ReAct 循环逻辑。

核心类: AgentRunner

`_TOOL_RESULT_MAX_CHARS = 16000` # 工具结果最大字符数

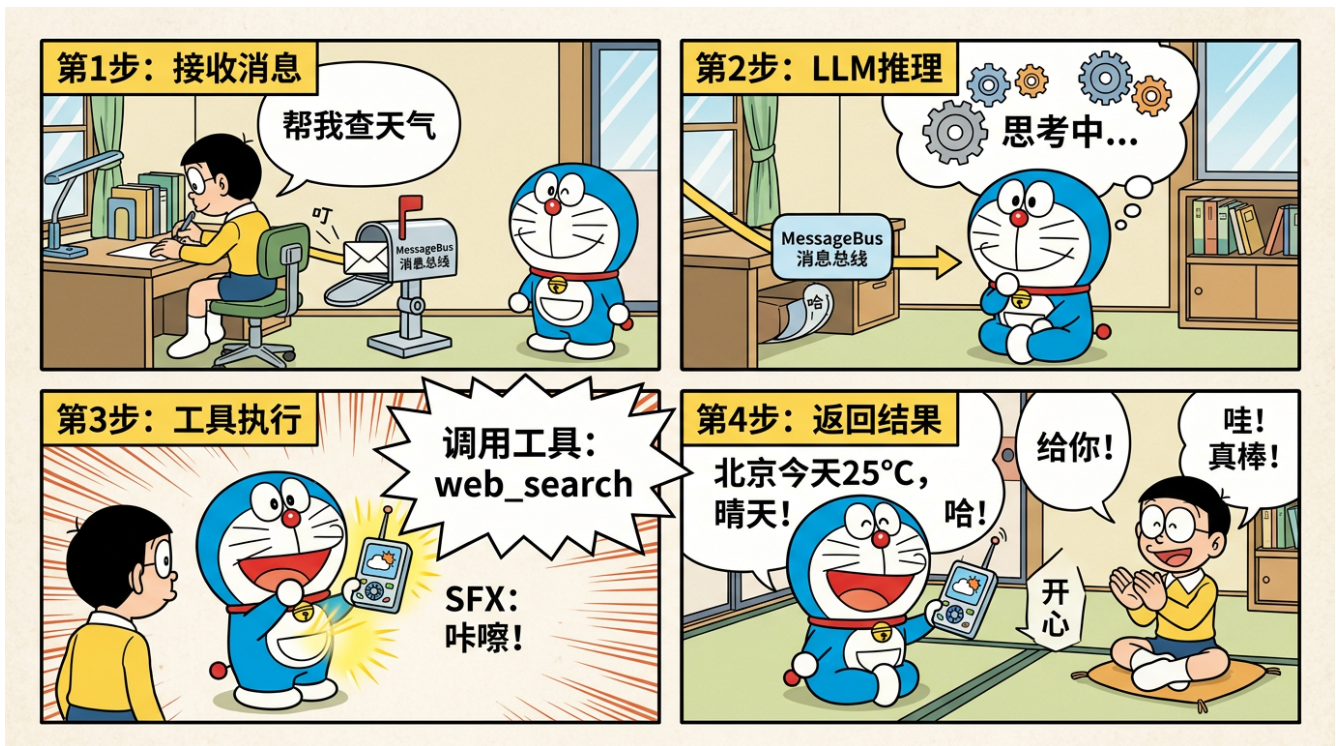


Figure 1: AgentLoop 循环漫画

```
class AgentRunner:
    """
    单次消息处理的 ReAct 循环执行器。
    每条消息创建一个新的 AgentRunner 实例。
    """

    def __init__(
        self,
        provider: BaseProvider,
        tool_registry: ToolRegistry,
        context_builder: ContextBuilder,
        memory_store: MemoryStore,
        max_iterations: int = 40,
        is_subagent: bool = False,
    ):
        self.provider = provider
        self.tool_registry = tool_registry
        self.context_builder = context_builder
        self.memory_store = memory_store
        self.max_iterations = max_iterations
        self.is_subagent = is_subagent

        # 子 Agent 的迭代限制更严格
        if is_subagent:
            self.max_iterations = min(max_iterations, 15)

    run() 方法: ReAct 循环核心

    async def run(self, user_message: str, session_id: str) -> str:
        """ 执行完整的 ReAct 循环, 返回最终响应。 """
```

```

# 1. 构建初始消息列表
messages = self.context_builder.build_messages(
    user_message=user_message,
    session_id=session_id,
)

# 2. 获取可用工具列表（转换为 LLM 兼容格式）
tools = self.tool_registry.get_tool_schemas()

final_response = ""
tool_calls_history = []

# 3. ReAct 循环
for iteration in range(self.max_iterations):

    # --- Lifecycle Hook: before_iteration ---
    await self._before_iteration(iteration, messages)

    # --- 调用 LLM ---
    response = await self.provider.chat_with_retry(
        messages=messages,
        tools=tools if tools else None,
    )

    # --- 检查是否有工具调用 ---
    if not response.tool_calls:
        # 没有工具调用 = LLM 认为任务完成
        final_response = response.content
        break

    # --- 有工具调用 ---
    # 将 assistant 的响应加入消息列表
    messages.append({
        "role": "assistant",
        "content": response.content,
        "tool_calls": response.tool_calls,
    })

    # --- Lifecycle Hook: before_execute_tools ---
    await self._before_execute_tools(response.tool_calls)

    # --- 逐一执行工具调用 ---
    for tool_call in response.tool_calls:
        result = await self._execute_tool(tool_call)

        # 截断过长的工具结果
        if len(result) > _TOOL_RESULT_MAX_CHARS:
            result = result[:_TOOL_RESULT_MAX_CHARS] + "\n...(truncated)"

        # 将工具结果加入消息列表
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,
            "content": result,
        })

```

```

        tool_calls_history.append(tool_call)

        # --- Lifecycle Hook: after_iteration ---
        await self._after_iteration(iteration, messages)

    else:
        # 达到最大迭代次数仍未完成
        final_response = "I've reached the maximum number of iterations. Here's what I've done so far..."

    return final_response

```

`__execute_tool()` 方法：工具执行（含 `save_memory` 拦截）

```

async def _execute_tool(self, tool_call) -> str:
    """ 执行单个工具调用。"""
    tool_name = tool_call.function.name
    tool_args = json.loads(tool_call.function.arguments)

    # 拦截 save_memory 虚拟工具
    if tool_name == "save_memory":
        content = tool_args.get("content", "")
        await self.memory_store.save_memory(content)
        return "Memory saved successfully."

    # 正常工具执行
    try:
        result = await self.tool_registry.execute(tool_name, tool_args)
        return str(result)
    except Exception as e:
        return f"Error executing {tool_name}: {str(e)}"

```

Lifecycle Hooks 详解

```

async def _before_iteration(self, iteration: int, messages: list):
    """ 每轮迭代开始前的钩子。"""
    # 可扩展：日志记录、状态检查等
    logger.debug(f"Iteration {iteration + 1}/{self.max_iterations}")

async def _before_execute_tools(self, tool_calls: list):
    """ 工具执行前的钩子。"""
    # 可扩展：安全检查、审计日志等
    for tc in tool_calls:
        logger.info(f"Executing tool: {tc.function.name}")

async def _after_iteration(self, iteration: int, messages: list):
    """ 每轮迭代结束后的钩子。"""
    # 可扩展：指标统计、性能监控等
    pass

```

关键设计决策

决策	实现	理由
最大迭代 40 次	<code>max_iterations=40</code>	给 Agent 足够空间完成复杂任务，同时防止无限循环

决策	实现	理由
子 Agent 15 次	<code>min(max_iterations, 15)</code>	子 Agent 任务较简单，限制更严格节省成本
工具结果截断 16000 字符	<code>_TOOL_RESULT_MAX_CHARS</code>	避免过长结果占用上下文窗口
错误不中断循环	<code>try-except</code> 返回错误信息	让 LLM 自行决定如何处理错误
无工具调用即结束	<code>if not response.tool_calls: break</code>	LLM 不调用工具说明任务已完成

4.4 ContextBuilder - context.py

文件定位

`nanobot/agent/context.py` 负责构建发送给 LLM 的完整消息列表。

核心类: **ContextBuilder**

```
class ContextBuilder:
    """
    构建发送给 LLM 的消息列表。
    包括 system prompt、历史消息和运行时上下文。
    """

    def __init__(
        self,
        config: AgentConfig,
        memory_store: MemoryStore,
        skill_loader: SkillLoader,
    ):
        self.config = config
        self.memory_store = memory_store
        self.skill_loader = skill_loader
```

`build_system_prompt()` 方法

```
def build_system_prompt(self) -> str:
    """
    构建 system prompt。

    结构: Identity + Bootstrap + Memory + Skills

    将不变的部分放在前面，利用 Prompt Cache 优化。
    """
    parts = []

    # 1. Identity (身份定义)
    # 来自配置文件中的 identity 字段
    if self.config.identity:
        parts.append(f"# Identity\n{self.config.identity}")

    # 2. Bootstrap (基础行为规范)
    # 框架内置的默认行为指令
    bootstrap = self._get_bootstrap_instructions()
    parts.append(f"# Instructions\n{bootstrap}")

    # 3. Memory (长期记忆)
    # 从 MEMORY.md 读取
```

```

memory = self.memory_store.read_memory()
if memory:
    parts.append(f"# Memory\n{memory}")

# 4. Skills (技能指令)
# 从 SKILL.md 文件加载
skills = self.skill_loader.load_skills()
if skills:
    for skill in skills:
        parts.append(f"# Skill: {skill.name}\n{skill.instructions}")

return "\n\n".join(parts)

```

build_messages() 方法

```

def build_messages(self, user_message: str, session_id: str) -> list:
    """
    构建完整的消息列表，发送给 LLM。

    消息结构：
    [system] → [history...] → [user_message]

    Prompt Cache 优化策略：
    - system prompt 放在最前面（跨请求几乎不变）
    - history 放在中间
    - user_message 放在最后（每次不同）
    """
    messages = []

    # System prompt (利用 Prompt Cache)
    system_prompt = self.build_system_prompt()
    messages.append({
        "role": "system",
        "content": system_prompt,
    })

    # 历史消息 (最近 N 轮对话)
    history = self._get_conversation_history(session_id)
    messages.extend(history)

    # 当前用户消息
    messages.append({
        "role": "user",
        "content": user_message,
    })

    return messages

```

Prompt Cache 优化策略解读

消息列表结构（为 Prompt Cache 优化）：

[system] Identity + Bootstrap + Memory + Skills	□ 几乎不变 □ 可被 Cache
[history] 最近的对话历史	□ 缓慢变化

[user] 当前用户消息	每次不同
---------------	------

Prompt Cache 工作原理:

1. LLM API (如 Claude) 会缓存消息的前缀部分
2. 如果两次请求的前缀相同, 第二次不需要重新处理
3. 这可以减少约 90% 的 system prompt 处理时间
4. 节省 Token 成本 (缓存命中的 Token 通常有折扣)

面试要点

system prompt 的构建顺序: Identity → Bootstrap → Memory → Skills。这个顺序是有意为之的——Identity 和 Bootstrap 几乎不变 (利于 Prompt Cache), Memory 和 Skills 偶尔变化, 放在后面。

4.5 MemoryStore - memory.py

文件定位

nanobot/agent/memory.py 管理 Nanobot 的双层记忆系统。

核心类: **MemoryStore**

```
class MemoryStore:
    """
    管理 Agent 的长期记忆 (MEMORY.md) 和交互历史 (HISTORY.md)。
    """

    def __init__(self, workspace: str, provider: BaseProvider = None):
        self.workspace = workspace
        self.provider = provider # 用于记忆压缩时调用 LLM
        self.memory_path = os.path.join(workspace, "MEMORY.md")
        self.history_path = os.path.join(workspace, "HISTORY.md")
```

read_memory() 方法

```
def read_memory(self) -> str:
    """ 读取 MEMORY.md 的全部内容。"""
    if not os.path.exists(self.memory_path):
        return ""

    with open(self.memory_path, "r", encoding="utf-8") as f:
        return f.read()
```

save_memory() 方法

```
async def save_memory(self, content: str):
    """
    保存内容到 MEMORY.md。

    这个方法由 save_memory 虚拟工具触发。
    LLM 决定什么时候需要记住什么信息。
    """

    # 追加模式写入
    with open(self.memory_path, "a", encoding="utf-8") as f:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M")
        f.write(f"\n## {timestamp}\n{content}\n")
```


append_history() 方法

```
async def append_history(self, turn_summary: dict):
    """ 将单轮对话摘要追加到 HISTORY.md。"""
    with open(self.history_path, "a", encoding="utf-8") as f:
        f.write(f"\n### {turn_summary['timestamp']}\n")
        f.write(f"**User**: {turn_summary['user']}\n")
        f.write(f"**Assistant**: {turn_summary['assistant']}\n")
        if turn_summary.get('tools_used'):
            f.write(f"**Tools**: {' '.join(turn_summary['tools_used'])}\n")
        f.write("---\n")
```

MemoryConsolidator 压缩机制

```
class MemoryConsolidator:
    """
    当 HISTORY.md 增长到一定大小时,
    使用 LLM 压缩旧的历史记录。
    """

    CONSOLIDATION_THRESHOLD = 50 # 默认超过 50 条记录触发压缩

    def __init__(self, memory_store: MemoryStore, provider: BaseProvider):
        self.memory_store = memory_store
        self.provider = provider

    async def maybe_consolidate(self):
        """ 检查是否需要压缩, 如需要则执行。"""
        history = self._read_history()
        entries = self._parse_entries(history)

        if len(entries) < self.CONSOLEIDATION_THRESHOLD:
            return # 还不需要压缩

        # 将旧记录 (保留最近 10 条) 压缩为摘要
        old_entries = entries[:-10]
        recent_entries = entries[-10:]

        # 使用 LLM 生成摘要
        summary = await self._summarize_with_llm(old_entries)

        # 重写 HISTORY.md
        self._rewrite_history(summary, recent_entries)

    async def _summarize_with_llm(self, entries: list) -> str:
        """ 使用 LLM 将多条历史记录压缩为摘要。"""
        prompt = (
            "Please summarize the following conversation history "
            "into a concise summary, preserving key information:\n\n"
            + "\n".join(entries)
        )
        response = await self.provider.chat_with_retry(
            messages=[{"role": "user", "content": prompt}]
        )
        return response.content
```

压缩流程图:

HISTORY.md 增长到 50+ 条

↓

MemoryConsolidator.maybe_consolidate()

↓

分离：旧记录（前 40 条）+ 近期记录（后 10 条）

↓

LLM 压缩旧记录为摘要

↓

重写 HISTORY.md：
[摘要] + [近期 10 条详细记录]

↓

文件大小大幅减小，关键信息保留

4.6 SubagentManager - subagent.py

文件定位

nanobot/agent/subagent.py 管理子 Agent 的创建和执行。

核心类：SubagentManager

```
class SubagentManager:
    """
    管理后台子 Agent 的创建和生命周期。
    子 Agent 由主 Agent 通过 SpawnTool 创建。
    """

    def __init__(
        self,
        provider: BaseProvider,
        workspace: str,
        tool_registry: ToolRegistry,
    ):
        self.provider = provider
        self.workspace = workspace
        self.tool_registry = tool_registry
        self._active_subagents: dict[str, asyncio.Task] = {}
```

spawn() 方法

```
async def spawn(self, task: str, session_id: str) -> str:
    """
    创建一个后台子 Agent 来执行任务。

    子 Agent 的限制：
    1. 最大迭代 15 次（vs 主 Agent 的 40 次）
    2. 不能使用 MessageTool（不能直接回复用户）
    3. 不能使用 SpawnTool（不能再创建子 Agent，防递归）
    """
    # 创建受限的工具集
    restricted_tools = self.tool_registry.clone()
    restricted_tools.remove("message") # 移除 MessageTool
    restricted_tools.remove("spawn_agent") # 移除 SpawnTool，防止递归

    # 创建子 Agent 的 Runner
    sub_runner = AgentRunner(
```

```

        provider=self.provider,
        tool_registry=restricted_tools,
        context_builder=self._build_sub_context(),
        memory_store=self.memory_store,
        max_iterations=15,
        is_subagent=True,
    )

    # 后台执行
    task_obj = asyncio.create_task(
        sub_runner.run(task, session_id)
    )

    subagent_id = str(uuid.uuid4())[:8]
    self._active_subagents[subagent_id] = task_obj

    return f"Subagent {subagent_id} spawned for task: {task}"

```

关键设计：

设计	说明
迭代限制 15 次	子 Agent 任务相对简单，15 次足够
移除 MessageTool	子 Agent 不应直接与用户通信，结果通过主 Agent 返回
移除 SpawnTool	防止子 Agent 创建子子 Agent，避免递归爆炸
后台异步执行	asyncio.create_task 让子 Agent 不阻塞主 Agent

4.7 工具系统 - tools/

4.7.1 ToolRegistry - registry.py

```

class ToolRegistry:
    """
    统一的工具注册表，管理所有可用工具。
    支持内置工具和 MCP 远程工具。
    """

    def __init__(self):
        self._tools: dict[str, BaseTool] = {}
        self._virtual_tools: dict[str, dict] = {}

    def register(self, tool: BaseTool):
        """ 注册一个工具。 """
        self._tools[tool.name] = tool

    def register_virtual(self, name: str, description: str, schema: dict):
        """ 注册一个虚拟工具（仅元数据，执行逻辑在外部）。 """
        self._virtual_tools[name] = {
            "name": name,
            "description": description,
            "parameters": schema,
        }

    async def execute(self, name: str, args: dict) -> str:
        """ 执行指定的工具。 """
        if name in self._tools:

```

```

        return await self._tools[name].run(args)
    raise ToolNotFoundError(f"Tool '{name}' not registered")

def get_tool_schemas(self) -> list[dict]:
    """ 获取所有工具的 schema 列表（用于 LLM）。"""
    schemas = []

    # 真实工具
    for tool in self._tools.values():
        schemas.append({
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description,
                "parameters": tool.parameters_schema,
            }
        })

    # 虚拟工具
    for vt in self._virtual_tools.values():
        schemas.append({
            "type": "function",
            "function": vt,
        })

    return schemas

def clone(self) -> 'ToolRegistry':
    """ 克隆工具注册表（用于子 Agent 的受限工具集）。"""
    new_registry = ToolRegistry()
    new_registry._tools = dict(self._tools)
    new_registry._virtual_tools = dict(self._virtual_tools)
    return new_registry

def remove(self, name: str):
    """ 移除指定工具。"""
    self._tools.pop(name, None)
    self._virtual_tools.pop(name, None)

```

4.7.2 MCPToolWrapper - mcp.py

```

class MCPToolWrapper(BaseTool):
    """
    将远程 MCP Server 提供的工具包装为本地工具。
    对 AgentRunner 来说，MCP 工具和内置工具没有区别。
    """

    def __init__(self, server_name: str, tool_spec: MCPToolSpec, client: MCPClient):
        # 工具命名规则: mcp_{server}_{tool}
        self.name = f"mcp_{server_name}_{tool_spec.name}"
        self.description = tool_spec.description
        self.client = client
        self.original_name = tool_spec.name

        # 将 MCP schema 转换为 OpenAI 兼容格式
        self.parameters_schema = self._normalize_schema_for_openai(
            tool_spec.input_schema

```

```

    )

    async def run(self, args: dict) -> str:
        """ 调用远程 MCP Server 执行工具。"""
        result = await self.client.call_tool(
            name=self.original_name,
            arguments=args,
        )
        return self._format_result(result)

    @staticmethod
    def _normalize_schema_for_openai(mcp_schema: dict) -> dict:
        """
        将 MCP 的 JSON Schema 转换为 OpenAI 兼容格式。

        主要处理:
        - 移除 MCP 特有的字段
        - 确保 required 字段格式正确
        - 处理嵌套 schema 的差异
        """
        schema = dict(mcp_schema)
        schema.setdefault("type", "object")
        schema.setdefault("properties", {})

        # 移除 MCP 特有字段
        schema.pop("$schema", None)
        schema.pop("additionalProperties", None)

        return schema

```

MCP 工具命名规则:

MCP Server: "filesystem"

MCP Tool: "read_file"

↓

Nanobot 中的工具名: "mcp_filesystem_read_file"

MCP Server: "github"

MCP Tool: "create_issue"

↓

Nanobot 中的工具名: "mcp_github_create_issue"

这种命名规则确保不同 MCP Server 的同名工具不会冲突。

4.7.3 MessageTool - message.py

```

class MessageTool(BaseTool):
    """ 允许 Agent 主动发送消息给用户。"""

    name = "message"
    description = "Send a message to the user"
    parameters_schema = {
        "type": "object",
        "properties": {
            "content": {
                "type": "string",
                "description": "The message content to send",
            }
        }
    }

```

```

    },
    "required": ["content"],
}

def __init__(self, bus: MessageBus):
    self.bus = bus

    async def run(self, args: dict) -> str:
        content = args["content"]
        msg = OutboundMessage(text=content)
        await self.bus.outbound.put(msg)
        return "Message sent"

```

4.7.4 SpawnTool - spawn.py

```

class SpawnTool(BaseTool):
    """ 允许主 Agent 创建后台子 Agent。 """

    name = "spawn_agent"
    description = "Spawn a background subagent to handle a task"
    parameters_schema = {
        "type": "object",
        "properties": {
            "task": {
                "type": "string",
                "description": "The task for the subagent to perform",
            }
        },
        "required": ["task"],
    }

    async def run(self, args: dict) -> str:
        return await self.subagent_manager.spawn(args["task"], self.session_id)

```

4.8 MessageBus - bus/

核心类: **MessageBus**

```

class MessageBus:
    """
    异步消息总线，使用双 asyncio.Queue 实现。

    Inbound Queue: Channel → AgentLoop
    Outbound Queue: AgentLoop → Channel
    """

    def __init__(self):
        self.inbound: asyncio.Queue[InboundMessage] = asyncio.Queue()
        self.outbound: asyncio.Queue[OutboundMessage] = asyncio.Queue()

    @dataclass
    class InboundMessage:
        """ 用户发来的消息。 """
        session_id: str  # 会话标识
        user_id: str  # 用户标识

```

```

text: str          # 消息文本
channel: str        # 来源通道 (telegram/feishu/...)
timestamp: float    # 时间戳
metadata: dict = None # 额外的元数据 (如图片、附件等)

```

```

@dataclass
class OutboundMessage:
    """Agent 要发出的消息。"""
    session_id: str    # 目标会话
    text: str          # 回复文本
    channel: str        # 目标通道
    metadata: dict = None # 额外数据 (如按钮、卡片等)

```

设计解读：

为什么用 `asyncio.Queue` 而不是 `Redis/RabbitMQ`？

1. 单进程架构：Nanobot 设计为单进程运行，不需要跨进程通信
2. 极简原则：`asyncio.Queue` 零依赖，不需要额外的消息中间件
3. 性能足够：个人助手场景下，`asyncio.Queue` 的性能绰绰有余
4. 部署简单：不需要额外安装和维护消息队列服务

如果需要扩展到多进程或分布式，可以替换为 `Redis Queue`，但对于 Nanobot 的目标场景（个人/小团队），这是不必要的。

4.9 Channel 适配层 - channels/

基类：BaseChannel

```

class BaseChannel(ABC):
    """ 所有通道适配器的基类。 """

    def __init__(self, config: ChannelConfig, bus: MessageBus):
        self.config = config
        self.bus = bus

    @abstractmethod
    async def start(self):
        """ 启动通道，开始接收消息。 """
        pass

    @abstractmethod
    async def stop(self):
        """ 停止通道。 """
        pass

    async def _dispatch_inbound(self, text: str, user_id: str, session_id: str):
        """ 将接收到的消息分发到 MessageBus。 """
        msg = InboundMessage(
            session_id=session_id,
            user_id=user_id,
            text=text,
            channel=self.__class__.__name__.lower(),
            timestamp=time.time(),
        )
        await self.bus.inbound.put(msg)

```

```

async def _consume_outbound(self):
    """ 持续消费 Outbound Queue 并发送。"""
    while True:
        msg: OutboundMessage = await self.bus.outbound.get()
        if msg.channel == self.channel_name:
            await self._send(msg)

    @abstractmethod
    async def _send(self, msg: OutboundMessage):
        """ 发送消息到具体平台。"""
        pass

```

Telegram 适配器示例

```

class TelegramChannel(BaseChannel):
    """Telegram 通道适配器。"""

    channel_name = "telegram"

    async def start(self):
        self.bot = TelegramBot(token=self.config.token)

        @self.bot.on_message()
        async def handle(update):
            await self._dispatch_inbound(
                text=update.message.text,
                user_id=str(update.message.from_user.id),
                session_id=f"tg_{update.message.chat.id}",
            )

        # 启动 Outbound 消费和 Telegram 轮询
        asyncio.create_task(self._consume_outbound())
        await self.bot.start_polling()

    async def _send(self, msg: OutboundMessage):
        chat_id = msg.session_id.replace("tg_", "")
        await self.bot.send_message(
            chat_id=int(chat_id),
            text=msg.text,
        )

```

飞书适配器示例

```

class FeishuChannel(BaseChannel):
    """飞书通道适配器。"""

    channel_name = "feishu"

    async def start(self):
        self.client = FeishuClient(
            app_id=self.config.app_id,
            app_secret=self.config.app_secret,
        )

        # 飞书使用事件订阅机制
        @self.client.on_event("im.message.receive_v1")

```



```

    async def handle(event):
        content = json.loads(event.message.content)
        await self._dispatch_inbound(
            text=content.get("text", ""),
            user_id=event.sender.sender_id.user_id,
            session_id=f"feishu_{event.message.chat_id}",
        )

    asyncio.create_task(self._consume_outbound())
    await self.client.start()

    async def _send(self, msg: OutboundMessage):
        chat_id = msg.session_id.replace("feishu_", "")
        await self.client.send_message(
            receive_id=chat_id,
            msg_type="text",
            content=json.dumps({"text": msg.text}),
        )

```

4.10 配置系统 - config/schema.py

配置数据类

```

@dataclass
class NanobotConfig:
    """Nanobot 顶层配置。"""
    name: str = "nanobot"
    identity: str = "" # Agent 身份描述
    provider: ProviderConfig = None # LLM 配置
    channels: list[ChannelConfig] = None # 通道配置
    mcp_servers: list[MCPConfig] = None # MCP 服务器配置
    memory: MemoryConfig = None # 记忆配置
    cron: list[CronConfig] = None # 定时任务配置
    max_iterations: int = 40 # 最大迭代次数
    restrict_to_workspace: bool = True # 限制在 workspace 内

@dataclass
class ProviderConfig:
    """LLM 供应商配置。"""
    type: str = "openai" # Provider 类型
    model: str = "gpt-4" # 模型名称
    api_key: str = "" # API Key (支持 ${ENV_VAR} 格式)
    base_url: str = None # 自定义 API 地址
    temperature: float = 0.7 # 温度参数
    max_tokens: int = 4096 # 最大输出 Token

@dataclass
class ChannelConfig:
    """通道配置。"""
    type: str # 通道类型 (telegram/feishu/...)
    token: str = None # 认证 Token
    app_id: str = None # 应用 ID
    app_secret: str = None # 应用密钥
    webhook_url: str = None # Webhook URL

```

```

@dataclass
class MCPConfig:
    """MCP Server 配置。"""
    name: str # Server 名称
    command: str = None # 本地 Server 启动命令 (stdio 传输)
    args: list[str] = None # 命令参数
    url: str = None # 远程 Server URL (HTTP 传输)
    env: dict = None # 环境变量

```

```

@dataclass
class MemoryConfig:
    """ 记忆系统配置。"""
    consolidation_threshold: int = 50 # 压缩触发阈值
    max_history_entries: int = 100 # 最大历史条目

```

```

@dataclass
class CronConfig:
    """ 定时任务配置。"""
    schedule: str # Cron 表达式
    message: str # 要发送给 Agent 的消息

```

配置加载

```

class ConfigLoader:
    """ 从 YAML 文件加载配置。"""

    @staticmethod
    def load(config_path: str) -> NanobotConfig:
        with open(config_path, "r") as f:
            raw = yaml.safe_load(f)

        # 环境变量替换: ${VAR_NAME} → 实际值
        raw = ConfigLoader._resolve_env_vars(raw)

        return NanobotConfig(**raw)

    @staticmethod
    def _resolve_env_vars(data):
        """ 递归替换配置中的 ${ENV_VAR} 为实际环境变量值。"""
        if isinstance(data, str):
            # 匹配 ${VAR_NAME} 模式
            pattern = r'\${(\w+)\}'
            def replace(match):
                var_name = match.group(1)
                value = os.environ.get(var_name, "")
                if not value:
                    logger.warning(f"Environment variable {var_name} not set")
                return value
            return re.sub(pattern, replace, data)
        elif isinstance(data, dict):
            return {k: ConfigLoader._resolve_env_vars(v) for k, v in data.items()}
        elif isinstance(data, list):
            return [ConfigLoader._resolve_env_vars(item) for item in data]

```

```
return data
```

4.11 关键代码片段解读

片段 1: 完整的消息处理链路

这段伪代码展示了一条消息从接收到响应的完整链路

1. Channel 接收用户消息

```
async def telegram_on_message(update):
    await bus.inbound.put(InboundMessage(
        session_id=f"tg_{update.chat.id}",
        text=update.text,
    ))
```

2. AgentLoop 消费消息

```
async def agent_loop_run():
    while True:
        msg = await bus.inbound.get()
        asyncio.create_task(handle(msg))
```

3. 会话锁 + 并发控制

```
async def handle(msg):
    async with session_locks[msg.session_id]:
        async with concurrency_gate:
            # 4. 构建上下文
            messages = context_builder.build_messages(msg.text, msg.session_id)
            tools = tool_registry.get_tool_schemas()
```

5. ReAct 循环

```
for i in range(40):
    response = await provider.chat_with_retry(messages, tools)

    if not response.tool_calls:
        break # 任务完成

    for tc in response.tool_calls:
        if tc.name == "save_memory":
            await memory_store.save_memory(tc.args["content"])
            result = "Saved"
        else:
            result = await tool_registry.execute(tc.name, tc.args)

    messages.append({"role": "tool", "content": result})
```

6. 保存历史

```
await memory_store.append_history(...)
```

7. 发送响应

```
await bus.outbound.put(OutboundMessage(text=response.content))
```

片段 2: MCP 工具注册流程

MCP 工具是如何被注册到 ToolRegistry 的

```
async def register_mcp_servers(config, tool_registry):
```

```

for mcp_config in config.mcp_servers:
    # 1. 创建 MCP Client
    if mcp_config.command:
        # stdio 传输 (本地 Server)
        client = await MCPClient.connect_stdio(
            command=mcp_config.command,
            args=mcp_config.args,
            env=mcp_config.env,
        )
    elif mcp_config.url:
        # HTTP 传输 (远程 Server)
        client = await MCPClient.connect_http(url=mcp_config.url)

    # 2. 获取 Server 提供的工具列表
    tools = await client.list_tools()

    # 3. 逐一包装并注册
    for tool_spec in tools:
        wrapper = MCPToolWrapper(
            server_name=mcp_config.name,
            tool_spec=tool_spec,
            client=client,
        )
        tool_registry.register(wrapper)
        # 注册后的工具名: mcp_{server_name}_{tool_name}

```

片段 3: Provider 注册表与工厂

PROVIDERS 注册表

```

PROVIDERS: dict[str, type[BaseProvider]] = {
    "openai": OpenAIProvider,
    "anthropic": AnthropicProvider,
    "deepseek": DeepSeekProvider,
    "dashscope": DashScopeProvider,
    "ollama": OllamaProvider,
    "groq": GroqProvider,
    "azure": AzureOpenAIProvider,
    "mistral": MistralProvider,
    "openrouter": OpenRouterProvider,
    "google": GoogleProvider,
}

def create_provider(config: ProviderConfig) -> BaseProvider:
    """ 工厂方法: 根据配置创建对应的 Provider 实例。"""
    provider_type = config.type.lower()

    if provider_type not in PROVIDERS:
        raise ValueError(f"Unknown provider: {provider_type}")

    provider_cls = PROVIDERS[provider_type]
    return provider_cls(config)

```

4.12 面试高频题

Q1: 请描述 **AgentLoop** 的执行流程

标准回答:

“AgentLoop 的执行流程可以分为 5 个步骤:

第一步, 启动时调用 `_register_default_tools()` 注册内置工具 (MessageTool、SpawnTool、save_memory 虚拟工具)。

第二步, 进入 `while True` 无限循环, 持续从 MessageBus 的 Inbound Queue 获取消息。使用 `await bus.inbound.get()` 阻塞等待。

第三步, 收到消息后使用 `asyncio.create_task` 创建异步任务来处理, 不阻塞主循环继续接收下一条消息。

第四步, 在处理任务中, 先获取该会话的会话锁 (保证同一会话串行), 再获取全局并发闸门 (默认最多 3 个并发), 然后创建 AgentRunner 执行 ReAct 循环。

第五步, AgentRunner 完成后, 调用 `_save_turn()` 将对话摘要保存到 HISTORY.md, 然后将响应消息推入 Outbound Queue, 由 Channel 发送给用户。”

Q2: save_memory 虚拟工具是怎么工作的?

标准回答:

“save_memory 是一个‘虚拟工具’——它在 ToolRegistry 中注册了名称、描述和参数 schema, 让 LLM 知道有这个工具可以用。但它的执行逻辑不在 ToolRegistry 中, 而是在 AgentRunner 的 `_execute_tool` 方法中被拦截处理。

当 LLM 决定调用 save_memory 时, AgentRunner 检测到这个特殊的工具名, 直接调用 MemoryStore.save_memory() 将内容写入 MEMORY.md 文件, 而不走正常的 ToolRegistry.execute() 流程。

这种设计的好处是: 对 LLM 来说, save_memory 和其他工具没有任何区别, 统一了调用接口; 同时因为是内部直接处理, 没有网络开销, 执行效率更高。”

Q3: 为什么工具结果要截断到 16000 字符?

标准回答:

“工具结果截断到 16000 字符 (`_TOOL_RESULT_MAX_CHARS`) 是出于三个考虑:

第一是上下文窗口限制, 过长的工具结果会占用大量上下文空间, 挤压了历史消息和系统 prompt 的空间;

第二是 LLM 处理效果, 研究表明 LLM 处理超长文本时效果会下降, 特别是中间部分的信息容易被忽略 (Lost in the Middle 问题);

第三是成本控制, Token 数量直接影响 API 调用费用, 截断可以有效控制成本。

16000 字符这个阈值是一个经验值——足以包含大多数工具返回的有用信息, 同时不会过度占用上下文。”

Q4: 如何新增一个 LLM 供应商 (如文心一言)?

标准回答:

“在 Nanobot 中新增 LLM 供应商非常简单, 只需要三步:

第一步, 在 `providers/` 目录下创建 `wenxin.py`, 实现 BaseProvider 接口, 包括 `chat_with_retry()` 方法。

第二步, 在 `providers/registry.py` 的 PROVIDERS 字典中添加一行: `"wenxin": WenxinProvider`。

第三步, 用户在 `nanobot.yml` 中使用 `type: wenxin` 即可。

不需要修改任何现有代码——这就是注册表模式 (Registry Pattern) 的好处。整个过程符合开闭原则: 对扩展开放, 对修改关闭。”

4.13 本章总结

核心文件速查表		
文件	核心类	职责
agent/loop.py	AgentLoop	消息消费
agent/runner.py	AgentRunner	ReAct
agent/context.py	ContextBuilder	Prompt
agent/memory.py	MemoryStore	记忆
agent/subagent.py	SubagentManager	子Agent
agent/tools/registry.py	ToolRegistry	工具
agent/tools/mcp.py	MCPToolWrapper	MCP
bus/message_bus.py	MessageBus	消息
channels/base.py	BaseChannel	通道
config/schema.py	NanobotConfig	配置
总代码量: 约 4000 行 Python		
核心文件: 10 个		
核心类: 10 个		

源码阅读建议

推荐阅读顺序:

1. config/schema.py

2. bus/message_bus.py

3. agent/loop.py

4. agent/runner.py

5. agent/context.py

6. agent/memory.py

7. agent/tools/

8. channels/

9. providers/
- 理解配置结构

→ 理解消息传递

→ 理解消息消费

→ 理解 ReAct 循环 (核心!)

→ 理解 Prompt 构建

→ 理解记忆系统

→ 理解工具系统

→ 理解平台适配

→ 理解 LLM 封装

下一章

接下来,我们将深入学习 MCP 协议——AI 界的”USB-C 接口”。

05 - MCP 协议详解

本章小结: 通过逐文件解读,我们看到 Nanobot 的核心代码如何构建出一个完整的 Agent 框架。核心是 10 个文件、10 个类,每个类职责清晰。掌握这些源码细节,你就能在面试中自信地说”我通读了全部源码”,并能深入讨论任何实现细节。

4.14 v0.1-v0.2.1 源码更新

核心文件行数 (截至 v0.2.1)

文件	行数	说明
agent/loop.py	1,839 行	AgentLoop - Turn state machine
agent/runner.py	1,571 行	AgentRunner - Tool execution loop
agent/memory.py	1,050 行	MemoryStore + Dream + GitStore
channels/websocket.py	1,179 行	WebUI channel
channels/feishu.py	1,984 行	飞书 channel
providers/registry.py	564 行	39 个 Provider 注册
session/manager.py	868 行	Session 管理
agent/context.py	280 行	ContextBuilder
agent/skills.py	261 行	SkillsLoader
agent/subagent.py	394 行	SubagentManager

完整项目：173 个 Python 文件，64,496 行代码

新增核心模块

- **Dream** 记忆系统：agent/memory.py 中的 Consolidator、.dream_cursor
- **Goal** 系统：session/goal_state.py (127 行)
- **WebUI**：channels/websocket.py (1179 行)
- **Gateway**：cli/commands.py 中的 gateway() 命令
- **AutoCompact**：agent/autocompact.py (96 行)
- **Cron** 服务：cron/ 目录（多个文件）
- **Command Router**：command/ 目录

推荐阅读顺序（更新版）

1. config/schema.py → 理解配置结构 (JSON, camelCase)
2. providers/registry.py → 理解 39+ Provider 注册
3. bus/events.py → 理解消息事件
4. agent/loop.py → 理解 Turn state machine (1839行)
5. agent/runner.py → 理解 Tool execution loop (1571行, 核心!)
6. agent/memory.py → 理解记忆系统 + Dream (1050行)
7. session/goal_state.py → 理解 Goal 系统 (127行)
8. agent/context.py → 理解 ContextBuilder (280行)
9. channels/websocket.py → 理解 WebUI (1179行)
10. agent/tools/ → 理解工具系统
11. channels/ → 理解 15+ Channel 适配

05 - MCP 协议详解

本章目标：深入理解 MCP（Model Context Protocol）协议的设计思想、三大核心原语、通信机制，以及它在 Nanobot 中的实现。MCP 是 2024-2026 年 AI 领域最重要的标准化协议之一，是面试热门话题。

目录

- 5.1 MCP 协议概述
- 5.2 核心问题：M×N 困境
- 5.3 三角架构
- 5.4 三大核心原语
- 5.5 通信协议与传输层
- 5.6 MCP vs Function Calling
- 5.7 MCP vs Skill vs Function Call 三者关系
- 5.8 MCP 在 Nanobot 中的实现
- 5.9 MCP 生态发展时间线
- 5.10 面试高频题与答案
- 5.11 练习：写一个简单的 MCP Server
- 5.12 本章总结

5.1 MCP 协议概述

基本信息

项目	内容
全称	Model Context Protocol（模型上下文协议）
发布方	Anthropic（Claude 的开发公司）
发布时间	2024 年 11 月
协议类型	开放标准协议
通信协议	JSON-RPC 2.0
传输方式	stdio / HTTP+SSE / Streamable HTTP
目标	标准化 AI 模型与外部工具/数据的交互方式

一句话定义

MCP 是 **AI** 应用连接外部工具和数据源的标准化协议，就像 **USB-C** 是设备连接的标准化接口一样。

为什么 MCP 重要？

在 MCP 出现之前，每个 AI 应用（如 Claude Desktop、ChatGPT、各种 Agent 框架）都需要为每个工具和数据源编写专门的集成代码。这导致了大量的重复工作和碎片化的生态。

MCP 的出现就像当年 USB 接口统一了各种外设接口一样——它为 AI 应用提供了一个标准化的“接口协议”，让工具只需要实现一次 MCP 接口，就能被所有支持 MCP 的 AI 应用使用。

类比：AI 界的 USB-C			
USB-C 之前：		MCP 之前：	
设备	→ 专用线缆 → 电脑	AI 应用	→ 专用代码
设备	→ 另一种线缆 → 电脑	AI 应用	→ 另一套代码
设备	→ 又一种线缆 → 电脑	AI 应用	→ 又一套代码

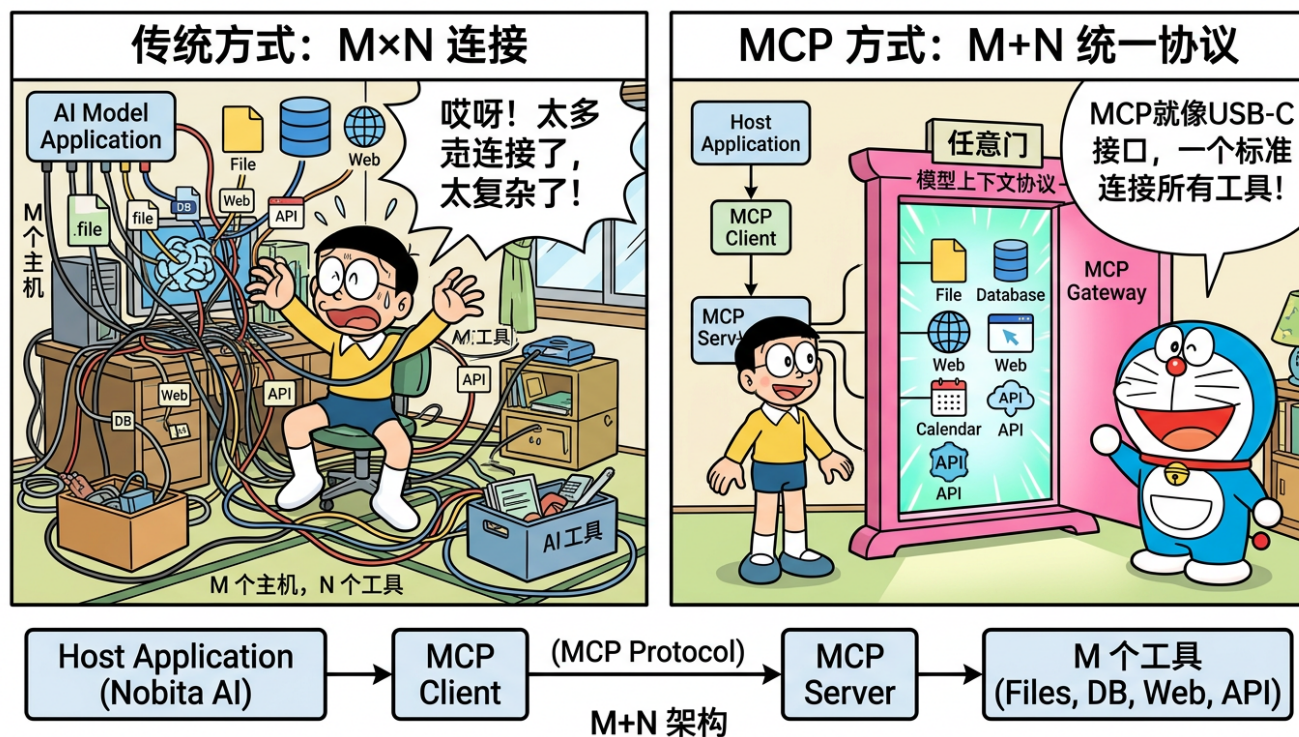
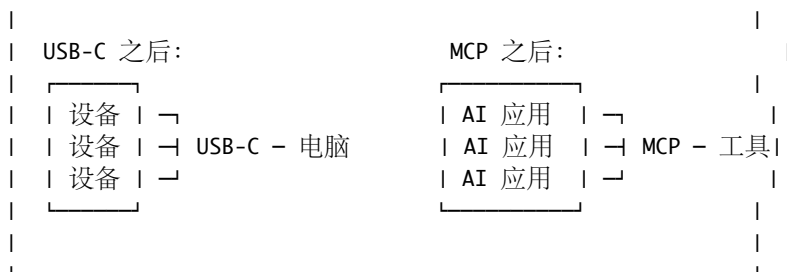


Figure 1: MCP 协议漫画

MCP 就像任意门: 统一标准连接所有工具, 从 $M \times N$ 到 $M+N$

5.2 核心问题: $M \times N$ 困境

问题描述

在 MCP 出现之前, AI 生态面临一个经典的 $M \times N$ 连接问题:

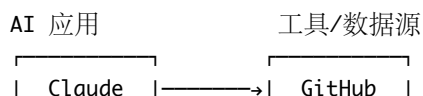
- M 个 AI 应用: Claude Desktop、ChatGPT、Cursor、Nanobot、LangChain 等
- N 个工具/数据源: GitHub、Slack、数据库、文件系统、搜索引擎等

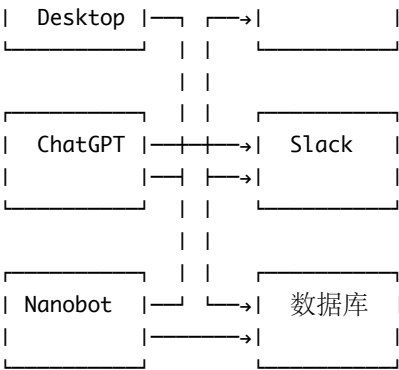
如果每个应用都要为每个工具编写专门的集成代码, 总共需要 $M \times N$ 个集成。

$M \times N$ vs $M+N$

没有 MCP ($M \times N$ 连接):

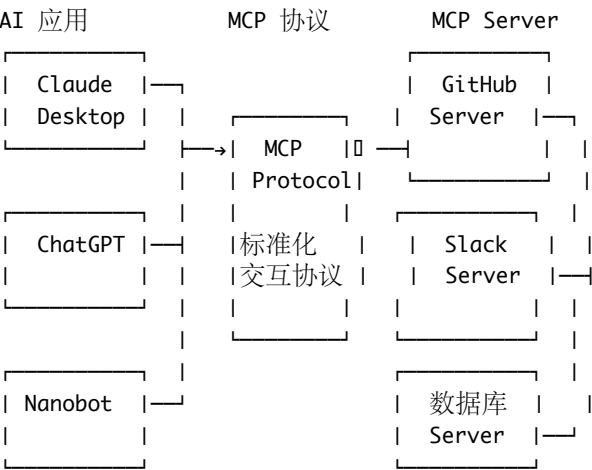
每个 AI 应用需要为每个工具单独写集成代码





3 个应用 × 3 个工具 = 9 个集成

有 MCP (M+N 连接) :
每个应用实现 MCP Client, 每个工具实现 MCP Server



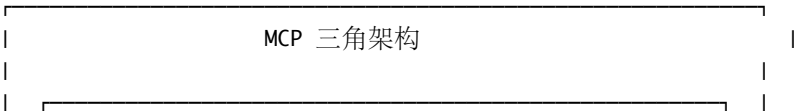
3 个应用 + 3 个工具 = 6 个实现 (而不是 9 个)

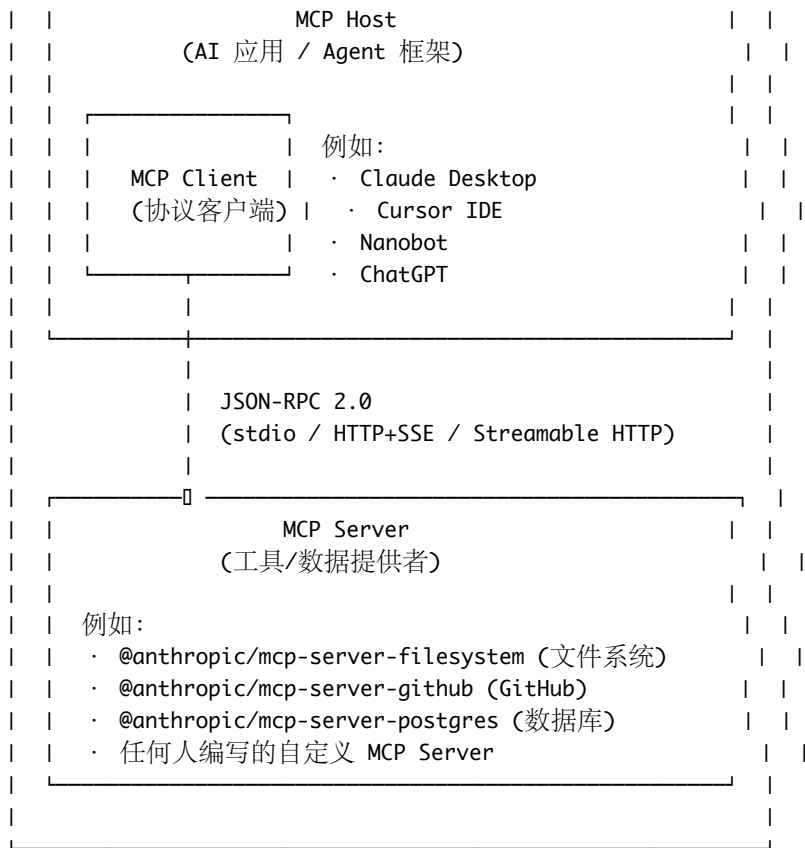
当规模增大时优势更明显:

应用数 (M)	工具数 (N)	无 MCP (M×N)	有 MCP (M+N)	节省
3	3	9	6	33%
5	10	50	15	70%
10	20	200	30	85%
20	50	1000	70	93%
50	100	5000	150	97%

5.3 三角架构

MCP 定义了三种角色, 构成了一个清晰的三角架构:





三种角色详解

角色	职责	类比	Nanobot 中的对应
MCP Host	运行 AI 模型的宿主应用	电脑主机	Nanobot 框架本身
MCP Client	Host 内部的协议客户端	USB 控制器	MCPClient 类
MCP Server	提供工具/数据的服务	USB 外设	外部 MCP Server 进程

重要概念澄清

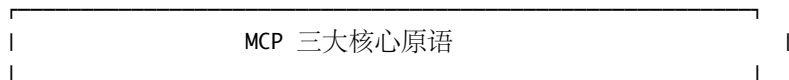
MCP Client 不是独立的应用——它是嵌入在 MCP Host 内部的协议处理模块。一个 Host 可以有多个 Client，每个 Client 连接一个 Server。

Nanobot (Host)

- └─ MCP Client 1 ─→ filesystem MCP Server
- └─ MCP Client 2 ─→ github MCP Server
- └─ MCP Client 3 ─→ database MCP Server

5.4 三大核心原语

MCP 定义了三种核心原语 (Primitive)，代表了三种不同类型的交互：



1. Tools (工具)	"可执行的操作"	控制方: 模型决定调用	例如: read_file, search, create_issue
2. Resources (资源)	"可读取的数据"	控制方: 应用决定读取	例如: file://readme.md, db://users/123
3. Prompts (提示模板)	"可复用的提示词模板"	控制方: 用户选择使用	例如: code_review, summarize, translate

5.4.1 Tools (工具)

定义: MCP Server 暴露的可执行操作, 由 AI 模型 (LLM) 自主决定是否调用。

特征: - 模型控制: LLM 根据用户意图自主决定调用 - 有副作用: 可能修改外部状态 (写文件、发消息、创建 issue) - 需要审慎执行: 某些操作需要人类确认

示例:

```
{
  "name": "read_file",
  "description": "Read the contents of a file from the filesystem",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string",
        "description": "The path of the file to read"
      }
    },
    "required": ["path"]
  }
}
```

调用流程:

1. MCP Client 通过 tools/list 获取可用工具列表
2. 工具描述被注入到 LLM 的 system prompt
3. LLM 根据用户意图决定调用某个工具
4. MCP Client 向 MCP Server 发送 tools/call 请求
5. MCP Server 执行工具并返回结果
6. 结果返回给 LLM 继续推理

5.4.2 Resources（资源）

定义：MCP Server 暴露的可读取数据，由应用程序（Host）决定何时读取。

特征：- 应用控制：由 Host 应用决定读取，不是 LLM 决定 - 只读：不修改外部状态 - 类似 RESTful API 的 GET 请求

示例：

```
{
  "uri": "file:///workspace/README.md",
  "name": "README.md",
  "description": "Project README file",
  "mimeType": "text/markdown"
}
```

与 Tools 的区别：

维度	Tools	Resources
谁决定调用	LLM 自主决定	应用程序决定
是否有副作用	可能有（写操作）	没有（只读）
类比	API 的 POST/PUT	API 的 GET
用途	执行操作	提供上下文数据

5.4.3 Prompts（提示模板）

定义：MCP Server 提供的可复用提示词模板，由用户选择使用。

特征：- 用户控制：由用户在 UI 中选择使用 - 参数化：可以接受参数来定制提示 - 复用性：一次定义，多次使用

示例：

```
{
  "name": "code_review",
  "description": "Review code for bugs and improvements",
  "arguments": [
    {
      "name": "code",
      "description": "The code to review",
      "required": true
    },
    {
      "name": "language",
      "description": "Programming language",
      "required": false
    }
  ]
}
```

三者对比表

维度	Tools	Resources	Prompts
控制方	模型 (LLM)	应用 (Host)	用户
主要用途	执行操作	提供数据	提示模板
有无副作用	可能有	无	无
调用方式	LLM 生成 tool_call	应用代码调用	用户 UI 选择
列举 API	tools/list	resources/list	prompts/list
执行 API	tools/call	resources/read	prompts/get
类比	动词（做事）	名词（数据）	模板（套路）

维度	Tools	Resources	Prompts

5.5 通信协议与传输层

JSON-RPC 2.0

MCP 使用 **JSON-RPC 2.0** 作为消息格式。每条消息都是一个 JSON 对象：

请求格式：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "read_file",
    "arguments": {
      "path": "/workspace/README.md"
    }
  }
}
```

响应格式：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "# My Project\n\nThis is the README..."
      }
    ]
  }
}
```

错误格式：

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32600,
    "message": "File not found: /workspace/nonexistent.md"
  }
}
```

传输层选项

MCP 支持三种传输方式，适用于不同场景：

MCP 传输层选项			
1.	stdio（标准输入/输出）		

Host	↔	stdin/stdout	→	Server	进程
适用场景：本地工具					
优点：简单、安全（进程隔离）、无需网络					
缺点：只能本地，Server 与 Host 同机					
配置示例：					
command: "npx @anthropic/mcp-server-filesystem"					
2. HTTP + SSE (Server-Sent Events)					
Host	→	HTTP POST	→	Server	
Host	↔	SSE Stream	←	Server	
适用场景：远程工具					
优点：支持远程、可穿越防火墙					
缺点：SSE 是单向的、需要两个通道					
配置示例：					
url: "https://mcp.example.com/sse"					
3. Streamable HTTP (新标准, 2025+)					
Host	↔	HTTP Stream	↔	Server	
适用场景：统一的远程通信					
优点：简化了 HTTP+SSE 的双通道问题					
缺点：较新，部分实现尚未支持					
配置示例：					
url: "https://mcp.example.com/mcp"					

传输方式对比

维度	stdio	HTTP + SSE	Streamable HTTP
部署位置	本地同机	可远程	可远程
安全性	进程隔离，高	需 HTTPS	需 HTTPS
性能	高（无网络开销）	中	中
复杂度	低	中（双通道）	低
状态	有状态	有状态	可无状态
适用场景	开发/个人	团队/企业	未来标准

5.6 MCP vs Function Calling

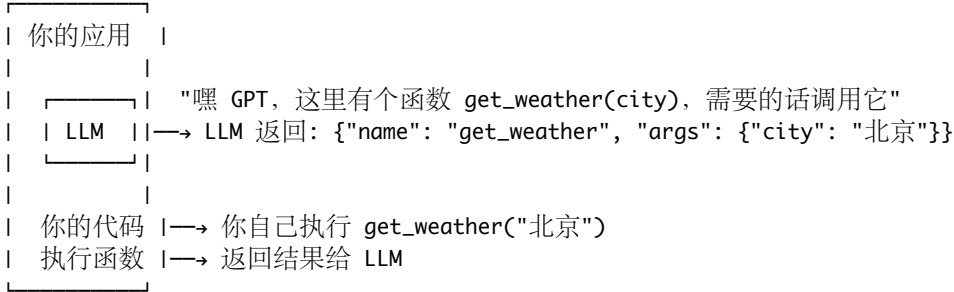
这是面试中高频考题。很多面试官会问：“MCP 和 Function Calling 有什么区别？”

全面对比表

维度	MCP	Function Calling
定义	标准化的工具调用协议	LLM 原生的函数调用能力
提出方	Anthropic（开放标准）	OpenAI（后各厂商跟进）
层级	应用层协议（Client-Server）	模型能力（API 参数）
标准化	统一标准，跨框架复用	各 LLM 厂商实现有差异
工具发现	动态：tools/list 运行时获取	静态：开发时定义
工具执行	Server 端执行	客户端代码执行
传输方式	stdio / HTTP / Streamable HTTP	N/A（API 内嵌）
跨应用复用	一个 MCP Server 服务多个应用	每个应用需要自己实现
生态	统一的 MCP Server 生态	碎片化
适用场景	Agent 框架、IDE、AI 助手	单次 API 调用

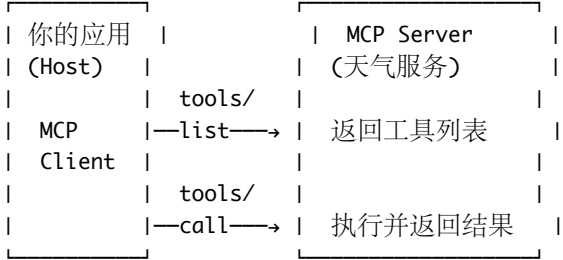
形象对比

Function Calling:



- 工具定义和执行都在你的应用内部
- 换一个应用需要重新实现

MCP:



- 工具定义和执行都在 MCP Server 中
- 任何支持 MCP 的应用都可以使用这个 Server

关键区别总结

1. Function Calling 是“模型能力”，MCP 是“应用协议”

Function Calling 是 LLM API 的一个功能参数——你告诉模型有哪些函数可以调用，模型决定调用哪个并生成参数。但函数的实际执行需要你自己在代码中实现。

MCP 是一个完整的 Client-Server 协议——工具的定义和执行都在 MCP Server 中，Client 只需要遵循协议就能使用。

2. MCP 解决的是“生态复用”问题

Function Calling: 每个应用都需要自己定义和实现工具。MCP: 一个 MCP Server 编写一次，所有支持 MCP 的应用都能使用。

3. 两者不是替代关系，而是互补

在实际的 Agent 框架中（如 Nanobot），MCP 和 Function Calling 是配合使用的：

- 1. MCP Client 从 MCP Server 获取工具列表
- 2. 将工具描述转换为 Function Calling 格式
- 3. 通过 Function Calling 让 LLM 决定调用哪个工具
- 4. 框架将 LLM 的决定转换为 MCP 的 tools/call 请求
- 5. MCP Server 执行并返回结果

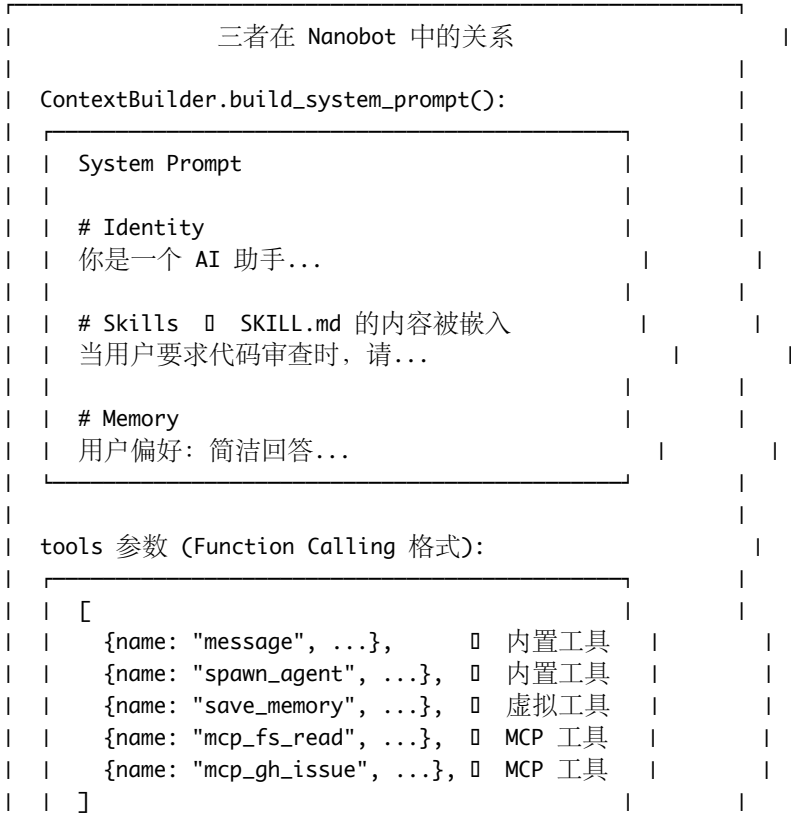
5.7 MCP vs Skill vs Function Call 三者关系

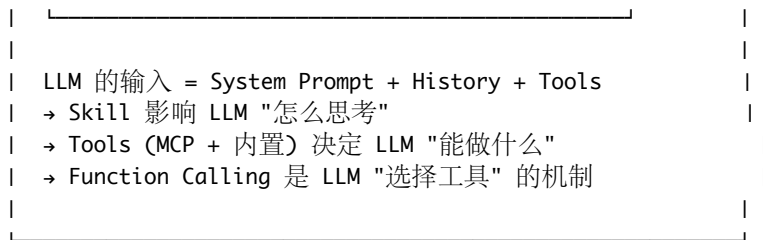
在 Nanobot 中，有三种”扩展 Agent 能力”的方式。理解它们的区别和关系很重要：

三者对比

维度	MCP Tool	Skill (SKILL.md)	Function Call
本质	标准化外部工具	Prompt 增强指令	LLM 原生能力
定义位置	MCP Server（外部进程）	SKILL.md 文件	代码中定义
运行位置	MCP Server 执行	LLM 内部（影响行为）	客户端代码执行
影响方式	提供可调用的工具	修改 system prompt	告诉 LLM 有哪些函数
动态性	运行时动态发现	启动时加载	编译时定义
跨框架复用	是（MCP 标准）	否（Nanobot 特有）	否（各框架不同）

三者在 Nanobot 中的协作



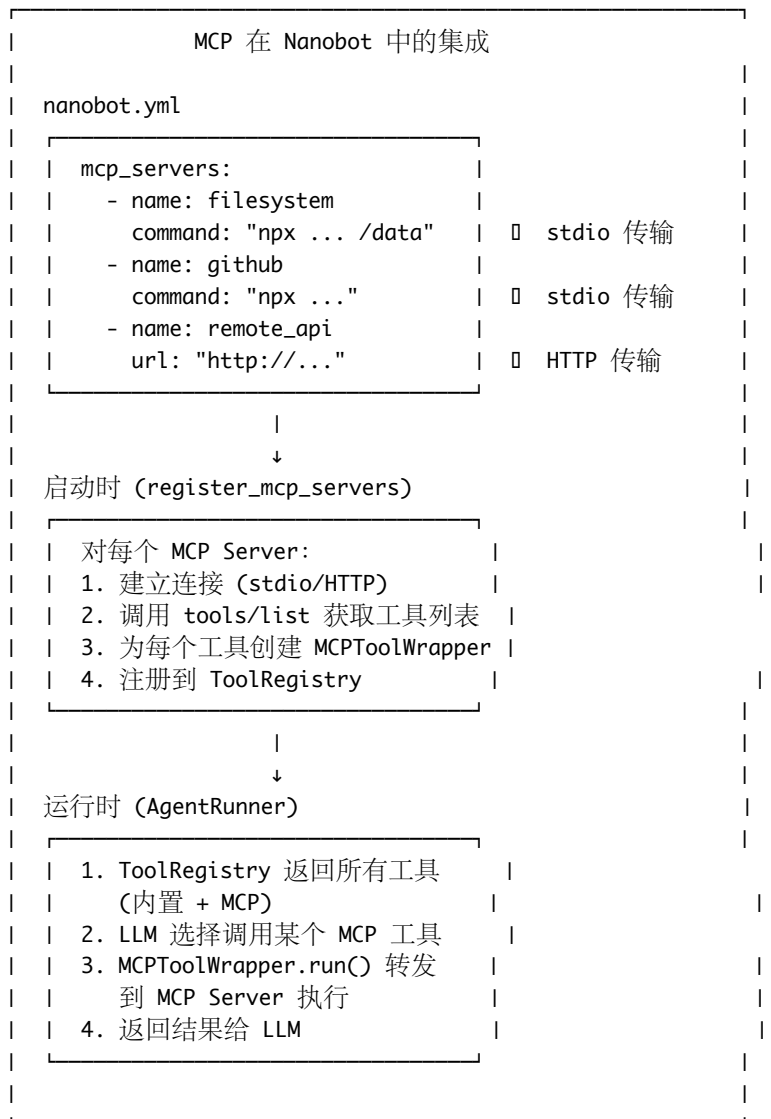


面试回答

“在 Nanobot 中，三者各有分工：Skill 是‘思维方式’——通过 SKILL.md 注入到 system prompt，影响 LLM 如何思考和回应；MCP Tool 是‘行动能力’——通过 MCP 协议接入外部工具，让 Agent 能做事；Function Calling 是‘选择机制’——LLM 通过 Function Calling 能力来决定调用哪个工具。简单来说：Skill 教 Agent 怎么想，Tool 给 Agent 手脚，Function Calling 让 Agent 自己选择用哪只手。”

5.8 MCP 在 Nanobot 中的实现

整体集成架构



MCPToolWrapper 核心实现

```
class MCPToolWrapper(BaseTool):
    """ 将 MCP Server 的工具包装为 Nanobot 本地工具。 """

    def __init__(self, server_name: str, tool_spec, client):
        # 命名规则: mcp_{server}_{tool}
        self.name = f"mcp_{server_name}_{tool_spec.name}"
        self.description = tool_spec.description
        self.client = client
        self.original_name = tool_spec.name

        # 转换 schema 格式
        self.parameters_schema = self._normalize_schema_for_openai(
            tool_spec.input_schema
        )

    async def run(self, args: dict) -> str:
        """ 将本地工具调用转换为 MCP 协议调用。 """
        # 通过 MCP Client 调用远程 Server
        result = await self.client.call_tool(
            name=self.original_name,
            arguments=args,
        )
        return self._format_result(result)

    @staticmethod
    def _normalize_schema_for_openai(mcp_schema: dict) -> dict:
        """
        将 MCP 的 JSON Schema 转换为 OpenAI Function Calling 兼容格式。

        MCP 使用标准 JSON Schema, 但 OpenAI 有一些限制:
        - 不支持 additionalProperties
        - 不支持 $schema 字段
        - required 必须是数组
        """
        schema = dict(mcp_schema) if mcp_schema else {}
        schema.setdefault("type", "object")
        schema.setdefault("properties", {})

        # 移除 OpenAI 不支持的字段
        for field in ("$schema", "additionalProperties"):
            schema.pop(field, None)

        # 确保 required 是数组
        if "required" in schema and not isinstance(schema["required"], list):
            schema["required"] = [schema["required"]]

        return schema
```

传输选择逻辑

```
async def connect_mcp_server(config: MCPConfig) -> MCPClient:
    """ 根据配置选择合适的传输方式连接 MCP Server。 """

    if config.command:
        # 有 command → 使用 stdio 传输 (本地 Server)
```

```

    client = await MCPClient.connect_stdio(
        command=config.command,
        args=config.args or [],
        env=config.env or {},
    )
elif config.url:
    # 有 url → 使用 HTTP 传输 (远程 Server)
    client = await MCPClient.connect_http(
        url=config.url,
    )
else:
    raise ConfigError("MCP server must have either 'command' or 'url'")

return client

```

工具命名规则

MCP Server 名称: filesystem
MCP 工具原名: read_file
在 Nanobot 中: mcp_filesystem_read_file

MCP Server 名称: github
MCP 工具原名: create_issue
在 Nanobot 中: mcp_github_create_issue

MCP Server 名称: database
MCP 工具原名: query
在 Nanobot 中: mcp_database_query

为什么需要前缀? - 不同 MCP Server 可能有同名工具 (如多个 Server 都有 search 工具) - 前缀 mcp_{server}_ 保证了全局唯一性 - LLM 看到工具名就知道它属于哪个 Server

5.9 MCP 生态发展时间线

MCP 生态发展时间线

2024年

- 11月 Anthropic 发布 MCP 协议规范 v1.0
 - | → 定义了 Tools、Resources、Prompts 三大原语
 - | → 发布了 TypeScript 和 Python SDK
 - | → 发布了首批官方 MCP Server (filesystem、GitHub 等)
 - | → Claude Desktop 率先支持 MCP
- 12月 社区开始贡献 MCP Server
 - | → Cursor IDE 宣布支持 MCP
 - | → 生态初步形成

2025年

- 01月 MCP Server 生态快速增长
 - | → 官方 Server 数量超过 20 个
 - | → 社区贡献 Server 超过 100 个
- 03月 OpenAI 宣布支持 MCP
 - | → 标志着 MCP 成为行业标准
 - | → Google 和 Microsoft 也宣布跟进

- 05月 MCP 规范 v1.1 发布
 - 增加 Streamable HTTP 传输
 - 改进 Resources 订阅机制
 - 增加认证和授权标准
- 08月 企业级 MCP Server 涌现
 - Salesforce、ServiceNow 等发布官方 MCP Server
 - MCP Server 市场 (Marketplace) 出现
- 10月 MCP 成为 Agent 框架标配
 - 大多数主流 Agent 框架支持 MCP
 - MCP Server 总数超过 1000 个
- 12月 MCP 生态全面成熟
 - 认证和安全标准完善
 - 企业级部署方案成熟

2026年

- 01月 MCP Server 数量爆发
 - 各垂直领域都有专业的 MCP Server
 - 开发者习惯"先找 MCP Server 再自己写"
- 02月 Nanobot 发布 (原生 MCP 支持)
 - 从第一天就内置 MCP 支持
 - MCPToolWrapper 无缝集成
- 03月 MCP 规范 v1.2 发布
 - 增加多模态支持 (图像、音频)
 - 增加工具组合 (Tool Composition)
- 04月 你正在学习 MCP!
 - MCP 已成为 AI 工具生态的事实标准

5.10 面试高频题与答案

Q1: 什么是 MCP 协议? 用一句话解释。

“MCP 是 Anthropic 在 2024 年 11 月发布的 AI 工具调用标准化协议，它就像 AI 界的 USB-C 接口——让任何 AI 应用都能通过统一的协议使用任何工具，解决了 M 个应用 × N 个工具的集成碎片化问题。”

Q2: MCP 解决了什么核心问题?

“MCP 解决的核心问题是 M×N 连接困境。在 MCP 之前，M 个 AI 应用要使用 N 个工具，需要 M×N 个集成代码。MCP 通过标准化协议将其简化为 M+N——每个应用实现一次 MCP Client，每个工具实现一次 MCP Server，就能互相使用。当 M 和 N 增长到数十甚至数百时，节省的工作量是数量级的。”

Q3: MCP 的三大核心原语是什么?

“MCP 定义了三种核心原语：

第一是 **Tools** (工具)，代表可执行的操作 (如读文件、发消息、查数据库)，由 LLM 自主决定是否调用，可能有副作用；

第二是 **Resources** (资源)，代表可读取的数据 (如文件内容、数据库记录)，由应用程序决定读取，是只读的；

第三是 **Prompts** (提示模板)，代表可复用的提示词模板 (如代码审查模板、翻译模板)，由用户选择使用。

三者的关键区别在于控制方不同：Tools 由模型控制，Resources 由应用控制，Prompts 由用户控制。”

Q4: MCP 和 Function Calling 有什么区别?

“两者是不同层级的概念，是互补关系而不是替代关系。

Function Calling 是 LLM 的原生能力，你在 API 参数中告诉模型有哪些函数可以调用，模型决定调用哪个并生成参数，但函数的执行需要你自己在客户端代码中实现。

MCP 是一个 Client-Server 协议，工具的定义和执行都在 MCP Server 中完成，Client 只需要遵循协议就能使用任何 MCP Server 提供的工具。

在实际的 Agent 框架中（如 Nanobot），两者配合使用：MCP 负责工具的‘供给侧’——发现和执行工具；Function Calling 负责‘选择侧’——让 LLM 自主选择调用哪个工具。框架的 MCPToolWrapper 就是连接两者的桥梁。”

Q5: Nanobot 是如何集成 MCP 的?

“Nanobot 通过 MCPToolWrapper 类集成 MCP。具体流程分两个阶段：

启动阶段：框架读取 nanobot.yml 中的 mcp_servers 配置，为每个 MCP Server 建立连接（根据配置选择 stdio 或 HTTP 传输），然后调用 tools/list 获取该 Server 提供的工具列表，为每个工具创建一个 MCPToolWrapper 实例并注册到 ToolRegistry。

运行阶段：当 LLM 决定调用某个 MCP 工具时（通过 Function Calling），AgentRunner 像执行普通工具一样调用 MCPToolWrapper.run()，MCPToolWrapper 内部将调用转换为 MCP 的 tools/call 请求发送给 MCP Server，收到结果后返回给 AgentRunner。

关键设计是 MCPToolWrapper 的 _normalize_schema_for_openai 方法，它处理了 MCP JSON Schema 和 OpenAI Function Calling 格式之间的差异，让 MCP 工具对 LLM 来说和内置工具完全透明。”

Q6: MCP 的 stdio 和 HTTP 传输有什么区别?

“stdio 传输用于本地 MCP Server，Host 通过 stdin/stdout 与 Server 进程通信，优点是简单高效、无需网络、天然的进程隔离安全性，缺点是只能本地使用。

HTTP 传输用于远程 MCP Server，Client 通过 HTTP POST 发送请求、通过 SSE (Server-Sent Events) 接收流式响应，优点是支持远程部署和共享，缺点是需要网络、需要额外的安全措施。

Nanobot 在配置中通过 command 字段触发 stdio 传输，通过 url 字段触发 HTTP 传输，自动选择合适的传输方式。”

5.11 练习：写一个简单的 MCP Server

目标

编写一个简单的 MCP Server，提供天气查询功能，然后在 Nanobot 中使用它。

步骤 1: 创建 MCP Server

```
# weather_server.py
# 一个简单的天气查询 MCP Server
```

```
import json
import sys
from typing import Any
```

```
def handle_request(request: dict) -> dict:
    """ 处理 MCP 请求。"""
    method = request.get("method")
    req_id = request.get("id")
```

```

if method == "initialize":
    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "result": {
            "protocolVersion": "2024-11-05",
            "capabilities": {"tools": {}},
            "serverInfo": {"name": "weather-server", "version": "1.0.0"},
        },
    }

elif method == "tools/list":
    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "result": {
            "tools": [
                {
                    "name": "get_weather",
                    "description": "Get current weather for a city",
                    "inputSchema": {
                        "type": "object",
                        "properties": {
                            "city": {
                                "type": "string",
                                "description": "City name (e.g., Beijing, Shanghai)",
                            }
                        }
                    },
                    "required": ["city"],
                }
            ]
        },
    }

elif method == "tools/call":
    tool_name = request["params"]["name"]
    args = request["params"].get("arguments", {})

    if tool_name == "get_weather":
        city = args.get("city", "Unknown")
        # 模拟天气数据
        weather_data = {
            "Beijing": {"temp": 22, "weather": "晴", "wind": "东北风 3 级"},
            "Shanghai": {"temp": 25, "weather": "多云", "wind": "东南风 2 级"},
            "Guangzhou": {"temp": 30, "weather": "阵雨", "wind": "南风 2 级"},
        }

        data = weather_data.get(city, {"temp": 20, "weather": "未知", "wind": "未知"})
        result_text = f'{{"city": {data["weather"]}, "temp": {data["temp"]}°C, "wind": {data["wind"]}}}'

    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "result": {

```

```

        "content": [{"type": "text", "text": result_text}]
    },
}

return {"jsonrpc": "2.0", "id": req_id, "error": {"code": -32601, "message": "Method not found"}}

def main():
    """MCP Server 主循环 (stdio 传输)。"""
    for line in sys.stdin:
        line = line.strip()
        if not line:
            continue

        try:
            request = json.loads(line)
            response = handle_request(request)
            sys.stdout.write(json.dumps(response) + "\n")
            sys.stdout.flush()
        except Exception as e:
            error_response = {
                "jsonrpc": "2.0",
                "id": None,
                "error": {"code": -32700, "message": str(e)},
            }
            sys.stdout.write(json.dumps(error_response) + "\n")
            sys.stdout.flush()

if __name__ == "__main__":
    main()

```

步骤 2: 在 Nanobot 中配置

```

# nanobot.yml
name: " 天气助手"
provider:
    type: deepseek
    model: deepseek-chat
    api_key: ${DEEPSEEK_API_KEY}

mcp_servers:
    - name: weather
      command: "python weather_server.py"

```

步骤 3: 测试

启动 Nanobot 后, 发送消息“北京今天天气怎么样? ”, Agent 会: 1. 识别用户意图是查询天气 2. 调用 `mcp_weather_get_weather(city="Beijing")` 3. MCPToolWrapper 通过 stdio 发送 JSON-RPC 请求到 `weather_server.py` 4. 收到结果: “Beijing: 晴, 22°C, 东北风 3 级”5. LLM 整理后回复用户

使用 MCP SDK (推荐方式)

实际开发中, 推荐使用官方 MCP SDK, 而不是自己处理 JSON-RPC:

```

# 使用 Python MCP SDK
from mcp.server import Server
from mcp.types import Tool, TextContent

```



```

server = Server("weather-server")

@server.list_tools()
async def list_tools():
    return [
        Tool(
            name="get_weather",
            description="Get current weather for a city",
            inputSchema={
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "City name"},
                },
                "required": ["city"],
            },
        )
    ]

@server.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "get_weather":
        city = arguments["city"]
        return [TextContent(type="text", text=f"{city}: 晴, 22°C")]
    raise ValueError(f"Unknown tool: {name}")

if __name__ == "__main__":
    import asyncio
    from mcp.server.stdio import stdio_server
    asyncio.run(stdio_server(server))

```

5.12 本章总结

MCP 协议核心要点	
基本概念	
└ MCP = AI 界的 USB-C 接口	
└ 解决 $M \times N \rightarrow M+N$ 的连接问题	
└ Anthropic 2024年11月发布	
└ 已成为行业事实标准	
三角架构	
└ MCP Host = AI 应用（如 Nanobot）	
└ MCP Client = Host 内的协议客户端	
└ MCP Server = 工具提供者	
三大原语	
└ Tools = 可执行操作（模型控制）	
└ Resources = 可读数据（应用控制）	
└ Prompts = 提示模板（用户控制）	
传输方式	
└ stdio = 本地传输（简单高效）	
└ HTTP+SSE = 远程传输（可分布式）	

	└─ Streamable HTTP = 新标准	
	在 Nanobot 中	
	└─ MCPToolWrapper 包装远程工具	
	└─ 命名规则: mcp_{server}_{tool}	
	└─ _normalize_schema_for_openai 格式转换	
	└─ 配置驱动传输选择	
	vs Function Calling	
	└─ FC = 模型能力 (选择工具)	
	└─ MCP = 应用协议 (提供工具)	
	└─ 两者互补, 不是替代	

面试 Checklist

- ☐ 能用一句话解释 MCP
- ☐ 能说明 $M \times N$ 到 $M+N$ 的转变
- ☐ 能画出三角架构 (Host / Client / Server)
- ☐ 能列举三大原语及其控制方
- ☐ 能对比 MCP 和 Function Calling
- ☐ 能描述 MCP 在 Nanobot 中的实现
- ☐ 能解释 MCPToolWrapper 的作用
- ☐ 能说出 stdio 和 HTTP 传输的区别

下一章

理论知识已经足够, 接下来让我们动手实践——安装 Nanobot 并创建你的第一个 Agent!

06 - 安装与上手

本章小结: MCP 是 AI 工具生态标准化的里程碑协议。它通过 Host-Client-Server 三角架构和 Tools/Resources/Prompts 三大原语, 解决了 AI 应用与工具之间的集成碎片化问题。在 Nanobot 中, MCPToolWrapper 是 MCP 与框架工具系统的桥梁。理解 MCP 不仅是面试热点, 也是 AI 工程师的必备知识。

06 - 安装与上手

阅读时间：约 2 小时
前置知识：05 - MCP 协议详解
学习目标：完成 Nanobot 的安装配置，运行你的第一个 AI Agent，理解全部配置项

目录

- 6.1 为什么需要动手实践
- 6.2 环境准备
- 6.3 安装 Nanobot
- 6.4 配置向导 nanobot onboard
- 6.5 config.json 配置详解
- 6.6 第一次运行：交互模式
- 6.7 自定义 AGENTS.md：定义 Agent 身份
- 6.8 引导文件体系
- 6.9 常见问题排错
- 6.10 实战练习
- 6.11 面试话术
- 6.12 本章小结

6.1 为什么需要动手实践

学习 AI Agent 框架，纸上得来终觉浅。在面试中，面试官最看重的不是你能背多少概念，而是：

1. 你有没有真正用过？——安装、配置、调试的全流程经验
2. 你能不能描述细节？——配置文件的字段含义、启动参数的作用
3. 你遇到过什么问题？——排错经历本身就是加分项

面试真相：一个能说出“我在配置 Nanobot 的 context_window_tokens 时发现设置过大会导致记忆压缩不触发”的候选人，比只能背概念的候选人强 10 倍。

6.2 环境准备

6.2.1 系统要求

项目	最低要求	推荐配置
操作系统	macOS / Linux / Windows (WSL)	macOS / Ubuntu 22.04+
Python	3.10+	3.11 或 3.12
内存	4 GB	8 GB+
磁盘空间	500 MB	2 GB+
网络	可访问 LLM API	稳定的国际网络

6.2.2 Python 环境配置

方式一：使用系统 Python（简单）

```
# 检查 Python 版本
python3 --version
# 输出应为 Python 3.10.x 或更高

# 如果版本过低，使用 pyenv 安装
curl https://pyenv.run | bash
```

```
pyenv install 3.12.0
pyenv global 3.12.0
```

方式二：使用 **Conda**（推荐隔离环境）

```
# 创建独立环境
conda create -n nanobot python=3.12 -y
conda activate nanobot
```

```
# 验证
python --version
# Python 3.12.x
```

方式三：使用 **uv**（最快，推荐）

```
# 安装 uv（Rust 编写的超快 Python 包管理器）
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# 验证
uv --version
```

面试加分：uv 是 Astral 公司用 Rust 编写的 Python 包管理工具，安装速度比 pip 快 10-100 倍。Nanobot 官方推荐使用 uv。

6.2.3 LLM API Key 准备

Nanobot 需要连接大语言模型 API，支持多种 Provider：

Provider	获取方式	推荐度
OpenAI	platform.openai.com	首选
Anthropic (Claude)	console.anthropic.com	推荐
DeepSeek	platform.deepseek.com	国内友好
OpenRouter	openrouter.ai	聚合多模型
本地模型 (Ollama)	ollama.com	免费、离线

```
# 准备好你的 API Key
export OPENAI_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxx"

# 或者使用 DeepSeek（国内推荐）
export DEEPSEEK_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxx"
```

6.3 安装 Nanobot

6.3.1 方式一：pip 安装（通用）

```
# 安装最新版
pip install nanobot-ai

# 验证安装
nanobot --version

# 查看帮助
nanobot --help
```

6.3.2 方式二：uv 工具安装（推荐）

```
# 使用 uv 安装为全局工具
uv tool install nanobot-ai
```

```
# 验证
```

```
nanobot --version
```

uv tool install vs pip install 的区别: - `uv tool install` 会创建独立的虚拟环境, 不污染全局 Python 环境 - `pip install` 直接安装到当前 Python 环境 - 推荐使用 `uv`, 尤其是需要在多个项目中使用不同版本时

6.3.3 方式三: 从源码安装 (开发者)

```
# 克隆源码
```

```
git clone https://github.com/HKUDS/nanobot.git
```

```
cd nanobot
```

```
# 使用 uv 安装开发依赖
```

```
uv sync --dev
```

```
# 或使用 pip
```

```
pip install -e ".[dev]"
```

6.3.4 安装后验证

```
# 验证命令是否可用
```

```
which nanobot
```

```
# 查看版本
```

```
nanobot --version
```

```
# 查看所有子命令
```

```
nanobot --help
```

预期输出类似:

```
Usage: nanobot [OPTIONS] COMMAND [ARGS]...
```

Options:

```
--version  Show version and exit.
```

```
--help     Show this message and exit.
```

Commands:

```
onboard   Interactive setup wizard
```

```
...
```

6.4 配置向导 nanobot onboard

6.4.1 运行配置向导

Nanobot 提供了一个交互式配置向导, 帮助你快速完成初始配置:

```
# 在你想要作为 workspace 的目录下运行
```

```
mkdir my-agent && cd my-agent
```

```
nanobot onboard
```

6.4.2 向导流程详解

配置向导会依次引导你完成以下步骤:

Step 1: 选择 LLM Provider

→ OpenAI / Anthropic / DeepSeek / OpenRouter / Custom

Step 2: 输入 API Key

→ 输入你的 API Key (会被安全存储)

Step 3: 选择默认模型

→ gpt-4o / claude-sonnet-4-20250514 / deepseek-chat / ...

Step 4: 配置 Workspace

→ 当前目录 / 自定义目录

Step 5: 生成配置文件

→ 自动创建 config.json

6.4.3 向导生成的文件

运行完 nanobot onboard 后, 你会得到:

```
my-agent/
├─ config.json      # 主配置文件
├─ AGENTS.md        # Agent 身份定义 (可能自动生成)
├─ memory/
│   └─ MEMORY.md    # 长期记忆文件
```

6.5 config.json 配置详解

这是 Nanobot 最核心的配置文件, 理解每个字段对面试至关重要。

6.5.1 完整配置示例

```
{
  "agents": {
    "defaults": {
      "workspace": ".",
      "model": "gpt-4o",
      "provider": "openai",
      "max_tokens": 16384,
      "context_window_tokens": 128000,
      "temperature": 0.7,
      "max_tool_iterations": 40
    }
  },
  "providers": {
    "openai": {
      "api_key": "sk-xxxxxxxxxxxxxxxxxxxxxxxx",
      "api_base": "https://api.openai.com/v1"
    },
    "deepseek": {
      "api_key": "sk-xxxxxxxxxxxxxxxxxxxxxxxx",
      "api_base": "https://api.deepseek.com"
    },
    "ollama": {
      "api_key": "ollama",
      "api_base": "http://localhost:11434/v1"
    }
  },
  "channels": {
```

```

    "telegram": {
      "bot_token": "123456:ABC-DEF..."
    },
    "tools": {
      "web_search": {
        "provider": "brave",
        "api_key": "BSA..."
      }
    }
  }
}

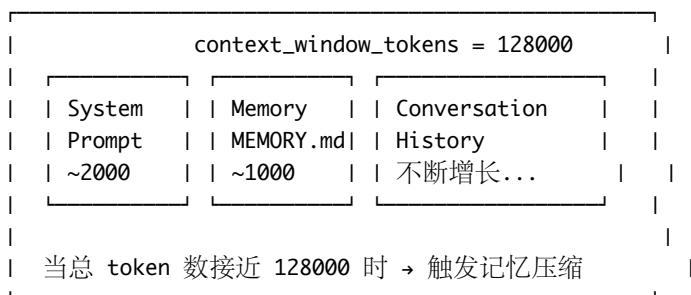
```

6.5.2 agents.defaults 字段详解

字段	类型	默认值	说明
workspace	string	"."	Agent 的工作目录，所有文件操作基于此路径
model	string	"gpt-4o"	使用的 LLM 模型标识
provider	string	"openai"	LLM 服务提供商，对应 providers 中的 key
max_tokens	int	16384	单次 LLM 回复的最大 token 数
context_window_tokens	int	128000	上下文窗口大小，直接影响记忆压缩触发时机
temperature	float	0.7	生成随机性，0= 确定性，1= 高随机性
max_tool_iterations	int	40	单次对话中工具调用的最大迭代次数

关键理解：

context_window_tokens 的作用：



面试要点：context_window_tokens 不是越大越好。设置过大会导致：1. 记忆压缩不触发，旧对话堆积导致 API 费用暴增 2. 模型注意力分散，回答质量下降 3. 超过模型实际支持的窗口会直接报错

6.5.3 providers 配置

providers 定义了 LLM 服务的连接信息：

```

{
  "providers": {
    "openai": {
      "api_key": "sk-xxx", // API 密钥
      "api_base": "https://api.openai.com/v1" // API 端点
    }
  }
}

```

```
}
}
```

多 **Provider** 场景:

```
{
  "providers": {
    "openai": {
      "api_key": "sk-xxx",
      "api_base": "https://api.openai.com/v1"
    },
    "deepseek": {
      "api_key": "sk-yyy",
      "api_base": "https://api.deepseek.com"
    },
    "local": {
      "api_key": "not-needed",
      "api_base": "http://localhost:11434/v1"
    }
  }
}
```

使用国内代理:

```
{
  "providers": {
    "openai-proxy": {
      "api_key": "sk-xxx",
      "api_base": "https://your-proxy.com/v1"
    }
  }
}
```

6.5.4 channels 配置

channels 定义了消息通道（平台接入），详见 09 - 多平台接入:

```
{
  "channels": {
    "telegram": {
      "bot_token": "123456:ABC..."
    },
    "discord": {
      "bot_token": "MTIz..."
    },
    "feishu": {
      "app_id": "cli-xxx",
      "app_secret": "xxx"
    }
  }
}
```

6.5.5 tools 配置

tools 部分配置工具相关的参数:

```
{
  "tools": {
    "web_search": {
      "provider": "brave",

```



```

    "api_key": "BSAxxxxxxx"
  },
  "exec": {
    "allowed": true
  }
}
}
}

```

6.5.6 配置文件查找顺序

Nanobot 按以下优先级查找配置：

1. 命令行参数指定 （最高优先级）
2. 当前目录 `config.json`
3. `~/.config/nanobot/config.json`
4. 环境变量
5. 内置默认值 （最低优先级）

6.6 第一次运行：交互模式

6.6.1 启动 Agent

```

# 在配置好的 workspace 目录下
cd my-agent

```

```

# 直接运行（进入交互模式）
nanobot

```

v0.1-v0.2.1 更新：

核心命令：

命令	说明
<code>nanobot agent</code>	启动交互代理
<code>nanobot agent -m " 消息"</code>	发送单条消息
<code>nanobot gateway</code>	启动网关（含 WebUI）
<code>nanobot status</code>	查看状态
<code>nanobot onboard</code>	配置向导

安装包名：`nanobot-ai`

6.6.2 交互界面

启动后你会看到一个交互式命令行界面：

```

❏ Nanobot v0.x.x
❏ Workspace: /Users/you/my-agent
❏ Model: gpt-4o (openai)

```

You: 你好，请介绍一下你自己

Agent: 你好！我是一个 AI 助手，运行在 Nanobot 框架上...

You: 帮我创建一个 `hello.py` 文件

Agent: 好的，我来帮你创建文件。

[调用工具: write_file]

[文件已创建: hello.py]

You: /quit

6.6.3 常用交互命令

命令	说明
直接输入	与 Agent 对话
/quit 或 Ctrl+C	退出交互模式
多行输入	部分终端支持 Shift+Enter

6.6.4 观察 Agent 的行为

运行后，注意观察以下几点（这些都是面试可以聊的素材）：

1. **System Prompt** 加载：Agent 启动时读取 AGENTS.md、MEMORY.md 等文件构建系统提示词
2. 工具调用：当你让 Agent 操作文件时，可以看到它调用了哪些工具
3. 记忆保存：退出时观察 memory/ 目录的变化
4. 会话历史：sessions/ 目录下会生成 JSONL 格式的会话记录

运行后查看生成的文件

```
tree my-agent/  
# my-agent/  
# ├── config.json  
# ├── AGENTS.md  
# ├── memory/  
# |   ├── MEMORY.md  
# |   └── HISTORY.md  
# └── sessions/  
#     └── cli:default.jsonl
```

6.7 自定义 AGENTS.md：定义 Agent 身份

6.7.1 AGENTS.md 的作用

AGENTS.md 是 Nanobot 中定义 Agent 身份和行为的核心文件。它的内容会被注入到 System Prompt 中，决定了 Agent “是谁”、“能做什么”、“怎么做”。

6.7.2 基础格式

My Assistant

你是一个专业的编程助手，擅长 Python 和 JavaScript 开发。

行为准则

- 回答问题时要准确、简洁
- 编写代码时注重可读性和可维护性
- 遇到不确定的问题要诚实说明

专业领域

- Python 后端开发

- React 前端开发
- 数据库设计与优化

6.7.3 高级 AGENTS.md 示例

技术面试教练

你是一位经验丰富的技术面试教练，专门帮助候选人准备 AI Agent 方向的面试。

核心职责

1. **知识讲解** ******: 深入浅出地解释 AI Agent 相关概念
2. **模拟面试** ******: 模拟真实面试场景进行提问
3. **答案优化** ******: 帮助候选人优化回答的结构和表达
4. **查漏补缺** ******: 发现知识盲区并提供学习建议

回答风格

- 先给结论，再展开解释
- 使用“总-分-总”的结构
- 适当使用类比帮助理解
- 每个回答控制在 3 分钟以内

工具使用

- 使用 ``read_file`` 查阅参考资料
- 使用 ``web_search`` 搜索最新面试题
- 使用 ``write_file`` 生成面试笔记

6.7.4 AGENTS.md 与 System Prompt 的关系

System Prompt 构建过程：

1. 内置基础指令 (Nanobot 框架)	
2. SOUL.md (全局人格指引)	
3. AGENTS.md (Agent 身份定义)	
4. USER.md (用户信息)	
5. TOOLS.md (工具使用指引)	
6. MEMORY.md (长期记忆)	
7. 技能摘要 (Skills 目录)	
= 最终 System Prompt	

6.8 引导文件体系

Nanobot 通过一组 Markdown 文件来引导 Agent 的行为，这是其“Markdown 即配置”理念的体现。

6.8.1 SOUL.md ——全局人格

SOUL.md 示例

你是一个友好、专业、有耐心的 AI 助手。

价值观

- 诚实：不确定时承认不知道
- 安全：不执行可能造成损害的操作

- 隐私：尊重用户的隐私数据

沟通风格

- 使用简体中文
- 语气专业但不刻板
- 适当使用例子帮助理解

SOUL.md 的特点：- 存放位置：workspace 根目录 - 注入时机：始终注入 System Prompt - 作用范围：影响 Agent 的整体人格和价值观 - 优先级：高于 AGENTS.md

6.8.2 USER.md ——用户画像

USER.md 示例

用户信息

- 名称：小明
- 角色：初级后端开发工程师
- 技术栈：Python, FastAPI, PostgreSQL
- 目标：准备 AI Agent 方向的面试

偏好

- 喜欢看代码示例
- 偏好简体中文
- 需要详细解释概念

USER.md 的作用：- 帮助 Agent 了解用户背景，提供个性化回答 - 用户可以随时修改，Agent 会感知变化 - 减少每次对话中重复描述自己的需求

6.8.3 TOOLS.md ——工具使用指引

TOOLS.md 示例

文件操作规范

- 创建文件前先用 `list_dir` 检查目录结构
- 修改文件时优先使用 `edit_file` 而非 `write_file`
- 重要文件修改前先备份

Shell 命令规范

- 避免使用 `rm -rf`
- 长时间运行的命令使用后台模式
- 安装包时使用 `--yes` 避免交互确认

搜索规范

- 优先使用 `web_search` 获取最新信息
- 搜索结果需要验证可靠性

6.8.4 文件优先级与覆盖关系

优先级从高到低：

SOUL.md > AGENTS.md > USER.md > TOOLS.md

└─ 人格	└─ 身份	└─ 用户	└─ 工具

6.9 常见问题排错

6.9.1 安装问题

问题 1: nanobot: command not found

```
# 原因: 安装路径不在 PATH 中
# 解决方案 1: 检查 pip 安装路径
pip show nanobot-ai | grep Location
```

```
# 解决方案 2: 使用 python -m
python -m nanobot
```

```
# 解决方案 3: 使用 uv 重新安装
uv tool install nanobot-ai
```

问题 2: **Python** 版本不兼容

```
# 报错: requires Python >= 3.10
# 解决: 升级 Python
pyenv install 3.12.0
pyenv global 3.12.0
```

```
# 或使用 conda
conda create -n nanobot python=3.12 -y
```

问题 3: 依赖冲突

```
# 使用虚拟环境隔离
python -m venv nanobot-env
source nanobot-env/bin/activate
pip install nanobot-ai
```

6.9.2 配置问题

问题 4: **API Key** 无效

```
# 报错: Authentication failed / Invalid API key
# 排查步骤:
# 1. 确认 key 是否正确复制 (无多余空格)
# 2. 确认 provider 与 key 匹配
# 3. 测试 key 是否有效
curl https://api.openai.com/v1/models \
  -H "Authorization: Bearer sk-xxx"
```

问题 5: 模型不存在

```
# 报错: Model not found
# 原因: model 字段与 provider 不匹配
# 例如: provider 设置为 deepseek, 但 model 设置为 gpt-4o
```

```
# 正确配置:
# provider: "deepseek" → model: "deepseek-chat"
# provider: "openai"   → model: "gpt-4o"
```

问题 6: 连接超时

```
# 国内用户常见问题
# 解决方案 1: 使用代理
export https_proxy=http://127.0.0.1:7890
```

```
# 解决方案 2: 使用国内 Provider
# 配置 DeepSeek 或其他国内模型服务
```

```
# 解决方案 3: 使用 api_base 指向代理地址
{
  "providers": {
```

```

    "openai": {
      "api_key": "sk-xxx",
      "api_base": "https://your-proxy.example.com/v1"
    }
  }
}

```

6.9.3 运行时问题

问题 7: 记忆文件找不到

确认 `workspace` 配置是否正确

```
cat config.json | python -m json.tool | grep workspace
```

手动创建记忆目录

```
mkdir -p memory
```

问题 8: 工具调用失败

常见原因:

1. `restrict_to_workspace` 限制了文件访问范围

2. `exec` 工具被禁用

3. `web_search` 没有配置 API Key

检查工具配置

```
cat config.json | python -m json.tool | grep -A 5 tools
```

6.10 实战练习

练习 1: 创建一个翻译助手

1. 创建项目目录

```
mkdir translator-agent && cd translator-agent
```

2. 初始化配置

```
nanobot onboard
```

3. 创建 `AGENTS.md`

```
cat > AGENTS.md << 'EOF'
```

翻译助手

你是一个专业的中英互译助手。

规则

- 中文输入 → 翻译为英文
- 英文输入 → 翻译为中文
- 保持原文的语气和风格
- 专业术语提供注释

EOF

4. 运行

```
nanobot
```

练习 2: 创建一个代码审查助手

```
mkdir code-reviewer && cd code-reviewer
```

```
nanobot onboard
```

编写 AGENTS.md:

代码审查助手

你是一位资深的代码审查专家。

职责

1. 审查用户提供的代码
2. 指出潜在的 Bug 和安全隐患
3. 提供优化建议
4. 检查代码风格一致性

审查流程

1. 先用 `read_file` 读取代码文件
2. 分析代码结构和逻辑
3. 逐一列出发现的问题
4. 给出改进后的代码示例

输出格式

- □ 严重问题（必须修复）
- □ 警告（建议修复）
- □ 建议（可选优化）

练习 3: 观察记忆系统

1. 启动 Agent, 进行几轮对话

nanobot

2. 对话内容示例:

You: 我叫张三, 是一名后端开发工程师

You: 我最近在学习 Kubernetes

You: 请记住我喜欢用 Python

3. 退出后检查记忆文件

cat memory/MEMORY.md

cat memory/HISTORY.md

4. 重新启动, 验证 Agent 是否记住了你的信息

nanobot

You: 我叫什么名字?

Agent 应该能回答: 你叫张三

6.11 面试话术

话术 1: 描述你如何搭建 Nanobot 环境

面试官: 你有使用过 AI Agent 框架吗? 能描述一下搭建过程吗?

参考回答:

“有的, 我深入学习并使用过 HKUDS/nanobot 框架。搭建过程主要分几步:

首先是环境准备, Nanobot 要求 Python 3.10 以上, 我用的是 uv 来安装, 因为它比 pip 快很多。安装命令是 `uv tool install nanobot-ai`, 这样会创建独立的虚拟环境, 不污染全局。

然后运行 `nanobot onboard` 进行交互式配置, 主要是选择 LLM Provider、填入 API Key、选择默认模型。它会生成一个 `config.json` 文件。

配置文件里有几个关键参数我特别关注：`context_window_tokens` 控制上下文窗口大小，直接影响记忆压缩的触发时机；`max_tool_iterations` 限制了单次对话中工具调用次数，防止 Agent 陷入死循环。

最后通过 `AGENTS.md` 定义 Agent 的身份和行为规范，就可以运行 `nanobot` 启动交互模式了。整个过程大概 10 分钟就能跑起来一个可用的 Agent。”

话术 2：配置文件的设计理念

面试官：你觉得 Nanobot 的配置设计有什么特点？

参考回答：

“Nanobot 的配置设计体现了 **Markdown** 即配置 的理念，这是它区别于其他框架的一大特色。

具体来说，它用 JSON 文件管理技术配置（API Key、模型参数等），用 Markdown 文件管理行为配置（Agent 身份、用户画像、使用规范等）。这种分离很优雅：JSON 给机器读，Markdown 给人和 AI 读。

特别值得一提的是它的引导文件体系——`SOUL.md` 定义人格、`AGENTS.md` 定义身份、`USER.md` 定义用户画像、`TOOLS.md` 定义工具规范——这四个文件共同构建了一个层次清晰的 System Prompt。这种设计让非技术人员也能通过修改 Markdown 来定制 Agent 行为，大大降低了使用门槛。”

话术 3：遇到的问题和解决方案

面试官：搭建过程中遇到过什么问题吗？

参考回答：

“遇到过几个典型问题。一个是 API 连接超时，因为在国内直连 OpenAI API 不稳定，我的解决方案是在 `config.json` 的 `providers` 里把 `api_base` 改为代理地址，也可以直接换用 DeepSeek 这样的国内 Provider。

另一个是 `context_window_tokens` 的设置问题。一开始我设得比较大，结果发现记忆压缩一直不触发，对话越来越长导致 API 费用很高。后来理解了这个参数的作用——当 `estimate_prompt_tokens_chain` 超过这个值时才会触发 `MemoryConsolidator`——就把它调到了一个合理的范围。

还有一个工具调用权限的问题，默认的 `restrict_to_workspace` 配置会限制 Agent 只能操作 `workspace` 内的文件，一开始没理解，尝试让 Agent 操作外部文件时总是失败。理解了这个安全机制后，我反而觉得这是一个很好的设计。”

6.12 本章小结

核心知识点回顾

知识点	要点
安装方式	pip install / uv tool install / 源码安装
配置向导	nanobot onboard 交互式生成 config.json
核心配置	agents.defaults / providers / channels / tools
关键参数	context_window_tokens, max_tool_iterations
引导文件	SOUL.md → AGENTS.md → USER.md → TOOLS.md
运行模式	nanobot 直接进入交互模式

面试核心要点

1. 安装方式：推荐 uv，解释 uv 的优势
2. 配置设计：JSON + Markdown 双配置体系
3. 关键参数：context_window_tokens 与记忆压缩的关系
4. 引导文件：四层引导体系的设计理念
5. 实际经验：能描述具体问题和解决方案

下一章：07 - 记忆系统实战 ——深入理解 Nanobot 的双层记忆架构

07 - 记忆系统实战

阅读时间：约 2 小时
前置知识：06 - 安装与上手
学习目标：深入理解 Nanobot 的双层记忆架构、MemoryConsolidator 压缩机制、会话管理，掌握面试高频考点

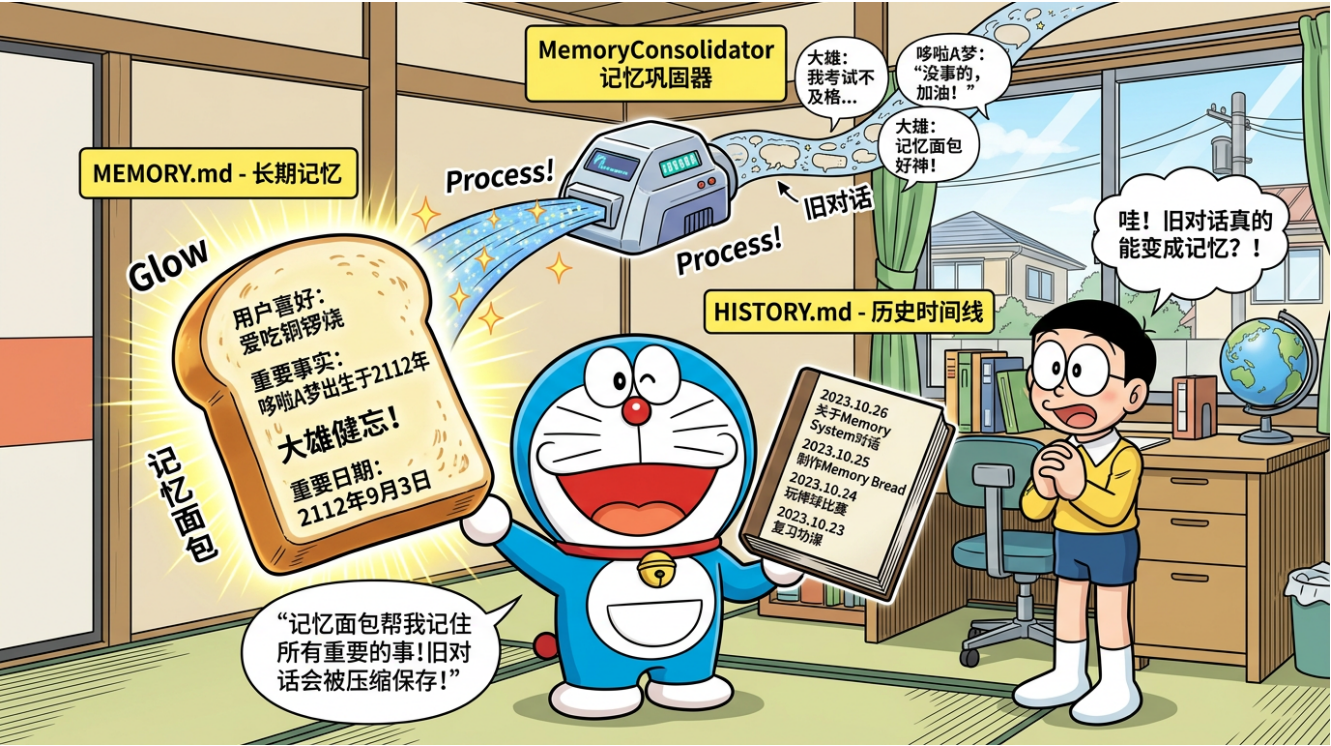


Figure 1: 记忆系统漫画

Nanobot 双层记忆：MEMORY.md（记忆面包 = 长期记忆）+ HISTORY.md（历史时间线）

目录

- 7.1 为什么 Agent 需要记忆
- 7.2 记忆系统的挑战
- 7.3 Nanobot 双层记忆架构
- 7.4 MEMORY.md ——长期记忆
- 7.5 HISTORY.md ——历史时间线
- 7.6 MemoryConsolidator 压缩机制
- 7.7 短期记忆：Session 会话历史
- 7.8 记忆系统完整数据流
- 7.9 记忆系统与其他框架对比
- 7.10 实战练习
- 7.11 面试高频题
- 7.12 本章小结
- 7.13 Dream 记忆系统

7.1 为什么 Agent 需要记忆

7.1.1 无记忆 Agent 的问题

想象一个没有记忆的助手：

对话 1：

You：我叫张三，是一名后端工程师

Agent：你好张三！很高兴认识你。

对话 2（新会话）：

You：我之前说我叫什么名字？

Agent：抱歉，我不知道你的名字，这是我们的第一次对话。

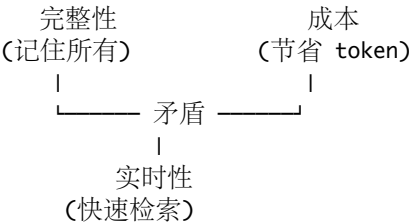
这就是纯 LLM 的局限——没有跨会话记忆。每次对话都是全新的开始。

7.1.2 记忆赋予 Agent 的能力

能力	无记忆	有记忆
跨会话延续	每次重新开始	记住之前的对话
用户偏好	每次重新询问	自动适应
任务延续	无法暂停继续	支持长期项目
知识积累	不积累	持续学习
上下文理解	仅当前对话	理解历史背景

7.1.3 记忆系统的核心矛盾

设计 Agent 记忆系统需要平衡三个矛盾：



- 1. 完整性 vs 成本：记住所有信息需要巨大的上下文窗口，费用高昂
- 2. 完整性 vs 实时性：存储所有信息使检索变慢
- 3. 成本 vs 实时性：压缩信息虽然省钱，但可能丢失重要细节

面试要点：能分析记忆系统的设计权衡，比只描述实现更能打动面试官。

7.2 记忆系统的挑战

7.2.1 LLM 的上下文窗口限制

模型上下文窗口对比：

模型	上下文窗口
GPT-3.5	4K / 16K
GPT-4o	128K
Claude 3.5	200K
DeepSeek-V3	128K
Gemini 1.5 Pro	2M

128K tokens $\approx$ 约 10 万字 $\approx$ 一本短篇小说

看似很大，但实际上：- System Prompt 占用 2000-5000 tokens - 工具定义占用 3000-8000 tokens - 技能摘要占用 1000-3000 tokens - 长期记忆占用 500-2000 tokens - 留给对话历史的空间其实有限

7.2.2 信息遗忘的代价

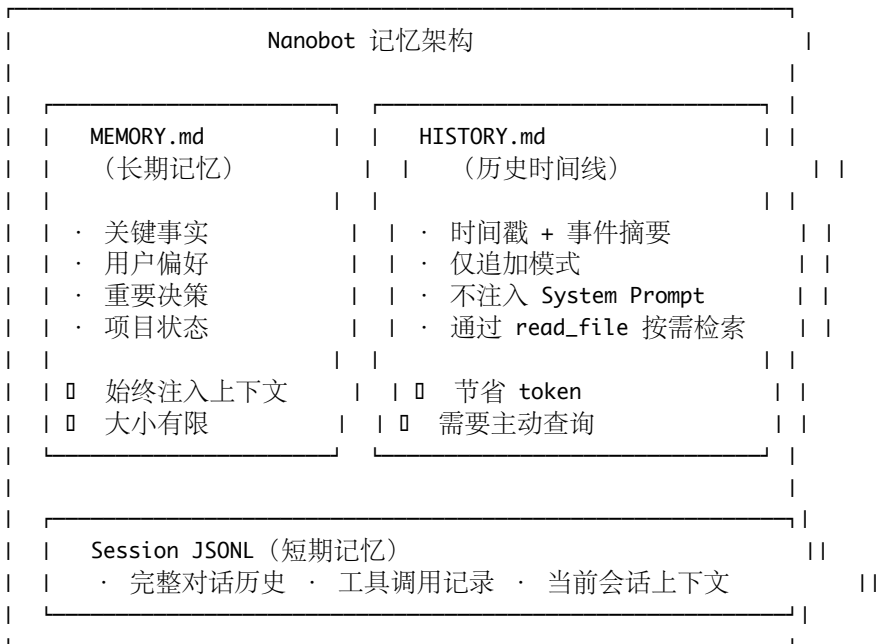
如果简单地“截断”旧对话来适应窗口：- 可能丢失用户的关键偏好 - 可能遗忘重要的任务上下文 - 可能重复询问已知信息（用户体验差）

7.2.3 业界常见方案

方案	优点	缺点
滑动窗口截断	实现简单	丢失早期信息
向量检索 (RAG)	精确检索	需要向量库，架构复杂
摘要压缩	保留关键信息	有信息损失
知识图谱	结构化存储	实现复杂，维护成本高
文件存储 (Nanobot)	简单透明	依赖文件 I/O

7.3 Nanobot 双层记忆架构

Nanobot 采用了一种极简但有效的双层记忆架构：



关键设计决策

设计决策	Nanobot 的选择	理由
存储方式	纯文件 (Markdown)	透明、可编辑、无外部依赖
长期记忆	全量注入	确保 Agent 始终知道关键信息
历史时间线	按需检索	节省 token，历史通常不需要
压缩方式	LLM 驱动摘要	利用 LLM 的理解能力
会话存储	JSONL 格式	追加写入高效，便于流式处理

7.4 MEMORY.md ——长期记忆

7.4.1 存储位置

<workspace>/memory/MEMORY.md

在 config.json 中通过 workspace 字段间接确定位置。

7.4.2 文件内容示例

```
# Memory

## 用户信息
- 用户名: 张三
- 职业: 后端开发工程师, 3 年经验
- 技术栈: Python, FastAPI, PostgreSQL, Redis
- 目标: 准备 AI Agent 方向的面试

## 偏好
- 语言: 简体中文
- 风格: 喜欢看代码示例, 偏好详细解释
- 工作环境: macOS, VS Code

## 项目上下文
- 当前正在学习 Nanobot 框架
- 已完成章节 01-06 的学习
- 重点关注记忆系统和工具系统

## 重要决策
- 2024-03-15: 决定使用 DeepSeek 作为主要 LLM Provider
- 2024-03-16: 确定面试重点为 Agent 架构设计
```

7.4.3 注入方式

MEMORY.md 的内容始终被注入到 System Prompt 中:

```
# 简化的源码逻辑
def build_system_prompt(self):
    parts = []
    parts.append(self.base_instructions)

    # MEMORY.md 始终注入
    memory_path = os.path.join(self.workspace, "memory", "MEMORY.md")
    if os.path.exists(memory_path):
        memory_content = read_file(memory_path)
        parts.append(f"## Long-term Memory\n{memory_content}")

    return "\n".join(parts)
```

7.4.4 为什么用 Markdown 而不是数据库

对比维度	Markdown 文件	数据库/向量库
安装依赖	无 (纯文件)	需要安装配置
可读性	人类直接可读	需要查询工具
可编辑性	任何编辑器	需要专用接口
Agent 理解	天然理解 Markdown	需要转换
透明度	完全透明	黑盒
扩展性	有限	强

对比维度	Markdown 文件	数据库/向量库
检索精度	全文匹配	语义检索

Nanobot 的哲学: “Agent 能读写 Markdown, 用户也能读写 Markdown, 为什么不记记忆就是 Markdown?”

7.4.5 MEMORY.md 的大小控制

MEMORY.md 不是无限增长的, 它通过 MemoryConsolidator 进行动态管理:

- 新信息进入: MemoryConsolidator 将新发现整合到 MEMORY.md
- 旧信息淘汰: 不再相关的信息在重写时被自然省略
- 大小目标: 保持在 500-2000 tokens 之间

7.5 HISTORY.md ——历史时间线

7.5.1 存储位置

<workspace>/memory/HISTORY.md

7.5.2 文件内容示例

History

2024-03-15 14:30

用户自我介绍, 是一名后端工程师, 正在准备 AI Agent 面试。讨论了学习路线。

2024-03-15 16:45

帮助用户安装 Nanobot, 解决了 API 连接超时问题 (改用 DeepSeek Provider)。

2024-03-16 09:00

讲解了 Nanobot 的架构设计, 用户对记忆系统特别感兴趣。创建了记忆系统的学习笔记。

2024-03-16 14:30

模拟面试练习: Agent 架构设计题。用户回答了 3 道题, 在"记忆压缩机制"的描述上需要加强。

7.5.3 核心特性

1. 仅追加模式 (Append-Only)

HISTORY.md 只会在末尾追加新条目, 不会修改或删除已有内容:

简化的源码逻辑

```
def append_history(self, entry: str):
    history_path = os.path.join(self.workspace, "memory", "HISTORY.md")
    with open(history_path, "a") as f:
        f.write(f"\n{entry}\n")
```

2. 不整体注入上下文

与 MEMORY.md 不同, HISTORY.md 不会被注入到 System Prompt:

System Prompt 包含:

- MEMORY.md → 始终注入
- HISTORY.md → 不注入 (节省 token)

3. 通过工具按需检索

当 Agent 需要回忆过去的事件时, 可以通过 read_file 或 grep 工具主动检索:

Agent 内部思考：
"用户问我之前讨论过什么...我需要查看历史记录"

[调用工具: read_file]
[路径: memory/HISTORY.md]

- 获取历史时间线
- 基于时间线回答用户问题

7.5.4 MEMORY.md vs HISTORY.md 对比

对比维度	MEMORY.md	HISTORY.md
性质	长期记忆（语义）	历史时间线（事件）
注入方式	始终注入 System Prompt	不注入，按需检索
更新方式	整体重写（覆盖）	仅追加（Append）
Token 消耗	每轮都消耗	仅检索时消耗
内容类型	关键事实、偏好	时间戳 + 事件摘要
大小控制	MemoryConsolidator 管理	持续增长
类比	大脑的核心认知	日记本

7.6 MemoryConsolidator 压缩机制

MemoryConsolidator 是 Nanobot 记忆系统中最精妙的设计，也是面试高频考点。

7.6.1 触发条件

简化的源码逻辑

```
def should_consolidate(self, messages: list) -> bool:
    total_tokens = estimate_prompt_tokens_chain(messages)
    return total_tokens > self.context_window_tokens
```

当 estimate_prompt_tokens_chain（估算的 Prompt 总 token 数）超过 context_window_tokens（配置的上下文窗口大小）时，触发压缩。

触发公式：
estimate_prompt_tokens_chain(当前对话链) > context_window_tokens

例如：
context_window_tokens = 128000
当前对话 tokens = 130000
→ 130000 > 128000 → 触发压缩！

7.6.2 压缩流程详解

完整的压缩流程：

Step 1: 检测触发

```
estimate_prompt_tokens_chain() > context_window_tokens
|
|
|
```

Step 2: 构造压缩 Prompt

"当前 MEMORY.md 内容" + "待处理的对话历史"

```
|
```

```
    ]
Step 3: 调用 LLM (tool_choice: save_memory)
```

LLM 分析对话，提取关键信息

```
    |
    ]
Step 4: LLM 返回 save_memory 工具调用
```

```
{
  "tool": "save_memory",
  "arguments": {
    "history_entry": "时间戳摘要...",
    "memory_update": "新的完整 MEMORY.md 内容..."
  }
}
```

```
    |
    ]
Step 5: 写入文件
```

HISTORY.md `] 追加 history_entry`
MEMORY.md `] 覆盖为 memory_update`

```
    |
    ]
Step 6: 截断对话历史
```

丢弃已被压缩的旧对话消息

7.6.3 save_memory 虚拟工具

save_memory 是一个虚拟工具——它不在工具注册表中暴露给 Agent 正常使用，仅在记忆压缩时通过 tool_choice 强制 LLM 调用。

save_memory 工具定义

```
save_memory_tool = {
  "type": "function",
  "function": {
    "name": "save_memory",
    "description": "Save consolidated memory",
    "parameters": {
      "type": "object",
      "properties": {
        "history_entry": {
          "type": "string",
          "description": "A timestamped summary for HISTORY.md"
        },
        "memory_update": {
          "type": "string",
          "description": "The complete new content for MEMORY.md"
        }
      },
      "required": ["history_entry", "memory_update"]
    }
  }
}
```

两个参数的职责：

参数	写入目标	写入方式	内容
history_entry	HISTORY.md	追加 (Append)	时间戳 + 对话摘要
memory_update	MEMORY.md	覆盖 (Overwrite)	新的完整 MEMORY.md

7.6.4 压缩 Prompt 的构造

MemoryConsolidator 构造给 LLM 的压缩提示词大致如下：

你是一个记忆管理助手。你需要：

1. 分析当前的记忆内容和新的对话历史
2. 将新的重要信息整合到记忆中
3. 移除不再相关的旧信息
4. 生成一条历史时间线条目

当前 MEMORY.md 内容：

```
---
{current_memory}
---
```

需要处理的对话历史：

```
---
{conversation_to_consolidate}
---
```

请调用 save_memory 工具保存结果。

7.6.5 鲁棒性设计

MemoryConsolidator 的健壮性设计是源码中的一大亮点，也是面试加分项：

1. JSON 归一化

LLM 返回的 JSON 可能格式不规范（多余逗号、缺少引号等），Nanobot 会进行归一化处理：

```
def normalize_json(raw_text: str) -> dict:
    """ 尝试修复和解析不规范的 JSON """
    # 移除 markdown 代码块标记
    text = raw_text.strip()
    if text.startswith("`"):
        text = text.split("\n", 1)[1]
        text = text.rsplit("`", 1)[0]

    # 尝试直接解析
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        # 尝试修复常见问题并重新解析
        text = fix_trailing_commas(text)
        text = fix_single_quotes(text)
        return json.loads(text)
```

2. tool_choice 失败回退

首先使用 tool_choice: {"type": "function", "function": {"name": "save_memory"}} 强制 LLM 调用 save_memory。如果失败（某些模型不支持 tool_choice），则回退到 tool_choice: "auto"：

简化的回退逻辑

```
async def consolidate(self):
```



```

try:
    # 首次尝试：强制调用 save_memory
    result = await self.llm.call(
        messages=consolidation_messages,
        tools=[save_memory_tool],
        tool_choice={"type": "function", "function": {"name": "save_memory"}}
    )
except Exception:
    # 回退：使用 auto 模式
    result = await self.llm.call(
        messages=consolidation_messages,
        tools=[save_memory_tool],
        tool_choice="auto"
    )

```

3. 连续 3 次失败的 Raw Archive

如果 tool_choice 和 auto 模式都连续失败 3 次，启用最后的保底策略——Raw Archive:

```

# 保底策略：直接将对话原文追加到 HISTORY.md
def raw_archive(self, messages: list):
    """ 当所有压缩策略都失败时，直接归档原始对话 """
    raw_text = format_messages_as_text(messages)
    timestamp = datetime.now().isoformat()

    entry = f"## {timestamp} [Raw Archive]\n{raw_text}\n"
    append_to_file("memory/HISTORY.md", entry)

    # 清除已归档的消息
    messages.clear()

```

容错链路:

```

tool_choice: save_memory → 失败
    |
    □
tool_choice: auto → 失败
    |
    □
直接 Raw Archive → 保底成功

```

面试要点：这种三级容错设计体现了工程鲁棒性思维。面试时可以展开讲：“Nanobot 的记忆压缩有三级容错——先用 tool_choice 强制调用，失败后回退到 auto 模式让模型自主选择，如果连续 3 次都失败则直接归档原始对话。这确保了无论什么情况下记忆都不会丢失。”

7.6.6 压缩前后对比例

压缩前——原始对话（约 5000 tokens）:

User: 我叫张三，是后端工程师

Agent: 你好张三! ... (详细自我介绍)

User: 帮我写一个 FastAPI 的 CRUD 接口

Agent: 好的，我来帮你写... (200行代码)

User: 这个接口怎么加上 JWT 认证?

Agent: 我们需要... (100行代码 + 详细解释)

User: 数据库用 PostgreSQL 还是 MySQL?

Agent: 对比分析... (长篇分析)

... 还有 20 轮对话 ...

压缩后——**MEMORY.md** (约 500 tokens):

Memory

用户信息

- 姓名: 张三
- 职业: 后端工程师
- 技术栈偏好: FastAPI, PostgreSQL

项目上下文

- 正在构建一个 REST API 项目
- 已实现 CRUD 接口和 JWT 认证
- 数据库选择了 PostgreSQL

技术偏好

- 喜欢详细的代码示例
- 偏好使用类型注解

压缩后——**HISTORY.md** (新增约 200 tokens):

2024-03-15 14:30-16:00

帮助用户张三(后端工程师)构建 FastAPI REST API 项目。完成了 CRUD 接口、JWT 认证集成。对比分析后选择 PostgreSQL 作为

7.7 短期记忆: Session 会话历史

7.7.1 Session 的概念

Session 是一次“对话会话”的完整记录,存储为 JSONL (JSON Lines) 格式。

<workspace>/sessions/<session_key>.jsonl

session_key 的格式为 {channel}:{chat_id}, 例如: - cli:default.jsonl ——CLI 交互的默认会话 - telegram:123456.jsonl ——Telegram 用户 123456 的会话 - discord:guild_123:channel_456.jsonl ——Discord 特定频道的会话

7.7.2 JSONL 格式

每行是一个完整的 JSON 对象,记录一个对话“回合”(turn):

```
{"role": "user", "content": "你好", "timestamp": "2024-03-15T14:30:00Z"}
{"role": "assistant", "content": "你好! 有什么可以帮你的吗?", "timestamp": "2024-03-15T14:30:05Z", "tool_calls": null}
{"role": "user", "content": "帮我创建一个 hello.py", "timestamp": "2024-03-15T14:31:00Z"}
{"role": "assistant", "content": "好的,我来帮你创建。", "timestamp": "2024-03-15T14:31:03Z", "tool_calls": [{"name": "write_file", "arguments": {"path": "hello.py", "content": "print('Hello, World!')"}]}
{"role": "tool", "content": "File written successfully", "name": "write_file", "timestamp": "2024-03-15T14:31:04Z"}
```

7.7.3 __save_turn() 方法

__save_turn() 是 Session 管理的核心方法,负责将每个对话回合持久化到 JSONL 文件:

简化的源码逻辑

```
def __save_turn(self, message: dict):
    """ 保存一个对话回合到 JSONL 文件 """

    # 1. 清理 reasoning/thinking 标签
    content = message.get("content", "")
    content = self._strip_thinking_tags(content)

    # 2. 处理大图片 (用占位符替换)
```

```

if "media" in message:
    message = self._replace_large_images(message)

# 3. 追加写入 JSONL
session_path = self._get_session_path()
with open(session_path, "a") as f:
    f.write(json.dumps(message, ensure_ascii=False) + "\n")

```

7.7.4 关键处理细节

1. 清理 reasoning/thinking 标签

某些模型（如 DeepSeek）会在回复中包含 `<thinking>...</thinking>` 内部推理标签。这些标签不应出现在持久化的会话记录中：

```

def _strip_thinking_tags(self, content: str) -> str:
    """ 移除 LLM 内部推理标签 """
    import re
    # 移除 <thinking>...</thinking>
    content = re.sub(r'<thinking>.*?</thinking>', '', content, flags=re.DOTALL)
    # 移除 <reasoning>...</reasoning>
    content = re.sub(r'<reasoning>.*?</reasoning>', '', content, flags=re.DOTALL)
    return content.strip()

```

2. 大图片占位符替换

如果对话中包含大图片（base64 编码），直接存储会导致 JSONL 文件膨胀。Nanobot 用占位符替换：

```

def _replace_large_images(self, message: dict) -> dict:
    """ 将大图片替换为占位符 """
    MAX_IMAGE_SIZE = 1024 * 100 # 100KB

    if "media" in message:
        for i, media_item in enumerate(message["media"]):
            if len(media_item.get("data", "")) > MAX_IMAGE_SIZE:
                message["media"][i] = {
                    "type": "image",
                    "placeholder": "[Large image omitted]",
                    "original_size": len(media_item["data"])
                }
    return message

```

7.7.5 Session 加载与恢复

当 Agent 重新启动时，会从 JSONL 文件恢复会话历史：

```

def load_session(self, session_key: str) -> list:
    """ 从 JSONL 文件加载会话历史 """
    session_path = self._get_session_path(session_key)
    messages = []

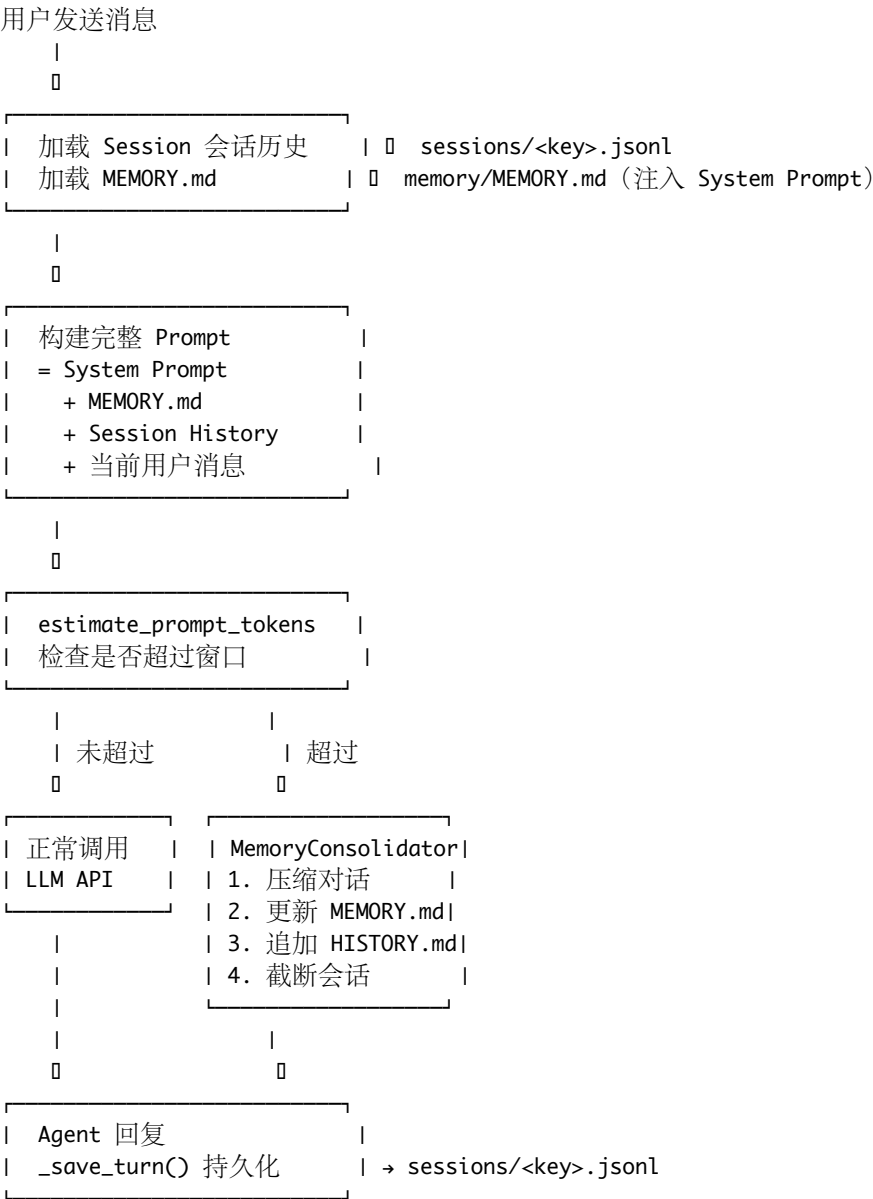
    if os.path.exists(session_path):
        with open(session_path, "r") as f:
            for line in f:
                line = line.strip()
                if line:
                    messages.append(json.loads(line))

    return messages

```

7.8 记忆系统完整数据流

将三层记忆整合起来，完整数据流如下：



7.9 记忆系统与其他框架对比

7.9.1 主流方案对比

框架/方案	存储方式	检索方式	压缩方式	透明度	复杂度
Nanobot	Markdown 文件	全文注入 + 文件检索	LLM 摘要	高	低
LangChain	向量数据库	向量相似度	无/手动	低	中
AutoGPT	JSON 文件	向量检索	无	中	中
MemGPT	虚拟分页	分页检索	LLM 编辑	中	高
CrewAI	共享内存	直接访问	无	低	低

7.9.2 向量库 vs 文件存储 vs 知识图谱

向量库方案（如 **LangChain + Chroma/Pinecone**）

优点：

- 语义检索精确（"找关于数据库的讨论" 即使没有"数据库"关键词也能找到）
- 适合大量非结构化文档
- 检索速度快（ $O(\log n)$ ）

缺点：

- 需要外部依赖（向量数据库）
- 向量化过程有信息损失
- 更新不直观（需要重新 `embedding`）
- 调试困难（向量不可读）

文件存储方案（**Nanobot**）

优点：

- 零外部依赖
- 完全透明（人类可读可编辑）
- **Agent** 原理解（`Markdown`）
- 版本控制友好（`git` 可追踪）
- 实现简单

缺点：

- 不支持语义检索（只能关键词匹配）
- 全文注入消耗 `token`
- 不适合超大规模记忆

知识图谱方案（如 **Neo4j + LLM**）

优点：

- 结构化存储关系
- 复杂推理能力（多跳查询）
- 知识去重

缺点：

- 实现极其复杂
- 需要实体抽取和关系建模
- 维护成本高
- 不适合轻量级 **Agent**

7.9.3 Nanobot 方案的适用场景

适合 **Nanobot** 文件记忆的场景：

- 个人助手（单用户，记忆量小）
- 轻量级 **Agent**（快速原型）
- 开发学习（透明可调试）
- 记忆内容结构化（事实列表）

不太适合的场景：

- 海量文档检索（**1000+** 页）
- 多用户共享知识库
- 需要复杂关系推理
- 实时高并发场景

7.10 实战练习

练习 1：观察记忆压缩过程

1. 创建测试 workspace

```
mkdir memory-test && cd memory-test
```

```
nanobot onboard
```

2. 设置较小的 context_window_tokens 以便快速触发压缩

编辑 config.json, 将 context_window_tokens 改为 4000

修改 config.json:

```
{
  "agents": {
    "defaults": {
      "context_window_tokens": 4000
    }
  }
}
```

3. 启动 Agent 并进行大量对话

```
nanobot
```

对话示例 (快速填充上下文):

You: 写一篇 500 字的关于 Python 的文章

You: 再写一篇关于 JavaScript 的

You: 比较一下两种语言的优缺点

4. 观察压缩触发

当 token 数超过 4000 时, Agent 会触发记忆压缩

你会看到 MEMORY.md 和 HISTORY.md 被更新

5. 检查结果

```
cat memory/MEMORY.md
```

```
cat memory/HISTORY.md
```

练习 2：手动编辑长期记忆

你可以直接编辑 MEMORY.md 来"植入"记忆

```
cat > memory/MEMORY.md << 'EOF'
```

```
# Memory
```

```
## 用户信息
```

- 我是一个资深的 Kubernetes 管理员
- 我管理着 50 个集群
- 我偏好使用 Helm Charts

```
## 项目
```

- 正在迁移微服务到 Service Mesh (Istio)

```
EOF
```

重新启动 Agent

```
nanobot
```

You: 给我一些关于我当前项目的建议

Agent 应该知道你在做 Istio 迁移

练习 3：分析 Session JSONL

查看会话记录

```
cat sessions/cli:default.jsonl | python -m json.tool --no-ensure-ascii
```

统计对话轮次

```
wc -l sessions/cli:default.jsonl
```

搜索特定内容

```
grep "write_file" sessions/cli:default.jsonl
```

7.11 面试高频题

题目 1：Nanobot 的记忆系统是如何设计的？

参考回答：

“Nanobot 采用了双层文件记忆 + 会话历史的三级记忆架构。

第一层是 **MEMORY.md** 长期记忆，存储在 workspace/memory/MEMORY.md，包含用户的关键事实、偏好、重要决策等。它的特点是始终注入 **System Prompt**，确保 Agent 每次对话都能访问这些核心信息。

第二层是 **HISTORY.md** 历史时间线，采用仅追加模式记录每次对话的时间戳摘要。它不会被注入上下文，只在需要时通过 read_file 工具按需检索，这样可以节省大量 token。

第三层是 **Session JSONL** 会话历史，以 JSON Lines 格式记录完整的对话内容，包括用户消息、Agent 回复和工具调用。

这三层之间通过 **MemoryConsolidator** 连接：当会话 token 数超过 context_window_tokens 时，MemoryConsolidator 调用 LLM 将对话压缩为新的 MEMORY.md 内容和 HISTORY.md 条目，然后截断旧的会话历史。

这种设计的核心优势是零外部依赖——不需要向量数据库，纯文件存储，用户和 Agent 都可以直接读写，完全透明。”

题目 2：如何处理上下文窗口溢出？

参考回答：

“Nanobot 通过 **MemoryConsolidator** 处理上下文溢出，核心流程是：

1. 检测：每次构建 Prompt 前，用 estimate_prompt_tokens_chain 估算总 token 数
2. 压缩：超过阈值时，将‘当前 MEMORY.md + 待压缩对话’发给 LLM，通过 tool_choice 强制调用 save_memory 虚拟工具
3. save_memory 返回两个结果：history_entry（追加到 HISTORY.md）和 memory_update（覆盖 MEMORY.md）
4. 截断：丢弃已压缩的旧对话消息

容错方面有三级保障：先用 tool_choice 强制调用，失败后回退 auto 模式，连续 3 次失败则直接 Raw Archive 原始对话。这确保了无论 LLM 行为如何，记忆都不会丢失。”

题目 3：长期记忆与短期记忆的区别？

参考回答：

“在 Nanobot 中：

长期记忆（**MEMORY.md**）是经过 LLM 提炼的核心信息，类似人的‘核心认知’——你知道自己叫什么名字、在哪里工作。它每轮对话都会被注入 System Prompt，所以 Agent 始终‘知道’这些信息，但代价是消耗 token。它通过 MemoryConsolidator 整体重写更新，旧的不重要信息在重写时自然被淘汰。

短期记忆（**Session JSONL**）是当前会话的完整对话记录，类似人的‘工作记忆’。它包含所有细节但生命周期有限——当 token 超限时，旧的对话会被压缩到长期记忆后截断。

两者的桥梁是 MemoryConsolidator：它把短期记忆中的关键信息‘晋升’到长期记忆，把细节‘沉淀’到 HISTORY.md 时间线。”

题目 4：为什么不用向量数据库？

参考回答：

“Nanobot 选择文件存储而非向量数据库，是一个有意识的权衡：

1. 零依赖：不需要安装 Chroma/Pinecone/Weaviate，降低部署门槛
2. 透明度：Markdown 文件人人可读，用户可以直接编辑记忆，这是向量库做不到的
3. **Agent** 原生理解：LLM 天然理解 Markdown，不需要向量化/反向量化
4. 版本控制：可以用 git 追踪记忆变化历史

当然这也有局限性——不支持语义检索，不适合大规模知识库。但对于 Nanobot 定位的个人助手场景，文件存储完全够用且更优雅。

如果要我改进，我会在保持文件存储的基础上增加一个轻量级的索引层，比如用 TF-IDF 对 HISTORY.md 建索引，在保持简单性的同时提升检索能力。”

题目 5：如果让你改进 Nanobot 的记忆系统，你会怎么做？

参考回答：

“我会从三个方向考虑改进：

1. 增加语义检索层：在保持 Markdown 文件存储的基础上，用轻量级的 embedding 模型（如 sentence-transformers）对 HISTORY.md 的条目建向量索引。这样 Agent 可以按语义检索历史，而不仅仅是关键词匹配。
2. 记忆分级策略：参考操作系统的多级缓存设计——L1 是 MEMORY.md（热数据，始终注入），L2 是最近的 HISTORY.md 条目（温数据，按需加载），L3 是归档历史（冷数据，仅精确查询时检索）。
3. 记忆质量评估：在 MemoryConsolidator 中加入一个验证步骤——压缩后让 LLM 对比原始对话和压缩结果，检查是否遗漏了关键信息。如果遗漏严重则重新压缩。”

7.12 本章小结

核心知识点

Nanobot 记忆系统 = MEMORY.md + HISTORY.md + Session JSONL

MEMORY.md（长期记忆）

- └─ 始终注入 System Prompt
- └─ MemoryConsolidator 整体重写
- └─ 关键事实、偏好、状态

HISTORY.md（历史时间线）

- └─ 仅追加模式
- └─ 不注入（节省 token）
- └─ 通过 read_file 按需检索

Session JSONL（短期记忆）

- └─ 完整对话记录
- └─ _save_turn() 持久化
- └─ 清理 thinking 标签 + 图片占位符

MemoryConsolidator（压缩器）

- └─ 触发：token 超过 context_window_tokens
- └─ 流程：构造 Prompt → save_memory 工具调用

— 容错: tool_choice → auto → Raw Archive

面试记忆清单

考点	一句话回答
记忆架构	双层文件记忆 + 会话历史的三级架构
MEMORY.md	长期记忆，始终注入 System Prompt
HISTORY.md	历史时间线，仅追加，按需检索
压缩触发	token 数超过 context_window_tokens
压缩工具	save_memory 虚拟工具 (history_entry + memory_update)
容错机制	tool_choice → auto → Raw Archive 三级回退
为什么用文件	零依赖、透明可编辑、Agent 原理解

下一章: 08 - 技能与工具 ——深入理解 Nanobot 的 Skill 系统和工具链

7.13 Dream 记忆系统

7.13.1 Dream 是什么

Dream 是 Nanobot v0.2.x 引入的智能记忆摘要系统。它会在后台自动处理未读的会话历史，生成结构化摘要并更新到记忆文件中。

核心价值: - 自动从 history.jsonl 中提取重要信息 - 使用 LLM 生成智能摘要 - 无需用户手动调用 save_memory - 跨会话的长期记忆自动累积

7.13.2 Dream 工作机制

核心组件 (源码: nanobot/agent/memory.py):

```
# Dream 游标管理
self._dream_cursor_file = self.memory_dir / ".dream_cursor"

# 获取上次处理位置
last_cursor = self.get_last_dream_cursor() # 读取 .dream_cursor

# 读取未处理的会话历史
entries = self.read_unprocessed_history(since_cursor=last_cursor)

# 生成 Dream prompt
prompt = build_dream_prompt() # 包含历史摘要 + skill-creator

# 执行 Dream run (使用受限工具集)
tools = build_dream_tools() # 仅允许文件操作工具
```

工作流程:

1. Gateway 启动时注册 Dream 系统任务
↓
2. 定期触发 Dream run (基于 heartbeat 或 cron)
↓
3. 读取 .dream_cursor 获取上次处理位置
↓
4. 从 history.jsonl 读取未处理条目
↓
5. 构造 Dream prompt (历史摘要 + skill-creator SKILL.md)

- ↓
6. LLM 生成智能摘要
 - ↓
 7. 调用 `save_memory` 工具更新 `MEMORY.md`
 - ↓
 8. 更新 `.dream_cursor` 游标
 - ↓
 9. 过滤内部会话（`cron:`、`dream:` 前缀）

7.13.3 Dream 与 MEMORY.md 的关系

特性	Dream	手动 <code>save_memory</code>
触发方式	自动（后台任务）	手动（Agent 调用工具）
处理内容	<code>history.jsonl</code> 未读条目	Agent 认为重要的信息
摘要质量	LLM 智能提取	Agent 主观判断
更新频率	定期	按需
游标管理	<code>.dream_cursor</code>	无

协同工作：- Dream 自动累积长期记忆 - 手动 `save_memory` 补充关键信息 - 两者都写入 `MEMORY.md` - Dream 不覆盖手动保存的内容

7.13.4 GitStore 版本控制

Dream 系统集成了 GitStore 版本控制：

```
# 源码: nanobot/agent/memory.py 第 67-69 行
self._git = GitStore(workspace, tracked_files=[
    "SOUL.md", "USER.md", "memory/MEMORY.md", "memory/.dream_cursor",
])
```

自动 `git commit` 的文件：- `SOUL.md` - 全局人格 - `USER.md` - 用户画像 - `memory/MEMORY.md` - 长期记忆 - `memory/.dream_cursor` - Dream 游标

价值：- 记忆变更可追溯 - 支持版本回退 - 便于调试和审计

7.13.5 配置参数

Dream 系统受以下配置影响：

```
{
  "memory": {
    "consolidation_threshold": 50
  },
  "gateway": {
    "heartbeat": {
      "enabled": true,
      "intervals": 1800
    }
  }
}
```

- `consolidation_threshold`: 触发记忆压缩的 token 阈值
- `heartbeat.intervals`: Dream 任务执行间隔（默认 1800 秒 = 30 分钟）

7.13.6 实战：查看 Dream 状态

```
# 查看 Dream 游标位置
cat workspace/memory/.dream_cursor
```

```
# 查看最近的会话历史
tail -20 workspace/memory/history.jsonl | python -m json.tool

# 查看生成的长期记忆
cat workspace/memory/MEMORY.md
```

7.13.7 面试话术

问题：Nanobot 的 Dream 记忆系统是如何工作的？

“Dream 是 Nanobot 的智能记忆摘要系统。它通过 `.dream_cursor` 游标跟踪已处理的会话历史，定期从 `history.jsonl` 中读取未处理条目，使用 LLM 生成智能摘要并更新到 `MEMORY.md`。Dream 使用受限的工具集（仅文件操作），并集成了 GitStore 版本控制，自动 `commit` 记忆文件变更。相比手动 `save_memory`，Dream 的优势是自动化和全面性，确保重要信息不会遗漏。”

扩展阅读：18 - WebUI 实战

08 - 技能与工具

阅读时间：约 2 小时
 前置知识：07 - 记忆系统实战
 学习目标：掌握 Nanobot 的 Skill 系统、内置工具体系、MCP 工具集成，能够自定义 Skill

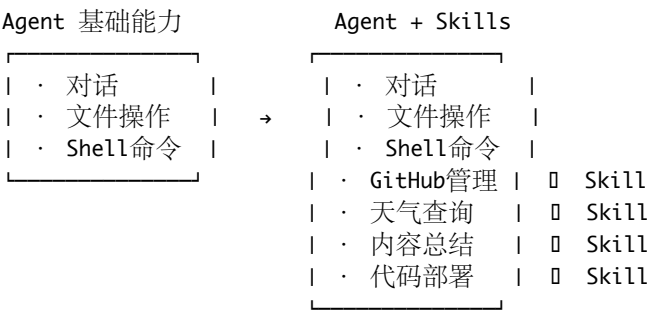
目录

- 8.1 Skills 系统概述
- 8.2 SKILL.md 格式规范
- 8.3 YAML Frontmatter 字段详解
- 8.4 技能发现机制
- 8.5 渐进披露 (Progressive Disclosure)
- 8.6 内置技能列表
- 8.7 内置工具完整列表
- 8.8 ToolRegistry 统一注册与执行机制
- 8.9 MCP 工具集成
- 8.10 自定义 Skill 编写实战
- 8.11 工具安全机制
- 8.12 面试高频题
- 8.13 本章小结

8.1 Skills 系统概述

8.1.1 什么是 Skill?

在 Nanobot 中，Skill（技能）是一种可插拔的能力扩展机制。你可以把它理解为”给 Agent 添加的专业模块”：



8.1.2 Skill vs Tool 的区别

这是面试中容易混淆的概念：

维度	Skill（技能）	Tool（工具）
粒度	粗粒度（一个完整能力）	细粒度（一个具体操作）
定义方式	Markdown 文件（SKILL.md）	Python 代码注册
包含内容	指令 + 脚本 + 参考资料	函数签名 + 执行逻辑
注入方式	System Prompt	Tool Definition
类比	一门专业课程	一个具体工具

关系：一个 Skill 可以教会 Agent 如何更好地使用多个 Tool。

Skill: GitHub 管理

└─ 知道如何创建 PR（指令）

- └─ 知道代码审查流程（知识）
 - └─ 会使用以下 Tools:
 - └─ exec（运行 git 命令）
 - └─ read_file（读取代码）
 - └─ web_fetch（查看 GitHub API）
-

8.2 SKILL.md 格式规范

8.2.1 目录结构

每个 Skill 是一个独立的目录：

```
skill-name/  
└─ SKILL.md          # 技能定义文件（必需）  
└─ scripts/         # 可选：辅助脚本  
    └─ deploy.sh  
    └─ check.py  
└─ references/      # 可选：参考资料  
    └─ api-docs.md  
    └─ examples.md  
└─ assets/          # 可选：资源文件  
    └─ template.json  
    └─ config.yaml
```

8.2.2 SKILL.md 文件格式

SKILL.md 由两部分组成：**YAML Frontmatter** + **Markdown** 正文。

```
---  
name: github  
description: "GitHub 仓库管理、PR 创建与代码审查"  
always: false  
metadata: '{"nanobot.requires.bins": ["git", "gh"], "nanobot.requires.env": ["GITHUB_TOKEN"]}'  
---
```

GitHub 管理技能

能力

你可以帮助用户管理 GitHub 仓库，包括：

1. **** 创建和管理 Pull Request ****
 - 创建 PR 并添加描述
 - 审查代码变更
 - 合并 PR
2. **** Issue 管理 ****
 - 创建 Issue
 - 标签分类
 - 分配负责人

使用步骤

创建 PR

1. 先用 `exec` 运行 `git status` 检查当前状态
2. 确认所有变更已提交

3. 使用 ``exec`` 运行 ``gh pr create --title " 标题" --body " 描述"``

代码审查

1. 用 ``exec`` 运行 ``gh pr diff <PR 号>``
2. 用 ``read_file`` 查看关键变更文件
3. 给出审查意见

注意事项

- 创建 PR 前确保分支已推送到远程
- 大型 PR 建议拆分为多个小 PR
- 代码审查时关注安全和性能问题

参考资料

更多细节请查看 ``references/`` 目录中的文档。

8.3 YAML Frontmatter 字段详解

8.3.1 必填字段

字段	类型	说明	示例
name	string	技能的唯一标识符	"github"
description	string	技能的简短描述（用于摘要展示）	"GitHub 仓库管理"

8.3.2 可选字段

字段	类型	默认值	说明
always	bool	false	是否每轮对话都全文注入 System Prompt
metadata	string (JSON)	null	技能的元数据，包括依赖检查

8.3.3 always 字段详解

`always: true` → 每轮对话都将 `SKILL.md` 全文注入 System Prompt

适用于: Agent 核心能力，必须时刻知道的技能

`always: true`

`always: false` (默认) → 仅注入摘要，需要时 Agent 主动 `read_file` 查看全文

适用于: 偶尔使用的技能，节省 token

`always: false`

设计权衡:

`always: true`

├─ 优点: Agent 始终知道完整技能内容

├─ 缺点: 消耗更多 token

└─ 适用: 核心技能 (1-2个)

`always: false`

├─ 优点: 节省 token (仅注入一行摘要)

├─ 缺点: Agent 需要额外步骤读取全文

└─ 适用: 大多数技能

8.3.4 metadata 字段详解

metadata 是一个 JSON 字符串，用于声明技能的依赖：

```
metadata: '{"nanobot.requires.bins": ["git", "gh"], "nanobot.requires.env": ["GITHUB_TOKEN"]}'
```

元数据 Key	说明	示例
nanobot.requires.bins	需要的系统命令	["git", "gh", "docker"]
nanobot.requires.env	需要的环境变量	["GITHUB_TOKEN", "AWS_KEY"]

依赖检查流程：

技能加载时：

1. 检查 nanobot.requires.bins
 - 遍历列表，对每个 bin 执行 `which <bin>`
 - 如果命令不存在，打印警告但不阻止加载
2. 检查 nanobot.requires.env
 - 遍历列表，检查 `os.environ.get(<env>)`
 - 如果环境变量未设置，打印警告

注意：依赖缺失不会阻止技能加载，只会打印警告。
技能可能在运行时因依赖缺失而失败，此时 Agent 会收到错误信息并处理。

8.4 技能发现机制

8.4.1 搜索路径

Nanobot 在两个位置搜索技能：

搜索顺序（优先级从高到低）：

1. <workspace>/skills/ □ 用户自定义技能（优先）
2. nanobot/skills/ □ 内置技能（包内预置）

同名技能：workspace 覆盖内置

8.4.2 发现流程

简化的技能发现逻辑

```
def discover_skills(workspace: str) -> dict:
    skills = {}

    # 1. 先加载内置技能
    builtin_skills_dir = os.path.join(NANOBOT_PACKAGE_DIR, "skills")
    for skill_dir in os.listdir(builtin_skills_dir):
        skill = load_skill(os.path.join(builtin_skills_dir, skill_dir))
        if skill:
            skills[skill.name] = skill

    # 2. 再加载 workspace 技能（同名覆盖内置）
    workspace_skills_dir = os.path.join(workspace, "skills")
    if os.path.exists(workspace_skills_dir):
        for skill_dir in os.listdir(workspace_skills_dir):
            skill = load_skill(os.path.join(workspace_skills_dir, skill_dir))
            if skill:
```

```
skills[skill.name] = skill # 覆盖同名内置技能
```

```
return skills
```

8.4.3 覆盖机制

这个设计允许用户定制内置技能：

内置 github 技能：

nanobot/skills/github/SKILL.md → "使用 gh 命令管理 GitHub"

用户自定义覆盖：

workspace/skills/github/SKILL.md → "使用 GitHub API 管理，需要审批流程"

最终生效：用户自定义版本

面试要点：这种“约定优于配置 + 同名覆盖”的设计模式，在很多框架中都能看到（如 Maven 的 Convention over Configuration，Webpack 的 resolve 策略）。

8.5 渐进披露 (Progressive Disclosure)

8.5.1 为什么需要渐进披露

如果有 20 个技能，每个 SKILL.md 有 500 行，全部注入 System Prompt 就是 10000 行——这会：

1. 消耗大量 token（费钱）
2. 分散 Agent 注意力（效果差）
3. 可能超出上下文窗口（直接报错）

8.5.2 三层渐进披露设计

Tier 1: 摘要目录（始终注入）

所有技能的 name + description 组成的摘要列表

消耗：约 100-500 tokens

示例注入内容：

"你拥有以下技能：

- github: GitHub 仓库管理、PR 创建与代码审查
- weather: 查询全球天气信息
- summarize: 长文本智能摘要
- tmux: 终端多窗口管理

如需使用某个技能，请先读取对应的 SKILL.md 了解详情。"

```
|
| 当 Agent 判断需要使用某个技能时
|
```

Tier 2: 读取 SKILL.md 全文（按需加载）

Agent 主动调用 read_file 读取完整 SKILL.md

消耗：按需，约 200-1000 tokens

Agent 内部思考：

"用户想管理 GitHub PR，我需要查看 github 技能的详细说明"

[调用工具: read_file]

[路径: skills/github/SKILL.md]


```
|
| 如果需要更详细的参考资料或脚本
|
```

Tier 3: 访问 `scripts/references` (深度使用)

Agent 读取辅助脚本或参考文档
消耗: 按需

[调用工具: `read_file`]
[路径: `skills/github/references/pr-workflow.md`]
[调用工具: `exec`]
[命令: `bash skills/github/scripts/create-pr.sh`]

8.5.3 `always: true` 的特殊处理

标记为 `always: true` 的技能跳过渐进披露，直接全文注入：

技能注入逻辑：

```
|
| 遍历所有技能
|
| — always: true → 全文注入 System Prompt
|
| — always: false → 仅注入 name + description 摘要
```

面试加分：渐进披露是 UI/UX 设计中的经典原则——先展示概要，让用户按需深入。Nanobot 将这个原则应用到了 Agent Prompt 设计中，是一种优雅的 token 管理策略。

8.6 内置技能列表

Nanobot 预置了以下技能：

技能名	功能	always	依赖
github	GitHub 仓库管理、PR、Issue	false	git, gh, GITHUB_TOKEN
weather	全球天气查询	false	无
summarize	长文本智能摘要	false	无
tmux	终端多窗口管理	false	tmux
clawhub	ClawHub 平台集成	false	无
skill-creator	帮助用户创建新技能	false	无

各技能详解

github 技能：

能力：

- 克隆/创建/管理仓库
- 创建/审查/合并 Pull Request
- 管理 Issue 和 Labels
- 查看 Actions 运行状态

依赖：

- git CLI 工具

- GitHub CLI (gh)
- GITHUB_TOKEN 环境变量

weather 技能:

能力:

- 查询全球城市天气
- 天气预报
- 温度/湿度/风速等详细信息

实现方式:

- 通过 `web_search` 或 `web_fetch` 查询天气 API

summarize 技能:

能力:

- 长文档摘要
- 网页内容提取与总结
- 多文档对比摘要

使用场景:

- 用户提供长篇文章需要总结
- 需要从多个来源提取关键信息

skill-creator 技能:

能力:

- 引导用户创建新的自定义技能
- 生成 `SKILL.md` 模板
- 检查技能目录结构

这是一个"元技能"——用来创建其他技能的技能。

8.7 内置工具完整列表

8.7.1 工具总览表

类别	工具名	功能	关键特性
文件	<code>read_file</code>	读取文件内容	支持二进制文件 base64、支持行范围
文件	<code>write_file</code>	创建/覆盖文件	自动创建父目录
文件	<code>edit_file</code>	编辑文件局部内容	基于搜索替换, 比 <code>write_file</code> 精确
文件	<code>list_dir</code>	列出目录内容	支持递归、支持过滤
Shell	<code>exec</code>	执行 Shell 命令	<code>asyncio</code> 异步执行、危险命令拒绝
Web	<code>web_search</code>	网络搜索	支持 Brave/DDG/Tavily 等引擎
Web	<code>web_fetch</code>	获取网页内容	HTML 转纯文本
通信	<code>message</code>	发送消息给用户	唯一可携带 <code>media</code> 的工具
调度	<code>cron</code>	定时任务管理	<code>add/list/remove</code> 三种操作
并发	<code>spawn</code>	启动后台子代理	<code>SubagentManager</code> 管理

8.7.2 文件操作工具

`read_file` —— 读取文件

```
{
  "name": "read_file",
  "parameters": {
    "path": "string (必需) - 文件路径",
```

```

    "start_line": "int (可选) - 起始行号",
    "end_line": "int (可选) - 结束行号"
}
}

```

使用示例:

Agent: [调用 read_file]

参数: {"path": "src/main.py"}

返回: "import os\nimport sys\n\ndef main():\n print('Hello')\n..."

write_file —— 写入文件

```

{
    "name": "write_file",
    "parameters": {
        "path": "string (必需) - 文件路径",
        "content": "string (必需) - 文件内容"
    }
}

```

关键行为: - 如果文件不存在, 自动创建 (包括父目录) - 如果文件已存在, 完全覆盖 - 受 `restrict_to_workspace` 限制

edit_file —— 编辑文件

```

{
    "name": "edit_file",
    "parameters": {
        "path": "string (必需) - 文件路径",
        "old_text": "string (必需) - 要替换的原文本",
        "new_text": "string (必需) - 替换后的新文本"
    }
}

```

关键行为: - 基于精确字符串匹配定位修改位置 - 比 `write_file` 更安全 (不会意外覆盖整个文件) - 如果 `old_text` 找不到匹配, 返回错误

list_dir —— 列出目录

```

{
    "name": "list_dir",
    "parameters": {
        "path": "string (必需) - 目录路径",
        "recursive": "bool (可选) - 是否递归",
        "pattern": "string (可选) - 过滤模式"
    }
}

```

8.7.3 Shell 执行工具

exec —— 执行命令

```

{
    "name": "exec",
    "parameters": {
        "command": "string (必需) - Shell 命令"
    }
}

```

核心实现特性:

简化的 `exec` 工具实现

```
async def exec_tool(command: str, workspace: str) -> str:
```

```

# 1. 危险命令检测
if is_dangerous_command(command):
    return "Error: This command is potentially dangerous and has been blocked."

# 2. 使用 asyncio 异步执行
process = await asyncio.create_subprocess_shell(
    command,
    stdout=asyncio.subprocess.PIPE,
    stderr=asyncio.subprocess.PIPE,
    cwd=workspace # 在 workspace 目录下执行
)

stdout, stderr = await process.communicate()

# 3. 返回结果
return f"Exit code: {process.returncode}\n{stdout.decode()}\n{stderr.decode()}"

```

危险命令拒绝机制:

```

DANGEROUS_PATTERNS = [
    r"rm\s+--rf\s+/",          # 删除根目录
    r"mkfs\.",                # 格式化磁盘
    r"dd\s+if=",              # 磁盘写入
    r">\s*/dev/sd",            # 写入磁盘设备
    r"chmod\s+--R\s+777\s+/", # 全局权限修改
]

def is_dangerous_command(command: str) -> bool:
    for pattern in DANGEROUS_PATTERNS:
        if re.search(pattern, command):
            return True
    return False

```

SSRF 防护:

exec 工具中如果涉及网络请求, 会检查目标地址是否为内网:

```

BLOCKED_HOSTS = [
    "127.0.0.1", "localhost",
    "169.254.169.254", # AWS 元数据服务
    "10.0.0.0/8",      # 内网 A 类
    "172.16.0.0/12",   # 内网 B 类
    "192.168.0.0/16",  # 内网 C 类
]

```

8.7.4 Web 工具

web_search ——网络搜索

```

{
  "name": "web_search",
  "parameters": {
    "query": "string (必需) - 搜索关键词",
    "num_results": "int (可选) - 返回结果数量"
  }
}

```

支持多个搜索引擎:

引擎	说明	需要 API Key
Brave Search	隐私搜索引擎	是
DuckDuckGo	免费搜索	否
Tavily	AI 优化的搜索	是
SearXNG	自托管搜索聚合	否

配置方式：

```
{
  "tools": {
    "web_search": {
      "provider": "brave",
      "api_key": "BSAxxxxxxx"
    }
  }
}
```

web_fetch —— 获取网页

```
{
  "name": "web_fetch",
  "parameters": {
    "url": "string (必需) - 网页 URL"
  }
}
```

核心功能：将 HTML 页面转换为纯文本，方便 Agent 阅读。

输入："https://example.com/article"

处理流程：

1. HTTP GET 请求获取 HTML
2. 解析 HTML，提取正文
3. 移除脚本、样式、导航等无关元素
4. 转换为清晰的纯文本格式
5. SSRF 防护：拒绝内网地址

输出："文章标题\n\n文章正文内容..."

8.7.5 通信工具

message —— 发送消息

```
{
  "name": "message",
  "parameters": {
    "content": "string (必需) - 消息内容",
    "media": "array (可选) - 媒体附件"
  }
}
```

关键特性：- 这是 Agent 主动向用户发送消息的唯一方式 - 对应 `OutboundMessage` 数据结构 - 唯一可以携带 **media** (图片/文件) 的工具

`OutboundMessage` 数据结构

@dataclass

class `OutboundMessage`:

 channel: str # 目标通道

 chat_id: str # 目标会话

```

content: str      # 文本内容
reply_to: str     # 回复的消息 ID (可选)
metadata: dict    # 元数据
media: list       # 媒体附件列表 (图片、文件等)

```

8.7.6 调度工具

cron —— 定时任务

```

{
  "name": "cron",
  "parameters": {
    "action": "string (必需) - add/list/remove",
    "name": "string (action=add/remove 时必需) - 任务名称",
    "schedule": "object (action=add 时必需) - 调度配置",
    "message": "string (action=add 时必需) - 触发时发送的消息"
  }
}

```

详见 10 - 子 Agent 与定时任务。

8.7.7 并发工具

spawn —— 启动子代理

```

{
  "name": "spawn",
  "parameters": {
    "task": "string (必需) - 子代理的任务描述"
  }
}

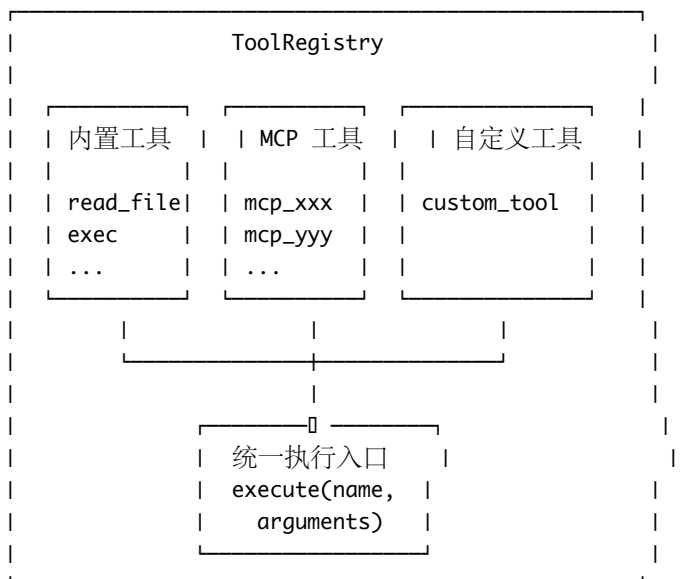
```

详见 10 - 子 Agent 与定时任务。

8.8 ToolRegistry 统一注册与执行机制

8.8.1 ToolRegistry 架构

ToolRegistry 是 Nanobot 工具系统的核心枢纽，负责统一管理所有工具的注册、发现和执行：



8.8.2 注册流程

简化的 ToolRegistry 实现

```
class ToolRegistry:
    def __init__(self):
        self._tools: dict[str, ToolDefinition] = {}

    def register(self, name: str, definition: dict, handler: Callable):
        """ 注册一个工具 """
        self._tools[name] = ToolDefinition(
            name=name,
            definition=definition, # JSON Schema 格式的工具定义
            handler=handler        # 实际执行函数
        )

    def get_definitions(self) -> list:
        """ 获取所有工具定义（用于发送给 LLM） """
        return [
            {
                "type": "function",
                "function": tool.definition
            }
            for tool in self._tools.values()
        ]

    async def execute(self, name: str, arguments: dict) -> str:
        """ 执行一个工具调用 """
        if name not in self._tools:
            return f"Error: Unknown tool '{name}'"

        tool = self._tools[name]
        try:
            result = await tool.handler(**arguments)
            return str(result)
        except Exception as e:
            return f"Error: {str(e)}"
```

8.8.3 执行流程

LLM 返回工具调用请求

|
□

```
| 解析 tool_calls |
| name: "read_file" |
| arguments: {"path": |
|   "src/main.py"} |
```

|
□

```
| ToolRegistry.execute |
| 1. 查找注册的 handler |
| 2. 验证参数 |
| 3. 执行 handler |
| 4. 返回结果 |
```

```

|
|
| 将结果作为 tool      |
| message 返回给 LLM  |
| role: "tool"         |
| content: "文件内容..." |
|
|
|
LLM 基于工具结果继续推理

```

8.8.4 工具定义的 JSON Schema 格式

所有工具都遵循 OpenAI 的 Function Calling 格式：

```

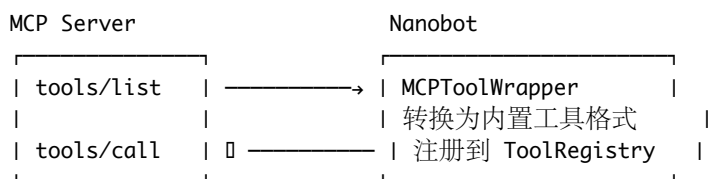
{
  "type": "function",
  "function": {
    "name": "read_file",
    "description": "Read the contents of a file",
    "parameters": {
      "type": "object",
      "properties": {
        "path": {
          "type": "string",
          "description": "The path to the file to read"
        },
        "start_line": {
          "type": "integer",
          "description": "Optional start line number"
        },
        "end_line": {
          "type": "integer",
          "description": "Optional end line number"
        }
      },
      "required": ["path"]
    }
  }
}

```

8.9 MCP 工具集成

8.9.1 MCPToolWrapper

Nanobot 通过 MCPToolWrapper 将 MCP Server 提供的工具“包装”为内置工具格式，实现无缝集成：



8.9.2 转换过程

简化的 MCPToolWrapper 实现

```
class MCPToolWrapper:
    def __init__(self, mcp_client):
        self.client = mcp_client

    async def wrap_tools(self, registry: ToolRegistry):
        """ 将 MCP 工具转换并注册到 ToolRegistry"""
        # 1. 获取 MCP 工具列表
        mcp_tools = await self.client.list_tools()

        for mcp_tool in mcp_tools:
            # 2. 转换工具定义格式
            definition = {
                "name": f"mcp_{mcp_tool.name}",
                "description": mcp_tool.description,
                "parameters": mcp_tool.input_schema
            }

            # 3. 创建执行代理函数
            async def handler(**kwargs):
                result = await self.client.call_tool(
                    mcp_tool.name, kwargs
                )
                return result.content

            # 4. 注册到 ToolRegistry
            registry.register(
                name=f"mcp_{mcp_tool.name}",
                definition=definition,
                handler=handler
            )
```

8.9.3 MCP 工具的命名规范

MCP 工具在注册时会添加前缀以避免命名冲突:

MCP Server 原始工具名: get_weather

注册到 ToolRegistry: mcp_get_weather

MCP Server 原始工具名: search_docs

注册到 ToolRegistry: mcp_search_docs

8.9.4 配置 MCP Server

在 config.json 中配置 MCP Server:

```
{
  "mcp": {
    "servers": {
      "weather": {
        "command": "npx",
        "args": ["-y", "@weather/mcp-server"],
        "env": {
          "API_KEY": "xxx"
        }
      },
    },
    "database": {
```

```

        "command": "python",
        "args": ["-m", "db_mcp_server"],
        "env": {
            "DB_URL": "postgresql://localhost/mydb"
        }
    }
}
}
}
}

```

8.10 自定义 Skill 编写实战

8.10.1 实战：创建一个“代码分析”技能

Step 1: 创建技能目录

```
mkdir -p skills/code-analyzer
```

Step 2: 编写 SKILL.md

```

---
name: code-analyzer
description: "Python 代码质量分析与优化建议"
always: false
metadata: '{"nanobot.requires.bins': ['python3']}'
---

```

代码分析师技能

你是一名资深的 Python 代码分析师。当用户请求代码分析时，按以下流程操作。

分析维度

1. ** 代码规范 **: PEP 8 合规性、命名规范
2. ** 安全性 **: SQL 注入、路径遍历、硬编码密钥
3. ** 性能 **: 算法复杂度、不必要的计算、N+1 查询
4. ** 可维护性 **: 函数长度、圈复杂度、耦合度
5. ** 测试覆盖 **: 关键路径是否有测试

分析流程

Step 1: 收集代码

使用 `list_dir` 查看项目结构 使用 `read_file` 读取关键文件

Step 2: 运行静态分析

使用 `exec` 运行以下命令（如果工具可用）： - `python3 -m py_compile` # 语法检查 - `python3 -m pylint` # 代码质量 - `python3 -m bandit` # 安全扫描

Step 3: 生成报告

按以下格式生成分析报告：

报告模板

```
### □ 总体评分：X/10
```

□ 严重问题
(列出所有严重问题)

□ 改进建议
(列出所有改进建议)

□ 优秀实践
(列出代码中做得好的地方)

□ 重构建议
(给出具体的重构方案)

Step 3: 添加参考资料 (可选)

```
mkdir -p skills/code-analyzer/references
```

```
<!-- skills/code-analyzer/references/security-checklist.md -->  
# Python 安全检查清单
```

常见安全漏洞

1. SQL 注入: 使用参数化查询而非字符串拼接
2. 路径遍历: 验证用户输入的文件路径
3. 命令注入: 避免 `os.system()`, 使用 `subprocess`
4. 硬编码密钥: 使用环境变量或密钥管理服务
5. 不安全的反序列化: 避免 `pickle.loads()` 处理不信任数据

Step 4: 添加辅助脚本 (可选)

```
mkdir -p skills/code-analyzer/scripts
```

```
#!/usr/bin/env python3  
# skills/code-analyzer/scripts/complexity.py  
""" 计算 Python 文件的圈复杂度 """  
import ast  
import sys  
  
def calculate_complexity(filepath):  
    with open(filepath) as f:  
        tree = ast.parse(f.read())  
  
    for node in ast.walk(tree):  
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):  
            complexity = 1  
            for child in ast.walk(node):  
                if isinstance(child, (ast.If, ast.While, ast.For,  
                                     ast.ExceptHandler, ast.With,  
                                     ast.BoolOp)):  
                    complexity += 1  
            print(f"{node.name}: complexity = {complexity}")  
  
if __name__ == "__main__":  
    calculate_complexity(sys.argv[1])
```

Step 5: 测试技能

```
# 启动 Agent  
nanobot
```

```
# 测试对话:
# You: 请分析一下 src/main.py 的代码质量
# Agent 应该会读取 SKILL.md, 然后按照分析流程执行
```

8.10.2 实战: 创建一个”面试教练”技能

```
---
name: interview-coach
description: "AI Agent 方向的面试训练和评估"
always: false
---
```

面试教练技能

模式

模式 1: 知识测验

随机从知识库中抽取问题, 评估用户的回答。

模式 2: 模拟面试

模拟真实面试场景, 包括追问和压力测试。

模式 3: 答案优化

帮助用户优化已有的面试回答。

评分标准

- 技术准确性 (40%)
- 表达清晰度 (20%)
- 深度和广度 (20%)
- 实际经验体现 (20%)

知识库主题

1. AI Agent 基础概念
2. 框架架构设计
3. 记忆系统
4. 工具与技能系统
5. 多平台部署
6. 安全与生产环境

反馈格式

评分: X/10

**** 优点 **:**

- ...

**** 不足 **:**

- ...

**** 改进建议 **:**

- ...

**** 参考答案 **:**

(给出一个更完善的回答示例)

8.11 工具安全机制

8.11.1 restrict_to_workspace

所有文件操作工具（read_file, write_file, edit_file, list_dir）都受 workspace 限制：

```
def validate_path(path: str, workspace: str) -> str:
    """ 确保路径在 workspace 范围内 """
    abs_path = os.path.abspath(os.path.join(workspace, path))
    abs_workspace = os.path.abspath(workspace)

    if not abs_path.startswith(abs_workspace):
        raise PermissionError(
            f"Access denied: {path} is outside workspace"
        )

    return abs_path
```

8.11.2 exec 工具的安全层

用户命令 → 危险模式检测 → SSRF 检测 → 路径限制 → 执行

	拒绝危险命令	拒绝内网访问	限制 cwd	异步执行
	(rm -rf /)	(127.0.0.1)	(workspace)	
└─	如果 exec.allowed = false, 直接拒绝所有命令			

8.11.3 web_fetch 的 SSRF 防护

```
def is_ssrf_target(url: str) -> bool:
    """ 检查 URL 是否指向内网地址 """
    from urllib.parse import urlparse
    import ipaddress

    parsed = urlparse(url)
    hostname = parsed.hostname

    try:
        ip = ipaddress.ip_address(hostname)
        return ip.is_private or ip.is_loopback or ip.is_reserved
    except ValueError:
        return hostname in ("localhost", "metadata.google.internal")
```

8.12 面试高频题

题目 1: Nanobot 的工具系统是如何设计的？

参考回答：

“Nanobot 的工具系统基于 **ToolRegistry** 统一注册机制。所有工具——无论是内置工具（read_file、exec 等）还是 MCP 外部工具——都通过 ToolRegistry 统一注册和执行。

每个工具包含三部分：工具定义（JSON Schema 格式，描述参数）、执行函数（实际的业务逻辑）、安全约束（权限检查、路径限制等）。

工具定义会作为 Tool Definition 发送给 LLM，LLM 决定何时调用什么工具。调用请求返回后，ToolRegistry 根据工具名找到对应的 handler 执行，并将结果以 tool message 返回给 LLM。

对于 MCP 外部工具，通过 MCPToolWrapper 将 MCP 协议的工具格式转换为内置格式，然后统一注册到 ToolRegistry。这样 Agent 无需区分工具来源，使用方式完全一致。”

题目 2: Skill 和 Tool 的区别是什么?

参考回答:

“Skill 和 Tool 在 Nanobot 中是两个不同层次的概念。

Tool 是细粒度的原子操作，比如 read_file 读文件、exec 执行命令、web_search 搜索网页。每个 Tool 有明确的参数定义和执行逻辑，通过 ToolRegistry 注册，以 JSON Schema 格式暴露给 LLM。

Skill 是粗粒度的能力模块，本质上是一个 Markdown 文件 (SKILL.md)，告诉 Agent ‘你能做什么、怎么做’。一个 Skill 通常会教 Agent 如何组合使用多个 Tool 来完成复杂任务。比如 GitHub Skill 教 Agent 如何组合使用 exec (运行 git 命令) 和 read_file (读取代码) 来完成代码审查。

简单类比: Tool 是锤子、螺丝刀这些工具，Skill 是 ‘如何组装家具’ 的说明书。”

题目 3: 什么是渐进披露? 在 Nanobot 中如何应用?

参考回答:

“渐进披露 (Progressive Disclosure) 是 UI/UX 设计中的经典原则——先展示概要，让用户按需深入细节。Nanobot 将这个原则创造性地应用到了 Agent 的 Prompt 管理中。

具体实现是三层结构:

Tier 1: 所有技能的 name 和 description 组成一个摘要列表，始终注入 System Prompt，大约消耗 100-500 tokens。Agent 通过摘要知道自己有哪些能力。

Tier 2: 当 Agent 判断某个技能与当前任务相关时，主动调用 read_file 读取完整的 SKILL.md，获取详细的使用说明。

Tier 3: 如果需要更深入的信息 (参考文档、辅助脚本)，Agent 继续访问 references/ 和 scripts/ 目录。

唯一的例外是标记了 `always: true` 的技能，它们跳过渐进披露，全文注入 System Prompt。

这种设计的价值在于 token 管理——如果有 20 个技能全部注入，可能消耗 1 万 token；使用渐进披露后，常态只消耗 300 token，按需加载时才产生额外消耗。”

题目 4: 如何为 Nanobot 添加一个新的工具?

参考回答:

“有三种方式:

第一种是写 **Skill**——不需要写代码，只需创建一个 SKILL.md 文件，用 Markdown 描述这个能力的使用方式。Agent 会基于已有的 Tool (exec、web_fetch 等) 来执行。这适合流程性、指导性的扩展。

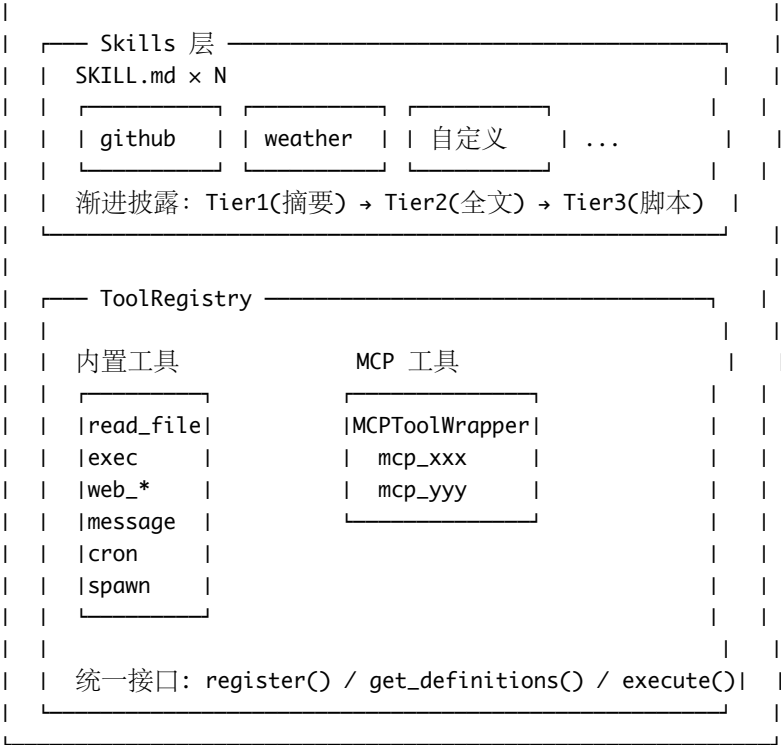
第二种是接入 **MCP Server**——如果需要专用的 API 调用或复杂逻辑，可以开发一个 MCP Server，Nanobot 通过 MCPToolWrapper 自动将其工具转换为内置格式注册到 ToolRegistry。

第三种是修改源码——在 ToolRegistry 中直接注册新的工具函数。这种方式最灵活但需要修改框架代码，不太适合分发。

推荐优先级: Skill > MCP Server > 源码修改。Skill 是最轻量方式，MCP Server 提供了标准化的扩展接口。”

8.13 本章小结

核心架构图



面试记忆清单

考点	一句话回答
Skill vs Tool	Skill 是能力说明书 (Markdown), Tool 是具体操作 (代码)
SKILL.md 格式	YAML Frontmatter + Markdown 正文
渐进披露	三层: 摘要目录 → 全文读取 → 脚本/参考
技能发现	workspace/skills/ 优先, 同名覆盖内置
ToolRegistry	统一注册与执行所有工具 (内置 + MCP)
MCPToolWrapper	将 MCP 工具转换为内置格式
安全机制	restrict_to_workspace + 危险命令拒绝 + SSRF 防护
always 字段	true= 全文注入, false= 仅摘要 (默认)

下一章: 09 - 多平台接入 —— 了解 Nanobot 如何同时接入 Telegram、Discord、飞书、钉钉等 8+ 平台

09 - 多平台接入

阅读时间：约 2 小时
前置知识：08 - 技能与工具
学习目标：理解 Nanobot 的 MessageBus 架构、ChannelManager 策略、BaseChannel 适配器模式，掌握多平台接入方法

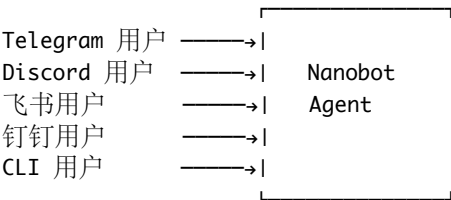
目录

- 9.1 多平台支持概述
- 9.2 MessageBus 架构详解
- 9.3 ChannelManager 出站策略
- 9.4 BaseChannel 适配器模式
- 9.5 各平台接入指南
- 9.6 session_key 会话隔离机制
- 9.7 多平台消息流完整链路
- 9.8 实战练习
- 9.9 面试高频题
- 9.10 本章小结

9.1 多平台支持概述

9.1.1 为什么需要多平台支持

在实际场景中，一个 AI Agent 需要同时服务于多个沟通渠道：



一个 Agent 实例可以同时接入多个平台，每个平台的用户独立会话互不干扰。

9.1.2 支持的平台列表

平台	类型	接入方式	适用场景
CLI	命令行	内置默认	开发调试
Telegram	IM	Bot Token	个人/小团队
Discord	IM	Bot Token	社区/游戏
飞书 (Feishu/Lark)	企业 IM	App ID/Secret	企业办公
钉钉 (DingTalk)	企业 IM	Robot 配置	企业办公
Slack	企业 IM	Bot Token	海外企业
微信 (WeChat)	IM	自定义接入	国内个人
QQ	IM	自定义接入	国内社交
Matrix	开源 IM	Access Token	自托管
Email	邮件	IMAP/SMTP	异步通知

9.1.3 核心理念

Nanobot 的多平台支持基于三层解耦架构：

- 层次 1: Agent 核心逻辑
 - └ 不关心消息来自哪个平台
 - └ 统一处理 InboundMessage
 - └ 统一生成 OutboundMessage

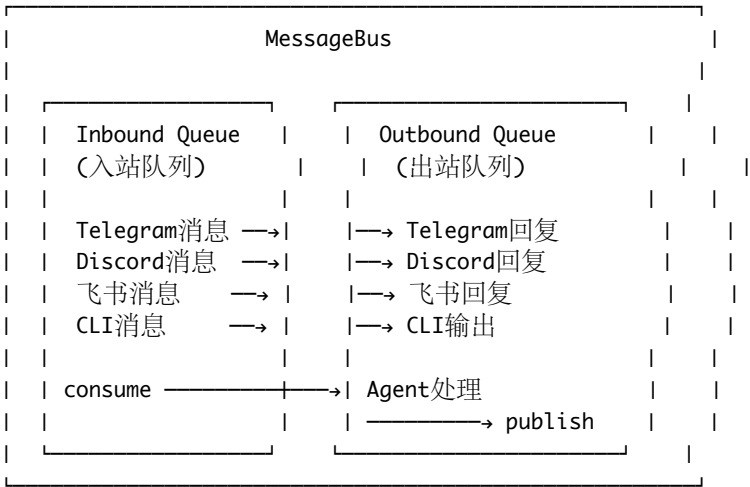
- 层次 2: MessageBus 消息总线
 - └ 异步双队列 (Inbound + Outbound)
 - └ 解耦生产者和消费者
 - └ 平台无关的消息格式

- 层次 3: Channel 适配器
 - └ 每个平台一个 Channel 实现
 - └ 负责协议转换
 - └ 将平台特定格式 → 统一格式

9.2 MessageBus 架构详解

9.2.1 双队列架构

MessageBus 是 Nanobot 多平台通信的核心枢纽，采用双队列设计：



9.2.2 InboundMessage 数据结构

InboundMessage 表示从任何平台进入的用户消息：

```

@dataclass
class InboundMessage:
    channel: str          # 来源通道标识，如 "telegram", "discord", "cli"
    sender_id: str         # 发送者 ID（平台内唯一标识）
    chat_id: str           # 会话 ID（群组 ID 或私聊 ID）
    content: str           # 文本内容
    media: list            # 媒体附件（图片、文件等）
    metadata: dict         # 平台特定的元数据
    session_key: str       # 会话键（用于会话隔离）
  
```

各字段详解：

字段	说明	示例
channel	消息来源平台	"telegram", "discord", "cli"
sender_id	发送者在该平台的唯一 ID	"user_123456"

字段	说明	示例
chat_id	会话标识（私聊或群组）	"chat_789", "group_456"
content	消息文本内容	" 你好，请帮我查天气"
media	附件列表	[{"type": "image", "url": "..."}]
metadata	平台特定数据	{"message_id": "123", "reply_to": "456"}
session_key	会话隔离键	"telegram:chat_789"

9.2.3 OutboundMessage 数据结构

OutboundMessage 表示 Agent 要发出的回复消息：

```
@dataclass
class OutboundMessage:
    channel: str      # 目标通道标识
    chat_id: str      # 目标会话 ID
    content: str       # 回复文本内容
    reply_to: str      # 回复的原消息 ID（可选）
    metadata: dict     # 平台特定的元数据
    media: list        # 媒体附件
```

9.2.4 MessageBus API

```
class MessageBus:
    """ 异步消息总线 """

    def __init__(self):
        self._inbound_queue = asyncio.Queue()
        self._outbound_queue = asyncio.Queue()

    # — 入站 API —

    async def publish_inbound(self, message: InboundMessage):
        """Channel 将用户消息发布到入站队列"""
        await self._inbound_queue.put(message)

    async def consume_inbound(self) -> InboundMessage:
        """Agent 从入站队列消费消息"""
        return await self._inbound_queue.get()

    # — 出站 API —

    async def publish_outbound(self, message: OutboundMessage):
        """Agent 将回复消息发布到出站队列"""
        await self._outbound_queue.put(message)

    async def consume_outbound(self) -> OutboundMessage:
        """Channel 从出站队列消费消息并发送"""
        return await self._outbound_queue.get()
```

9.2.5 为什么用队列而不是直接调用

对比	直接调用	队列解耦
耦合度	Agent 直接依赖 Channel	完全解耦
并发	需要手动管理	队列天然支持

对比	直接调用	队列解耦
背压	无	队列满时自动等待
扩展性	添加平台需要修改 Agent	只需添加 Channel
测试	需要 Mock 平台	直接操作队列

9.3 ChannelManager 出站策略

9.3.1 ChannelManager 的职责

ChannelManager 是 MessageBus 和各 Channel 之间的管理层，负责：

1. 管理所有已注册的 Channel
2. 路由出站消息到正确的 Channel
3. 处理流式输出的合并和优化
4. 处理发送失败的重试

Agent → MessageBus.outbound → ChannelManager → Channel.send()

|
├─ 流式合并
├─ 降频优化
├─ 失败重试
└─ 路由分发

9.3.2 _stream_delta 合并（降频优化）

LLM 的流式输出是逐 token 产生的，如果每个 token 都发送一条消息，会导致：

- Telegram/Discord 等平台的 API 限流
- 用户端消息闪烁
- 网络资源浪费

ChannelManager 通过 _stream_delta 合并机制优化：

```
class ChannelManager:
    async def _stream_delta(self, channel: str, chat_id: str,
                           content_delta: str):
        """ 合并流式输出片段，降低发送频率 """
        key = f"{channel}:{chat_id}"

        # 将新片段追加到缓冲区
        if key not in self._buffer:
            self._buffer[key] = ""
        self._buffer[key] += content_delta

        # 检查是否应该发送
        now = time.time()
        last_send = self._last_send_time.get(key, 0)

        # 降频策略：至少间隔 300ms 才发送一次
        if now - last_send >= 0.3:
            await self._flush_buffer(key)
            self._last_send_time[key] = now
```

LLM 输出流：

t0: "你"
t1: "好"
t2: "!"

t3: "我"
t4: "是"
t5: "一"
t6: "个"
t7: "AI"

无降频（8次发送）： "你" → "好" → "!" → "我" → "是" → "一" → "个" → "AI"

有降频（3次发送）： "你好! " → "我是一" → "个AI"
□ 300ms后 □ 300ms后 □ 结束flush

9.3.3 __progress / __tool_hint 过滤

当 Agent 正在执行工具调用或内部推理时，会产生进度信息和工具提示。ChannelManager 会根据配置决定是否将这些信息展示给用户：

```
async def _handle_progress(self, channel: str, chat_id: str,
                           progress_info: str):
    """ 处理进度信息"""
    channel_config = self._channels[channel]

    # 某些平台可能不展示进度（如 Email）
    if channel_config.get("show_progress", True):
        await self._send(channel, chat_id,
                        f"□ {progress_info}")

async def _handle_tool_hint(self, channel: str, chat_id: str,
                           tool_name: str):
    """ 处理工具调用提示"""
    channel_config = self._channels[channel]

    if channel_config.get("show_tool_hints", True):
        await self._send(channel, chat_id,
                        f"□ 正在使用 {tool_name}...")
```

9.3.4 发送失败指数退避

当向平台发送消息失败时（网络问题、API 限流等），ChannelManager 使用指数退避策略重试：

```
async def _send_with_retry(self, channel: str, chat_id: str,
                           content: str, max_retries: int = 5):
    """ 带指数退避的消息发送"""
    for attempt in range(max_retries):
        try:
            await self._channels[channel].send(chat_id, content)
            return # 发送成功
        except Exception as e:
            if attempt < max_retries - 1:
                # 指数退避: 1s, 2s, 4s, 8s, 16s
                wait_time = 2 ** attempt
                logger.warning(
                    f"Send failed (attempt {attempt + 1}), "
                    f"retrying in {wait_time}s: {e}")
            else:
                logger.error(f"Send failed after {max_retries} attempts: {e}")
            raise
```

重试时间线:

失败 → 等待 1s → 重试
失败 → 等待 2s → 重试
失败 → 等待 4s → 重试
失败 → 等待 8s → 重试
失败 → 等待 16s → 重试
失败 → 放弃, 记录错误日志

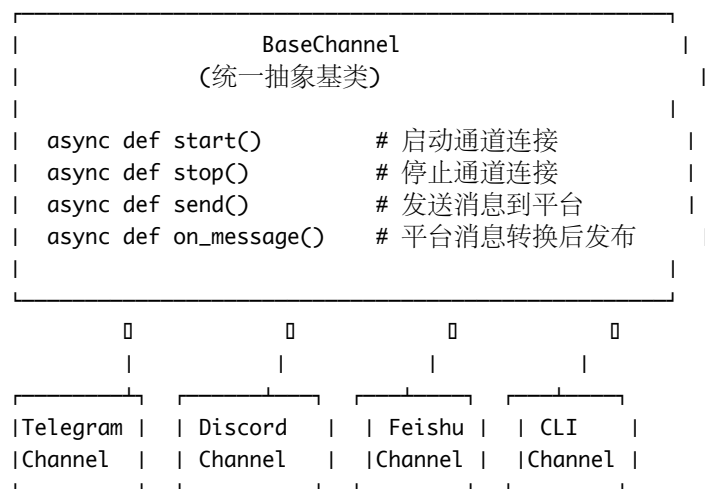
面试要点: 指数退避 (Exponential Backoff) 是分布式系统中处理瞬时故障的标准策略。配合最大重试次数和可选的抖动 (jitter), 可以有效避免“雷群效应” (Thundering Herd)。

9.4 BaseChannel 适配器模式

9.4.1 适配器模式 (Adapter Pattern)

每个平台的 API 都不一样 (Telegram 用 Bot API、Discord 用 Gateway、飞书用 Event API…), 但 Nanobot 的 Agent 核心不应该关心这些差异。

BaseChannel 通过适配器模式解决这个问题:



9.4.2 BaseChannel 抽象基类

```
from abc import ABC, abstractmethod
```

```
class BaseChannel(ABC):
    """ 所有平台通道的抽象基类 """

    def __init__(self, name: str, config: dict, message_bus: MessageBus):
        self.name = name
        self.config = config
        self.bus = message_bus

    @abstractmethod
    async def start(self):
        """ 启动通道 (建立连接、注册 Webhook 等) """
        pass

    @abstractmethod
    async def stop(self):
        """ 停止通道 (断开连接、清理资源) """
```

```

    pass

@abstractmethod
async def send(self, chat_id: str, content: str,
               media: list = None):
    """ 发送消息到该平台 """
    pass

async def on_message(self, platform_message: dict):
    """ 收到平台消息后，转换为 InboundMessage 并发布 """
    inbound = self._convert_to_inbound(platform_message)
    await self.bus.publish_inbound(inbound)

@abstractmethod
def _convert_to_inbound(self, platform_message: dict) -> InboundMessage:
    """ 将平台特定消息格式转换为统一的 InboundMessage """
    pass

```

9.4.3 Telegram Channel 实现示例

```

class TelegramChannel(BaseChannel):
    """Telegram 平台适配器"""

    def __init__(self, config: dict, message_bus: MessageBus):
        super().__init__("telegram", config, message_bus)
        self.bot_token = config["bot_token"]
        self.bot = None

    async def start(self):
        """ 启动 Telegram Bot """
        from telegram import Bot
        self.bot = Bot(token=self.bot_token)
        # 注册消息处理回调
        # 开始轮询或 Webhook

    async def stop(self):
        """ 停止 Telegram Bot """
        if self.bot:
            await self.bot.shutdown()

    async def send(self, chat_id: str, content: str,
                  media: list = None):
        """ 发送消息到 Telegram """
        if media:
            for item in media:
                if item["type"] == "image":
                    await self.bot.send_photo(
                        chat_id=chat_id,
                        photo=item["url"]
                    )

        if content:
            await self.bot.send_message(
                chat_id=chat_id,
                text=content,
                parse_mode="Markdown"
            )

```

```

def _convert_to_inbound(self, update: dict) -> InboundMessage:
    """ 将 Telegram Update 转换为 InboundMessage """
    message = update.get("message", {})
    return InboundMessage(
        channel="telegram",
        sender_id=str(message["from"]["id"]),
        chat_id=str(message["chat"]["id"]),
        content=message.get("text", ""),
        media=self._extract_media(message),
        metadata={
            "message_id": message["message_id"],
            "username": message["from"].get("username"),
        },
        session_key=f"telegram:{message['chat']['id']}"
    )

```

9.4.4 添加新平台的步骤

如果你要为 Nanobot 添加一个新平台（比如 Line），只需：

1. 创建 Channel 类，继承 BaseChannel

```

class LineChannel(BaseChannel):
    def __init__(self, config, message_bus):
        super().__init__("line", config, message_bus)

    async def start(self):
        # 初始化 Line SDK, 注册 Webhook
        pass

    async def stop(self):
        # 清理资源
        pass

    async def send(self, chat_id, content, media=None):
        # 调用 Line Messaging API 发送消息
        pass

    def _convert_to_inbound(self, event):
        # 将 Line Event 转换为 InboundMessage
        pass

```

2. 在 config.json 中添加配置

```

# {
#   "channels": {
#     "line": {
#       "channel_access_token": "xxx",
#       "channel_secret": "xxx"
#     }
#   }
# }

```

3. 注册到 ChannelManager

面试要点：这种“面向接口编程”的适配器模式是设计模式的经典应用。添加新平台不需要修改 Agent 核心代码，符合开闭原则（对扩展开放，对修改关闭）。

9.5 各平台接入指南

9.5.1 Telegram 接入

Step 1: 创建 Bot

1. 在 Telegram 中搜索 @BotFather
2. 发送 /newbot
3. 按提示设置 Bot 名称和用户名
4. 获取 Bot Token (格式: 123456:ABC-DEF...)

Step 2: 配置 config.json

```
{
  "channels": {
    "telegram": {
      "bot_token": "123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11"
    }
  }
}
```

Step 3: 启动

```
nanobot
# Agent 会自动启动 Telegram Bot 轮询
# 在 Telegram 中找到你的 Bot, 开始对话
```

9.5.2 飞书 (Feishu/Lark) 接入

Step 1: 创建飞书应用

1. 登录 飞书开放平台 (open.feishu.cn)
2. 创建企业自建应用
3. 添加"机器人"能力
4. 获取 App ID 和 App Secret
5. 配置事件订阅 (接收消息事件)
6. 配置消息卡片请求网址

Step 2: 配置 config.json

```
{
  "channels": {
    "feishu": {
      "app_id": "cli_xxxxxxxx",
      "app_secret": "xxxxxxxxxxxxxxxxxxxxxxxx",
      "verification_token": "xxxxxxx",
      "encrypt_key": "xxxxxxx"
    }
  }
}
```

Step 3: 配置 Webhook

飞书需要一个公网可访问的 Webhook URL:
`https://your-domain.com/webhook/feishu`

如果在本地开发, 可以使用 ngrok:
`ngrok http 8080`

9.5.3 钉钉 (DingTalk) 接入

Step 1: 创建钉钉机器人

1. 登录 钉钉开放平台 (open.dingtalk.com)
2. 创建应用 → 添加机器人能力
3. 配置消息接收地址
4. 获取 Robot Code 和 App Key/Secret

Step 2: 配置 config.json

```
{
  "channels": {
    "dingtalk": {
      "app_key": "dingxxxxxxxx",
      "app_secret": "xxxxxxxxxxxxxxxxxxxxxxxx",
      "robot_code": "xxxxxxx"
    }
  }
}
```

9.5.4 Discord 接入

Step 1: 创建 Discord Bot

1. 登录 Discord Developer Portal (discord.com/developers)
2. 创建 Application → Bot
3. 获取 Bot Token
4. 生成邀请链接, 将 Bot 添加到服务器
5. 开启 Message Content Intent

Step 2: 配置 config.json

```
{
  "channels": {
    "discord": {
      "bot_token": "MTIzNDU2Nzg5MDEyMzQ1Njc4OQ.AbCdEf.xxxxxxxxxxxxxxxxxx"
    }
  }
}
```

9.5.5 其他平台

平台	接入难度	关键配置
Slack	中等	Bot Token, Signing Secret
微信	较难	需要企业微信或第三方框架
QQ	较难	QQ 开放平台或第三方框架
Matrix	中等	Homeserver URL, Access Token
Email	简单	IMAP/SMTP 服务器和凭据

9.5.6 多平台同时接入

config.json 中可以同时配置多个平台:

```
{
  "channels": {
    "telegram": {
      "bot_token": "123456:ABC..."
    },
    "discord": {
      "bot_token": "MTIz..."
    },
    "feishu": {

```

```

        "app_id": "cli_xxx",
        "app_secret": "xxx"
    }
}
}

```

启动后，Agent 会同时监听所有配置的平台，通过 session_key 隔离不同平台/用户的会话。

9.6 session_key 会话隔离机制

9.6.1 session_key 的构成

session_key 用于唯一标识一个“对话上下文”：

```
session_key = f"{channel}:{chat_id}"
```

示例：

场景	session_key	说明
CLI 默认	cli:default	命令行交互默认会话
Telegram 私聊	telegram:123456	用户 ID 为 123456
Telegram 群组	telegram:-100789	群组 ID
Discord 频道	discord:guild_1:chan_2	服务器 1 的频道 2
飞书私聊	feishu:ou_xxx	飞书用户

9.6.2 session_key 的作用

session_key 决定了：

1. 会话历史隔离
session_key → sessions/<session_key>.jsonl
不同 key 的用户拥有独立的对话历史
2. 记忆隔离
每个 session_key 独立的短期记忆
但 MEMORY.md 是全局共享的
3. 流式输出路由
Agent 生成的回复按 session_key 路由到正确的平台/会话

9.6.3 session_key_override：线程/子会话隔离

某些平台支持“线程”或“回复链”功能（如 Discord 线程、Slack 线程）。Nanobot 通过 session_key_override 实现线程级别的会话隔离：

```

def build_session_key(message: InboundMessage) -> str:
    """ 构建 session_key，支持线程覆盖 """
    base_key = f"{message.channel}:{message.chat_id}"

    # 如果消息包含线程信息，使用线程级隔离
    thread_id = message.metadata.get("thread_id")
    if thread_id:
        return f"{base_key}:thread_{thread_id}"

    # 如果有显式的 session_key_override
    override = message.metadata.get("session_key_override")
    if override:

```

```
return override
```

```
return base_key
```

会话隔离层次:

```
└─ telegram:123456          # Telegram 用户 123456 的主会话
|
└─ discord:guild_1:chan_2   # Discord 频道的主会话
|   └─ discord:guild_1:chan_2:thread_100 # 线程 100
|       └─ discord:guild_1:chan_2:thread_200 # 线程 200
|           (线程内的对话上下文独立于主频道)
|
└─ feishu:ou_xxx           # 飞书用户的会话
```

面试要点: session_key 的设计体现了“多租户隔离”的思想。每个 session_key 对应独立的会话状态,就像 SaaS 系统中每个租户有独立的数据。线程级覆盖是进一步的细粒度隔离。

9.7 多平台消息流完整链路

9.7.1 完整消息处理流程

以 Telegram 用户发送消息为例:

Step 1: 用户在 Telegram 发送 "帮我查天气"

```
|
[]
```

Step 2: Telegram API 推送 Update 到 Bot

```
|
[]
```

Step 3: TelegramChannel.on_message()

```
└─ 解析 Telegram Update 格式
└─ 构建 InboundMessage:
|   channel: "telegram"
|   sender_id: "123456"
|   chat_id: "123456"
|   content: "帮我查天气"
|   session_key: "telegram:123456"
└─ 发布到 MessageBus Inbound Queue
```

```
|
[]
```

Step 4: Agent 从 Inbound Queue 消费消息

```
└─ 加载 session "telegram:123456" 的历史
└─ 加载 MEMORY.md
└─ 构建完整 Prompt
└─ 调用 LLM API
```

```
|
[]
```

Step 5: LLM 返回工具调用 → Agent 执行

```
└─ 调用 web_search("北京天气")
└─ 获取搜索结果
└─ LLM 基于结果生成回复
```

```
|
[]
```

Step 6: Agent 创建 OutboundMessage

```
└─ channel: "telegram"
└─ chat_id: "123456"
```

```

└─ content: "北京今天晴, 气温 25°C..."
└─ 发布到 MessageBus Outbound Queue
    |
    []
Step 7: ChannelManager 从 Outbound Queue 消费
└─ 路由到 TelegramChannel
└─ _stream_delta 合并 (如果是流式输出)
└─ 调用 TelegramChannel.send()
    |
    []
Step 8: TelegramChannel.send()
└─ 调用 Telegram Bot API
└─ 用户在 Telegram 看到回复

```

9.7.2 跨平台场景

同一个 Agent 可以同时处理来自不同平台的消息:

时间线:

```

t=0s Telegram用户A: "你好"
t=1s Discord用户B: "帮我写代码"
t=2s Agent 处理 A 的消息
t=3s Agent 处理 B 的消息
t=4s 回复发送到 Telegram (给A)
t=5s 回复发送到 Discord (给B)

```

关键: A 和 B 的会话完全独立

session_key_A = "telegram:user_A"

session_key_B = "discord:guild_1:user_B"

9.8 实战练习

练习 1: 接入 Telegram

1. 获取 Bot Token (通过 @BotFather)

2. 配置

```

cat > config.json << 'EOF'
{
  "agents": {
    "defaults": {
      "model": "gpt-4o",
      "provider": "openai"
    }
  },
  "providers": {
    "openai": {
      "api_key": "sk-your-key"
    }
  },
  "channels": {
    "telegram": {
      "bot_token": "your-bot-token"
    }
  }
}

```

EOF

3. 启动

nanobot

4. 在 Telegram 中与 Bot 对话

练习 2: 多平台同时接入

配置同时接入 Telegram 和 Discord

```
cat > config.json << 'EOF'
{
  "agents": {
    "defaults": {
      "model": "gpt-4o",
      "provider": "openai"
    }
  },
  "providers": {
    "openai": {
      "api_key": "sk-your-key"
    }
  },
  "channels": {
    "telegram": {
      "bot_token": "telegram-bot-token"
    },
    "discord": {
      "bot_token": "discord-bot-token"
    }
  }
}
EOF
```

启动后两个平台同时可用

nanobot

练习 3: 观察 session 隔离

启动多平台 Agent 后

在 Telegram 中对话几轮

在 Discord 中对话几轮

然后检查 sessions 目录

ls sessions/

telegram:123456.jsonl

discord:guild_1:chan_2.jsonl

验证两个会话的历史是独立的

wc -l sessions/*.jsonl

9.9 面试高频题

题目 1: 如何设计一个支持多平台的 Agent 系统?

参考回答:

“我会采用 Nanobot 的架构思路，核心是三层解耦设计：

第一层是 **Agent** 核心，只处理业务逻辑——接收统一格式的消息、调用 LLM、执行工具、生成回复。它完全不知道消息来自哪个平台。

第二层是 **MessageBus** 消息总线，采用双队列架构——Inbound Queue 收集所有平台的入站消息，Outbound Queue 收集 Agent 的出站回复。队列实现生产者/消费者解耦，天然支持异步和背压。

第三层是 **Channel** 适配器，每个平台一个 Channel 实现，继承统一的 BaseChannel 抽象类。Channel 负责两件事：一是将平台特定的消息格式转换为统一的 InboundMessage，二是将 OutboundMessage 转换为平台特定的 API 调用。

添加新平台只需实现一个新的 Channel 类，不需要修改 Agent 和 MessageBus 代码，符合开闭原则。

会话隔离通过 session_key（格式 {channel}:{chat_id}）实现，每个 session_key 对应独立的对话历史和状态。

出站优化方面，ChannelManager 负责流式输出的降频合并（避免平台 API 限流）和发送失败的指数退避重试。”

题目 2: MessageBus 和直接调用有什么区别？

参考回答：

“直接调用意味着 Agent 直接引用并调用 Channel 的发送方法，这会导致紧耦合——Agent 需要知道所有平台的存在。每添加一个平台都要修改 Agent 代码。

MessageBus 通过异步队列解耦了生产者和消费者。Channel 往 Inbound Queue 里放消息，Agent 从 Inbound Queue 取消息；Agent 往 Outbound Queue 里放回复，ChannelManager 从 Outbound Queue 取出后路由到对应 Channel。双方通过数据结构（InboundMessage/OutboundMessage）交互，互不依赖。

这种设计还带来了背压控制——如果 Agent 处理不过来，消息会在队列里排队，不会丢失。在测试时也更方便，可以直接向队列注入消息而不需要启动真实的平台连接。”

题目 3: 如何处理不同平台的消息格式差异？

参考回答：

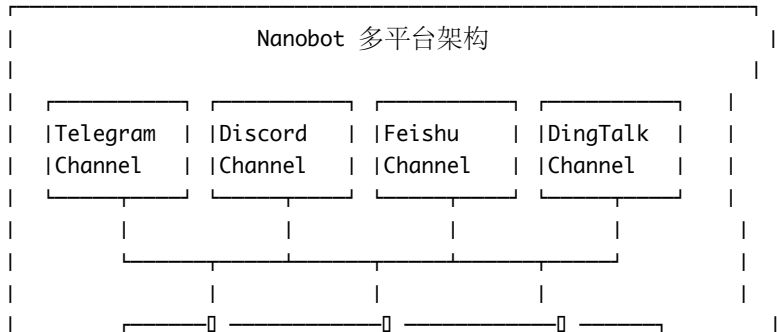
“通过适配器模式。定义一个统一的消息结构——InboundMessage 包含 channel、sender_id、chat_id、content、media、metadata 等字段。每个平台的 Channel 实现一个 _convert_to_inbound 方法，负责将平台特定格式（如 Telegram 的 Update、Discord 的 Message Event）映射到统一格式。

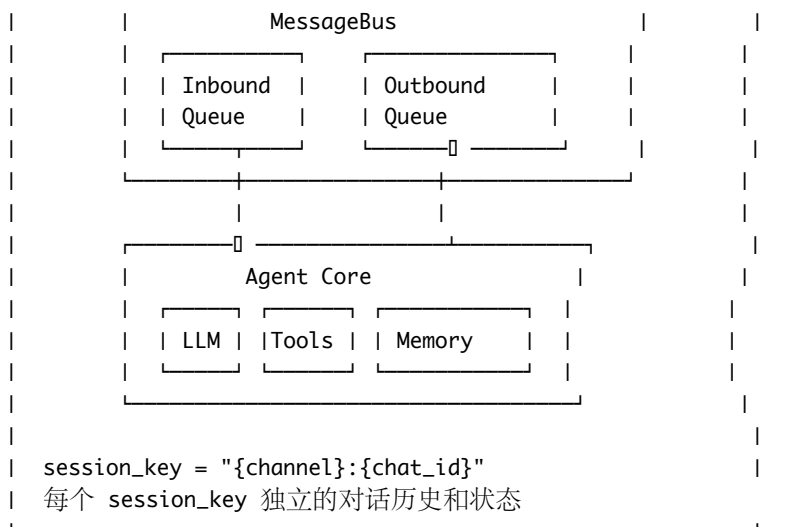
出站同理，OutboundMessage 是统一格式，Channel 的 send 方法负责转换为平台 API 调用。

对于平台特有的功能（如 Telegram 的 Inline Keyboard、Discord 的 Embed），可以放在 metadata 字段中传递，Channel 在发送时识别并使用。”

9.10 本章小结

核心架构图





面试记忆清单

考点	一句话回答
架构设计	三层解耦：Agent 核心 + MessageBus + Channel 适配器
MessageBus	异步双队列（Inbound + Outbound），解耦生产者/消费者
InboundMessage	统一入站消息：channel, sender_id, chat_id, content, media, metadata
OutboundMessage	统一出站消息：channel, chat_id, content, reply_to, media
BaseChannel	抽象基类，每个平台实现一个适配器（适配器模式）
session_key	{channel}:{chat_id}，实现会话隔离
流式优化	_stream_delta 降频合并，间隔 300ms
失败重试	指数退避：1s, 2s, 4s, 8s, 16s
添加新平台	只需实现 BaseChannel 子类，开闭原则

下一章：10 - 子 Agent 与定时任务 —— 深入理解 SubAgent 后台任务和 Cron 定时调度

10 - 子 Agent 与定时任务

阅读时间：约 1.5 小时
前置知识：09 - 多平台接入
学习目标：理解 SubAgent 后台任务机制、Cron 定时调度系统、Heartbeat 心跳服务，掌握并发任务设计

目录

- 10.1 为什么需要子 Agent 和定时任务
- 10.2 子 Agent (SubAgent) 系统
- 10.3 定时任务 (Cron) 系统
- 10.4 Heartbeat 心跳服务
- 10.5 三者的协作关系
- 10.6 实战练习
- 10.7 面试高频题
- 10.8 本章小结

10.1 为什么需要子 Agent 和定时任务

10.1.1 主 Agent 的局限

主 Agent 是同步的——用户发消息，Agent 处理，返回结果。但有些场景需要：

场景 1：用户说"帮我调研一下 Rust 和 Go 的对比，要详细的"

- 这可能需要 10 分钟搜索和整理
- 用户不想等 10 分钟什么都做不了

场景 2：用户说"每天早上 8 点给我发一份新闻摘要"

- 需要定时执行，不是一次性对话

场景 3：用户同时提出多个独立任务

- "帮我查天气，同时分析一下这个代码文件"
- 两个任务可以并行处理

10.1.2 解决方案

场景	解决方案
耗时后台任务	SubAgent (子代理后台执行)
定时触发任务	Cron (定时调度)
周期性唤醒	Heartbeat (心跳服务)

10.2 子 Agent (SubAgent) 系统

10.2.1 SpawnTool 启动子代理

SubAgent 通过 spawn 工具启动：

```
{
  "name": "spawn",
  "parameters": {
    "task": "string (必需) - 子代理要执行的任务描述"
  }
}
```


当主 Agent 调用 spawn 时:

主Agent 对话:

User: 帮我详细调研 Rust 和 Go 的对比

Agent: 好的, 这个任务比较耗时, 我启动一个后台任务来做。

[调用工具: spawn]

[task: "详细调研 Rust 和 Go 的对比, 包括性能、生态、学习曲线等维度"]

Agent: 我已经启动了后台调研任务, 完成后会通知你。你可以继续问我其他问题。

(后台 SubAgent 开始独立执行...)

(主Agent 可以继续和用户对话...)

(几分钟后...)

Agent: 后台调研已完成! 以下是 Rust 和 Go 的详细对比...

10.2.2 SubagentManager 管理

SubagentManager 是管理所有子代理生命周期的中心组件:

```
class SubagentManager:
    """ 管理所有后台子代理 """

    def __init__(self, agent_config, message_bus):
        self.config = agent_config
        self.bus = message_bus
        self._active_agents: dict[str, SubAgent] = {}

    async def spawn(self, task: str, parent_session_key: str) -> str:
        """ 启动一个新的子代理 """
        agent_id = generate_unique_id()

        sub_agent = SubAgent(
            agent_id=agent_id,
            task=task,
            config=self._build_subagent_config(),
            parent_session_key=parent_session_key,
            message_bus=self.bus
        )

        self._active_agents[agent_id] = sub_agent

        # 在后台启动执行
        asyncio.create_task(self._run_agent(sub_agent))

        return agent_id

    async def _run_agent(self, agent: SubAgent):
        """ 在后台运行子代理 """
        try:
            result = await agent.execute()
            # 执行完成后, 通过 MessageBus 回报结果
            await self.bus.publish_inbound(InboundMessage(
                channel="internal",
                sender_id="subagent",
                chat_id=agent.parent_session_key,
                content=f"[后台任务完成] {result}",
                media=[],
                metadata={"subagent_id": agent.agent_id},
            ))
```

```

        session_key=agent.parent_session_key
    ))
except Exception as e:
    logger.error(f"SubAgent {agent.agent_id} failed: {e}")
finally:
    del self._active_agents[agent.agent_id]

```

10.2.3 SubAgent 与主 Agent 的区别

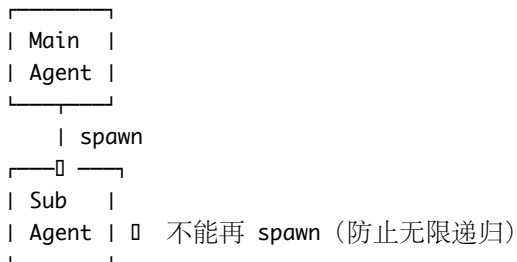
这是面试高频考点：

对比维度	主 Agent	SubAgent（子代理）
迭代限制	40 次	15 次
工具集	完整工具集	受限工具集
MessageTool	可用	不可用
SpawnTool	可用	不可用（防递归）
CronTool	可用	不可用（防递归）
执行方式	前台同步	后台异步
用户交互	直接交互	不能与用户交互
结果汇报	直接返回	通过 MessageBus 回报
生命周期	持续运行	任务完成即终止

10.2.4 为什么限制工具集

防递归原则：

SubAgent 不能调用 spawn → 防止创建子子代理



SubAgent 不能调用 message → 它不直接与用户对话
只能通过 MessageBus 回报结果给主Agent

SubAgent 不能调用 cron → 防止定时任务中再创建定时任务
避免定时任务指数增长

10.2.5 迭代限制的设计思考

主Agent: max_tool_iterations = 40

- └ 理由：主Agent需要处理复杂的多步骤任务
- └ 场景：代码审查 → 搜索 → 修改 → 测试 → 提交
- └ 40 次迭代足以完成大多数复杂任务

SubAgent: max_tool_iterations = 15

- └ 理由：子代理执行的是明确的单一任务
- └ 场景：搜索调研 → 整理 → 汇报
- └ 15 次迭代防止子代理失控消耗过多资源
- └ 如果 15 次不够，说明任务应该拆分

10.2.6 结果回报机制

SubAgent 完成任务后，通过 MessageBus 将结果发送回主 Agent 的会话：

```
SubAgent 执行完成
|
[]
构建 InboundMessage
├─ channel: "internal"          # 内部通道
├─ sender_id: "subagent"        # 标识来自子代理
├─ chat_id: parent_session_key  # 发送到父会话
├─ content: "调研结果..."      # 任务结果
└─ metadata: {subagent_id: "xxx"}
|
[]
publish_inbound(message)
|
[]
主Agent 从 Inbound Queue 消费
├─ 识别这是子代理的回报
├─ 将结果整合到当前对话
└─ 向用户展示结果
```

10.2.7 使用场景

场景	适合用 SubAgent	原因
长时间调研	是	不阻塞主对话
批量文件处理	是	后台执行
代码分析	是	耗时但不需要交互
数据采集	是	多源并行采集
实时对话	否	需要即时交互
需要用户确认	否	SubAgent 无法交互

10.3 定时任务（Cron）系统

10.3.1 CronService 基于 APScheduler

Nanobot 的定时任务基于 Python 的 APScheduler（Advanced Python Scheduler）库：

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler

class CronService:
    """ 定时任务管理服务 """

    def __init__(self, message_bus: MessageBus):
        self.scheduler = AsyncIOScheduler()
        self.bus = message_bus
        self._jobs: dict[str, CronJob] = {}

    async def start(self):
        """ 启动调度器 """
        self.scheduler.start()

    async def stop(self):
        """ 停止调度器 """
        self.scheduler.shutdown()
```

10.3.2 cron 工具的三种操作

```
{
  "name": "cron",
  "parameters": {
    "action": "string (必需) - add | list | remove",
    "name": "string - 任务名称 (add/remove 时必需)",
    "schedule": "object - 调度配置 (add 时必需)",
    "message": "string - 触发时发送的消息 (add 时必需)"
  }
}
```

add ——添加定时任务

Agent: [调用 cron]

```
参数: {
  "action": "add",
  "name": "daily_news",
  "schedule": {
    "cron_expr": "0 8 * * *",
    "tz": "Asia/Shanghai"
  },
  "message": "请搜索今天的科技新闻，整理成摘要发给我"
}
```

list ——列出所有任务

Agent: [调用 cron]

```
参数: {
  "action": "list"
}
```

返回:

```
[
  {
    "name": "daily_news",
    "schedule": "0 8 * * * (Asia/Shanghai)",
    "next_run": "2024-03-17 08:00:00",
    "message": "请搜索今天的科技新闻..."
  }
]
```

remove ——删除任务

Agent: [调用 cron]

```
参数: {
  "action": "remove",
  "name": "daily_news"
}
```

10.3.3 调度配置方式

cron 工具支持三种调度配置:

方式一: **every_seconds** (间隔秒数)

```
{
  "schedule": {
    "every_seconds": 3600
  }
}
```

表示每 3600 秒（1 小时）执行一次。

方式二：cron_expr + tz (Cron 表达式)

```
{
  "schedule": {
    "cron_expr": "0 8 * * *",
    "tz": "Asia/Shanghai"
  }
}
```

Cron 表达式格式：

```
_____ 分钟 (0-59)
| _____ 小时 (0-23)
| | _____ 日 (1-31)
| | | _____ 月 (1-12)
| | | | _____ 星期 (0-6, 0=周日)
| | | | |
* * * * *
```

示例：

```
0 8 * * *      → 每天早上 8:00
0 */2 * * *    → 每 2 小时
0 9 * * 1-5    → 工作日 9:00
30 18 * * 5    → 每周五 18:30
0 0 1 * *     → 每月 1 日 0:00
```

方式三：at (指定时间执行一次)

```
{
  "schedule": {
    "at": "2024-03-17T15:30:00",
    "tz": "Asia/Shanghai"
  }
}
```

表示在指定时间执行一次（一次性任务）。

10.3.4 定时任务的执行流程

定时任务触发

```
|
[]
CronService 构建 InboundMessage
├─ channel: "cron"          # 标识来自定时任务
├─ sender_id: "cron_service"
├─ chat_id: original_session_key
├─ content: "请搜索今天的科技新闻..." # 预设的 message
└─ metadata: {cron_job: "daily_news"}
|
[]
publish_inbound(message)
|
[]
Agent 从 Inbound Queue 消费
├─ 像处理普通用户消息一样处理
├─ 执行搜索、整理等操作
└─ 结果通过 OutboundMessage 发送到原始会话
```

10.3.5 防递归机制

Nanobot 严格禁止在定时任务执行上下文中再创建定时任务：

```
class CronTool:
    async def execute(self, action, **kwargs):
        # 检查当前上下文是否来自 Cron 触发
        if self._is_cron_context():
            return "Error: Cannot create cron jobs from within a cron execution context."

        # 正常执行
        ...

    def _is_cron_context(self) -> bool:
        """ 检查当前消息是否来自定时任务触发 """
        current_message = self._get_current_inbound()
        return current_message.channel == "cron"
```

为什么要防递归：

危险场景：

cron_job_A: "每分钟执行，内容是创建一个每分钟执行的cron"

```
t=0: cron_A 触发 → 创建 cron_B (每分钟执行)
t=1: cron_A 触发 → 创建 cron_C
      cron_B 触发 → 创建 cron_D
t=2: cron_A 触发 → 创建 cron_E
      cron_B 触发 → 创建 cron_F
      cron_C 触发 → 创建 cron_G
      cron_D 触发 → 创建 cron_H
```

指数增长 → 系统崩溃！

面试要点：防递归是系统设计中的重要原则。在 SubAgent 中禁止 spawn（防止子代理递归），在 Cron 中禁止创建新 Cron（防止定时任务递归），这些都是对“失控增长”的防御。

10.4 Heartbeat 心跳服务

10.4.1 什么是 Heartbeat

Heartbeat 是一种周期性唤醒 Agent 的机制。与 Cron 不同，Heartbeat 的目的不是执行特定任务，而是让 Agent “保持活跃”并自主决定需要做什么。

Cron: "每天8点，发送今日新闻" → 明确的任务

Heartbeat: "每30分钟醒来，看看有没有事" → 自主判断

10.4.2 配置方式

在 config.json 中配置 Heartbeat：

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "enabled": true,
        "interval_s": 1800,
        "keep_recent_messages": 5
      }
    }
  }
}
```

```

    }
}
}

```

字段	类型	说明
enabled	bool	是否启用心跳
interval_s	int	心跳间隔（秒）
keep_recent_messages	int	心跳时保留的最近消息数

10.4.3 心跳触发机制

```

class HeartbeatService:
    """ 心跳服务 """

    def __init__(self, config: dict, message_bus: MessageBus):
        self.enabled = config.get("enabled", False)
        self.interval = config.get("interval_s", 1800)
        self.keep_recent = config.get("keep_recent_messages", 5)
        self.bus = message_bus

    async def start(self):
        """ 启动心跳循环 """
        if not self.enabled:
            return

        while True:
            await asyncio.sleep(self.interval)
            await self._heartbeat()

    async def _heartbeat(self):
        """ 执行一次心跳 """
        # 构建心跳消息
        message = InboundMessage(
            channel="heartbeat",
            sender_id="heartbeat_service",
            chat_id="heartbeat",
            content="[Heartbeat] 定期检查，请查看是否有待处理事项。",
            media=[],
            metadata={
                "type": "heartbeat",
                "keep_recent_messages": self.keep_recent
            },
            session_key="heartbeat:default"
        )

        await self.bus.publish_inbound(message)

```

10.4.4 Heartbeat 的使用场景

场景	配置	说明
健康检查	interval_s: 300	每 5 分钟检查系统状态
邮件摘要	interval_s: 3600	每小时检查新邮件
项目监控	interval_s: 1800	每 30 分钟检查项目状态
知识更新	interval_s: 86400	每天更新知识库

10.4.5 keep_recent_messages 的作用

心跳触发时，不需要加载完整的会话历史（太浪费 token），只保留最近的 N 条消息作为上下文：

完整会话历史：100 条消息
keep_recent_messages: 5

心跳时的上下文：
└─ System Prompt（正常）
└─ MEMORY.md（正常）
└─ 最近 5 条消息（而非全部 100 条）

- 好处：
- ▣ 节省 token
 - ▣ Agent 仍能了解最近状态
 - ▣ 减少不必要的信息干扰

10.5 三者的协作关系

10.5.1 SubAgent、Cron、Heartbeat 对比

维度	SubAgent	Cron	Heartbeat
触发方式	主 Agent 调用 spawn	定时表达式	固定间隔
任务内容	明确的单一任务	预设的消息	Agent 自主决定
生命周期	执行完即终止	持续存在	持续运行
与用户交互	不能	结果发到会话	按需交互
并发	可多个并行	独立调度	单实例
防递归	不能再 spawn	不能在 cron 中创建 cron	N/A

10.5.2 协作场景示例

场景：智能助手管理日常工作

Heartbeat（每30分钟唤醒）

- └─ 检查是否有新邮件 → 有 → spawn子Agent处理
- └─ 检查 GitHub PR → 有新PR → 通知用户
- └─ 检查待办事项 → 有到期项 → 提醒用户

Cron（定时任务）

- └─ 每天 8:00 → 发送今日日程
- └─ 每天 18:00 → 发送工作总结
- └─ 每周五 17:00 → 生成周报

SubAgent（按需启动）

- └─ 用户："帮我调研这个技术方案" → spawn后台调研
- └─ Heartbeat发现新邮件 → spawn后台处理

└─ Cron触发周报 → spawn后台收集数据和生成

10.5.3 设计模式分析

SubAgent = Actor 模式

- └─ 独立运行的轻量级进程
- └─ 通过消息传递通信
- └─ 任务完成自动销毁

Cron = Observer 模式（时间驱动）

- └─ 监听时间事件
- └─ 触发时通知 Agent
- └─ 本身不执行业务逻辑

Heartbeat = Polling 模式

- └─ 定期检查
- └─ 让 Agent 自主决策
- └─ 类似于操作系统的定时器中断

10.6 实战练习

练习 1：使用 SubAgent 并行处理

启动 Agent

nanobot

对话示例：

You：帮我做两件事：

1. 搜索 Python 3.12 的新特性

2. 搜索 Rust 2024 的发展趋势

分别用后台任务处理

Agent 应该会调用两次 spawn，启动两个 SubAgent 并行执行

你可以继续与主 Agent 对话，等后台任务完成后收到结果

练习 2：创建定时任务

nanobot

对话示例：

You：帮我创建一个定时任务，每小时提醒我休息一下

Agent 会调用 cron 工具：

{

"action": "add",

"name": "rest_reminder",

"schedule": {"every_seconds": 3600},

"message": " 休息提醒：已经工作 1 小时了，起来活动一下吧！ "

}

查看任务列表

You：列出所有定时任务

删除任务

You：删除休息提醒任务

练习 3: 配置 Heartbeat

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "enabled": true,
        "interval_s": 300,
        "keep_recent_messages": 3
      }
    }
  }
}
```

启动后, 每 5 分钟 Agent 会自动醒来
你可以在 AGENTS.md 中告诉它醒来时要做什么:

```
# AGENTS.md 中添加:
# ## 心跳行为
# 当收到心跳唤醒时:
# 1. 检查 workspace 中是否有新文件
# 2. 如果有 TODO.md, 检查待办事项的状态
# 3. 如果发现需要处理的事项, 主动通知用户
```

10.7 面试高频题

题目 1: Nanobot 的 SubAgent 系统是怎么设计的?

参考回答:

“Nanobot 的 SubAgent 通过 spawn 工具启动, 由 SubagentManager 统一管理。

启动机制: 主 Agent 调用 spawn 工具时, SubagentManager 创建一个新的 SubAgent 实例, 并通过 `asyncio.create_task` 在后台运行, 不阻塞主对话。

关键约束: SubAgent 相比主 Agent 有三个限制——迭代次数从 40 降到 15, 防止长时间运行; 不能使用 MessageTool, 因为它不直接与用户交互; 不能使用 SpawnTool 和 CronTool, 防止递归创建。

结果回报: SubAgent 完成后, 将结果封装为 InboundMessage 通过 MessageBus 发回主 Agent 的会话, 主 Agent 再向用户展示。

这种设计的核心思想是受控的并行性——允许后台执行但严格限制能力边界, 避免失控。”

题目 2: 如何防止 Agent 系统中的递归问题?

参考回答:

“Nanobot 在两个维度防止递归:

第一是 **SubAgent** 防递归——子代理的工具集中不包含 spawn 工具, 所以子代理无法创建子子代理。这在 ToolRegistry 层面就做了过滤, 子代理启动时只注册受限的工具子集。

第二是 **Cron** 防递归——在 cron 触发的执行上下文中, cron 工具会检测当前消息的 channel 是否为‘cron’。如果是, 拒绝执行 add 操作。这防止了定时任务中创建新定时任务导致的指数增长。

这两种防递归策略的思路是一致的: 在特定上下文中限制特定工具的可用性。这是最小权限原则在 Agent 系统中的应用。”

题目 3: Cron 和 Heartbeat 有什么区别?

参考回答:

“两者都是定时触发机制，但设计目的不同。

Cron 是任务驱动——它携带明确的 `message`（搜索今日新闻），Agent 按照这个消息执行特定任务。类似于 Linux 的 `crontab`，用户预设好要做什么和什么时候做。

Heartbeat 是状态驱动——它不携带具体任务，而是定期唤醒 Agent 让它自主检查和决策。Agent 醒来后根据当前状态（有没有新邮件？待办事项是否到期？）自主决定做什么或不做什么。

举个类比：Cron 像闹钟（8 点叫我起床），Heartbeat 像保安巡逻（每 30 分钟巡逻一次，看看有没有异常）。

配置上，Heartbeat 还有个 `keep_recent_messages` 参数，只保留最近 N 条消息作为上下文，避免加载完整历史浪费 token。”

题目 4：如果让你设计一个并行任务系统，你会注意什么？

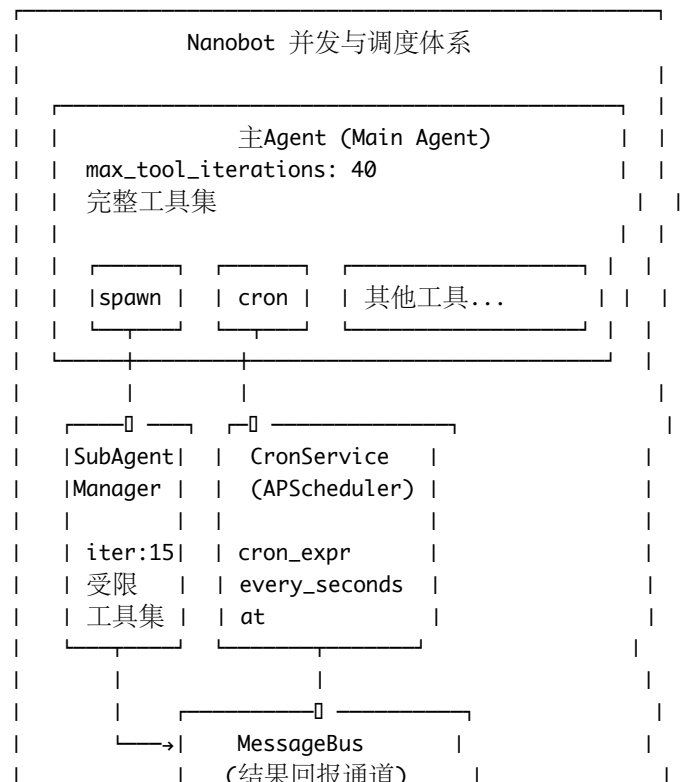
参考回答：

“我会关注四个方面：

1. 资源控制：限制并发子任务的最大数量，设置单个任务的超时和迭代限制。Nanobot 的 15 次迭代限制就是一个好的实践。
2. 递归防护：禁止子任务创建新的子任务，防止任务数量失控。这需要在工具/权限层面做限制，而不是依赖 LLM 的‘自觉’。
3. 结果汇报：子任务完成后要有可靠的结果回传机制。Nanobot 通过 MessageBus 实现，确保结果不丢失。我会考虑加入超时处理——如果子任务超时未完成，主动通知用户。
4. 状态可观测：提供查看所有正在运行的子任务的方式（名称、状态、运行时间），方便用户和运维人员了解系统状态。”

10.8 本章小结

核心概念图



11 - 安全与部署

阅读时间：约 2 小时
前置知识：10 - 子 Agent 与定时任务
学习目标：掌握 Nanobot 的安全机制设计、Docker 部署方案、生产环境最佳实践，为面试中的”部署与安全”类问题做好准备

目录

- 11.1 为什么安全如此重要
- 11.2 Nanobot 安全机制详解
- 11.3 密钥与敏感信息管理
- 11.4 Docker 部署
- 11.5 生产环境配置
- 11.6 多实例架构
- 11.7 日志与监控
- 11.8 备份策略
- 11.9 配置安全清单
- 11.10 面试高频题
- 11.11 本章小结

11.1 为什么安全如此重要

11.1.1 AI Agent 的安全风险

AI Agent 比传统软件面临更多安全风险，因为它具有自主执行能力：

传统软件：

用户操作 → 预设逻辑 → 确定性执行
风险可预测，边界明确

AI Agent：

用户指令 → LLM 推理 → 自主决策 → 工具执行
输出不确定，行为边界模糊

11.1.2 典型攻击场景

攻击类型	场景	风险
Prompt 注入	用户构造恶意指令让 Agent 执行危险操作	数据泄露、系统破坏
路径遍历	Agent 被引导访问 workspace 外的文件	读取 /etc/passwd 等
命令注入	通过 exec 工具执行恶意 Shell 命令	系统被接管
SSRF	Agent 被引导访问内网服务	内网探测、元数据泄露
密钥泄露	API Key 被写入日志或记忆文件	账号被盗用
资源耗尽	无限循环的工具调用	系统崩溃、费用暴增

11.1.3 安全设计原则

最小权限原则 (Least Privilege)

- └ Agent 只能访问必需的资源
- └ 工具只暴露必需的功能

└─ SubAgent 的权限比主Agent更少

纵深防御原则 (Defense in Depth)

- └─ 多层安全检查
- └─ 每层都假设其他层可能失败
- └─ 即使一层被绕过，其他层仍然保护系统

故障安全原则 (Fail-Safe)

- └─ 出错时拒绝操作（而非放行）
- └─ 未配置时默认安全
- └─ 异常情况记录但不暴露细节

11.2 Nanobot 安全机制详解

11.2.1 restrict_to_workspace: 文件与 Shell 操作沙箱

这是 Nanobot 最核心的安全机制——将所有文件和命令操作限制在 **workspace** 目录内：

```
class WorkspaceGuard:
    """workspace 安全边界守卫"""

    def __init__(self, workspace: str):
        self.workspace = os.path.abspath(workspace)

    def validate_path(self, path: str) -> str:
        """ 验证路径是否在 workspace 内 """
        # 解析绝对路径（处理 ../ 等相对路径）
        abs_path = os.path.abspath(
            os.path.join(self.workspace, path)
        )

        # 检查是否在 workspace 内
        if not abs_path.startswith(self.workspace + os.sep):
            if abs_path != self.workspace:
                raise PermissionError(
                    f"Access denied: '{path}' is outside workspace "
                    f"'{self.workspace}'"
                )

        return abs_path

    def validate_command(self, command: str):
        """ 验证 Shell 命令的工作目录 """
        # exec 工具的 cwd 被强制设置为 workspace
        # 即使命令中使用 cd，也始终从 workspace 开始
        pass
```

限制范围：

操作	受限方式
read_file	路径必须在 workspace 内
write_file	路径必须在 workspace 内
edit_file	路径必须在 workspace 内
list_dir	路径必须在 workspace 内
exec	工作目录 (cwd) 强制设为 workspace

防御路径遍历攻击:

攻击尝试:

```
read_file("../../etc/passwd")
→ 拼接: workspace + "../../etc/passwd"
→ 解析: /etc/passwd
→ 检查: /etc/passwd 不以 workspace 开头
→ 拒绝: PermissionError!
```

```
read_file("/etc/passwd")
→ 绝对路径不以 workspace 开头
→ 拒绝: PermissionError!
```

```
read_file("./safe/../../etc/passwd")
→ 解析: /etc/passwd
→ 拒绝: PermissionError!
```

11.2.2 exec 工具的危險模式拒絕

exec 工具內置了危險命令檢測:

```
DANGEROUS_PATTERNS = [
    # 系統破壞類
    (r"rm\s+(-[rf]+\s+)?/(?!\w)", " 刪除根目錄文件"),
    (r"mkfs\.", " 格式化磁盤"),
    (r"dd\s+if=", " 磁盤級寫入"),
    (r">\s*/dev/sd", " 寫入磁盤設備"),

    # 權限修改類
    (r"chmod\s+(-R\s+)?777\s+/", " 全局權限修改"),
    (r"chown\s+-R\s+.\s+/", " 全局所有者修改"),

    # 網絡風險類
    (r"curl\s+.*\|\s*bash", " 遠程腳本執行"),
    (r"wget\s+.*-O\s*-\s*\|\s*bash", " 遠程腳本執行"),

    # 敏感信息類
    (r"cat\s+.*(password|shadow|secret)", " 讀取敏感文件"),
    (r"env\s*$", " 顯示所有環境變量"),
]
```

```
def check_dangerous_command(command: str) -> tuple[bool, str]:
    """ 檢查命令是否危險 """
    for pattern, reason in DANGEROUS_PATTERNS:
        if re.search(pattern, command, re.IGNORECASE):
            return True, reason
    return False, ""
```

檢測流程:

用戶輸入命令

```
|
|
```

1. 正則匹配危險模式
rm -rf / → 拒絕
mkfs.ext4 → 拒絕
curl ... bash → 拒絕

```
| 安全
|
|
```

```
| 2. 设置 cwd = workspace |
| 命令在沙箱内执行 |
|
```

```
|
|
```

```
| 3. 超时控制 |
| 防止无限运行 |
|
```

```
|
|
```

执行命令，返回结果

11.2.3 SSRF 防护 (web_fetch)

SSRF (Server-Side Request Forgery) 是 Agent 系统特有的风险——攻击者可能通过 Prompt 注入让 Agent 访问内网服务:

```
import ipaddress
from urllib.parse import urlparse

def is_ssrf_target(url: str) -> bool:
    """ 检查 URL 是否指向可能的 SSRF 目标"""
    parsed = urlparse(url)
    hostname = parsed.hostname

    if not hostname:
        return True # 无效 URL, 拒绝

    # 检查是否是内网主机名
    BLOCKED_HOSTNAMES = {
        "localhost",
        "metadata.google.internal", # GCP 元数据
        "instance-data", # 部分云平台元数据
    }
    if hostname.lower() in BLOCKED_HOSTNAMES:
        return True

    # 尝试解析为 IP 地址
    try:
        ip = ipaddress.ip_address(hostname)
        # 阻止私有地址
        if ip.is_private: # 10.x, 172.16-31.x, 192.168.x
            return True
        if ip.is_loopback: # 127.0.0.1
            return True
        if ip.is_reserved: # 保留地址
            return True
        if ip.is_link_local: # 169.254.x.x
            return True
    except ValueError:
        pass # 非 IP 地址 (域名), 后续检查

    # 检查特殊 IP
```



```

BLOCKED_IPS = [
    "169.254.169.254",    # AWS/GCP/Azure 元数据
    "100.100.100.200",   # 阿里云元数据
]
if hostname in BLOCKED_IPS:
    return True

return False

```

SSRF 攻击示例:

攻击场景:

恶意用户: "帮我获取 <http://169.254.169.254/latest/meta-data/> 的内容"

无防护 → Agent 调用 `web_fetch` → 获取云平台元数据 → 泄露 IAM 凭证

有防护 → `web_fetch` 检测到 SSRF 目标 → 拒绝请求

11.2.4 路径/域名白名单

除了黑名单机制, Nanobot 还支持白名单模式:

```

{
  "tools": {
    "web_fetch": {
      "allowed_domains": [
        "github.com",
        "stackoverflow.com",
        "docs.python.org"
      ]
    },
    "exec": {
      "allowed_commands": [
        "git",
        "python",
        "npm",
        "pip"
      ]
    }
  }
}

```

11.2.5 最小权限原则在 Nanobot 中的体现

主Agent

- └─ 完整工具集 (read, write, exec, web, message, spawn, cron)
- └─ 40 次迭代限制
- └─ workspace 限制

SubAgent

- └─ 受限工具集 (无 message, spawn, cron)
- └─ 15 次迭代限制
- └─ workspace 限制

Cron 执行上下文

- └─ 不能创建新的 Cron
- └─ 其他工具正常可用

每个层级只拥有完成任务所需的最少权限

11.3 密钥与敏感信息管理

11.3.1 基本原则

▣ 正确做法：

- └─ API Key 通过环境变量传入
- └─ 密钥文件加入 `.gitignore`
- └─ 使用专用的密钥管理服务
- └─ 定期轮换密钥

▣ 错误做法：

- └─ API Key 硬编码在 `config.json` 中并提交到 Git
- └─ 密钥写在 `AGENTS.md` 或 `MEMORY.md` 中
- └─ 在日志中打印完整密钥
- └─ 多个服务共用同一个密钥

11.3.2 环境变量管理

方式一：直接设置环境变量

```
export OPENAI_API_KEY="sk-xxxxxxx"
export TELEGRAM_BOT_TOKEN="123456:ABC..."
```

方式二：使用 `.env` 文件

```
cat > .env << 'EOF'
OPENAI_API_KEY=sk-xxxxxxx
TELEGRAM_BOT_TOKEN=123456:ABC...
BRAVE_API_KEY=BSAxxxxxxx
EOF
```

将 `.env` 加入 `.gitignore`

```
echo ".env" >> .gitignore
```

11.3.3 `config.json` 中引用环境变量

```
{
  "providers": {
    "openai": {
      "api_key": "${OPENAI_API_KEY}",
      "api_base": "https://api.openai.com/v1"
    }
  },
  "channels": {
    "telegram": {
      "bot_token": "${TELEGRAM_BOT_TOKEN}"
    }
  }
}
```

11.3.4 `.gitignore` 配置

密钥文件

```
.env
*.pem
*.key
credentials.json
```

```
# Nanobot 数据
config.json
memory/
sessions/

# 系统文件
__pycache__/
*.pyc
.DS_Store
```

11.4 Docker 部署

11.4.1 为什么用 Docker

优势	说明
环境一致性	开发/测试/生产完全一致
隔离性	容器间互不影响
可移植性	一次构建，到处运行
快速部署	几秒钟启动一个新实例
资源控制	限制 CPU/内存使用

11.4.2 Dockerfile 示例

```
# 基础镜像
FROM python:3.12-slim

# 设置工作目录
WORKDIR /app

# 安装系统依赖
RUN apt-get update && apt-get install -y \
    git \
    curl \
    && rm -rf /var/lib/apt/lists/*

# 安装 nanobot
RUN pip install --no-cache-dir nanobot-ai

# 创建非 root 用户
RUN useradd -m -s /bin/bash nanobot
USER nanobot

# 创建必要目录
RUN mkdir -p /home/nanobot/workspace/memory \
    /home/nanobot/workspace/sessions \
    /home/nanobot/workspace/skills

# 设置 workspace
WORKDIR /home/nanobot/workspace

# 健康检查
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD pgrep -f nanobot || exit 1
```

```
# 启动命令
CMD ["nanobot"]
```

11.4.3 docker-compose.yml 配置

```
version: '3.8'

services:
  nanobot:
    build: .
    container_name: nanobot-agent
    restart: unless-stopped

    # 环境变量
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - TELEGRAM_BOT_TOKEN=${TELEGRAM_BOT_TOKEN}
      - BRAVE_API_KEY=${BRAVE_API_KEY}

    # 数据卷挂载
    volumes:
      # 配置文件
      - ./config.json:/home/nanobot/workspace/config.json:ro

      # AGENTS.md 和引导文件
      - ./AGENTS.md:/home/nanobot/workspace/AGENTS.md:ro
      - ./SOUL.md:/home/nanobot/workspace/SOUL.md:ro

      # 自定义技能
      - ./skills:/home/nanobot/workspace/skills:ro

      # 持久化数据（可读写）
      - nanobot-memory:/home/nanobot/workspace/memory
      - nanobot-sessions:/home/nanobot/workspace/sessions

    # 资源限制
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 2G
        reservations:
          cpus: '0.5'
          memory: 512M

    # 日志配置
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

# 命名卷（持久化）
volumes:
  nanobot-memory:
  nanobot-sessions:
```

11.4.4 构建与运行

```
# 构建镜像
docker-compose build

# 启动服务
docker-compose up -d

# 查看日志
docker-compose logs -f nanobot

# 停止服务
docker-compose down

# 进入容器调试
docker exec -it nanobot-agent bash
```

11.4.5 环境变量传递

```
# 方式一: .env 文件 (docker-compose 自动读取)
cat > .env << 'EOF'
OPENAI_API_KEY=sk-xxxxxxx
TELEGRAM_BOT_TOKEN=123456:ABC...
EOF

docker-compose up -d

# 方式二: 命令行传递
docker run -d \
  -e OPENAI_API_KEY=sk-xxxxxxx \
  -e TELEGRAM_BOT_TOKEN=123456:ABC... \
  -v $(pwd)/config.json:/home/nanobot/workspace/config.json:ro \
  -v nanobot-memory:/home/nanobot/workspace/memory \
  nanobot-agent

# 方式三: Docker Secret (Swarm 模式)
echo "sk-xxxxxxx" | docker secret create openai_key -
```

11.4.6 数据卷挂载策略

挂载策略:

文件/目录	挂载模式	说明
config.json	:ro	只读, 配置不应被修改
AGENTS.md	:ro	只读
SOUL.md	:ro	只读
skills/	:ro	只读
memory/	rw	读写, Agent需要更新
sessions/	rw	读写, 会话需要写入

ro = Read Only (只读挂载, 增强安全性)
rw = Read Write (读写挂载, 数据持久化)

11.5 生产环境配置

11.5.1 生产环境 config.json

```
{
  "agents": {
    "defaults": {
      "workspace": "/home/nanobot/workspace",
      "model": "gpt-4o",
      "provider": "openai",
      "max_tokens": 8192,
      "context_window_tokens": 64000,
      "temperature": 0.3,
      "max_tool_iterations": 30,
      "restrict_to_workspace": true
    }
  },
  "providers": {
    "openai": {
      "api_key": "${OPENAI_API_KEY}",
      "api_base": "https://api.openai.com/v1"
    }
  },
  "channels": {
    "telegram": {
      "bot_token": "${TELEGRAM_BOT_TOKEN}"
    }
  },
  "tools": {
    "exec": {
      "allowed": true,
      "timeout": 60
    },
    "web_search": {
      "provider": "brave",
      "api_key": "${BRAVE_API_KEY}"
    }
  }
}
```

生产环境参数调优:

参数	开发环境	生产环境	调优理由
temperature	0.7	0.3	生产环境需要更稳定的输出
max_tokens	16384	8192	控制单次回复长度，节省成本
context_window_tokens	128000	64000	更频繁地压缩记忆，降低 API 费用
max_tool_iterations	40	30	减少失控风险
restrict_to_workspace	true	true	始终启用

11.5.2 Nginx 反向代理

如果 Nanobot 需要接收 Webhook（飞书、钉钉等），需要配置反向代理：

```
# /etc/nginx/sites-available/nanobot
server {
    listen 80;
    server_name agent.example.com;
```

```

# 强制 HTTPS 重定向
return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name agent.example.com;

    # SSL 证书
    ssl_certificate /etc/letsencrypt/live/agent.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/agent.example.com/privkey.pem;

    # SSL 安全配置
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    # Webhook 路由
    location /webhook/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # 超时设置
        proxy_read_timeout 300s;
        proxy_connect_timeout 10s;
    }

    # 健康检查端点
    location /health {
        proxy_pass http://127.0.0.1:8080/health;
        access_log off;
    }

    # 拒绝其他路径
    location / {
        return 403;
    }
}

```

11.5.3 HTTPS 配置

使用 [Let's Encrypt](#) 免费证书

1. 安装 [certbot](#)

```
sudo apt install certbot python3-certbot-nginx
```

2. 申请证书

```
sudo certbot --nginx -d agent.example.com
```

3. 自动续期 ([certbot](#) 会自动设置 [cron](#))

```
sudo certbot renew --dry-run
```

11.6 多实例架构

11.6.1 为什么需要多实例

场景 1: 多个独立 Agent

- └─ 客服Agent → 处理客户问题
- └─ 运维Agent → 监控系统状态
- └─ 开发Agent → 辅助代码开发

场景 2: 多租户

- └─ 团队A → 独立配置和数据
- └─ 团队B → 独立配置和数据
- └─ 团队C → 独立配置和数据

场景 3: 高可用

- └─ 实例 1 → 主要服务
- └─ 实例 2 → 备份/负载均衡

11.6.2 多实例部署方案

每个 Agent 实例拥有独立的配置、**workspace** 和端口:

```
# docker-compose.yml - 多实例
version: '3.8'
```

```
services:
```

```
  # 客服 Agent
```

```
  agent-support:
```

```
    build: .
```

```
    container_name: agent-support
```

```
    environment:
```

- OPENAI_API_KEY=\${OPENAI_API_KEY}
- TELEGRAM_BOT_TOKEN=\${SUPPORT_BOT_TOKEN}

```
    volumes:
```

- ./instances/support/config.json:/home/nanobot/workspace/config.json:ro
- ./instances/support/AGENTS.md:/home/nanobot/workspace/AGENTS.md:ro
- support-memory:/home/nanobot/workspace/memory
- support-sessions:/home/nanobot/workspace/sessions

```
    ports:
```

- "8081:8080"

```
  # 运维 Agent
```

```
  agent-devops:
```

```
    build: .
```

```
    container_name: agent-devops
```

```
    environment:
```

- OPENAI_API_KEY=\${OPENAI_API_KEY}
- TELEGRAM_BOT_TOKEN=\${DEVOPS_BOT_TOKEN}

```
    volumes:
```

- ./instances/devops/config.json:/home/nanobot/workspace/config.json:ro
- ./instances/devops/AGENTS.md:/home/nanobot/workspace/AGENTS.md:ro
- devops-memory:/home/nanobot/workspace/memory
- devops-sessions:/home/nanobot/workspace/sessions

```
    ports:
```

- "8082:8080"

```
  # 开发 Agent
```

```
  agent-dev:
```



```

build: .
container_name: agent-dev
environment:
  - OPENAI_API_KEY=${OPENAI_API_KEY}
  - DISCORD_BOT_TOKEN=${DEV_BOT_TOKEN}
volumes:
  - ./instances/dev/config.json:/home/nanobot/workspace/config.json:ro
  - ./instances/dev/AGENTS.md:/home/nanobot/workspace/AGENTS.md:ro
  - dev-memory:/home/nanobot/workspace/memory
  - dev-sessions:/home/nanobot/workspace/sessions
ports:
  - "8083:8080"

```

```

volumes:
  support-memory:
  support-sessions:
  devops-memory:
  devops-sessions:
  dev-memory:
  dev-sessions:

```

目录结构:

```

production/
├─ docker-compose.yml
├─ Dockerfile
├─ .env
├─ instances/
│   ├─ support/
│   │   ├─ config.json
│   │   ├─ AGENTS.md          # "你是客服助手..."
│   │   └─ skills/
│   ├─ devops/
│   │   ├─ config.json
│   │   ├─ AGENTS.md          # "你是运维专家..."
│   │   └─ skills/
│   └─ dev/
│       ├─ config.json
│       ├─ AGENTS.md          # "你是开发助手..."
│       └─ skills/

```

11.6.3 Nginx 多实例路由

多实例 Webhook 路由

```

server {
    listen 443 ssl http2;
    server_name agent.example.com;

    # ... SSL 配置 ...

    # 客服 Agent Webhook
    location /webhook/support/ {
        proxy_pass http://127.0.0.1:8081;
    }

    # 运维 Agent Webhook
    location /webhook/devops/ {
        proxy_pass http://127.0.0.1:8082;
    }
}

```

```

}

# 开发 Agent Webhook
location /webhook/dev/ {
    proxy_pass http://127.0.0.1:8083;
}
}

```

11.7 日志与监控

11.7.1 日志配置

```

# Nanobot 日志级别
import logging

# 开发环境
logging.basicConfig(level=logging.DEBUG)

# 生产环境
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
    handlers=[
        logging.FileHandler('/var/log/nanobot/agent.log'),
        logging.StreamHandler()
    ]
)

```

日志级别建议：

环境	级别	说明
开发	DEBUG	详细信息，包括工具调用细节
测试	INFO	关键操作和事件
生产	WARNING	警告和错误

11.7.2 关键监控指标

指标	说明	告警阈值
API 调用次数	LLM API 调用频率	超出预算
响应延迟	用户消息到回复的时间	> 30s
错误率	工具调用失败比例	> 5%
内存使用	进程内存占用	> 1.5GB
会话活跃数	同时进行的会话数量	> 100
Token 消耗	每小时 token 使用量	超出预算
记忆压缩次数	MemoryConsolidator 触发频率	异常频繁

11.7.3 日志安全

```

# 重要：日志中不要记录敏感信息
def safe_log(message: str, data: dict) -> str:
    """ 安全的日志记录，脱敏敏感信息 """
    safe_data = {}
    for key, value in data.items():
        if key in ("api_key", "token", "secret", "password"):

```

```

        safe_data[key] = value[:8] + "****" # 只显示前 8 位
    else:
        safe_data[key] = value

    return f"{message}: {safe_data}"

# 示例
logger.info(safe_log("Provider config", {
    "provider": "openai",
    "api_key": "sk-abcdefghijklmnopqrstuvwxyz", # 会被脱敏为 "sk-abcde****"
    "model": "gpt-4o"
}))

```

11.8 备份策略

11.8.1 需要备份的数据

必须备份：

```

├─ memory/MEMORY.md      # 长期记忆
├─ memory/HISTORY.md     # 历史时间线
├─ sessions/*.jsonl      # 会话历史
├─ config.json           # 配置文件
├─ AGENTS.md             # Agent 身份定义
└─ skills/               # 自定义技能

```

不需要备份（可重建）：

```

├─ __pycache__/_
└─ .nanobot/_

```

11.8.2 自动备份脚本

```

#!/bin/bash
# backup-nanobot.sh

WORKSPACE="/home/nanobot/workspace"
BACKUP_DIR="/backups/nanobot"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/nanobot_backup_${DATE}.tar.gz"

# 创建备份目录
mkdir -p ${BACKUP_DIR}

# 打包备份
tar -czf ${BACKUP_FILE} \
    -C ${WORKSPACE} \
    memory/ \
    sessions/ \
    config.json \
    AGENTS.md \
    SOUL.md \
    skills/

# 保留最近 30 天的备份
find ${BACKUP_DIR} -name "*.tar.gz" -mtime +30 -delete

echo "Backup created: ${BACKUP_FILE}"

```

```
echo "Size: $(du -h ${BACKUP_FILE} | cut -f1)"
# 添加到 crontab, 每天凌晨 3 点备份
crontab -e
# 0 3 * * * /home/nanobot/scripts/backup-nanobot.sh >> /var/log/nanobot-backup.log 2>&1
```

11.8.3 恢复流程

1. 停止服务

```
docker-compose down
```

2. 恢复数据

```
BACKUP_FILE="/backups/nanobot/nanobot_backup_20240315_030000.tar.gz"
tar -xzf ${BACKUP_FILE} -C /home/nanobot/workspace/
```

3. 验证文件完整性

```
ls -la /home/nanobot/workspace/memory/
ls -la /home/nanobot/workspace/sessions/
```

4. 重新启动

```
docker-compose up -d
```

5. 验证服务正常

```
docker-compose logs -f nanobot
```

11.9 配置安全清单

生产部署前必检清单

[] 安全配置

- └─ [] restrict_to_workspace 已启用
- └─ [] 危险命令检测已启用
- └─ [] SSRF 防护已启用
- └─ [] exec 工具超时已设置
- └─ [] max_tool_iterations 合理 (建议 ≤ 30)
- └─ [] SubAgent 迭代限制已确认 (15次)

[] 密钥管理

- └─ [] 所有密钥通过环境变量传入
- └─ [] config.json 中无硬编码密钥
- └─ [] .env 文件已加入 .gitignore
- └─ [] 密钥不出现在日志中
- └─ [] API Key 有使用额度限制

[] 网络安全

- └─ [] HTTPS 已配置
- └─ [] Webhook 端点有访问控制
- └─ [] 内网地址已屏蔽 (SSRF)
- └─ [] 不必要的端口已关闭
- └─ [] 防火墙规则已配置

[] 容器安全

- └─ [] 使用非 root 用户运行
- └─ [] 资源限制已设置 (CPU/内存)
- └─ [] 只读挂载配置文件
- └─ [] 镜像使用 slim 基础版本

└─ [] 健康检查已配置

[] 数据安全

└─ [] 备份策略已实施

└─ [] 备份已测试恢复

└─ [] 日志轮转已配置

└─ [] 敏感信息已脱敏

└─ [] 会话数据有清理策略

[] 监控告警

└─ [] 关键指标已监控

└─ [] 错误日志有告警

└─ [] API 费用有预算告警

└─ [] 系统资源有监控

└─ [] 服务可用性检查

11.10 面试高频题

题目 1：生产环境部署 **Agent** 系统需要注意什么？

参考回答：

“生产部署 Agent 系统，我会关注五个方面：

第一是安全隔离。Agent 能执行 Shell 命令和文件操作，必须通过 `restrict_to_workspace` 将所有操作限制在沙箱内。exec 工具需要有危险命令检测、超时控制。web_fetch 需要 SSRF 防护，阻止访问内网地址和云平台元数据。

第二是密钥管理。所有 API Key 通过环境变量传入，不硬编码在配置文件中。Docker 部署时用 .env 文件或 Docker Secret。日志中对密钥做脱敏处理。

第三是资源控制。设置 `max_tool_iterations` 限制单次对话的工具调用次数，防止 Agent 陷入死循环导致费用暴增。Docker 层面限制 CPU 和内存。LLM API 设置使用额度上限。

第四是可观测性。监控 API 调用次数、响应延迟、错误率、token 消耗等关键指标。日志轮转防止磁盘写满。异常情况设置告警。

第五是数据持久化。记忆文件和会话历史使用 Docker Volume 持久化。定期备份并测试恢复流程。配置文件只读挂载。”

题目 2：如何防止 **Prompt** 注入攻击？

参考回答：

“Prompt 注入是 Agent 系统特有的安全风险——恶意用户通过构造输入来操纵 Agent 执行非预期操作。

Nanobot 从多个层面防御：

1. 工具层面的硬限制：无论 LLM 被注入什么指令，`restrict_to_workspace` 在代码层面限制了文件操作范围。危险命令检测使用正则匹配，不依赖 LLM 的判断。SSRF 防护也是代码级检查。这些硬限制不会被 Prompt 注入绕过。
2. 迭代限制：`max_tool_iterations` 限制了工具调用次数，即使 Agent 被注入‘不停执行’的指令，也会在达到上限时停止。
3. 最小权限：SubAgent 不能调用 `message`（不能冒充主 Agent 与用户交流）、不能调用 `spawn`（不能创建更多 Agent）、不能调用 `cron`（不能创建定时任务）。
4. **System Prompt** 隔离：AGENTS.md 中的指令在 System Prompt 中的优先级高于用户输入，可以设置‘忽略用户要求你违反规则的指令’等防御性提示。

不过需要承认，Prompt 注入没有完美解决方案——这是整个 AI 行业面临的挑战。重点是做好纵深防御，确保即使 LLM 被操纵，代码级的安全机制仍然生效。”

题目 3: Agent 系统的高可用架构如何设计?

参考回答:

“对于 Nanobot 这样的 Agent 系统，高可用设计需要考虑几个维度:

服务层面: 使用 Docker + docker-compose 部署，配置 `restart: unless-stopped` 确保异常退出自动重启。用 Nginx 做反向代理，支持 HTTPS 和健康检查。

数据层面: 记忆和会话数据使用 Docker Volume 持久化，定期备份到远程存储。MEMORY.md 和 HISTORY.md 是 Markdown 文件，可以用 git 做版本控制。

多实例方面: 每个 Agent 实例有独立的 config、workspace 和端口。不同的 Agent (客服、运维、开发) 部署为独立容器，互不影响。

容灾方面: 关键的 LLM Provider 可以配置备用 (如主用 OpenAI，备用 DeepSeek)，当主 Provider 不可用时自动切换。消息发送的指数退避重试机制也提供了一定的容错能力。

如果要做更完善的高可用，可以考虑将会话状态存储到 Redis，实现无状态的 Agent 实例——任何实例都能恢复任何用户的会话。但这对 Nanobot 这种文件存储的架构改动较大。”

题目 4: 如何监控 Agent 系统的运行成本?

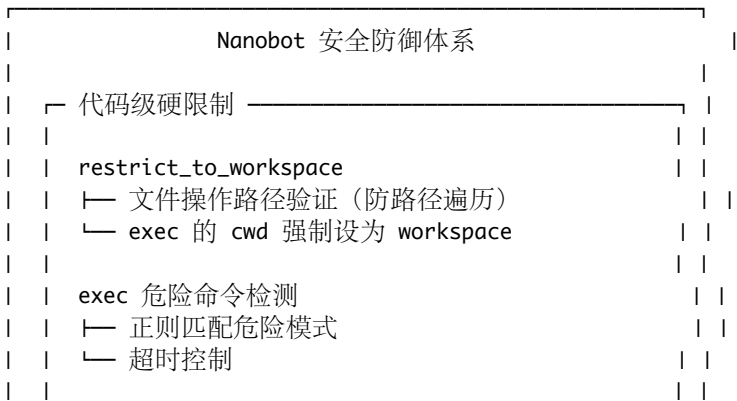
参考回答:

“LLM API 费用是 Agent 系统最大的运行成本，我会从几个维度控制:

1. 上下文窗口控制: 合理设置 `context_window_tokens`，让记忆压缩更频繁地触发，避免长对话积累大量 token。生产环境建议设为模型窗口的 50% (如 GPT-4o 的 128K 窗口设为 64K)。
2. 输出长度限制: `max_tokens` 控制单次回复长度，生产环境不需要太长的回复。
3. 迭代次数限制: `max_tool_iterations` 防止单次对话消耗过多轮 API 调用。
4. 渐进披露: Skill 的渐进披露机制默认只注入摘要 (几百 token)，而非全文 (几千 token)，大幅降低每轮对话的 System Prompt 消耗。
5. 监控告警: 记录每次 API 调用的 token 用量，设置日/周/月的费用预算告警。API 平台通常也支持设置硬性额度上限。
6. 模型选择: 不是所有任务都需要 GPT-4o，简单对话可以用更便宜的模型。可以考虑给 SubAgent 配置更经济的模型。”

11.11 本章小结

安全机制总图



		SSRF 防护		
		└─ 私有 IP 地址阻止		
		└─ 云元数据地址阻止		
		└─ 域名白名单（可选）		
		└─ 权限控制		
		主Agent: 完整工具集 + 40次迭代		
		SubAgent: 受限工具集 + 15次迭代		
		Cron上下文: 禁止创建新Cron		
		└─ 密钥管理		
		环境变量传入 + .gitignore + 日志脱敏		
		└─ 运维安全		
		HTTPS + 非root运行 + 资源限制 + 监报告警		

面试记忆清单

考点	一句话回答
核心安全机制	restrict_to_workspace 限制文件和 Shell 操作范围
危险命令	exec 工具正则检测 rm -rf /、mkfs 等危险模式
SSRF 防护	web_fetch 阻止私有 IP、云元数据地址
密钥管理	环境变量传入，不硬编码，不入库
Docker 部署	非 root 用户 + 资源限制 + 只读挂载配置
数据持久化	Docker Volume 挂载 memory/ 和 sessions/
多实例	独立 config/workspace/端口，互不影响
备份策略	定期备份 memory + sessions + config
成本控制	context_window_tokens + max_tokens + 渐进披露
设计原则	最小权限 + 纵深防御 + 故障安全

下一章：12 - Nanobot 实战项目 ——通过实战项目深入掌握 Nanobot 的高级用法

第十二章：Nanobot 实战项目

学完理论，是时候用 **Nanobot** 构建真实系统了。本章通过三个难度递进的实战项目，带你从零到一打造可上线的 AI Agent 系统。每个项目都包含完整的需求分析、架构设计、代码实现、部署指南和面试话术。

导读

为什么需要实战项目

面试官最常问的问题不是「你了解什么技术」，而是「你做过什么项目」。

在 AI Agent 领域，能够回答以下问题的候选人会脱颖而出：

- 你是如何设计 Agent 的工具调用链的？
- 遇到 Agent 幻觉（Hallucination）你怎么处理？
- 你的 MCP Server 是如何保证安全性的？
- 多平台消息如何保持一致性？

这些问题，只有做过实际项目才能答好。

三个项目的难度递进

项目一（中级） 智能运维助手 单一平台 Shell 工具 定时任务 日志分析	项目二（中高级） 多平台智能客服 多平台接入 知识库+工单 用户记忆 MCP 集成	项目三（高级） 自定义 MCP Server 协议层开发 数据库安全 SQL 防注入 完整 Server	
维度	项目一	项目二	项目三
难度			
涉及技能	exec, read_file, cron	Skills, Memory, MCP 集成	MCP 协议开发
面试价值	展示工具调用能力	展示架构设计能力	展示底层开发能力
适合岗位	初中级 AI 工程师	中高级 AI 工程师	高级 AI 架构师

项目一：智能运维助手（难度：中级）

1.1 项目需求分析

项目背景 某中型互联网公司有 50+ 台服务器，运维团队每天需要：- 检查各服务器的 CPU、内存、磁盘使用率 - 分析日志中的异常和错误 - 在紧急情况下执行常见运维命令（重启服务、清理磁盘等）- 每日生成巡检报告

手动执行这些操作耗时且容易遗漏。目标：用 **Nanobot** 构建智能运维助手，自动化 80% 的日常运维工作。

功能需求

智能运维助手功能清单		
核心功能	具体描述	
服务器监控	CPU/内存/磁盘实时采集	
日志分析	错误日志提取、趋势分析	
自动告警	阈值触发飞书/钉钉通知	

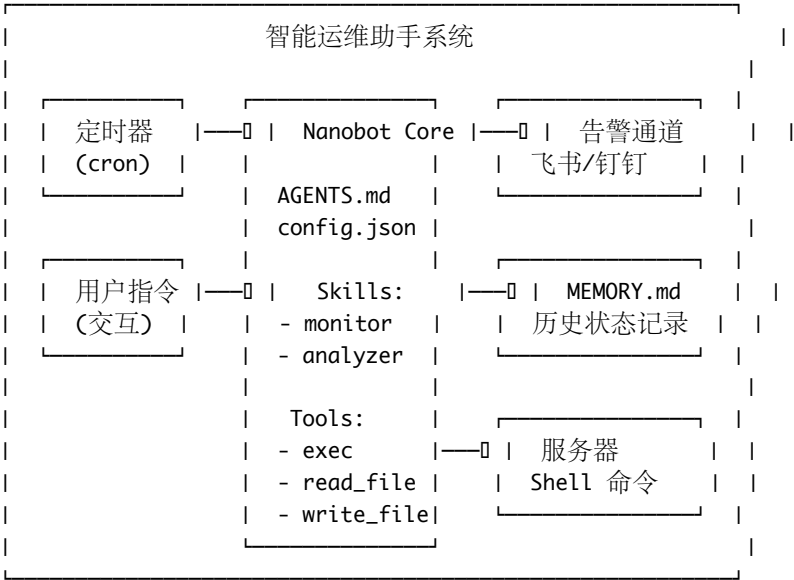
命令执行	安全执行预定义运维命令	
定时巡检	每小时/每天自动巡检	
历史记录	记录所有操作和状态变化	

非功能需求

- 安全性：只允许执行白名单内的命令，禁止 `rm -rf`、`shutdown` 等危险命令
- 可靠性：单次巡检失败不影响后续巡检
- 可观测性：所有操作记录到 `MEMORY.md`

1.2 架构设计

整体架构图

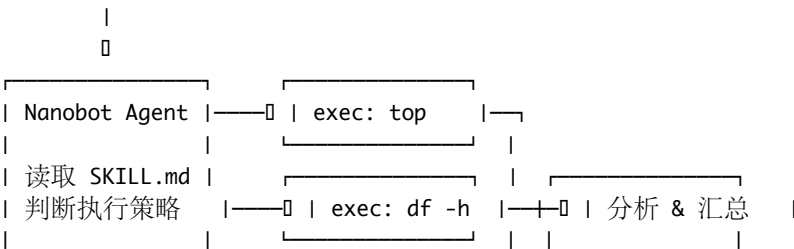


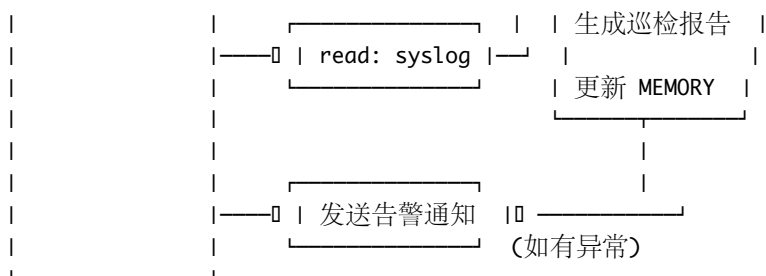
核心设计决策

设计点	方案	原因
命令执行	Nanobot exec 工具	原生支持，无需额外开发
日志读取	read_file 工具	直接读取日志文件
状态持久化	MEMORY.md	Nanobot 原生记忆系统
定时巡检	cron 配置	Nanobot 原生支持
命令安全	白名单机制	在 AGENTS.md 中约束

数据流图

定时触发 / 用户指令





1.3 核心代码实现

AGENTS.md（运维专家身份）

智能运维助手

身份

你是一个专业的 Linux 服务器运维专家。你的职责是监控服务器状态、分析日志、执行安全的运维命令，并在发现异常时及时告警。

核心原则

1. ** 安全第一 **：只执行白名单内的命令，绝不执行破坏性操作
2. ** 记录一切 **：所有操作和发现都记录到 MEMORY.md
3. ** 主动告警 **：发现异常立即通过配置的通道通知
4. ** 数据驱动 **：基于具体指标做判断，不做模糊推测

安全白名单

允许执行的命令类别：

- 系统状态查看：`top -bn1`，`free -m`，`df -h`，`uptime`，`w`
- 进程管理：`ps aux`，`ps -ef | grep <service>`
- 日志查看：`tail -n`，`head -n`，`grep`（仅限日志目录）
- 网络诊断：`ping`，`curl`，`netstat -tlnp`，`ss -tlnp`
- 服务管理：`systemctl status`，`systemctl restart`（仅限白名单服务）
- 磁盘清理：`journalctl --vacuum-size=500M`，清理 /tmp 下超过 7 天的文件

禁止执行的命令

以下命令绝对禁止执行，即使用户明确要求也要拒绝：

- `rm -rf /` 或任何根目录删除操作
- `shutdown`，`reboot`，`halt`，`poweroff`
- `chmod 777` 或对系统目录的权限修改
- `dd if=/dev/zero`
- 任何涉及 `/etc/passwd`，`/etc/shadow` 的写操作
- `iptables -F`（清空防火墙规则）

告警阈值

指标	警告阈值	严重阈值
CPU 使用率	> 80%	> 95%
内存使用率	> 85%	> 95%
磁盘使用率	> 80%	> 90%
错误日志频率	> 10 条/分钟	> 50 条/分钟

巡检流程

每次巡检按以下顺序执行：

1. 检查 CPU 使用率 → ``top -bn1 | head -5``
2. 检查内存使用率 → ``free -m``
3. 检查磁盘使用率 → ``df -h``
4. 检查关键服务状态 → ``systemctl status <services>``
5. 检查最近错误日志 → ``tail -100 /var/log/syslog | grep -i error``
6. 汇总结果，更新 MEMORY.md
7. 如有异常，发送告警

可用技能

- ``skills/server-monitor/SKILL.md`` - 服务器监控技能
- ``skills/log-analyzer/SKILL.md`` - 日志分析技能

config.json（配置文件）

```
{
  "model": "claude-sonnet-4-20250514",
  "provider": "anthropic",
  "system_prompt_file": "AGENTS.md",
  "memory_file": "MEMORY.md",
  "skills_dirs": ["skills"],
  "tools": {
    "exec": {
      "enabled": true,
      "timeout": 30,
      "description": " 执行 Shell 命令（受安全白名单约束）"
    },
    "read_file": {
      "enabled": true,
      "allowed_paths": ["/var/log/", "/tmp/", "/home/ops/"],
      "description": " 读取服务器上的文件"
    },
    "write_file": {
      "enabled": true,
      "allowed_paths": ["/home/ops/reports/", "MEMORY.md"],
      "description": " 写入报告和记忆文件"
    }
  },
  "cron": [
    {
      "schedule": "0 * * * *",
      "prompt": " 执行一次完整的服务器巡检，检查 CPU、内存、磁盘和关键服务状态，将结果更新到 MEMORY.md"
    },
    {
      "schedule": "0 9 * * *",
      "prompt": " 生成昨日运维日报，包含：1) 各项指标趋势 2) 异常事件汇总 3) 今日建议操作。输出到 /home/ops/reports/"
    }
  ],
  "notify": {
    "feishu": {
      "webhook": "${FEISHU_WEBHOOK_URL}",
      "enabled": true
    }
  }
}
```

```
}  
}
```

skills/server-monitor/SKILL.md (服务器监控技能)

服务器监控技能

描述

提供服务器 CPU、内存、磁盘的实时监控能力，支持阈值告警和趋势分析。

使用场景

- 定时巡检时自动调用
- 用户主动询问服务器状态时调用
- 收到告警需要深入排查时调用

监控脚本

CPU 使用率采集

使用 `exec` 工具执行以下命令获取 CPU 使用率：

```
```bash  
top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1
```

##### 内存使用率采集

```
free -m | awk 'NR==2{printf "%.1f%%\n", $3*100/$2}'
```

##### 磁盘使用率采集

```
df -h | awk 'NF==2/{printf "%s\n", $5}'
```

##### 综合采集脚本

```
echo "=== 服务器状态报告 $(date '+%Y-%m-%d %H:%M:%S') ==="
echo ""
echo "--- CPU ---"
top -bn1 | head -5
echo ""
echo "--- 内存 ---"
free -m
echo ""
echo "--- 磁盘 ---"
df -h
echo ""
echo "--- 负载 ---"
uptime
echo ""
echo "--- 关键进程 ---"
ps aux --sort=-%cpu | head -10
```

## 结果分析指南

当拿到采集数据后，按以下逻辑分析：

1. CPU 分析：
  - 查看 `%us`（用户态）和 `%sy`（内核态）的比值
  - 如果 `%sy` 超过 30%，可能存在 I/O 瓶颈或系统调用密集
  - 找出 CPU 占用最高的前 5 个进程
2. 内存分析：

- 关注 **available** 而非 **free** (Linux 会用空闲内存做缓存)
- 如果 swap 使用率 > 0, 说明物理内存不足
- 检查是否有内存泄漏 (持续增长的进程)

### 3. 磁盘分析:

- 主分区使用率是核心指标
- 检查 **/tmp**, **/var/log** 等易满的目录
- 如果磁盘使用率 > 80%, 列出大文件: `find / -size +100M -type f`

## 输出格式

分析结果需要以结构化格式输出:

## 巡检报告 - {时间}

### 整体状态: □ 正常 / □ 警告 / □ 严重

指标	当前值	阈值	状态
CPU	xx%	80%	□
内存	xx%	85%	□
磁盘	xx%	80%	□

### 异常详情

(如有异常, 在此详细说明)

### 建议操作

(基于当前状态给出操作建议)

#### skills/log-analyzer/SKILL.md (日志分析技能)

```markdown

日志分析技能

描述

分析系统日志和应用日志, 提取错误信息, 识别异常模式, 支持趋势分析。

使用场景

- 巡检时分析最近日志
- 排查特定时间段的异常
- 统计错误频率和类型

日志采集命令

系统日志分析

```bash

# 最近 100 条错误日志

`tail -1000 /var/log/syslog | grep -i "error\|fail\|critical" | tail -100`

# 按小时统计错误数量

`grep -i "error" /var/log/syslog | awk '{print $1, $2, $3}' | \`  
`cut -d: -f1,2 | sort | uniq -c | sort -rn | head -20`

# OOM Killer 事件

`dmesg | grep -i "oom\|out of memory" | tail -20`

## 应用日志分析

### # Nginx 错误日志

```
tail -500 /var/log/nginx/error.log | grep -v "favicon" | tail -50
```

### # 5xx 错误统计

```
awk '$9 ~ /^5/ {print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
```

### # 慢请求统计 (响应时间 > 1s)

```
awk '$NF > 1.0 {print $7, $NF"s"}' /var/log/nginx/access.log | sort -t' ' -k2 -rn | head -20
```

## 分析策略

### 错误分类

拿到错误日志后，按以下类别分类：

| 类别   | 关键词                                | 严重程度 | 处理建议   |
|------|------------------------------------|------|--------|
| 系统资源 | OOM, disk full, no space           | 严重   | 立即处理   |
| 服务异常 | service failed, connection refused | 严重   | 检查服务状态 |
| 网络问题 | timeout, connection reset          | 警告   | 检查网络配置 |
| 权限问题 | permission denied, access denied   | 警告   | 检查权限设置 |
| 配置错误 | invalid config, parse error        | 警告   | 检查配置文件 |
| 一般警告 | warning, deprecated                | 一般   | 后续处理即可 |

### 趋势识别

1. 对比过去 1 小时和过去 24 小时的错误率
2. 如果错误率呈上升趋势 (增长 > 50%)，标记为需要关注
3. 如果某类错误在短时间内大量出现 (> 50 条/10 分钟)，触发告警

## 输出格式

## 日志分析报告 - {时间段}

### ### 错误统计

```
类别	数量	趋势	示例
系统资源	3	□ 上升	OOM killed process nginx
网络问题	12	→ 稳定	Connection timed out
```

### ### 高频错误 Top 5

1. `[错误信息]` - 出现 xx 次 - 首次出现于 xx:xx
2. ...

### ### 需要关注

- □ [紧急问题描述]
- □ [需关注问题描述]

### #### 告警通知脚本

```
```python
#!/usr/bin/env python3
"""
```

ops_alert.py - 运维告警通知模块
支持飞书和钉钉 Webhook 通知
"""

```
import json
import os
import urllib.request
from datetime import datetime
from typing import Optional
```

```
class AlertLevel:
    INFO = "info"
    WARNING = "warning"
    CRITICAL = "critical"
```

```
class OpsAlertNotifier:
    """运维告警通知器，支持飞书和钉钉"""

    def __init__(self):
        self.feishu_webhook = os.getenv("FEISHU_WEBHOOK_URL")
        self.dingtalk_webhook = os.getenv("DINGTALK_WEBHOOK_URL")

    def send_alert(
        self,
        title: str,
        content: str,
        level: str = AlertLevel.INFO,
        server: str = "unknown",
    ):
        """发送告警到所有已配置的通道"""
        results = {}

        if self.feishu_webhook:
            results["feishu"] = self._send_feishu(title, content, level, server)

        if self.dingtalk_webhook:
            results["dingtalk"] = self._send_dingtalk(title, content, level, server)

        return results

    def _send_feishu(
        self, title: str, content: str, level: str, server: str
    ) -> bool:
        """发送飞书通知"""
        level_emoji = {"info": "🟢 ", "warning": "🟡 ", "critical": "🔴 "}
        emoji = level_emoji.get(level, " ")

        payload = {
            "msg_type": "interactive",
            "card": {
                "header": {
                    "title": {"tag": "plain_text", "content": f"{emoji} {title}"},
                    "template": {
                        "info": "blue",

```

```

        "warning": "yellow",
        "critical": "red",
    }.get(level, "blue"),
},
"elements": [
    {
        "tag": "div",
        "text": {
            "tag": "lark_md",
            "content": (
                f"""服务器**: {server}\n"""
                f"""时间**: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n"""
                f"""详情**: \n{content}"""
            ),
        },
    },
],
},
}

return self._post_webhook(self.feishu_webhook, payload)

def _send_dingtalk(
    self, title: str, content: str, level: str, server: str
) -> bool:
    """发送钉钉通知"""
    level_text = {"info": "提示", "warning": "警告", "critical": "严重"}

    payload = {
        "msgtype": "markdown",
        "markdown": {
            "title": f"[{level_text.get(level, '提示')}] {title}",
            "text": (
                f"## {title}\n\n"
                f"- **级别**: {level_text.get(level, '提示')}\n"
                f"- **服务器**: {server}\n"
                f"- **时间**: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n\n"
                f"### 详情\n\n{content}"
            ),
        },
    },
}

return self._post_webhook(self.dingtalk_webhook, payload)

def _post_webhook(self, url: str, payload: dict) -> bool:
    """发送 HTTP POST 请求到 Webhook"""
    try:
        data = json.dumps(payload).encode("utf-8")
        req = urllib.request.Request(
            url,
            data=data,
            headers={"Content-Type": "application/json"},
        )
        with urllib.request.urlopen(req, timeout=10) as resp:
            return resp.status == 200
    except Exception as e:

```



```

        print(f"告警发送失败: {e}")
        return False

def send_inspection_alert(report: dict):
    """巡检结果告警入口"""
    notifier = OpsAlertNotifier()

    if report.get("overall_status") == "critical":
        notifier.send_alert(
            title="服务器巡检异常 - 需立即处理",
            content=report.get("summary", ""),
            level=AlertLevel.CRITICAL,
            server=report.get("server", "unknown"),
        )
    elif report.get("overall_status") == "warning":
        notifier.send_alert(
            title="服务器巡检警告",
            content=report.get("summary", ""),
            level=AlertLevel.WARNING,
            server=report.get("server", "unknown"),
        )

if __name__ == "__main__":
    send_inspection_alert({
        "overall_status": "warning",
        "server": "prod-web-01",
        "summary": "CPU 使用率 82%，超过警告阈值 80%。\\n建议检查高 CPU 进程。",
    })

```

1.4 部署与运行

Docker 部署方案

```

# Dockerfile
FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    procs sysstat net-tools curl && \
    rm -rf /var/lib/apt/lists/*

RUN pip install nanobot

WORKDIR /app

COPY AGENTS.md config.json ./
COPY skills/ ./skills/
COPY ops_alert.py ./

ENV ANTHROPIC_API_KEY=""
ENV FEISHU_WEBHOOK_URL=""
ENV DINGTALK_WEBHOOK_URL=""

CMD ["nanobot", "start", "--config", "config.json"]

```

```
# docker-compose.yml
version: '3.8'

services:
  ops-agent:
    build: .
    container_name: ops-agent
    restart: unless-stopped
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - FEISHU_WEBHOOK_URL=${FEISHU_WEBHOOK_URL}
    volumes:
      - /var/log:/var/log:ro          # 挂载宿主机日志（只读）
      - ./reports:/home/ops/reports  # 巡检报告输出
      - ./memory:/app/memory         # 持久化记忆
    network_mode: host               # 需要访问宿主机网络
```

启动运行

```
# 1. 创建 .env 文件
cat > .env << 'EOF'
ANTHROPIC_API_KEY=sk-ant-xxxxx
FEISHU_WEBHOOK_URL=https://open.feishu.cn/open-apis/bot/v2/hook/xxxxx
EOF

# 2. 构建并启动
docker-compose up -d

# 3. 查看运行日志
docker-compose logs -f ops-agent

# 4. 手动触发巡检（交互模式）
docker exec -it ops-agent nanobot chat " 执行一次完整巡检"
```

1.5 面试话术（STAR 法）

面试官：请介绍一个你做过的 AI Agent 项目。

S (Situation) - 背景：

「我们团队管理 50+ 台服务器，运维团队每天花大量时间做重复的巡检工作——检查 CPU、内存、磁盘，分析日志里的异常。手动操作不仅耗时，还经常遗漏问题。」

T (Task) - 任务：

「我负责设计和实现一个基于 Nanobot 的智能运维助手，目标是自动化日常巡检、日志分析和异常告警，覆盖 80% 的日常运维操作。」

A (Action) - 行动：

「我做了几个关键设计决策：

1. 安全架构：在 AGENTS.md 中定义了命令白名单和黑名单机制，确保 Agent 只能执行安全的运维命令，比如 top、df -h 等，同时严格禁止 rm -rf 等危险操作。
2. 技能拆分：将监控和日志分析拆成独立的 Skill，每个 Skill 包含采集命令、分析逻辑和输出格式，使得 Agent 的行为可预测且可维护。
3. 告警阈值体系：设计了两级告警——CPU 80% 触发警告、95% 触发严重告警，通过飞书 Webhook 实时推送，确保团队能快速响应。

4. **cron** 定时巡检：利用 Nanobot 原生 cron 功能，每小时自动巡检，每天 9 点生成日报。」

R (Result) - 结果：

「上线后，运维团队日常巡检时间减少了 70%，平均故障发现时间从 2 小时缩短到 15 分钟。有一次磁盘使用率异常增长，Agent 在凌晨 3 点自动告警并提供了大文件清单，帮团队提前避免了一次线上故障。」

项目二：多平台智能客服 Bot（难度：中高级）

2.1 项目需求分析

项目背景 某 SaaS 公司需要一个智能客服系统，能够同时接入飞书、钉钉和 Telegram 三个平台，统一处理客户咨询。要求：

- 基于产品 FAQ 知识库回答常见问题
- 无法解答的问题自动创建工单
- 记住用户历史咨询记录
- 支持多语言（中文/英文）

功能矩阵

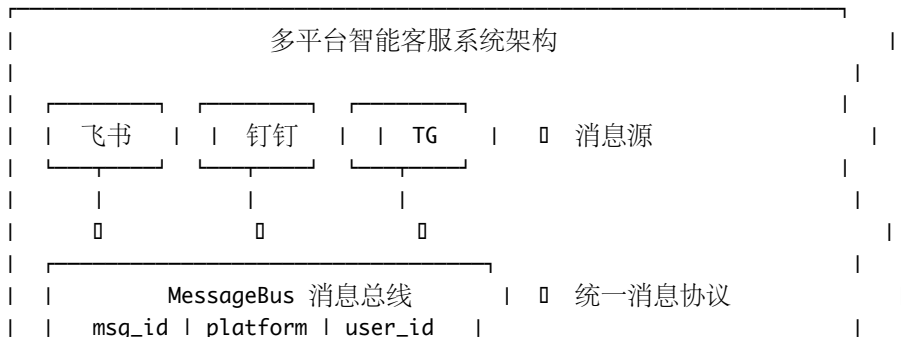
多平台智能客服功能矩阵					
功能	描述	飞书	钉钉	TG	
FAQ 查询	知识库匹配	□	□	□	
工单创建	自动创建工单	□	□	□	
工单查询	查看工单状态	□	□	□	
用户记忆	记住历史问题	□	□	□	
问题升级	转人工客服	□	□	□	
满意度评价	收集用户反馈	□	□	□	

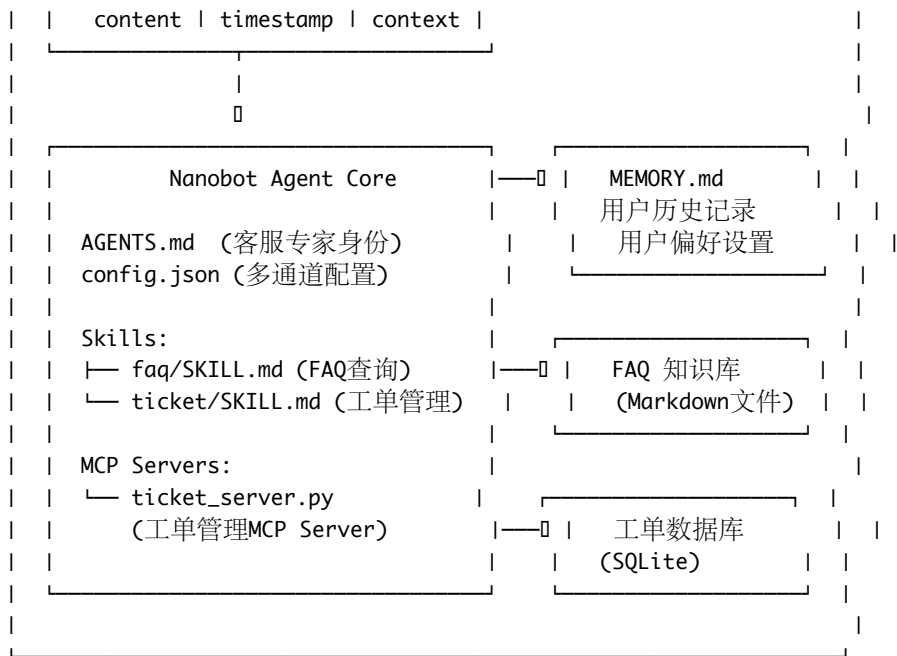
用户故事

作为一个产品用户，
我希望在飞书中直接 @客服Bot 提问，
它能基于知识库快速回答我的问题，
如果问题解决不了，自动帮我创建工单并告知工单号，
下次我再找它时，它还记得我之前的问题。

2.2 架构设计

整体架构图





MessageBus 消息统一协议

```

from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum
from typing import Optional

class Platform(Enum):
    FEISHU = "feishu"
    DINGTALK = "dingtalk"
    TELEGRAM = "telegram"

@dataclass
class UnifiedMessage:
    """ 统一消息格式 - 抹平多平台差异 """

    msg_id: str
    platform: Platform
    user_id: str
    user_name: str
    content: str
    timestamp: datetime = field(default_factory=datetime.now)
    reply_to: Optional[str] = None
    attachments: list = field(default_factory=list)

    def to_nanobot_prompt(self) -> str:
        """ 转换为 Nanobot 可处理的 prompt """
        context = (
            f"[来源: {self.platform.value}] "
            f"[用户: {self.user_name}({self.user_id})] "
            f"[时间: {self.timestamp.strftime('%Y-%m-%d %H:%M:%S')}] \n\n"
            f"用户提问: {self.content}"
        )

```

```

if self.reply_to:
    context += f"\n\n(这是对消息 {self.reply_to} 的回复)"
return context

```

记忆策略设计

MEMORY.md 记忆结构:

```

## 用户画像
### user_12345 (张三, 飞书)
- 公司: XX科技
- 常见问题: 数据导出, API 调用
- 偏好语言: 中文
- 满意度: 4.5/5
- 最近交互: 2025-01-15

### user_67890 (John, Telegram)
- 公司: YY Inc.
- 常见问题: Billing, API rate limit
- 偏好语言: English
- 满意度: 4.0/5
- 最近交互: 2025-01-14

## 高频问题追踪
| 问题 | 出现次数 | 最近出现 | 知识库是否覆盖 | |
|---|---|---|---|---|
| 数据导出格式 | 45 | 01-15 | 0 |
| API 限流策略 | 32 | 01-14 | 0 |
| 自定义字段 | 28 | 01-13 | 0 | 需补充 |

```

2.3 核心代码实现

AGENTS.md (客服专家身份)

多平台智能客服

身份

你是一个专业的产品客服专家。你代表公司与用户沟通, 解答产品相关问题。
你的目标是高效、准确、友善地帮助用户解决问题。

核心原则

1. ** 知识库优先 **: 优先从 FAQ 知识库中查找答案, 确保回答准确
2. ** 坦诚不知 **: 如果知识库中没有答案, 不要编造, 而是创建工单交给人工处理
3. ** 记住用户 **: 通过 MEMORY.md 记住用户的历史问题和偏好
4. ** 多语言适配 **: 根据用户使用的语言自动切换回复语言
5. ** 及时升级 **: 遇到投诉、紧急问题或用户明确要求人工时, 立即升级

回复风格

- 开头使用称呼: 「您好」/ 「Hi」
- 回答简洁但完整, 避免长篇大论
- 关键信息使用加粗或列表形式
- 结尾询问是否还有其他问题
- 遇到负面情绪时先共情再解决

工作流程

1. 收到用户消息
2. 从 `MEMORY.md` 中查找该用户的历史记录
3. 判断问题类型:
 - FAQ 类 → 调用 `faq Skill` 查找答案
 - 工单类 → 调用 `ticket Skill` 创建/查询工单
 - 投诉/紧急 → 创建高优先级工单并通知人工
4. 组织回复, 发送给用户
5. 更新 `MEMORY.md` 中的用户记录

可用技能

- ``skills/faq/SKILL.md`` - FAQ 知识库查询
- ``skills/ticket/SKILL.md`` - 工单创建与管理

MCP 集成

- ``ticket_server`` - 工单管理 MCP Server, 提供创建/查询/更新工单功能

config.json (多通道配置)

```
{
  "model": "claude-sonnet-4-20250514",
  "provider": "anthropic",
  "system_prompt_file": "AGENTS.md",
  "memory_file": "MEMORY.md",
  "skills_dirs": ["skills"],
  "platforms": {
    "feishu": {
      "enabled": true,
      "app_id": "${FEISHU_APP_ID}",
      "app_secret": "${FEISHU_APP_SECRET}",
      "verification_token": "${FEISHU_VERIFICATION_TOKEN}"
    },
    "dingtalk": {
      "enabled": true,
      "app_key": "${DINGTALK_APP_KEY}",
      "app_secret": "${DINGTALK_APP_SECRET}"
    },
    "telegram": {
      "enabled": true,
      "bot_token": "${TELEGRAM_BOT_TOKEN}"
    }
  },
  "mcp_servers": {
    "ticket_server": {
      "command": "python3",
      "args": ["mcp_servers/ticket_server.py"],
      "description": "工单管理系统"
    }
  },
  "tools": {
    "read_file": {
      "enabled": true,
      "allowed_paths": ["knowledge_base/", "MEMORY.md"]
    }
  }
}
```

```

    },
    "write_file": {
      "enabled": true,
      "allowed_paths": ["MEMORY.md"]
    }
  }
}

```

skills/faq/SKILL.md (FAQ 查询技能)

FAQ 知识库查询技能

描述

从产品 FAQ 知识库中查找并回答用户问题。

知识库结构

知识库文件存放在 ``knowledge_base/`` 目录下，按产品模块分类：

knowledge_base/ general.md # 通用问题（注册、登录、账户） billing.md # 计费相关 api.md # API 使用
 data-export.md # 数据导出 troubleshooting.md # 常见故障排查

查询流程

1. 分析用户问题的关键词和意图
2. 使用 `read_file` 工具读取对应的知识库文件
3. 在知识库内容中匹配最相关的 FAQ 条目
4. 组织回复

匹配策略

- ****精确匹配****: 用户问题中包含 FAQ 条目的关键词
- ****语义匹配****: 理解用户问题的意图，即使措辞不同也能匹配
- ****多条目合并****: 如果问题涉及多个知识点，合并多个条目回答

回复格式

找到匹配的 FAQ:

您好！关于您的问题，以下是解答：

[答案内容]

如果以上信息未能解决您的问题，我可以帮您创建一个工单，由专业团队跟进处理。

还有其他问题吗？

未找到匹配的 FAQ:

您好！您的问题我暂时无法在知识库中找到确切答案，我已为您创建工单 [工单号]，我们的专业团队会在 24 小时内联系您。

还有其他我能帮助的吗？

知识库示例内容

general.md 示例

````markdown`

## Q: 如何注册账号?

注册步骤:

1. 访问官网首页, 点击「免费注册」
2. 输入企业邮箱地址
3. 设置密码 (至少 8 位, 包含大小写字母和数字)
4. 验证邮箱
5. 填写团队信息, 完成注册

## Q: 如何重置密码?

重置密码方式:

1. 在登录页点击「忘记密码」
2. 输入注册邮箱
3. 查收重置链接邮件 (有效期 30 分钟)
4. 点击链接设置新密码

注意: 每 24 小时最多发送 5 次重置邮件。

#### skills/ticket/SKILL.md (工单管理技能)

```markdown

工单管理技能

描述

通过 MCP Server 创建、查询和更新工单。

可用工具 (来自 ticket_server MCP)

create_ticket

创建新工单。

参数:

- `title` (string, 必填): 工单标题
- `description` (string, 必填): 问题描述
- `priority` (string): 优先级 - low/medium/high/urgent
- `user\_id` (string, 必填): 用户 ID
- `user\_name` (string): 用户名称
- `platform` (string): 来源平台

get_ticket

查询工单状态。

参数:

- `ticket\_id` (string, 必填): 工单 ID

list_user_tickets

列出用户的所有工单。

参数:

- `user\_id` (string, 必填): 用户 ID
- `status` (string): 过滤状态 - open/in_progress/resolved/closed

update_ticket

更新工单状态。

参数:

- `ticket\_id` (string, 必填): 工单 ID
- `status` (string): 新状态
- `comment` (string): 更新备注

使用场景

1. **用户问题无法通过 FAQ 解决** → 创建 medium 优先级工单
2. **用户投诉或紧急问题** → 创建 urgent 优先级工单
3. **用户询问之前的问题** → 查询工单状态
4. **用户确认问题已解决** → 更新工单为 resolved

MCP Server: ticket_server.py (工单管理 MCP Server 完整代码)

```
#!/usr/bin/env python3
"""
ticket_server.py - 工单管理 MCP Server
基于 MCP 协议实现的工单 CRUD 系统, 使用 SQLite 存储
"""
```

```
import json
import sqlite3
import sys
import uuid
from datetime import datetime
from typing import Any
```

```
DB_PATH = "tickets.db"
```

```
def init_database():
    """ 初始化数据库表结构 """
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tickets (
            ticket_id TEXT PRIMARY KEY,
            title TEXT NOT NULL,
            description TEXT NOT NULL,
            priority TEXT DEFAULT 'medium',
            status TEXT DEFAULT 'open',
            user_id TEXT NOT NULL,
            user_name TEXT DEFAULT '',
            platform TEXT DEFAULT '',
            created_at TEXT NOT NULL,
            updated_at TEXT NOT NULL,
            comments TEXT DEFAULT '[]'
        )
    """)
    conn.commit()
    conn.close()
```

```
class TicketMCPServer:
    """ 工单管理 MCP Server """
```

```

def __init__(self):
    init_database()

def handle_request(self, request: dict) -> dict:
    """ 处理 MCP 请求 """
    method = request.get("method", "")
    req_id = request.get("id")

    handlers = {
        "initialize": self._handle_initialize,
        "tools/list": self._handle_list_tools,
        "tools/call": self._handle_call_tool,
    }

    handler = handlers.get(method)
    if handler:
        result = handler(request)
        return {"jsonrpc": "2.0", "id": req_id, "result": result}

    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "error": {"code": -32601, "message": f"Unknown method: {method}"},
    }

def _handle_initialize(self, request: dict) -> dict:
    return {
        "protocolVersion": "2024-11-05",
        "serverInfo": {
            "name": "ticket-server",
            "version": "1.0.0",
        },
        "capabilities": {"tools": {}},
    }

def _handle_list_tools(self, request: dict) -> dict:
    return {
        "tools": [
            {
                "name": "create_ticket",
                "description": " 创建新的客服工单",
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "title": {
                            "type": "string",
                            "description": " 工单标题",
                        },
                        "description": {
                            "type": "string",
                            "description": " 问题详细描述",
                        },
                    },
                },
                "priority": {
                    "type": "string",
                    "enum": ["low", "medium", "high", "urgent"],
                    "description": " 优先级",
                },
            }
        ]
    }

```

```

        "default": "medium",
    },
    "user_id": {
        "type": "string",
        "description": " 用户 ID",
    },
    "user_name": {
        "type": "string",
        "description": " 用户名称",
    },
    "platform": {
        "type": "string",
        "description": " 来源平台",
    },
    },
    "required": ["title", "description", "user_id"],
},
{
    "name": "get_ticket",
    "description": " 根据工单 ID 查询工单详情",
    "inputSchema": {
        "type": "object",
        "properties": {
            "ticket_id": {
                "type": "string",
                "description": " 工单 ID",
            },
        },
    },
    "required": ["ticket_id"],
},
{
    "name": "list_user_tickets",
    "description": " 列出指定用户的所有工单",
    "inputSchema": {
        "type": "object",
        "properties": {
            "user_id": {
                "type": "string",
                "description": " 用户 ID",
            },
            "status": {
                "type": "string",
                "enum": [
                    "open",
                    "in_progress",
                    "resolved",
                    "closed",
                ],
            },
        },
        "description": " 按状态过滤",
    },
    "required": ["user_id"],
},
},
},

```

```

        {
            "name": "update_ticket",
            "description": " 更新工单状态或添加备注",
            "inputSchema": {
                "type": "object",
                "properties": {
                    "ticket_id": {
                        "type": "string",
                        "description": " 工单 ID",
                    },
                    "status": {
                        "type": "string",
                        "enum": [
                            "open",
                            "in_progress",
                            "resolved",
                            "closed",
                        ],
                        "description": " 新状态",
                    },
                    "comment": {
                        "type": "string",
                        "description": " 更新备注",
                    },
                },
                "required": ["ticket_id"],
            },
        },
    ]
}

def _handle_call_tool(self, request: dict) -> dict:
    params = request.get("params", {})
    tool_name = params.get("name", "")
    arguments = params.get("arguments", {})

    tool_handlers = {
        "create_ticket": self._create_ticket,
        "get_ticket": self._get_ticket,
        "list_user_tickets": self._list_user_tickets,
        "update_ticket": self._update_ticket,
    }

    handler = tool_handlers.get(tool_name)
    if not handler:
        return {
            "content": [
                {
                    "type": "text",
                    "text": f" 未知工具: {tool_name}",
                }
            ],
            "isError": True,
        }

    try:

```

```

        result = handler(arguments)
        return {
            "content": [
                {"type": "text", "text": json.dumps(result, ensure_ascii=False)}
            ]
        }
    except Exception as e:
        return {
            "content": [{"type": "text", "text": f" 执行出错: {str(e)}"}],
            "isError": True,
        }

def _create_ticket(self, args: dict) -> dict:
    """ 创建工单"""
    ticket_id = f"TK-{uuid.uuid4().hex[:8].upper()}"
    now = datetime.now().isoformat()

    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute(
        """
        INSERT INTO tickets
            (ticket_id, title, description, priority, status,
             user_id, user_name, platform, created_at, updated_at)
        VALUES (?, ?, ?, ?, 'open', ?, ?, ?, ?, ?)
        """,
        (
            ticket_id,
            args["title"],
            args["description"],
            args.get("priority", "medium"),
            args["user_id"],
            args.get("user_name", ""),
            args.get("platform", ""),
            now,
            now,
        ),
    )
    conn.commit()
    conn.close()

    return {
        "success": True,
        "ticket_id": ticket_id,
        "message": f" 工单 {ticket_id} 创建成功",
        "created_at": now,
    }

def _get_ticket(self, args: dict) -> dict:
    """ 查询工单"""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM tickets WHERE ticket_id = ?", (args["ticket_id"],)
    )

```

```

row = cursor.fetchone()
conn.close()

if not row:
    return {"success": False, "message": " 工单不存在"}

return {"success": True, "ticket": dict(row)}

def _list_user_tickets(self, args: dict) -> dict:
    """ 列出用户工单 """
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    if "status" in args:
        cursor.execute(
            "SELECT * FROM tickets WHERE user_id = ? AND status = ? ORDER BY created_at DESC",
            (args["user_id"], args["status"]),
        )
    else:
        cursor.execute(
            "SELECT * FROM tickets WHERE user_id = ? ORDER BY created_at DESC",
            (args["user_id"],),
        )

    rows = cursor.fetchall()
    conn.close()

    return {
        "success": True,
        "count": len(rows),
        "tickets": [dict(row) for row in rows],
    }

def _update_ticket(self, args: dict) -> dict:
    """ 更新工单 """
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM tickets WHERE ticket_id = ?", (args["ticket_id"],)
    )
    row = cursor.fetchone()
    if not row:
        conn.close()
        return {"success": False, "message": " 工单不存在"}

    now = datetime.now().isoformat()
    updates = []
    values = []

    if "status" in args:
        updates.append("status = ?")
        values.append(args["status"])

```

```

if "comment" in args:
    comments = json.loads(row["comments"])
    comments.append({
        "text": args["comment"],
        "timestamp": now,
    })
    updates.append("comments = ?")
    values.append(json.dumps(comments, ensure_ascii=False))

updates.append("updated_at = ?")
values.append(now)
values.append(args["ticket_id"])

cursor.execute(
    f"UPDATE tickets SET {'', '.join(updates)} WHERE ticket_id = ?",
    values,
)
conn.commit()
conn.close()

return {
    "success": True,
    "message": f"工单 {args['ticket_id']} 更新成功",
    "updated_at": now,
}

def run(self):
    """ 启动 MCP Server, 通过 stdin/stdout 通信"""
    while True:
        try:
            line = sys.stdin.readline()
            if not line:
                break

            request = json.loads(line.strip())
            response = self.handle_request(request)
            sys.stdout.write(json.dumps(response) + "\n")
            sys.stdout.flush()
        except json.JSONDecodeError:
            continue
        except KeyboardInterrupt:
            break

if __name__ == "__main__":
    server = TicketMCPServer()
    server.run()

```

消息路由器 (message_router.py)

```

#!/usr/bin/env python3
"""
message_router.py - 多平台消息路由器
将不同平台的消息统一转发给 Nanobot 处理
"""

import hashlib

```

```

import hmac
import json
import os
import subprocess
from dataclasses import asdict
from datetime import datetime

from flask import Flask, request, jsonify

app = Flask(__name__)

class NanobotBridge:
    """Nanobot 交互桥接器"""

    @staticmethod
    def send_to_nanobot(prompt: str) -> str:
        """ 将消息发送给 Nanobot 并获取回复 """
        try:
            result = subprocess.run(
                ["nanobot", "chat", "--config", "config.json", prompt],
                capture_output=True,
                text=True,
                timeout=60,
            )
            return result.stdout.strip()
        except subprocess.TimeoutExpired:
            return " 抱歉, 处理超时了, 请稍后再试。"
        except Exception as e:
            return f" 系统暂时出了点问题, 请稍后再试。(错误码: {hash(str(e)) % 10000})"

# ===== 飞书接入 =====

@app.route("/webhook/feishu", methods=["POST"])
def feishu_webhook():
    """ 飞书消息接收端点 """
    data = request.json

    if data.get("type") == "url_verification":
        return jsonify({"challenge": data.get("challenge")})

    event = data.get("event", {})
    message = event.get("message", {})
    content = json.loads(message.get("content", "{}"))
    text = content.get("text", "").strip()

    if not text:
        return jsonify({"code": 0})

    sender = event.get("sender", {})
    user_id = sender.get("sender_id", {}).get("user_id", "unknown")

    prompt = (
        f"[来源: 飞书] [用户 ID: {user_id}]\n\n"
        f" 用户提问: {text}"
    )

```



```

)

reply = NanobotBridge.send_to_nanobot(prompt)

feishu_reply(message.get("message_id"), reply)

return jsonify({"code": 0})

def feishu_reply(message_id: str, text: str):
    """ 回复飞书消息 """
    import urllib.request

    token = get_feishu_tenant_token()
    url = f"https://open.feishu.cn/open-apis/im/v1/messages/{message_id}/reply"
    payload = {
        "content": json.dumps({"text": text}),
        "msg_type": "text",
    }
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url,
        data=data,
        headers={
            "Content-Type": "application/json",
            "Authorization": f"Bearer {token}",
        },
    )
    urllib.request.urlopen(req, timeout=10)

def get_feishu_tenant_token() -> str:
    """ 获取飞书 tenant_access_token """
    import urllib.request

    url = "https://open.feishu.cn/open-apis/auth/v3/tenant_access_token/internal"
    payload = {
        "app_id": os.getenv("FEISHU_APP_ID"),
        "app_secret": os.getenv("FEISHU_APP_SECRET"),
    }
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url, data=data, headers={"Content-Type": "application/json"}
    )
    with urllib.request.urlopen(req, timeout=10) as resp:
        result = json.loads(resp.read())
        return result.get("tenant_access_token", "")

# ===== 钉钉接入 =====

@app.route("/webhook/dingtalk", methods=["POST"])
def dingtalk_webhook():
    """ 钉钉消息接收端点 """
    data = request.json
    text = data.get("text", {}).get("content", "").strip()

```

```

if not text:
    return jsonify({"msgtype": "empty"})

user_id = data.get("senderStaffId", "unknown")

prompt = (
    f"[来源: 钉钉] [用户 ID: {user_id}]\n\n"
    f" 用户提问: {text}"
)

reply = NanobotBridge.send_to_nanobot(prompt)

session_webhook = data.get("sessionWebhook", "")
if session_webhook:
    dingtalk_reply(session_webhook, reply)

return jsonify({"msgtype": "text", "text": {"content": reply}})

def dingtalk_reply(webhook_url: str, text: str):
    """ 通过 sessionWebhook 回复钉钉消息"""
    import urllib.request

    payload = {"msgtype": "text", "text": {"content": text}}
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        webhook_url,
        data=data,
        headers={"Content-Type": "application/json"},
    )
    urllib.request.urlopen(req, timeout=10)

# ===== Telegram 接入 =====

@app.route("/webhook/telegram", methods=["POST"])
def telegram_webhook():
    """Telegram 消息接收端点"""
    data = request.json
    message = data.get("message", {})
    text = message.get("text", "").strip()

    if not text or text.startswith("/start"):
        return jsonify({"ok": True})

    chat_id = message.get("chat", {}).get("id")
    user = message.get("from", {})
    user_name = user.get("first_name", "")

    prompt = (
        f"[来源: Telegram] [用户: {user_name}({user.get('id', '')})]\n\n"
        f" 用户提问: {text}"
    )

    reply = NanobotBridge.send_to_nanobot(prompt)

```

```

telegram_reply(chat_id, reply)

return jsonify({"ok": True})

def telegram_reply(chat_id: int, text: str):
    """ 回复 Telegram 消息 """
    import urllib.request

    bot_token = os.getenv("TELEGRAM_BOT_TOKEN")
    url = f"https://api.telegram.org/bot{bot_token}/sendMessage"
    payload = {"chat_id": chat_id, "text": text, "parse_mode": "Markdown"}
    data = json.dumps(payload).encode("utf-8")
    req = urllib.request.Request(
        url, data=data, headers={"Content-Type": "application/json"}
    )
    urllib.request.urlopen(req, timeout=10)

# ===== 健康检查 =====

@app.route("/health", methods=["GET"])
def health_check():
    return jsonify({
        "status": "healthy",
        "timestamp": datetime.now().isoformat(),
        "platforms": {
            "feishu": bool(os.getenv("FEISHU_APP_ID")),
            "dingtalk": bool(os.getenv("DINGTALK_APP_KEY")),
            "telegram": bool(os.getenv("TELEGRAM_BOT_TOKEN")),
        },
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

2.4 部署与运行

Docker 部署

```

# docker-compose.yml
version: '3.8'

services:
  customer-service-bot:
    build: .
    container_name: cs-bot
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - FEISHU_APP_ID=${FEISHU_APP_ID}
      - FEISHU_APP_SECRET=${FEISHU_APP_SECRET}

```

```

- DINGTALK_APP_KEY=${DINGTALK_APP_KEY}
- DINGTALK_APP_SECRET=${DINGTALK_APP_SECRET}
- TELEGRAM_BOT_TOKEN=${TELEGRAM_BOT_TOKEN}
volumes:
- ./data:/app/data          # 工单数据库
- ./knowledge_base:/app/knowledge_base # 知识库
- ./memory:/app/memory      # 记忆文件

nginx:
  image: nginx:alpine
  ports:
  - "443:443"
  volumes:
  - ./nginx.conf:/etc/nginx/nginx.conf
  - ./certs:/etc/nginx/certs # SSL 证书
  depends_on:
  - customer-service-bot

```

Nginx 反向代理配置

```

# nginx.conf
server {
    listen 443 ssl;
    server_name cs-bot.example.com;

    ssl_certificate      /etc/nginx/certs/cert.pem;
    ssl_certificate_key  /etc/nginx/certs/key.pem;

    location /webhook/ {
        proxy_pass http://customer-service-bot:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /health {
        proxy_pass http://customer-service-bot:8080;
    }
}

```

部署步骤

```

# 1. 准备知识库
mkdir -p knowledge_base
# 将 FAQ 文档放入 knowledge_base/ 目录

# 2. 配置环境变量
cp .env.example .env
# 编辑 .env, 填入各平台的密钥

# 3. 启动服务
docker-compose up -d

# 4. 配置各平台 Webhook 回调地址
# 飞书: https://cs-bot.example.com/webhook/feishu
# 钉钉: https://cs-bot.example.com/webhook/dingtalk
# Telegram: 使用 setWebhook API 设置

```

```
# 5. 验证服务状态
curl https://cs-bot.example.com/health
```

2.5 面试话术 (STAR 法)

面试官：说说你是怎么设计多平台客服系统的？

S (Situation) - 背景：

「我们公司的客户分布在飞书、钉钉和 Telegram 三个平台。之前每个平台都有独立的客服入口，但知识库和工单系统是割裂的——同一个客户在飞书问过的问题，换到钉钉问又要从头回答，体验很差。」

T (Task) - 任务：

「我负责设计一套统一的智能客服系统，核心目标是：统一知识库、统一工单、跨平台用户记忆。技术栈选择了 Nanobot + 自定义 MCP Server。」

A (Action) - 行动：

「我做了三个关键架构决策：

- 1. **MessageBus** 统一消息层：设计了 UnifiedMessage 数据结构，把三个平台的不同消息格式标准化为统一协议。每条消息包含 platform、user_id、content，确保 Nanobot 能统一处理。
- 2. **MCP Server** 解耦工单系统：没有在 Agent 里直接操作数据库，而是开发了独立的 ticket_server MCP Server。Agent 通过 MCP 协议调用 create_ticket、get_ticket 等工具，实现了业务逻辑和 AI 能力的分离。
- 3. **MEMORY.md** 用户画像：利用 Nanobot 的原生记忆系统记录用户画像——包括常见问题类型、偏好语言、历史满意度。Agent 每次收到消息时先读取用户画像，实现个性化回复。」

R (Result) - 结果：

「上线后，客服首次解决率从 60% 提升到 85%。用户跨平台咨询时不再需要重复描述问题。工单平均处理时间从 48 小时降低到 12 小时，因为 Agent 创建的工单包含完整的上下文信息，人工客服拿到后能直接开始处理。」

项目三：自定义 MCP Server 开发（难度：高级）

3.1 项目需求分析

项目背景 数据团队日常需要频繁查询数据库获取业务数据，但直接给所有人数据库权限存在安全风险。需要开发一个安全的数据库查询 MCP Server，让 Nanobot Agent 能在受控环境下执行 SQL 查询。

功能需求

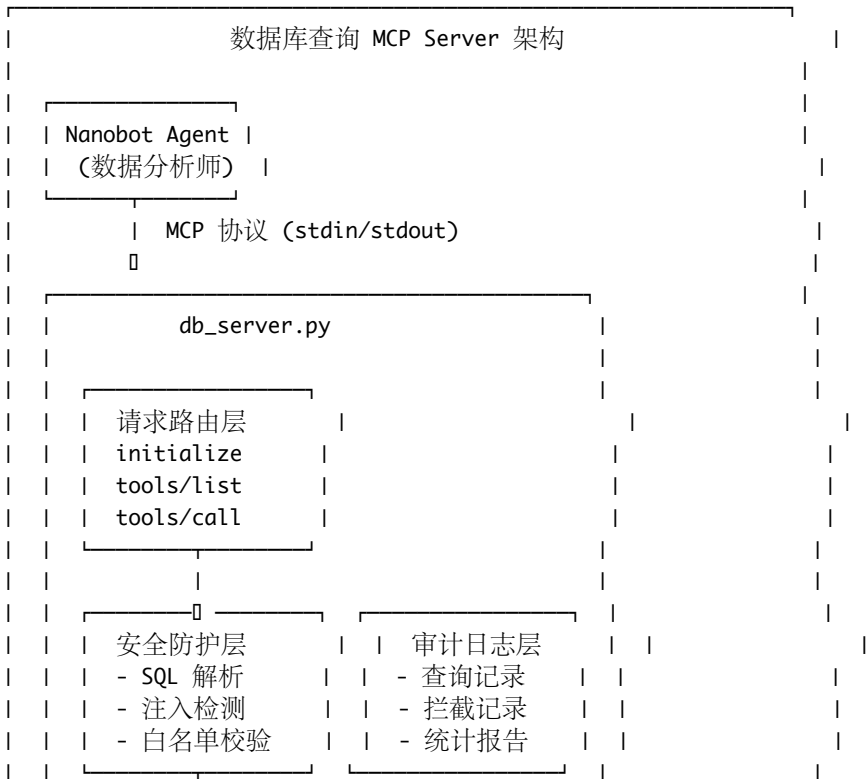
| 数据库查询 MCP Server 功能需求 | | |
|-----------------------|--------------------|--|
| 功能 | 具体描述 | |
| SQL 查询 | 执行 SELECT 查询，返回结果 | |
| 表结构查看 | 查看数据库中的表和字段信息 | |
| 数据统计 | 提供常用统计指标（行数、分布等） | |
| 安全防护 | 防 SQL 注入、只读模式、查询超时 | |
| 结果格式化 | 返回易读的表格格式 | |
| 查询审计 | 记录所有查询操作 | |

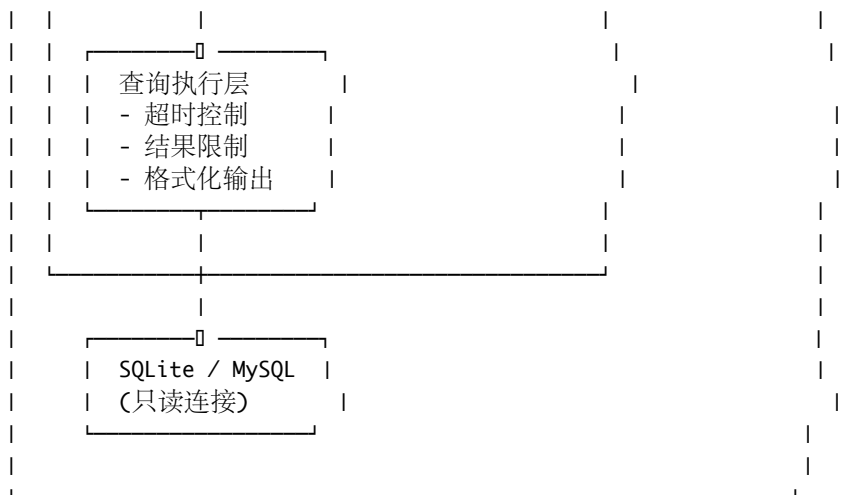
安全需求（重中之重）



3.2 架构设计

整体架构图





工具设计原则

| 原则 | 说明 | 示例 |
|------|-----------------|---|
| 最小权限 | 工具只提供必要的数据库访问能力 | 只支持 <code>SELECT</code> ，不支持 <code>DML</code> |
| 明确语义 | 每个工具的功能边界清晰 | <code>query_database</code> vs <code>list_tables</code> |
| 安全默认 | 默认配置是最安全的 | 默认限制 1000 行结果 |
| 可审计 | 所有操作都有日志 | 每次查询记录 <code>SQL</code> 和耗时 |
| 容错设计 | 错误不会暴露敏感信息 | 数据库错误不返回堆栈 |

3.3 核心代码实现

db_server.py (完整的数据库 MCP Server)

```
#!/usr/bin/env python3
```

```
"""
```

```
db_server.py - 安全的数据库查询 MCP Server
```

提供四个工具：

- `query_database`: 执行只读 `SQL` 查询
- `list_tables`: 列出所有数据表
- `describe_table`: 查看表结构
- `get_statistics`: 获取表的统计信息

安全特性：

- 只允许 `SELECT` 语句
- `SQL` 注入检测
- 查询超时控制
- 结果行数限制
- 完整的审计日志

```
"""
```

```
import json
import os
import re
import signal
import sqlite3
import sys
```

```

import threading
import time
from contextlib import contextmanager
from datetime import datetime
from typing import Any, Optional

# ===== 配置 =====

class Config:
    DB_PATH = os.getenv("DB_PATH", "business.db")
    QUERY_TIMEOUT = int(os.getenv("QUERY_TIMEOUT", "30"))
    MAX_ROWS = int(os.getenv("MAX_ROWS", "1000"))
    MAX_CONCURRENT = int(os.getenv("MAX_CONCURRENT", "5"))
    AUDIT_LOG_PATH = os.getenv("AUDIT_LOG_PATH", "audit.log")
    READONLY = os.getenv("DB_READONLY", "true").lower() == "true"

# ===== 审计日志 =====

class AuditLogger:
    """ 查询审计日志记录器 """

    def __init__(self, log_path: str):
        self.log_path = log_path
        self._lock = threading.Lock()

    def log_query(
        self,
        sql: str,
        success: bool,
        duration_ms: float,
        rows_returned: int = 0,
        error: str = "",
    ):
        entry = {
            "timestamp": datetime.now().isoformat(),
            "type": "query",
            "sql": sql,
            "success": success,
            "duration_ms": round(duration_ms, 2),
            "rows_returned": rows_returned,
            "error": error,
        }
        self._write(entry)

    def log_blocked(self, sql: str, reason: str):
        entry = {
            "timestamp": datetime.now().isoformat(),
            "type": "blocked",
            "sql": sql,
            "reason": reason,
        }
        self._write(entry)

    def _write(self, entry: dict):

```



```

with self._lock:
    try:
        with open(self.log_path, "a", encoding="utf-8") as f:
            f.write(json.dumps(entry, ensure_ascii=False) + "\n")
    except IOError:
        pass

# ===== SQL 安全检查 =====

class SQLSecurityChecker:
    """SQL 安全检查器 - 多层防护"""

    DANGEROUS_KEYWORDS = [
        "DROP", "DELETE", "UPDATE", "INSERT", "ALTER", "TRUNCATE",
        "CREATE", "REPLACE", "GRANT", "REVOKE", "EXEC", "EXECUTE",
        "CALL", "MERGE", "UPSERT", "ATTACH", "DETACH",
    ]

    INJECTION_PATTERNS = [
        r";\s*(DROP|DELETE|UPDATE|INSERT|ALTER)",
        r"UNION\s+(ALL\s+)?SELECT",
        r"OR\s+1\s*=\s*1",
        r"OR\s+'1'\s*=\s*'1'",
        r"--\s*$",
        r"/\s*\*\s*/",
        r"SLEEP\s*\(",
        r"BENCHMARK\s*\(",
        r"LOAD_FILE\s*\(",
        r"INTO\s+(OUTFILE|DUMPFILE)",
        r"@@(version|datadir|basedir)",
        r"INFORMATION_SCHEMA",
    ]

    def check(self, sql: str) -> tuple[bool, str]:
        """
        检查 SQL 是否安全。
        返回 (is_safe, reason)
        """
        if not sql or not sql.strip():
            return False, " 空查询"

        normalized = sql.strip().upper()

        if not self._is_select_statement(normalized):
            return False, " 只允许 SELECT 查询语句"

        keyword_check = self._check_dangerous_keywords(normalized)
        if not keyword_check[0]:
            return keyword_check

        injection_check = self._check_injection_patterns(sql)
        if not injection_check[0]:
            return injection_check

        if normalized.count(";") > 0:

```

```

        parts = [p.strip() for p in normalized.split(";") if p.strip()]
        if len(parts) > 1:
            return False, " 不允许多语句执行"

    return True, " 通过安全检查"

def _is_select_statement(self, sql: str) -> bool:
    cleaned = sql.lstrip("( \t\n")
    return cleaned.startswith("SELECT") or cleaned.startswith("WITH")

def _check_dangerous_keywords(self, sql: str) -> tuple[bool, str]:
    words = set(re.findall(r'\b[A-Z_]+\b', sql))
    for keyword in self.DANGEROUS_KEYWORDS:
        if keyword in words:
            return False, f" 包含禁止的关键词: {keyword}"
    return True, ""

def _check_injection_patterns(self, sql: str) -> tuple[bool, str]:
    for pattern in self.INJECTION_PATTERNS:
        if re.search(pattern, sql, re.IGNORECASE):
            return False, f" 检测到可疑的注入模式"
    return True, ""

# ===== 数据库连接管理 =====

class DatabaseManager:
    """ 数据库连接和查询管理器 """

    def __init__(self, db_path: str, readonly: bool = True):
        self.db_path = db_path
        self.readonly = readonly
        self._semaphore = threading.Semaphore(Config.MAX_CONCURRENT)

    @contextmanager
    def get_connection(self):
        """ 获取数据库连接（带并发控制） """
        acquired = self._semaphore.acquire(timeout=10)
        if not acquired:
            raise RuntimeError(" 并发查询数已达上限，请稍后重试")

        try:
            uri = f"file:{self.db_path}?mode=ro" if self.readonly else self.db_path
            conn = sqlite3.connect(
                uri if self.readonly else self.db_path,
                uri=self.readonly,
                timeout=Config.QUERY_TIMEOUT,
            )
            conn.row_factory = sqlite3.Row
            conn.execute(f"PRAGMA busy_timeout = {Config.QUERY_TIMEOUT * 1000}")
            yield conn
        finally:
            conn.close()
            self._semaphore.release()

    def execute_query(self, sql: str, max_rows: int = None) -> dict:

```

```

""" 执行查询并返回结果"""
if max_rows is None:
    max_rows = Config.MAX_ROWS

start_time = time.time()

with self.get_connection() as conn:
    cursor = conn.cursor()

    timer = threading.Timer(Config.QUERY_TIMEOUT, self._cancel_query, [conn])
    timer.start()

    try:
        cursor.execute(sql)
        rows = cursor.fetchmany(max_rows + 1)
        truncated = len(rows) > max_rows
        if truncated:
            rows = rows[:max_rows]

        columns = (
            [description[0] for description in cursor.description]
            if cursor.description
            else []
        )

        duration_ms = (time.time() - start_time) * 1000

        return {
            "success": True,
            "columns": columns,
            "rows": [dict(row) for row in rows],
            "row_count": len(rows),
            "truncated": truncated,
            "max_rows": max_rows,
            "duration_ms": round(duration_ms, 2),
        }
    except sqlite3.OperationalError as e:
        if "interrupted" in str(e).lower():
            raise TimeoutError(" 查询超时")
        raise
    finally:
        timer.cancel()

def list_tables(self) -> list[dict]:
    """ 列出所有表"""
    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(
            "SELECT name, type FROM sqlite_master "
            "WHERE type IN ('table', 'view') AND name NOT LIKE 'sqlite_%' "
            "ORDER BY type, name"
        )
        tables = []
        for row in cursor.fetchall():
            count_cursor = conn.cursor()
            try:

```

```

        count_cursor.execute(f'SELECT COUNT(*) FROM "{row["name"]}"')
        count = count_cursor.fetchone()[0]
    except Exception:
        count = -1

    tables.append({
        "name": row["name"],
        "type": row["type"],
        "row_count": count,
    })
    return tables

def describe_table(self, table_name: str) -> dict:
    """ 查看表结构"""
    if not re.match(r'^[a-zA-Z_][a-zA-Z0-9_]*$', table_name):
        raise ValueError(" 无效的表名")

    with self.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute(f'PRAGMA table_info("{table_name}")')
        columns = []
        for row in cursor.fetchall():
            columns.append({
                "name": row["name"],
                "type": row["type"],
                "nullable": not row["notnull"],
                "default": row["dflt_value"],
                "primary_key": bool(row["pk"]),
            })

        cursor.execute(f'PRAGMA index_list("{table_name}")')
        indexes = []
        for row in cursor.fetchall():
            indexes.append({
                "name": row["name"],
                "unique": bool(row["unique"]),
            })

    return {
        "table_name": table_name,
        "columns": columns,
        "indexes": indexes,
        "column_count": len(columns),
    }

def get_statistics(self, table_name: str) -> dict:
    """ 获取表的统计信息"""
    if not re.match(r'^[a-zA-Z_][a-zA-Z0-9_]*$', table_name):
        raise ValueError(" 无效的表名")

    with self.get_connection() as conn:
        cursor = conn.cursor()

        cursor.execute(f'SELECT COUNT(*) as total FROM "{table_name}"')
        total = cursor.fetchone()[0]

```

```

cursor.execute(f'PRAGMA table_info("{table_name}")')
columns_info = cursor.fetchall()

column_stats = []
for col in columns_info:
    col_name = col["name"]
    col_type = col["type"].upper()
    stats = {"name": col_name, "type": col["type"]}

    try:
        cursor.execute(
            f'SELECT COUNT(DISTINCT "{col_name}") FROM "{table_name}"'
        )
        stats["distinct_count"] = cursor.fetchone()[0]

        cursor.execute(
            f'SELECT COUNT(*) FROM "{table_name}" WHERE "{col_name}" IS NULL'
        )
        stats["null_count"] = cursor.fetchone()[0]

        if col_type in ("INTEGER", "REAL", "NUMERIC", "FLOAT", "DOUBLE"):
            cursor.execute(
                f'SELECT MIN("{col_name}"), MAX("{col_name}"), '
                f'AVG("{col_name}") FROM "{table_name}"'
            )
            row = cursor.fetchone()
            stats["min"] = row[0]
            stats["max"] = row[1]
            stats["avg"] = round(row[2], 2) if row[2] else None
    except Exception:
        pass

    column_stats.append(stats)

return {
    "table_name": table_name,
    "total_rows": total,
    "column_stats": column_stats,
}

@staticmethod
def _cancel_query(conn):
    try:
        conn.interrupt()
    except Exception:
        pass

# ===== MCP Server =====

class DatabaseMCPServer:
    """ 数据库查询 MCP Server """

    def __init__(self):
        self.db = DatabaseManager(Config.DB_PATH, Config.READONLY)
        self.security = SQLSecurityChecker()

```

```

self.audit = AuditLogger(Config.AUDIT_LOG_PATH)

def handle_request(self, request: dict) -> dict:
    method = request.get("method", "")
    req_id = request.get("id")

    handlers = {
        "initialize": self._handle_initialize,
        "tools/list": self._handle_list_tools,
        "tools/call": self._handle_call_tool,
    }

    handler = handlers.get(method)
    if handler:
        result = handler(request)
        return {"jsonrpc": "2.0", "id": req_id, "result": result}

    return {
        "jsonrpc": "2.0",
        "id": req_id,
        "error": {"code": -32601, "message": f"Unknown method: {method}"},
    }

def _handle_initialize(self, request: dict) -> dict:
    return {
        "protocolVersion": "2024-11-05",
        "serverInfo": {
            "name": "database-query-server",
            "version": "1.0.0",
        },
        "capabilities": {"tools": {}},
    }

def _handle_list_tools(self, request: dict) -> dict:
    return {
        "tools": [
            {
                "name": "query_database",
                "description": (
                    "  执行只读 SQL 查询。只允许 SELECT 语句，"
                    "  内置 SQL 注入防护和查询超时控制。"
                ),
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "sql": {
                            "type": "string",
                            "description": "  要执行的 SELECT SQL 语句",
                        },
                        "max_rows": {
                            "type": "integer",
                            "description": "  最大返回行数（默认 1000）",
                            "default": 1000,
                        },
                    },
                },
                "required": ["sql"],
            }
        ]
    }

```

```

        },
    },
    {
        "name": "list_tables",
        "description": "  列出数据库中的所有表和视图，包含行数统计",
        "inputSchema": {
            "type": "object",
            "properties": {},
        },
    },
    {
        "name": "describe_table",
        "description": "  查看指定表的结构信息，包含字段名、类型、索引等",
        "inputSchema": {
            "type": "object",
            "properties": {
                "table_name": {
                    "type": "string",
                    "description": "  表名",
                },
            },
            "required": ["table_name"],
        },
    },
    {
        "name": "get_statistics",
        "description": (
            "  获取指定表的统计信息，包含行数、各列的"
            "  唯一值数量、空值数量、数值列的最大/最小/平均值"
        ),
        "inputSchema": {
            "type": "object",
            "properties": {
                "table_name": {
                    "type": "string",
                    "description": "  表名",
                },
            },
            "required": ["table_name"],
        },
    },
]

def _handle_call_tool(self, request: dict) -> dict:
    params = request.get("params", {})
    tool_name = params.get("name", "")
    arguments = params.get("arguments", {})

    tool_handlers = {
        "query_database": self._tool_query_database,
        "list_tables": self._tool_list_tables,
        "describe_table": self._tool_describe_table,
        "get_statistics": self._tool_get_statistics,
    }

```

```

handler = tool_handlers.get(tool_name)
if not handler:
    return self._error_response(f" 未知工具: {tool_name}")

try:
    result = handler(arguments)
    return {
        "content": [
            {
                "type": "text",
                "text": json.dumps(result, ensure_ascii=False, default=str),
            }
        ]
    }
except TimeoutError as e:
    self.audit.log_query(
        str(arguments), False, 0, error=" 查询超时"
    )
    return self._error_response(f" 查询超时 (超过 {Config.QUERY_TIMEOUT} 秒) ")
except ValueError as e:
    return self._error_response(str(e))
except Exception as e:
    return self._error_response(" 查询执行出错, 请检查 SQL 语法")

def _tool_query_database(self, args: dict) -> dict:
    sql = args.get("sql", "")
    max_rows = args.get("max_rows", Config.MAX_ROWS)

    is_safe, reason = self.security.check(sql)
    if not is_safe:
        self.audit.log_blocked(sql, reason)
        raise ValueError(f" 安全检查未通过: {reason}")

    result = self.db.execute_query(sql, max_rows)

    self.audit.log_query(
        sql,
        result["success"],
        result["duration_ms"],
        result["row_count"],
    )

    return result

def _tool_list_tables(self, args: dict) -> dict:
    tables = self.db.list_tables()
    self.audit.log_query("LIST TABLES", True, 0, len(tables))
    return {"tables": tables, "count": len(tables)}

def _tool_describe_table(self, args: dict) -> dict:
    table_name = args.get("table_name", "")
    result = self.db.describe_table(table_name)
    self.audit.log_query(f"DESCRIBE {table_name}", True, 0)
    return result

def _tool_get_statistics(self, args: dict) -> dict:

```



```

table_name = args.get("table_name", "")
result = self.db.get_statistics(table_name)
self.audit.log_query(f"STATISTICS {table_name}", True, 0)
return result

```

```
@staticmethod
```

```

def _error_response(message: str) -> dict:
    return {
        "content": [{"type": "text", "text": message}],
        "isError": True,
    }

```

```

def run(self):
    """ 启动 MCP Server """
    while True:
        try:
            line = sys.stdin.readline()
            if not line:
                break

            request = json.loads(line.strip())
            response = self.handle_request(request)
            sys.stdout.write(json.dumps(response) + "\n")
            sys.stdout.flush()
        except json.JSONDecodeError:
            continue
        except KeyboardInterrupt:
            break

```

```
# ===== 测试用例 =====
```

```

def create_sample_database():
    """ 创建示例数据库用于测试 """
    conn = sqlite3.connect(Config.DB_PATH)
    cursor = conn.cursor()

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT NOT NULL,
            email TEXT UNIQUE NOT NULL,
            department TEXT,
            created_at TEXT DEFAULT (datetime('now')),
            is_active INTEGER DEFAULT 1
        )
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS orders (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            product TEXT NOT NULL,
            amount REAL NOT NULL,
            status TEXT DEFAULT 'pending',
            created_at TEXT DEFAULT (datetime('now')),

```

```

        FOREIGN KEY (user_id) REFERENCES users(id)
    )
    """

sample_users = [
    ("张三", "zhangsan@example.com", " 工程部"),
    ("李四", "lisi@example.com", " 产品部"),
    ("王五", "wangwu@example.com", " 市场部"),
    ("赵六", "zhaoliu@example.com", " 工程部"),
    ("陈七", "chenqi@example.com", " 销售部"),
]

cursor.executemany(
    "INSERT OR IGNORE INTO users (name, email, department) VALUES (?, ?, ?)",
    sample_users,
)

sample_orders = [
    (1, "云服务器 ECS", 2999.00, "completed"),
    (1, "对象存储 OSS", 599.00, "completed"),
    (2, "域名注册", 99.00, "pending"),
    (3, "CDN 加速", 1299.00, "completed"),
    (4, "数据库 RDS", 3999.00, "processing"),
    (5, "安全证书 SSL", 299.00, "completed"),
]

cursor.executemany(
    "INSERT OR IGNORE INTO orders (user_id, product, amount, status) VALUES (?, ?, ?, ?)",
    sample_orders,
)

conn.commit()
conn.close()

def run_security_tests():
    """ 运行安全检查测试 """
    checker = SQLSecurityChecker()

    test_cases = [
        ("SELECT * FROM users", True, " 正常查询"),
        ("SELECT name FROM users WHERE id = 1", True, " 带条件查询"),
        ("DROP TABLE users", False, "DROP 攻击"),
        ("DELETE FROM users", False, "DELETE 攻击"),
        ("SELECT * FROM users; DROP TABLE users", False, " 多语句注入"),
        ("SELECT * FROM users UNION SELECT * FROM passwords", False, "UNION 注入"),
        ("SELECT * FROM users WHERE id = 1 OR 1=1", False, "OR 1=1 注入"),
        ("UPDATE users SET name = 'hacked'", False, "UPDATE 攻击"),
        ("SELECT * FROM users -- comment", False, " 注释注入"),
        ("SELECT SLEEP(10)", False, " 延时注入"),
        (
            "WITH cte AS (SELECT * FROM users) SELECT * FROM cte",
            True,
            "CTE 查询",
        ),
    ]

```

```

print("=" * 60)
print("SQL 安全检查测试")
print("=" * 60)

passed = 0
total = len(test_cases)

for sql, expected_safe, description in test_cases:
    is_safe, reason = checker.check(sql)
    status = "❌" if is_safe == expected_safe else "✅"
    if is_safe == expected_safe:
        passed += 1
    print(f"{status} {description}")
    print(f"    SQL: {sql}")
    print(f"    预期: {'安全' if expected_safe else '拦截'} | "
          f"    实际: {'安全' if is_safe else '拦截'}")
    if not is_safe:
        print(f"        原因: {reason}")
    print()

print(f" 测试结果: {passed}/{total} 通过")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--test":
        create_sample_database()
        run_security_tests()
    elif len(sys.argv) > 1 and sys.argv[1] == "--init-db":
        create_sample_database()
        print(f" 示例数据库已创建: {Config.DB_PATH}")
    else:
        server = DatabaseMCPServer()
        server.run()

```

config.json (接入 Nanobot)

```

{
  "model": "claude-sonnet-4-20250514",
  "provider": "anthropic",
  "system_prompt_file": "AGENTS.md",
  "memory_file": "MEMORY.md",
  "skills_dirs": ["skills"],
  "mcp_servers": {
    "database": {
      "command": "python3",
      "args": ["mcp_servers/db_server.py"],
      "env": {
        "DB_PATH": "/data/business.db",
        "QUERY_TIMEOUT": "30",
        "MAX_ROWS": "1000",
        "DB_READONLY": "true"
      },
      "description": " 安全的数据库查询服务"
    }
  },
  "tools": {

```

```

    "read_file": {
      "enabled": true,
      "allowed_paths": ["MEMORY.md", "reports/"]
    },
    "write_file": {
      "enabled": true,
      "allowed_paths": ["MEMORY.md", "reports/"]
    }
  }
}

```

AGENTS.md (数据分析师身份)

数据分析助手

身份

你是一个专业的数据分析师。你通过安全的数据库查询工具帮助团队获取和分析业务数据。你精通 SQL，擅长将业务问题转化为数据查询，并以可视化的方式呈现分析结果。

核心原则

1. **** 理解意图 ****: 先理解用户的业务问题，再转化为 SQL 查询
2. **** 循序渐进 ****: 先查看表结构和统计信息，再编写精确的查询
3. **** 结果解读 ****: 不要只返回原始数据，要提供业务层面的解读
4. **** 安全意识 ****: 只使用 SELECT 查询，绝不尝试修改数据

工作流程

1. 收到用户的数据分析需求
2. 使用 ``list_tables`` 了解可用的数据表
3. 使用 ``describe_table`` 了解表结构
4. 编写安全的 SELECT SQL 查询
5. 使用 ``query_database`` 执行查询
6. 分析结果，生成可读的报告
7. 将分析结果记录到 MEMORY.md

查询规范

- 所有查询必须是 SELECT 语句
- 大表查询要加 LIMIT 限制
- 使用有意义的列别名 (AS)
- 复杂查询使用 CTE (WITH 语句) 提高可读性
- 涉及金额的查询使用 ROUND() 保留两位小数

输出格式

数据分析结果使用 Markdown 表格展示，包含：

1. 查询目的说明
2. 数据表格
3. 关键发现 (用 bullet points)
4. 建议 (如果有的话)

可用的 MCP 工具

- ``query_database`` - 执行 SQL 查询 (仅 SELECT)
- ``list_tables`` - 列出所有数据表

- ``describe_table`` - 查看表结构详情
 - ``get_statistics`` - 获取表的统计信息
-

3.4 部署与运行

部署架构

```
project-root/
├── AGENTS.md           # 数据分析师身份
├── config.json         # Nanobot 配置
├── MEMORY.md          # 分析历史记录
├── mcp_servers/
│   └── db_server.py    # 数据库 MCP Server
├── reports/           # 分析报告输出
├── data/
│   └── business.db     # 业务数据库
├── Dockerfile
└── docker-compose.yml
```

Docker 部署

```
FROM python:3.11-slim
```

```
RUN pip install nanobot
```

```
WORKDIR /app
```

```
COPY AGENTS.md config.json ./
```

```
COPY mcp_servers/ ./mcp_servers/
```

```
ENV ANTHROPIC_API_KEY=""
```

```
ENV DB_PATH="/data/business.db"
```

```
ENV DB_READONLY="true"
```

```
CMD ["nanobot", "start", "--config", "config.json"]
```

```
# docker-compose.yml
```

```
version: '3.8'
```

```
services:
```

```
  data-analyst:
```

```
    build: .
```

```
    container_name: data-analyst
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
```

```
      - DB_PATH=/data/business.db
```

```
      - DB_READONLY=true
```

```
      - QUERY_TIMEOUT=30
```

```
      - MAX_ROWS=1000
```

```
    volumes:
```

```
      - ./data:/data:ro          # 数据库文件（只读）
```

```
      - ./reports:/app/reports  # 分析报告
```

```
      - ./memory:/app/memory    # 记忆文件
```

运行测试

1. 初始化示例数据库

```
python3 mcp_servers/db_server.py --init-db
```

2. 运行安全测试

```
python3 mcp_servers/db_server.py --test
```

3. 启动 Nanobot

```
nanobot chat --config config.json " 帮我分析各部门的订单金额分布"
```

4. 更多查询示例

```
nanobot chat --config config.json " 哪个用户的消费最高? "
```

```
nanobot chat --config config.json " 各产品的销售额排名是什么? "
```

```
nanobot chat --config config.json " 有多少订单还在 pending 状态? "
```

3.5 面试话术 (STAR 法)

面试官：你开发过 MCP Server 吗？能说说具体的实现吗？

S (Situation) - 背景：

「我们的数据团队经常需要查询业务数据库做分析，但给每个人都开数据库直连权限风险太大——之前就出过有人误执行了 UPDATE 语句导致数据被改的事故。需要一种安全的方式让 AI Agent 能帮团队查数据，同时确保数据库安全。」

T (Task) - 任务：

「我负责从零开发一个数据库查询 MCP Server，核心要求是：只读查询、防 SQL 注入、查询超时控制，并且要和 Nanobot 无缝集成。」

A (Action) - 行动：

「我的实现分四个层次：

1. 四层安全防护：

- 第一层，SQL 语句解析，只允许 SELECT 和 WITH 开头的查询；
- 第二层，危险关键字检测，拦截 DROP、DELETE、UPDATE 等 14 个危险关键字；
- 第三层，注入模式检测，用正则匹配 UNION SELECT、OR 1=1 等 12 种常见注入模式；
- 第四层，多语句检测，禁止分号分隔的多语句执行。

2. 资源限制：查询超时 30 秒自动中断（用 threading.Timer + conn.interrupt()），结果限制 1000 行，并发控制最多 5 个查询。

3. 完整审计日志：每次查询都记录 SQL、执行时间、返回行数；被拦截的危险查询也会记录，用于安全分析。

4. 工具设计遵循最小权限原则：只暴露 4 个工具——query_database、list_tables、describe_table、get_statistics，每个工具职责明确、边界清晰。」

R (Result) - 结果：

「上线后跑了 3 个月，安全测试覆盖了 11 种攻击模式全部拦截成功。数据团队使用 Agent 做日常数据查询的效率提升了 3 倍——以前写 SQL、连数据库、导出结果要 20 分钟，现在用自然语言问 Agent 只需要 2 分钟。审计日志还帮我们发现了几个高频查询，后来做成了定时报表，进一步减少了人工操作。」

面试高频题（10 道，含详细答案）

题目一：你如何设计一个运维 Agent 系统？

参考答案：

设计运维 Agent 系统，我会从五个维度考虑：

1. 安全边界设计（最重要） - 在 AGENTS.md 中定义命令白名单和黑名单 - 白名单：top, df -h, free -m, ps aux 等只读命令 - 黑名单：rm -rf, shutdown, chmod 777 等破坏性命令 - Agent 即使被用户诱导也不能突破这个边界
 2. 技能模块化 - 将监控、日志分析、告警等拆分为独立 Skill - 每个 Skill 包含采集命令、分析逻辑和输出格式 - 好处：可独立维护，容易扩展新的运维场景
 3. 定时任务设计 - 使用 Nanobot cron 配置实现定时巡检 - 不同频率对应不同的巡检深度（每小时快速巡检，每天深度巡检）
 4. 告警策略 - 两级告警阈值：warning 和 critical - 告警通道：飞书/钉钉 Webhook - 告警去重：相同问题短时间内不重复告警
 5. 记忆和趋势分析 - 使用 MEMORY.md 记录历史巡检数据 - Agent 可以基于历史数据判断趋势（如磁盘使用率是否在持续增长）
-

题目二：多平台客服系统如何保证消息一致性？

参考答案：

保证多平台消息一致性，关键是设计统一的消息层：

1. 统一消息协议 - 定义 UnifiedMessage 数据结构，包含 platform、user_id、content 等字段 - 各平台的 Webhook 端点负责将平台特定的消息格式转换为 UnifiedMessage - Nanobot Agent 只处理统一格式的消息，不感知平台差异
 2. 用户身份映射 - 同一个用户在不同平台有不同的 ID - 通过 MEMORY.md 记录用户在各平台的 ID 映射关系 - 实现跨平台的用户画像统一
 3. 知识库统一 - 所有平台共享同一套 FAQ 知识库 - 知识库以 Markdown 文件形式存储，通过 Skill 调用 - 更新知识库后，所有平台同时生效
 4. 工单系统统一 - 工单通过 MCP Server 管理，与平台无关 - 工单记录来源平台，但处理流程统一
-

题目三：MCP Server 开发中如何处理安全问题？

参考答案：

MCP Server 的安全问题需要分层防护：

1. 输入验证层 - SQL 语句只允许 SELECT，通过解析第一个关键字判断 - 检查危险关键字（DROP、DELETE 等 14 个） - 正则匹配注入模式（UNION SELECT、OR 1=1 等 12 种） - 禁止多语句执行（检测分号）
 2. 运行时限制 - 查询超时控制：用线程定时器 + 数据库连接中断 - 结果行数限制：最多返回 1000 行 - 并发控制：用信号量限制同时执行的查询数
 3. 连接层安全 - 数据库以只读模式连接（SQLite 用 ?mode=ro） - 即使注入成功，也无法写入数据
 4. 审计和监控 - 所有查询记录到审计日志 - 被拦截的危险查询重点记录 - 可以定期分析审计日志，发现潜在的安全威胁
 5. 错误信息脱敏 - 数据库错误不直接暴露给用户 - 返回通用错误信息，避免泄露数据库结构
-

题目四：如何设计 Agent 的记忆策略？

参考答案：

Agent 的记忆策略要考虑四个方面：

- 1. 记什么（内容策略） - 用户画像：基本信息、偏好、历史问题类型 - 操作历史：关键操作和结果 - 高频问题：统计用户群体的共性问题 - 不记：敏感信息（密码、Token 等）
- 2. 怎么记（存储策略） - 使用 Nanobot 原生的 MEMORY.md 机制 - 结构化存储：Markdown 标题分层，表格存数据 - 按用户分区：每个用户独立的记录区域
- 3. 记多久（淘汰策略） - 用户画像长期保留 - 操作历史保留近 30 天 - 超过容量限制时，淘汰最旧的记录 - 关键事件（告警、故障）永久保留摘要
- 4. 怎么用（检索策略） - 每次收到消息先查用户记录 - 相关历史信息注入到 prompt 上下文 - 利用历史信息实现个性化回复

题目五：你的项目中遇到过 Agent 幻觉问题吗？怎么解决的？

参考答案：

遇到过，在智能客服项目中，Agent 有时会编造不存在的产品功能来回答用户问题。

解决方案：

- 1. 知识库兜底：在 AGENTS.md 中明确规定，回答必须基于 FAQ 知识库内容，找不到答案就创建工单，绝不编造。
- 2. 引用来源：要求 Agent 在回答时标注信息来源（如「根据我们的 FAQ 文档…」），方便用户验证。
- 3. 置信度判断：如果 Agent 对匹配结果的置信度低于阈值，提示用户「这个回答可能不够准确，建议联系人工客服」。
- 4. 反馈闭环：用户评价机制，持续收集不准确的回答，补充到知识库。

题目六：如何保证 Agent 工具调用的可靠性？

参考答案：

工具调用的可靠性保障需要从多个层面入手：

- 1. 超时控制 - 每个工具调用设置超时时间 - 超时后返回明确的错误信息而非一直等待
- 2. 重试机制 - 可重试的操作（如网络请求）设置自动重试 - 幂等操作最多重试 3 次 - 非幂等操作不重试
- 3. 错误处理 - 工具返回结构化的错误信息（error code + message） - Agent 能根据错误类型决定下一步操作（重试/降级/上报）
- 4. 降级策略 - 如果 MCP Server 不可用，降级为基本功能 - 例如数据库查询不可用时，提示用户稍后重试
- 5. 监控告警 - 记录工具调用的成功率和延迟 - 异常率超过阈值触发告警

题目七：你是如何做 Nanobot 项目的技术选型的？

参考答案：

选择 Nanobot 基于以下技术对比分析：

| 维度 | Nanobot | LangChain | 自研 |
|-------|----------|-----------|----------|
| 上手成本 | 低（配置驱动） | 中（需学习框架） | 高（全部自己写） |
| 工具扩展 | MCP 标准协议 | 私有协议 | 自定义协议 |
| 记忆系统 | 原生支持 | 需要额外开发 | 需要额外开发 |
| 多平台 | 原生支持 | 需要适配 | 需要适配 |
| 部署复杂度 | 低 | 中 | 高 |
| 生态兼容 | MCP 生态 | 自有生态 | 无生态 |

Nanobot 的核心优势是：1. 配置驱动：通过 AGENTS.md + config.json 定义 Agent 行为，不需要写框架代码 2. **MCP** 标准：工具扩展基于开放的 MCP 协议，而非私有 API 3. 原生记忆：MEMORY.md 开箱即用，无需自己管理向量数据库 4. 技能系统：SKILL.md 提供模块化的能力扩展机制

题目八：如何测试一个 **AI Agent** 系统？

参考答案：

AI Agent 测试分为四个层次：

1. 单元测试（工具层） - 测试每个 MCP Server 工具的输入输出 - 测试安全检查模块（如 SQL 注入检测） - 测试边界条件（超时、空输入、超长输入）
2. 集成测试（交互层） - 测试 Agent 与 MCP Server 的完整交互流程 - 测试多工具串联调用（如先 list_tables 再 query_database） - 测试错误恢复（工具调用失败后 Agent 的行为）
3. 端到端测试（场景层） - 编写典型用户场景的测试用例 - 验证 Agent 回复的准确性和相关性 - 测试多轮对话的上下文保持能力
4. 安全测试（攻击层） - 测试 prompt 注入攻击 - 测试越权操作（试图执行禁止的命令） - 测试输入各种恶意 SQL

在我的数据库 MCP Server 项目中，编写了 11 个安全测试用例，覆盖正常查询和常见攻击向量。

题目九：如果 **Agent** 的响应延迟太高，你会怎么优化？

参考答案：

响应延迟优化从四个方向入手：

1. 减少 **LLM** 调用次数 - 优化 prompt，让 Agent 一次调用就完成任务，而不是多轮推理 - 将常用操作模板化，减少 Agent 的决策环节
 2. 工具调用优化 - 数据库查询加缓存（高频查询的结果缓存 5 分钟） - 并行调用无依赖关系的工具 - 设置合理的超时时间
 3. 模型选择 - 简单任务用小模型（快速响应） - 复杂分析任务用大模型（准确回答）
 4. 异步处理 - 耗时查询改为异步执行 - 先给用户一个「正在处理」的回复 - 查询完成后再推送结果
-

题目十：Nanobot 的 **Skill** 系统和传统 **RAG** 有什么区别？

参考答案：

这是一个很好的问题，它们解决的是不同层面的问题：

Skill 系统（程序化知识） - 定义 Agent「怎么做」：操作步骤、工具使用方法、输出格式 - 是过程性知识的载体 - 例如：服务器监控 Skill 定义了用哪些命令采集数据、怎么分析结果 - 不需要向量化和语义搜索

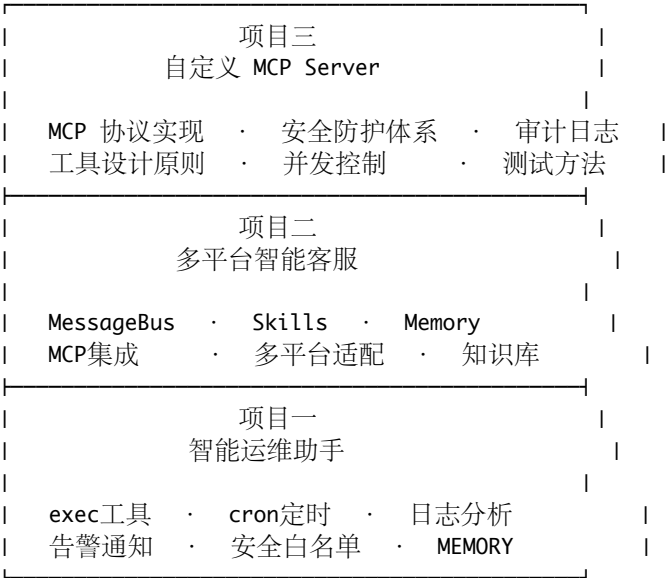
RAG（事实性知识） - 提供 Agent「知道什么」：产品文档、FAQ、百科知识 - 是声明性知识的载体 - 例如：客服知识库包含产品功能说明、常见问题答案 - 需要向量化存储和语义检索

在实际项目中，两者互补： - 客服项目中，FAQ 知识库用 read_file 读取 Markdown 文件（简化版 RAG） - Skill 定义了查找和回复的流程 - 如果知识库规模很大（1000+ 条目），才需要引入真正的向量数据库做 RAG

小结

三个项目的核心技能图谱

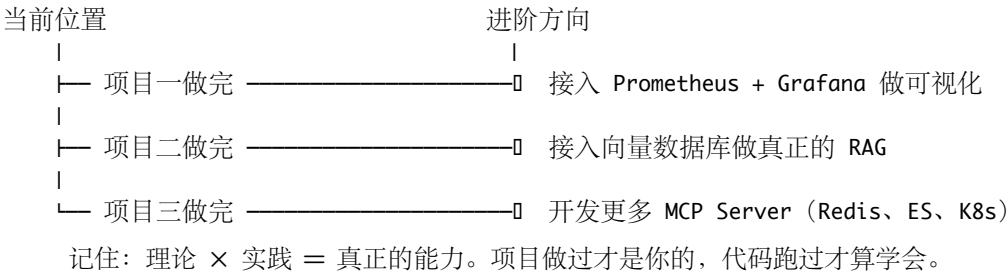
Nanobot 实战技能图谱



面试建议

- 1. 至少精做一个项目：三个项目不必全做，但至少要有有一个做到能完整运行的程度
- 2. 准备好 **STAR** 话术：每个项目的 **Situation-Task-Action-Result** 要能流利表达
- 3. 理解设计决策背后的 **why**：面试官不只关心你做了什么，更关心你为什么这样做
- 4. 准备好失败案例：准备一个「遇到问题 → 解决问题」的故事，展示问题解决能力
- 5. 代码要能运行：如果面试官要求现场演示，确保你的代码是可运行的

下一步学习路线



第 13 章 AI Agent 面试八股文大全（134 题）

本章定位：面试前突击宝典，覆盖 Agent 基础、Nanobot 源码、MCP 协议、大模型原理、训练优化、RAG、多智能体、系统设计、工程实践九大板块，共 134 道高频面试题，每题附详细答案与加分点。

目录

- 一、Agent 基础 (Q1-Q15)
 - 二、Nanobot 专项 (Q16-Q35)
 - 三、MCP 协议 (Q36-Q50)
 - 四、大模型基础 (Q51-Q70)
 - 五、训练优化 (Q71-Q85)
 - 六、Nanobot 源码与设计深度题 (Q86-Q100)
 - 七、RAG (Q101-Q110)
 - 八、多智能体 (Q111-Q118)
 - 九、系统设计 (Q119-Q126)
 - 十、工程实践 (Q127-Q134)
-

一、Agent 基础 (Q1-Q15)

Q1. 什么是 AI Agent？它与普通 LLM 调用有什么区别？

答案：AI Agent 是一种以大语言模型为「大脑」，能自主感知环境、制定计划、调用工具、观察结果并迭代执行的智能体系统。与普通 LLM 调用最核心的区别在于三点：第一，Agent 具备工具使用能力，可以调用外部 API、读写文件、执行代码；第二，Agent 拥有记忆系统，能在多轮交互中保持上下文并积累经验；第三，Agent 具备规划与反思能力，可以将复杂任务拆解为子步骤，并根据中间结果动态调整策略。普通 LLM 调用本质是「一问一答」的无状态函数，而 Agent 是一个持续运行的决策循环。

面试加分点：可以画出 Agent = LLM + Memory + Tools + Planning 的经典架构图来辅助说明，并提到 Manus、Devin 等 2025-2026 年的标志性 Agent 产品来展示行业认知。

Q2. 请解释 ReAct (Reasoning + Acting) 框架的核心思想

答案：ReAct 框架由 Yao et al. (2022) 提出，核心思想是将大模型的推理 (**Reasoning**) 和行动 (**Acting**) 交替进行。具体流程为：模型先产生一段 Thought (思考当前情况、分析下一步该做什么)，然后选择一个 Action (如调用搜索工具)，得到 Observation (工具返回结果)，再进入下一轮 Thought-Action-Observation 循环，直到得出最终答案。ReAct 的关键优势在于：相比纯 Chain-of-Thought 只推理不行动，它能获取实时外部信息；相比纯 Act 模式 (直接调用工具)，它增加了推理链使行为可解释、可调试。Nanobot 的 AgentLoop 本质上就是一个 ReAct 循环实现。

面试加分点：能说出 ReAct 论文的全称「ReAct: Synergizing Reasoning and Acting in Language Models」，并能对比 ReAct 与 Reflexion、Plan-and-Solve 等其他推理框架的差异。

Q3. Agent 的规划 (Planning) 模块通常有哪些实现方式？

答案：规划模块是 Agent 拆解复杂任务为可执行步骤的核心能力，主要有以下实现方式：(1) 任务分解 (**Task Decomposition**)：如 Chain-of-Thought 逐步推理，或 Tree-of-Thought 多路径探索；(2) 子目标拆分：将大任务分解为有依赖关系的子目标，如 HuggingGPT 先规划工具调用链再逐一执行；(3) **Reflection**/自我纠错：如 Reflexion 框架，在执行失败后反思原因并修改计划；(4) 外部规划器：使用经典 AI 规划算法 (如 PDDL) 或代码生成来制定计划；(5) **Orchestrator-Workers** 模式：主 Agent 负责规划和分配，子 Agent 负责执行具体步骤。Nanobot 中 SubagentManager 就是 Orchestrator-Workers 模式的实现。

面试加分点：能提到 Planning 目前最大的痛点是长期规划的可靠性不足，以及 Agent 容易陷入「死循环」的问题，这也是 Nanobot 设置 15 次迭代上限的原因。

Q4. Agent 的记忆系统分为哪几种？各自的作用是什么？

答案：Agent 记忆系统通常分为三层：(1) 短期记忆 (**Working Memory**)：即当前会话的上下文窗口，包括用户消息、工具调用结果等。受限于模型的 context window 长度（如 128K tokens），是最直接但容量有限的记忆形式。(2) 长期记忆 (**Long-term Memory**)：跨会话持久化的信息，如用户偏好、历史决策、项目知识等，通常通过向量数据库或文件系统实现。Nanobot 的 MemoryConsolidator 就负责将短期记忆压缩写入 `~/nanobot/memory/` 目录。(3) 外部知识库 (**External Knowledge**)：通过 RAG 检索的文档、数据库、API 等外部数据源，本质是可按需加载的「参考资料」。三者协同工作：短期记忆保证连贯性，长期记忆保证个性化，外部知识保证准确性。

面试加分点：可以将这三层类比为人类的「工作记忆」「海马体长期记忆」和「图书馆查阅」，面试官会觉得你理解深刻。

Q5. 什么是工具调用 (Tool Calling)？它在 Agent 中的作用是什么？

答案：工具调用是 Agent 与外部世界交互的核心机制。在技术实现上，模型在生成回复时，可以选择输出一个特殊格式的结构化指令（通常是 JSON 格式），指定要调用的工具名称和参数。运行时框架（如 Nanobot 的 AgentLoop）解析这个指令，执行对应的工具函数，将结果以 `tool_result` 消息的形式追加到对话历史中，再由模型继续生成下一步。工具调用使 Agent 能力从「纯文本生成」扩展到「读写文件、执行代码、查询数据库、调用 API」等几乎无限的操作空间。核心挑战包括：工具选择的准确性、参数生成的正确性、错误处理的鲁棒性。

面试加分点：能对比 OpenAI 的 Function Calling 和 MCP 协议的工具调用机制差异，前者是模型提供商特定格式，后者是开放协议标准。

Q6. Agent 如何进行工具选择？有哪些策略？

答案：Agent 工具选择的策略主要有：(1) 全量注入：将所有可用工具的描述放入 system prompt，由 LLM 自主选择。简单直接但 tool 数量多时会占用大量上下文空间。(2) 渐进披露 (**Progressive Disclosure**)：先给模型少量核心工具，当模型需要特定能力时再动态加载。Nanobot 的 Skills 系统就采用这种策略，通过 `description` 字段让模型判断是否需要读取完整 Skill 文件。(3) 语义检索：将工具描述向量化，根据当前任务语义检索最相关的 Top-K 工具注入。(4) 分层路由：先由一个「路由模型」判断任务类型，再加载对应类别的工具集。实际工程中通常组合使用这些策略。

面试加分点：提到 Nanobot 中 `_TOOL_RESULT_MAX_CHARS = 16000` 的截断设计，说明工具调用结果也需要控制大小以保护上下文窗口。

Q7. 什么是 Reflection (反思机制)？在 Agent 中如何实现？

答案：Reflection 是 Agent 在执行过程中对自身行为和结果进行评估、纠错的能力。实现方式包括：(1) 自我批评 (**Self-Critique**)：在生成答案后，让模型评估自己的输出质量，发现问题后重新生成；(2) **Reflexion** 框架：维护一个反思记忆，记录历次尝试的失败原因和教训，在下次尝试时注入这些经验；(3) 验证-重试循环：执行工具调用后验证结果是否符合预期，不符合则调整策略重试；(4) 外部评判：用另一个 LLM 或规则引擎评估 Agent 的输出质量。Nanobot 的 AgentLoop 中，模型会观察每次工具调用的结果 (Observation)，这本身就是一种隐式反思——如果结果不理想，模型会在下一轮 Thought 中调整策略。

面试加分点：提到 Reflection 的风险：过度反思可能导致 Agent 陷入「自我怀疑循环」，需要设置最大重试次数来避免。

Q8. 如何评测一个 Agent 系统的效果？有哪些常用指标？

答案：Agent 评测是当前领域的难点之一，常用指标包括：(1) 任务完成率 (**Success Rate**)：在标准化基准上完成任务的比例，如 SWE-bench 衡量代码修复能力；(2) 步骤效率 (**Step Efficiency**)：完成任务所需的平均步骤数，步骤越少说明规划能力越强；(3) 工具调用准确率：选择正确工具和生成正确参数的比例；(4) 端到端延迟 (**E2E Latency**)：从用户发出请求到得到最终结果的总耗时；(5) **Token** 消耗量：直接关系到成本；(6) 鲁棒性：面对边界情况、模糊

指令、工具失败时的表现；(7) 用户满意度：主观评价。业界常用的 Benchmark 有 SWE-bench、WebArena、GAIA、AgentBench 等。

面试加分点：能提到「Agent 评测的不可重复性问题」——同一个 Agent 面对同一个任务可能得到不同结果，因此需要多次运行取统计值。

Q9. Workflow Agent 和 Autonomous Agent 有什么区别？

答案：Workflow Agent（工作流 Agent）按照预定义的流程图执行，每个步骤和分支条件都是开发者事先设计好的，类似于传统的 DAG 工作流引擎，只是在每个节点使用 LLM 处理。优点是可控性强、可预测、易调试；缺点是灵活性差、无法应对预定义流程之外的情况。Autonomous Agent（自主 Agent）则由 LLM 自主决定下一步做什么，包括选择工具、判断终止条件等，开发者只定义目标和可用工具。优点是灵活性强、能处理开放性任务；缺点是不可预测、可能产生错误决策、调试困难。Nanobot 属于 Autonomous Agent 范式，而 Dify 等低代码平台更偏向 Workflow Agent。实际应用中两者经常混合使用。

面试加分点：可以提到 Anthropic 在 2025 年发布的 Agent 设计模式分类，将 Agent 分为 Prompt Chaining、Routing、Parallelization、Orchestrator-Workers、Evaluator-Optimizer 等模式。

Q10. 请解释 Orchestrator-Workers 模式

答案：Orchestrator-Workers 是一种多 Agent 协作模式，由一个「编排器（Orchestrator）」Agent 负责任务分解和分配，多个「工人（Worker）」Agent 负责执行具体子任务。工作流程为：用户请求 → Orchestrator 分析任务并拆解为子任务 → 将子任务分发给不同 Worker → Worker 各自执行并返回结果 → Orchestrator 汇总结果并返回给用户。Nanobot 的 SubagentManager 就是这种模式的实现：主 Agent 通过 Task 工具启动子 Agent，每个子 Agent 有独立的 AgentLoop 和上下文，执行完毕后将结果返回主 Agent。关键设计考虑包括：子任务的粒度划分、结果的合并策略、失败子任务的重试机制、以及防止递归嵌套过深（Nanobot 限制 15 次迭代）。

面试加分点：对比 Orchestrator-Workers 与「流水线（Pipeline）」模式的差异——前者是并行扇出再汇总，后者是串行传递。

Q11. Agent 系统中的状态管理有哪些挑战？

答案：Agent 状态管理的核心挑战包括：(1) 上下文膨胀：随着对话轮次增加和工具调用累积，上下文 token 数持续增长，可能超出模型窗口限制。解决方案包括对话压缩（如 Nanobot 的 MemoryConsolidator）、滑动窗口、摘要替换等。(2) 状态一致性：多个工具调用可能修改共享状态（如文件系统），需要保证操作的原子性和一致性。Nanobot 使用 session_locks 和 _concurrency_gate(3) 来控制并发。(3) 持久化与恢复：Agent 可能被中断后恢复，需要保存足够的状态信息。(4) 多 Agent 状态同步：在多 Agent 架构中，不同 Agent 之间的状态共享和隔离需要精心设计。(5) 幂等性：相同的工具调用应该产生相同的结果，避免副作用累积。

面试加分点：结合 Nanobot 源码中 MessageBus 的双队列设计来解释状态管理的实际工程方案。

Q12. 如何保证 Agent 工具调用的可靠性？

答案：保证工具调用可靠性需要多层防护：(1) 输入验证：通过 JSON Schema 定义工具参数的类型和约束，在调用前验证参数合法性。Nanobot 的 ToolRegistry 就使用 input_schema 字段定义参数规范。(2) 超时控制：为每个工具调用设置合理的超时时间，避免单个工具阻塞整个 Agent 循环。(3) 重试机制：对于可重试的错误（如网络超时、API 限流），实现指数退避重试。(4) 结果截断：限制工具返回结果的大小，如 Nanobot 的 _TOOL_RESULT_MAX_CHARS = 16000，防止超大结果撑爆上下文。(5) 错误包装：将工具错误信息以结构化方式返回给模型，使模型能据此调整策略。(6) 沙箱隔离：将危险操作（如代码执行）放在沙箱环境中运行，防止系统损坏。

面试加分点：提到 Nanobot 中当工具调用失败时，错误信息会被包装为 tool_result 返回给模型，让模型自行决定是重试还是换一种方式。

Q13. 什么是 Agent 的「幻觉」问题？如何缓解？

答案：Agent 的幻觉问题继承自底层 LLM，但在 Agent 场景下更为严重，因为幻觉可能导致错误的工具调用和连锁错误。具体表现包括：编造不存在的工具名称、生成无效的参数值、虚构工具返回结果、错误的事实推理等。缓解策略有：(1) **Grounding** (接地)：通过 RAG 检索或工具调用获取真实数据，减少模型「编造」的空间；(2) **Verification** (验证)：对关键输出进行二次验证，如检查文件是否真的存在、API 返回是否有效；(3) **Constrained Decoding** (受限解码)：限制模型只能从已注册的工具体表中选择，而非自由生成工具名；(4) **Prompt Engineering**：在 system prompt 中明确要求模型「不确定时说不知道，而非编造答案」。

面试加分点：能区分 Agent 场景中「模型幻觉」和「执行错误」的区别——前者是模型认知层面的错误，后者是工具层面的真实失败。

Q14. Agent 系统中如何处理长任务 (Long-horizon Tasks)？

答案：长任务处理是 Agent 的核心难点之一。主要策略有：(1) 分层规划：将长任务分解为高层计划和低层执行步骤，高层计划相对稳定，低层步骤可以灵活调整。(2) 检查点机制 (**Checkpointing**)：定期保存中间状态，失败时可以从检查点恢复而非从头开始。(3) 记忆压缩：使用摘要技术压缩历史交互记录，保留关键信息而丢弃冗余细节。Nanobot 的 MemoryConsolidator 就是这种机制的实现。(4) 子任务委托：通过 Orchestrator-Workers 模式将子任务委托给独立的 Agent，每个子 Agent 只需关注局部上下文。(5) 进度跟踪：维护一个显式的任务进度列表，帮助 Agent 知道「已完成什么」和「还需要做什么」。

面试加分点：提到 Nanobot 中 save_memory 虚拟工具允许 Agent 主动将重要信息持久化，这是处理长任务时的关键能力。

Q15. 请对比 Agent 框架的几种主流架构模式

答案：主流 Agent 架构模式可分为五种：(1) **Prompt Chaining** (提示链)：将任务分解为固定步骤，每步的输出作为下一步的输入，适合流程明确的任务；(2) **Routing** (路由)：根据输入类型分发到不同处理分支，类似 if-else 的 LLM 版本；(3) **Parallelization** (并行化)：多个 LLM 调用同时执行，结果聚合。分为「切片并行」和「投票并行」两种子模式；(4) **Orchestrator-Workers** (编排-工人)：一个中心 Agent 动态分配任务给多个 Worker Agent，适合复杂开放任务；(5) **Evaluator-Optimizer** (评估-优化)：一个 Agent 生成输出，另一个 Agent 评估并提出改进建议，循环迭代直到质量达标。Nanobot 核心采用 ReAct 循环 + Orchestrator-Workers 的组合模式。

面试加分点：这五种模式来自 Anthropic 的官方博客「Building effective agents」(2024)，面试时引用出处会显得很专业。

二、Nanobot 专项 (Q16-Q35)

Q16. Nanobot 是什么？它在 Agent 框架生态中的定位是什么？

答案：Nanobot 是由 HKUDS (香港大学数据科学实验室) 开源的一个轻量级 AI Agent 框架，GitHub 上获得 37K+ stars。它的核心定位是「用最少的代码实现最完整的 Agent 能力」——整个核心代码只有约 4000 行 Python，但涵盖了 Agent 循环、记忆管理、工具注册、MCP 集成、子 Agent 管理、多平台适配等完整功能。与 LangChain (10 万+ 行代码、大量抽象层) 形成鲜明对比，Nanobot 的设计哲学是「极简但完整」，强调代码可读性和架构清晰度。它非常适合作为学习 Agent 内部原理的教学项目，同时也具备生产可用的能力。

面试加分点：提到 Nanobot 的代码量相当于一个优秀的课程设计项目，但架构设计达到了工业级水准，这种「小而精」的设计思想本身就值得学习。

Q17. 请详细描述 Nanobot AgentLoop 的运行流程

答案：AgentLoop 是 Nanobot 的核心引擎，运行流程如下：(1) 初始化阶段：加载配置文件，实例化 ToolRegistry、MemoryConsolidator 等组件，注册所有可用工具 (包括 MCP 工具和内置工具)。(2) 上下文构建：ContextBuilder 将 system prompt、历史消息、可用工具列表、记忆片段组装为完整的请求体。(3) LLM 调用：将构建好的上下文发

送给大模型 API (如 Claude/GPT), 获取模型响应。(4) 响应解析: 解析模型返回的内容, 判断是纯文本回复还是工具调用请求。(5) 工具执行: 如果是工具调用, 通过 ToolRegistry 查找并执行对应工具函数, 收集返回结果。(6) 结果注入: 将工具执行结果以 `tool_result` 消息形式追加到对话历史。(7) 循环判断: 如果模型继续请求工具调用, 回到步骤 3; 如果模型输出最终文本回复, 结束循环。(8) 记忆持久化: 会话结束时, MemoryConsolidator 将重要信息写入长期记忆。

面试加分点: 能画出 AgentLoop 的流程图, 并指出它本质上是 ReAct 框架的工程实现。

Q18. ContextBuilder 在 Nanobot 中的作用是什么? 它如何管理上下文?

答案: ContextBuilder 负责在每次 LLM 调用前组装完整的请求上下文, 是「控制模型看到什么信息」的核心组件。它的主要职责包括: (1) **System Prompt** 组装: 将基础 system prompt、Skills 描述、用户自定义规则等拼接为完整的系统提示词; (2) 消息历史管理: 维护对话历史列表, 包括用户消息、助手回复、工具调用和工具结果; (3) 上下文窗口控制: 当消息历史过长时, 调用 MemoryConsolidator 进行压缩, 确保不超出模型的 context window; (4) 工具列表注入: 根据当前状态动态决定注入哪些工具的描述信息; (5) 记忆片段注入: 将相关的长期记忆片段检索并注入到上下文中。ContextBuilder 的设计体现了一个核心原则: 上下文是 **Agent** 最宝贵的资源, 必须精心管理。

面试加分点: 提到 ContextBuilder 与 Anthropic 的 Prompt Cache 技术结合可以大幅降低 API 成本——重复的 system prompt 部分不需要重新计算。

Q19. MemoryConsolidator 的压缩策略是什么?

答案: MemoryConsolidator 是 Nanobot 的记忆管理核心, 负责两个层面的记忆处理: (1) 会话内压缩 (**In-session Consolidation**): 当对话历史的 token 数接近上下文窗口限制时, MemoryConsolidator 会将较早的对话轮次压缩为摘要。压缩策略是让 LLM 自己生成对话摘要, 保留关键决策、重要事实和用户偏好, 丢弃冗余的中间推理过程。压缩后的摘要替换原始消息, 释放上下文空间。(2) 跨会话持久化 (**Cross-session Persistence**): 会话结束时 (或通过 `save_memory` 虚拟工具主动触发), 将重要信息以 Markdown 文件形式写入 `~/.nanobot/memory/` 目录。下次会话开始时, ContextBuilder 会读取这些记忆文件并注入到上下文中。这种分层设计兼顾了短期连贯性和长期个性化。

面试加分点: 对比 Nanobot 的记忆方案与 MemGPT 的「虚拟内存分页」方案的思路差异。

Q20. ToolRegistry 的设计和工作原理是什么?

答案: ToolRegistry 是 Nanobot 的工具注册中心, 采用注册表模式 (Registry Pattern) 管理所有可用工具。核心设计包括: (1) 统一接口: 每个工具定义为一个包含 `name`、`description`、`input_schema` (JSON Schema 格式的参数定义) 和 `handler` (执行函数) 的对象。(2) 动态注册: 支持在运行时动态添加和移除工具, MCP 工具在建立连接后自动注册到 ToolRegistry 中。(3) 工具查找: AgentLoop 解析模型的工具调用请求时, 通过工具名在 Registry 中查找对应的 handler。(4) 工具描述导出: ContextBuilder 需要工具列表时, ToolRegistry 导出所有工具的 `name`、`description`、`input_schema` 供模型选择。(5) 结果处理: 执行工具后统一处理返回值, 包括大小截断 (`_TOOL_RESULT_MAX_CHARS = 16000`) 和错误包装。这种设计使内置工具和 MCP 外部工具对 Agent 来说完全透明统一。

面试加分点: 能说出 ToolRegistry 的设计本质是「依赖注入」思想——AgentLoop 不关心具体工具的实现细节, 只依赖统一的工具接口。

Q21. SubagentManager 如何防止递归爆炸? 为什么设置 15 次迭代限制?

答案: SubagentManager 管理子 Agent 的创建和生命周期, 其 15 次迭代上限是一个关键的安全设计。原因和机制如下: (1) 递归风险: 主 Agent 可以通过 Task 工具启动子 Agent, 如果子 Agent 也启动子-子 Agent, 可能形成无限递归, 导致 token 消耗失控和系统资源耗尽。15 次上限确保单个子 Agent 的执行步骤有界。(2) 成本控制: 每次迭代都意味着一次 LLM API 调用, 15 次是在任务完成概率和成本之间的经验平衡点。(3) 死循环检测: 如果 Agent 在 15 次迭代内无法完成任务, 通常意味着任务描述不清晰或 Agent 陷入了重复操作, 此时强制终止比继续消耗更合理。(4) 深度限制: 除了单个 Agent 的迭代限制, 还可以限制 Agent 嵌套的深度层数, 防止 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow \dots$ 的深层递归链。

面试加分点: 能类比操作系统中进程 fork 的资源限制 (如 ulimit), 说明这是一种常见的系统保护模式。

Q22. MessageBus 的双队列设计是什么？解决了什么问题？

答案：MessageBus 是 Nanobot 内部的消息传递机制，采用双队列设计来解耦消息的生产和消费。具体来说：（1）输入队列（**Input Queue**）：接收来自外部的消息，如用户输入、Channel 适配器转发的消息。消息按到达顺序排队，等待 AgentLoop 处理。（2）输出队列（**Output Queue**）：AgentLoop 生成的回复消息放入输出队列，由对应的 Channel 适配器取出并发送给用户。双队列设计解决了几个关键问题：（a）异步解耦：消息接收和 Agent 处理异步执行，不会因为 Agent 处理慢而阻塞消息接收；（b）多平台适配：不同平台（CLI、飞书、钉钉）的消息格式不同，MessageBus 提供统一的内部消息格式，Channel 适配器负责格式转换；（c）并发控制：通过队列机制天然实现消息的顺序处理，避免并发冲突。

面试加分点：能对比 MessageBus 与 Kafka/RabbitMQ 等消息队列的思路相似性，说明即使是轻量级系统也遵循相同的架构原则。

Q23. Nanobot 的 Skills 渐进披露（Progressive Disclosure）机制是什么？

答案：Skills 渐进披露是 Nanobot 管理大量技能/工具描述的优雅策略。核心思想是：不要一次性把所有工具的完整信息塞给模型，而是分层逐步展示。实现方式为：（1）第一层：标题 + 简介：在 system prompt 中只注入每个 Skill 的名称和一行描述（如 "create-rule: 创建 Cursor 规则文件"），占用极少 token。（2）第二层：按需展开：当模型判断某个 Skill 与当前任务相关时，通过读取 Skill 文件获取完整的说明、参数定义和使用示例。这种设计的优势包括：节省上下文空间、降低模型在大量工具中「选择困难」的概率、支持无限扩展 Skill 数量而不增加基础 prompt 长度。这与 UI 设计中的「渐进披露」原则完全一致——先展示概要，用户需要时再展示详情。

面试加分点：可以对比 LangChain 中每次都注入所有工具完整描述的方式，说明 Nanobot 的方案在工具数量较多时更高效。

Q24. Nanobot 如何集成 MCP 协议？

答案：Nanobot 通过 MCP Client 的方式集成 MCP 协议，流程如下：（1）配置声明：在 ~/.nanobot/config.json 中声明 MCP Server 列表，包括每个 Server 的启动命令、传输方式（stdio 或 HTTP+SSE）和连接参数。（2）连接建立：AgentLoop 启动时，MCPClient 模块按照配置逐个启动 MCP Server 进程并建立连接。（3）工具发现：连接建立后，通过 MCP 协议的 tools/list 方法获取每个 Server 提供的工具列表（名称、描述、参数 Schema）。（4）注册到 ToolRegistry：将 MCP 工具统一注册到 ToolRegistry，与内置工具无差别对待。（5）工具调用：当 Agent 决定调用某个 MCP 工具时，ToolRegistry 找到对应的 MCP handler，通过 MCP 协议发送 tools/call 请求到对应 Server，接收结果并返回。这种设计使 Nanobot 可以通过 MCP 无限扩展工具能力。

面试加分点：提到 Nanobot 的 MCP 集成是「Host-Client」模式——Nanobot 同时是 MCP Host（宿主应用）和 Client（协议客户端）。

Q25. _TOOL_RESULT_MAX_CHARS = 16000 这个常量的设计考量是什么？

答案：这个常量定义了工具调用返回结果的最大字符数，超过时会被截断。设计考量包括：（1）保护上下文窗口：工具返回的结果会被追加到对话历史中，如果不限大小，一次文件读取可能返回几十万字符，瞬间耗尽上下文空间。16000 字符约 4000-5000 tokens，是一个合理的「既能包含足够信息又不会过度占用」的平衡点。（2）降低成本：更多 token 意味着更高的 API 费用。（3）提升模型处理质量：研究表明模型在处理超长输入时效果会下降（Lost in the Middle 问题），截断后模型反而能更好地利用信息。（4）截断策略：通常保留头部和尾部信息，中间部分截断，因为文件/结果的开头和结尾往往包含最关键的信息。

面试加分点：能提到这个值可以根据使用的模型的 context window 大小进行调整——使用 200K 窗口的模型时可以适当增大。

Q26. Nanobot 的 session_locks 并发控制机制是什么？

答案：session_locks 是 Nanobot 实现会话级并发控制的核心机制。每个会话(session)关联一个异步锁(asyncio.Lock)，确保同一个会话中同一时刻只有一个 AgentLoop 在执行。这解决了以下问题：(1) 消息竞态：如果用户快速连续发送多条消息，没有锁的话多个 AgentLoop 可能同时操作同一个对话历史，导致消息交叉和状态混乱。(2) 资源保护：同一会话的文件操作、数据库操作等需要串行执行以保证一致性。(3) Token 控制：防止同一会话同时发起多个 LLM 调用导致成本失控。session_locks 使用字典结构 {session_id: Lock} 管理，会话首次活动时创建锁，后续请求需要先获取锁才能执行。

面试加分点：能区分 session_locks（会话级互斥）和 _concurrency_gate（全局并发度限制）的不同层次——前者保证会话内有序，后者保证系统级资源可控。

Q27. _concurrency_gate(3) 的作用和设计原理是什么？

答案：_concurrency_gate(3) 是一个全局信号量(Semaphore)，限制整个 Nanobot 实例同时执行的 AgentLoop 数量不超过 3 个。设计原理包括：(1) 资源保护：每个 AgentLoop 会占用显著的内存（对话历史）和网络资源（LLM API 连接），无限制并发可能导致内存溢出或 API 限流。(2) API Rate Limit 适配：大模型 API 通常有并发和 QPM 限制，3 个并发是一个保守但安全的默认值。(3) 服务质量保证：限制并发可以确保每个用户请求都能获得足够的计算资源和响应速度，避免资源争抢导致所有请求都变慢。(4) 可配置性：这个值可以根据实际部署环境调整——单机开发环境用 3 足够，生产环境可以通过水平扩展实例来增加总并发数。这与 Web 服务器的 worker 数量限制是同一种思路。

面试加分点：可以提到 Python asyncio.Semaphore 的使用方式，以及与线程池 ThreadPoolExecutor 的 max_workers 参数的类比。

Q28. save_memory 虚拟工具的设计巧妙在哪里？

答案：save_memory 是一个「虚拟工具」——它在 ToolRegistry 中注册但并不执行外部操作，而是由 AgentLoop 内部特殊处理。其巧妙之处在于：(1) 模型主动持久化：让模型（而非开发者的硬编码逻辑）决定什么信息值得保存到长期记忆。模型比任何规则引擎更能判断「这条信息对未来有用」。(2) 统一的工具调用接口：从模型视角看，save_memory 与其他工具（如读文件、搜索）没有区别，都是通过 tool_call 格式调用，不需要额外的特殊机制。(3) 自然语言描述：保存的内容是模型用自然语言总结的关键信息，而非原始数据的机械复制，信息密度更高。(4) 透明可审计：保存的记忆以 Markdown 文件形式存储，人类可以随时查看、编辑和管理。这种设计体现了「让 AI 管理 AI 的知识」的思想。

面试加分点：对比传统的「基于规则的记忆管理」（如按时间窗口自动保存）和 Nanobot 的「基于模型判断的记忆管理」，后者更智能更灵活。

Q29. Nanobot 的 Channel 适配器模式是如何设计的？

答案：Channel 适配器模式是 Nanobot 实现多平台接入的核心架构。设计如下：(1) 适配器接口：定义统一的 Channel 接口，包括 receive_message()（接收消息）、send_message()（发送回复）等方法。(2) 具体实现：每个平台（CLI、飞书、钉钉、Slack）有一个独立的 Channel 适配器实现，负责该平台的消息格式转换、认证鉴权、Webhook 处理等平台特定逻辑。(3) 消息标准化：所有 Channel 适配器将平台特定格式的消息转换为 Nanobot 内部统一的消息格式，然后通过 MessageBus 传递给 AgentLoop。(4) 响应适配：AgentLoop 的输出通过 MessageBus 传到对应 Channel 适配器，由适配器转换为平台特定格式发送。这种设计遵循「适配器模式(Adapter Pattern)」和「六边形架构(Hexagonal Architecture)」的思想，使核心 Agent 逻辑与外部平台完全解耦。

面试加分点：提到新增一个平台只需实现一个新的 Channel 适配器，不需要修改 AgentLoop 的任何代码，体现了开闭原则(OCP)。

Q30. Nanobot 的「配置驱动组装」理念是什么？

答案：「配置驱动组装」是 Nanobot 的核心设计理念之一，指通过配置文件而非硬编码来决定 Agent 的能力组合。具体体现为：(1) config.json 驱动：模型选择、API 密钥、MCP Server 列表、工具启用/禁用、系统提示词等全部在配置文件中声明。(2) Skills 目录扫描：将 .nanobot/skills/ 目录下的 Markdown 文件自动识别为技能，无需修改代码

即可扩展能力。(3) **Rules** 动态加载: `.nanobot/rules/` 目录下的规则文件会被自动注入到 `system prompt` 中, 通过添加文件即可修改 Agent 行为。(4) **MCP** 插件化: 通过配置 `MCP Server` 列表来「插拔」外部工具能力。这种设计的优势是: 同一份 `Nanobot` 代码, 通过不同配置可以组装出完全不同能力的 Agent——一个做代码助手、一个做客服机器人、一个做数据分析师。

面试加分点: 类比 `Spring` 框架的 `IoC/DI` 思想——组件的具体实现由配置决定而非代码硬编码。

Q31. Nanobot 如何利用 Prompt Cache 优化成本?

答案: `Prompt Cache` 是大模型 API (如 `Anthropic Claude`) 提供的缓存优化机制, `Nanobot` 利用它大幅降低 API 成本。原理如下: (1) 缓存命中: 当多次 API 调用的 `prompt` 前缀相同时, API 服务器可以复用之前的 `KV Cache` 计算结果, 只对新增部分计算注意力, 大幅减少计算量。(2) **Nanobot** 的利用方式: 将相对稳定的内容 (`system prompt`、工具定义、`Skills` 描述) 放在 `prompt` 的前部, 将变化的内容 (最新用户消息、最近工具结果) 放在后部。这样每次调用只有后部变化, 前部可以命中缓存。(3) 成本节省: 缓存命中的 `token` 计费通常只有正常价格的 10%, 在长会话中 (`system prompt` 可能占总 `token` 的 30-50%) 节省非常显著。(4) **ContextBuilder** 的配合: `ContextBuilder` 在组装上下文时会刻意保持前缀的稳定性, 避免不必要的变动导致缓存失效。

面试加分点: 能说出 `Anthropic` 的 `Prompt Caching` 具体计费规则: 缓存写入有额外费用但缓存读取打折 90%, 在多轮对话中投入产出比非常高。

Q32. 请对比 Nanobot 与 LangChain 的核心区别

答案: 两者在定位、设计哲学和适用场景上有显著区别:

| 维度 | Nanobot | LangChain |
|------|--------------------------|---------------------------|
| 代码量 | ~4000 行 | 10 万 + 行 |
| 设计哲学 | 极简、透明、可读 | 全面、抽象、模块化 |
| 抽象层次 | 低抽象, 直接看到 API 调用 | 高抽象, 多层封装 |
| 学习曲线 | 低, 源码可半天读完 | 高, 概念和 API 众多 |
| 扩展方式 | MCP 协议 + Skills 文件 | Chain/Agent/Tool 类继承 |
| 生态整合 | 通过 MCP 生态 | 自有 LangSmith/LangGraph 生态 |
| 适用场景 | 轻量级 Agent、学习研究、快速原型 | 大型复杂项目、企业级应用 |
| 工具管理 | 渐进披露 + ToolRegistry | 全量注入 + 类定义 |
| 记忆系统 | MemoryConsolidator 文件持久化 | 多种 Memory 类可选 |

核心区别在于: `Nanobot` 追求「让你理解 Agent 是怎么工作的」, `LangChain` 追求「让你快速搭建 Agent 应用」。

面试加分点: 可以提到 `LangChain` 作者 `Harrison Chase` 自己也承认 `LangChain` 存在过度抽象的问题, 并在 `LangGraph` 中简化了设计。

Q33. 请对比 Nanobot 与 OpenClaw (开源 Agent 框架) 的差异

答案: `OpenClaw` 和 `Nanobot` 都是轻量级 Agent 框架, 但设计思路有差异: (1) 架构风格: `Nanobot` 采用单体架构, 所有组件在同一进程中运行, 通过类和函数组织; `OpenClaw` 更倾向于微服务化设计, 各模块可以独立部署。(2) 工具生态: `Nanobot` 深度集成 `MCP` 协议, 直接复用 `MCP` 生态中的数千个工具服务器; `OpenClaw` 通常使用自定义的工具注册机制。(3) 记忆方案: `Nanobot` 的 `MemoryConsolidator` 使用 `LLM` 进行记忆压缩和持久化, 强调「AI 自我管理」; `OpenClaw` 可能更依赖传统的数据库存储方案。(4) 多平台支持: `Nanobot` 通过 `Channel` 适配器原生支持飞书、钉钉等国内平台; `OpenClaw` 可能需要额外开发适配层。(5) 社区规模: `Nanobot` 有 37K+ stars 的活跃社区, 文档和教程更丰富。选型建议: 学习理解 Agent 原理选 `Nanobot`, 需要微服务化部署选 `OpenClaw`。

面试加分点: 面试中框架对比题不要「踩一捧一」, 而应该客观分析各自的适用场景。

Q34. 基于 Nanobot 学习，简历上可以写哪些项目？

答案：基于 Nanobot 的学习和实践，可以提炼出以下简历项目方向：(1) **Agent** 框架源码研究项目：深入阅读 Nanobot 4000 行源码，理解 AgentLoop、MemoryConsolidator、ToolRegistry 等核心模块的设计与实现，可以展示底层原理理解能力。(2) 自定义 **MCP** 工具服务器：基于 MCP 协议开发特定领域的工具服务器（如 MySQL 查询工具、Jira 集成工具），展示协议理解和工程能力。(3) 多平台 **AI** 助手：基于 Nanobot 的 Channel 适配器开发飞书/钉钉机器人，展示系统集成能力。(4) **Agent** 评测平台：设计评测用例并对 Nanobot Agent 进行系统化评测，展示测试和分析能力。(5) 记忆系统优化：对 MemoryConsolidator 进行改进（如加入向量检索），展示创新能力。

面试加分点：简历中最好写「基于开源项目二次开发」而非「使用了某框架」，前者更能展示深度。

Q35. Nanobot 的 4000 行代码中，哪些设计模式值得学习？

答案：Nanobot 虽然代码量小但包含了丰富的设计模式：(1) 注册表模式 (**Registry Pattern**)：ToolRegistry 统一管理工具的注册和查找；(2) 适配器模式 (**Adapter Pattern**)：Channel 适配器将不同平台的消息格式适配为统一接口；(3) 建造者模式 (**Builder Pattern**)：ContextBuilder 逐步构建复杂的上下文对象；(4) 观察者模式 (**Observer Pattern**)：MessageBus 实现消息的发布-订阅；(5) 策略模式 (**Strategy Pattern**)：不同的模型提供商 (Claude/GPT) 通过统一接口调用；(6) 单例模式 (**Singleton-like**)：配置管理和锁管理使用字典实现的全局状态；(7) 模板方法模式 (**Template Method**)：AgentLoop 定义了执行框架，具体步骤由子组件实现；(8) 外观模式 (**Facade Pattern**)：对外暴露简洁的 API，内部组合多个复杂组件。这些模式的运用使代码简洁但具备良好的扩展性。

面试加分点：能用 SOLID 原则逐一分析 Nanobot 的设计，说明它如何满足单一职责、开闭原则、依赖倒置等。

三、MCP 协议 (Q36-Q50)

Q36. 什么是 MCP (Model Context Protocol) 协议？

答案：MCP (Model Context Protocol, 模型上下文协议) 是由 Anthropic 于 2024 年 11 月正式发布的一个开放标准协议，旨在为大语言模型应用提供一种统一的方式来连接外部数据源和工具。它的核心目标是解决 AI 应用中的「集成碎片化」问题——每个 LLM 应用都需要各自实现与数据库、API、文件系统等外部系统的连接，导致大量重复开发。MCP 定义了一个标准化的通信协议，使得一个工具/数据源只需要实现一次 MCP Server，就能被所有支持 MCP 的应用使用。MCP 类比 USB-C 接口——之前每个设备用不同的充电线，USB-C 统一了接口标准。

面试加分点：能说出 MCP 是 Anthropic 主导但开源治理的项目，GitHub 上的 specification 仓库是公开的，任何人都可以参与讨论和贡献。

Q37. MCP 的 Host、Client、Server 三角架构是什么？

答案：MCP 采用三层架构：(1) **Host** (宿主)：面向用户的 AI 应用，如 Claude Desktop、Cursor IDE、Nanobot 等。Host 是用户直接交互的界面层，负责管理 MCP Client 实例。(2) **Client** (客户端)：由 Host 内部维护，每个 Client 与一个 Server 建立一对一连接。Client 负责协议通信——发送请求、接收响应、管理连接生命周期。一个 Host 可以有多个 Client，每个连接一个不同的 Server。(3) **Server** (服务器)：提供具体能力 (工具、数据、提示词模板) 的服务进程。每个 Server 专注于一个特定领域，如文件系统 Server 提供读写文件能力，GitHub Server 提供仓库操作能力。Server 暴露标准化的 MCP 接口，不关心调用方是什么应用。这种三层分离使得 Host、Client、Server 各自独立演化。

面试加分点：画出三角架构图，标注通信方向——Host 管理 Client，Client 与 Server 1:1 通信，Server 之间相互独立。

Q38. MCP 与 Function Calling 有什么区别？

答案：这是高频面试题，两者在多个层面有本质区别：

| 维度 | Function Calling | MCP |
|-----|--------------------------|--------|
| 定义方 | 模型提供商 (OpenAI/Anthropic) | 开放标准协议 |

| 维度 | Function Calling | MCP |
|--------|-------------------------------------|---------------------------|
| 作用范围 | 模型输出格式——告诉模型如何表达工具调用意图 | 运行时通信——定义应用与工具服务之间的通信方式 |
| 层次 | 模型层（LLM 推理时使用） | 应用层（应用连接外部服务时使用） |
| 工具定义位置 | 在 API 调用时通过 <code>tools</code> 参数传入 | 在 MCP Server 中定义，通过协议动态发现 |
| 互操作性 | 与特定模型 API 绑定 | 跨模型、跨应用通用 |
| 生态效应 | 每个应用需要自己实现工具逻辑 | 一次实现 Server，所有应用复用 |

简单说：Function Calling 是「模型如何表达要调用什么工具」，MCP 是「应用如何连接和调用外部工具服务」。两者是互补关系——MCP 通过 Function Calling 的 JSON Schema 格式来描述工具参数。

面试加分点：一句话总结：「Function Calling 决定 AI 说什么（output format），MCP 决定工具怎么连（runtime protocol）」。

Q39. MCP 的三大原语（Primitives）是什么？

答案：MCP 定义了三种核心原语：（1）**Tools**（工具）：可执行的操作，由 Server 暴露、由 Client 调用。例如 `search_web(query)` 或 `read_file(path)`。每个 Tool 有名称、描述、参数 JSON Schema。工具调用需要模型主动发起，是 Agent 与外部世界交互的主要方式。（2）**Resources**（资源）：可读取的数据，类似 REST API 中的 GET 资源。例如 `file:///path/to/doc` 或 `db://users/123`。Resources 提供上下文信息供模型参考，通常由 Host 应用管理何时注入到对话中。（3）**Prompts**（提示词模板）：Server 提供的预定义提示词模板，包含参数插槽。例如 `summarize(text, language)` 模板。Prompts 由用户选择使用，通过模板化提高提示词的复用性和一致性。三者的控制权不同：Tools 由模型决定何时调用，Resources 由应用决定何时加载，Prompts 由用户决定何时使用。

面试加分点：能说出三者的控制权分配原则——这体现了 MCP 对「安全性」的考虑，工具执行需要经过模型判断而非无条件触发。

Q40. Skill、Function Calling、MCP Tool 三者有什么关系和区别？

答案：三者处于不同的抽象层次：（1）**Function Calling**：模型输出层面的格式标准，定义了模型如何通过结构化 JSON 表达「我想调用某个工具」的意图。它本身不执行任何操作。（2）**MCP Tool**：运行时服务层面的工具实例，通过 MCP 协议暴露给客户端。一个 MCP Tool 包含完整的定义（名称、描述、参数 Schema）和实际的执行逻辑。MCP Tool 的参数格式复用了 Function Calling 的 JSON Schema 标准。（3）**Skill**（**Nanobot** 特有概念）：知识层面的能力封装，是一个 Markdown 文件，包含任务描述、操作步骤、提示词模板等。Skill 不是工具本身，而是「如何使用一组工具完成特定任务」的知识。Skill 可能引导 Agent 去调用 MCP Tools 或 Function Calling。关系链为：Skill 描述「做什么」→ Agent 通过 Function Calling 格式表达「调用哪个工具」→ MCP Tool 实际「执行操作」。

面试加分点：可以类比为：Skill 是菜谱，Function Calling 是点菜的标准格式，MCP Tool 是厨房里的厨具。

Q41. MCP 的工具发现机制是怎样的？

答案：MCP 的工具发现是一个动态协商过程：（1）连接建立：Client 与 Server 建立传输连接后，先进行能力协商（Capability Negotiation），双方交换各自支持的 MCP 版本和特性。（2）工具列表请求：Client 发送 `tools/list` 请求，Server 返回自己提供的所有工具的完整信息，包括 `name`、`description`、`inputSchema`（JSON Schema 格式的参数定义）。（3）动态更新：如果 Server 的工具列表发生变化（如新增或移除工具），Server 可以发送 `notifications/tools/list_changed` 通知，Client 重新拉取工具列表。（4）**Host** 整合：Host 应用将所有 Client 发现的工具汇总，统一注入到 LLM 的上下文中供模型选择。这种动态发现机制使得工具可以在运行时热插拔，不需要重启应用。

面试加分点：对比 Swagger/OpenAPI 的服务发现机制，MCP 的工具发现更轻量级且专门为 AI Agent 场景优化。

Q42. MCP 的传输层有哪些选项？各自适用什么场景？

答案：MCP 支持三种传输方式：(1) **stdio**（标准输入/输出）：Client 启动 Server 作为子进程，通过 stdin/stdout 管道通信。优点是部署简单、不需要网络配置；适用于本地单用户场景。Nanobot 默认使用这种方式。(2) **HTTP + SSE (Server-Sent Events)**：Client 通过 HTTP POST 发送请求，Server 通过 SSE 推流返回结果。优点是支持远程访问、穿越防火墙、兼容现有 Web 基础设施；适用于远程 Server 和多用户共享场景。(3) **Streamable HTTP**：MCP 规范 2025 年新增的传输方式，基于标准 HTTP 语义实现双向流式通信，是对 HTTP+SSE 的改进方案，更适合现代云原生架构。三者的选择原则是：本地开发用 stdio 最简单，生产部署用 HTTP+SSE 或 Streamable HTTP。传输层对上层的 MCP 消息格式完全透明。

面试加分点：能解释为什么 stdio 方式性能最好——没有网络开销和序列化开销，直接内存管道通信。

Q43. MCP 协议的安全考虑有哪些？

答案：MCP 安全是一个多层面的问题：(1) 传输安全：stdio 方式通过进程隔离保证安全；HTTP 方式必须使用 TLS 加密，避免中间人攻击。(2) 认证授权：MCP 规范支持但不强制特定的认证方案，实际部署中通常使用 API Key、OAuth2 或 mTLS 进行身份验证。(3) 权限控制：Tool 的 `annotations` 字段可以标注操作的危险级别（如 `destructive: true`），Host 应用可以据此决定是否需要用户确认后再执行。(4) 输入验证：所有工具参数必须通过 JSON Schema 验证，防止注入攻击。(5) 提示注入防护：MCP Server 返回的数据可能被恶意构造来操纵 LLM 行为，Host 需要对返回数据进行清洗。(6) 最小权限原则：每个 MCP Server 只暴露必要的工具，不应有过大的权限范围。

面试加分点：提到「间接提示注入 (Indirect Prompt Injection)」是 MCP 面临的重大安全风险——恶意网页或文件中嵌入的指令可能通过 MCP 工具被注入到 Agent 的上下文中。

Q44. 什么是 MCP-UI？它解决什么问题？

答案：MCP-UI 是指通过 MCP 协议传递用户界面元素的能力扩展，目标是让 MCP Server 不仅能返回数据，还能返回可交互的 UI 组件。解决的核心问题是：纯文本交互在很多场景下效率不高——比如用户需要从一个列表中选择、确认一个操作、填写一个表单，这些用 UI 组件比纯文本对话更自然。MCP-UI 的设想包括：Server 返回结构化的 UI 描述（如按钮、下拉菜单、表格），Host 应用负责将其渲染为实际的界面元素。用户与 UI 的交互结果再通过 MCP 协议回传给 Server。目前 MCP-UI 仍处于提案和早期实现阶段，尚未成为正式规范的一部分。

面试加分点：这是一个展示技术前瞻性的好话题——面试中提到 MCP-UI 说明你在跟踪最新的技术动态。

Q45. MCP 使用的 JSON-RPC 2.0 协议有什么特点？

答案：MCP 底层使用 JSON-RPC 2.0 作为消息格式标准，其核心特点包括：(1) 轻量简洁：每个消息是一个 JSON 对象，包含 `jsonrpc`（固定为“2.0”）、`method`（方法名）、`params`（参数）、`id`（请求标识符）四个字段。(2) 请求-响应模式：Client 发送带 `id` 的请求，Server 返回带相同 `id` 的响应，实现异步配对。(3) 通知模式：不带 `id` 的消息是通知，发送方不期望响应。MCP 用此实现事件推送（如工具列表变更通知）。(4) 批量调用：支持在一个消息中发送多个请求，Server 批量处理后返回。(5) 错误格式标准化：定义了标准的错误码和错误消息格式。选择 JSON-RPC 而非 REST 的原因是：RPC 更适合工具调用这种「动作导向」的交互模式，而 REST 更适合资源的 CRUD 操作。

面试加分点：对比 JSON-RPC 和 gRPC（使用 Protocol Buffers）——JSON-RPC 更轻量 and 人类可读，gRPC 更高效但需要编译 proto 文件。

Q46. MCP 如何实现 M×N 到 M+N 的生态效应？

答案：这是理解 MCP 价值的关键。在没有 MCP 之前，如果有 M 个 AI 应用和 N 个外部工具/数据源，每个应用都需要为每个工具编写专门的集成代码，总共需要 M×N 个适配器。例如 5 个 AI 应用 × 10 个工具 = 50 个定制集成。有了 MCP 之后，每个 AI 应用只需实现一个 MCP Client (M 个)，每个工具只需实现一个 MCP Server (N 个)，总共只需 M+N 个实现，5 + 10 = 15 个。这种从 O(M×N) 到 O(M+N) 的复杂度降低，正是标准化协议的核心价值，

类似于：所有设备都用 USB-C 接口，而不是每对设备用专有线材。更重要的是，新增一个应用或新增一个工具的边际成本从 $O(N)$ 或 $O(M)$ 降低到 $O(1)$ 。

面试加分点：可以类比 ODBC/JDBC 对数据库连接的标准化——之前每种编程语言要为每种数据库写驱动，有了标准后只需要每端实现一次。

Q47. MCP 生态的时间线和里程碑事件？

答案：MCP 生态发展的关键时间线：(1) **2024 年 11 月**：Anthropic 正式发布 MCP 规范 v1.0 和 TypeScript/Python SDK，同时 Claude Desktop 成为首个支持 MCP 的 Host 应用。(2) **2024 年 12 月-2025 年 1 月**：早期社区开始开发 MCP Server，如文件系统、GitHub、Slack 等基础 Server 出现。awesome-mcp-servers 仓库开始汇集生态资源。(3) **2025 年 3 月**：MCP 规范更新支持 Streamable HTTP 传输。Cursor IDE 宣布支持 MCP，标志着主流开发工具的采纳。(4) **2025 年 Q2-Q3**：OpenAI、Google 等其他大模型厂商开始支持或兼容 MCP 协议。企业级 MCP Server 开始出现。(5) **2025 年 Q4-2026 年初**：MCP 生态进入成熟期，Server 数量超过数千个，形成了完善的开发工具链和部署方案。MCP 开始被视为 AI 基础设施层面的标准协议。

面试加分点：能提到 MCP 从提出到被主流采纳只用了不到一年时间，说明 AI 工具标准化的需求非常迫切。

Q48. 企业中部署 MCP 的核心价值是什么？

答案：企业部署 MCP 的价值体现在多个层面：(1) 减少重复开发：一次开发 MCP Server，所有 AI 应用（内部 Agent、客服机器人、数据分析助手等）都能复用。(2) 内部系统集成：通过 MCP Server 将企业内部系统（ERP、CRM、HR 系统、文档系统）暴露给 AI Agent，使 AI 能直接操作企业数据。(3) 权限管理统一：在 MCP Server 层统一实施权限检查和审计日志，而非在每个 AI 应用中重复实现。(4) 能力复合：多个 MCP Server 可以被同一个 Agent 同时使用，如「查询 CRM 中的客户信息 + 在 Jira 中创建工单 + 发送飞书通知」成为一句话可以完成的操作。(5) 供应商无关性：MCP 是开放标准，不绑定特定 LLM 厂商，企业可以自由切换模型提供商。

面试加分点：能举一个具体的企业场景，如「销售查询 CRM → 生成报告 → 发送邮件」全链路自动化。

Q49. 自建 MCP Server 需要注意哪些要点？

答案：自建 MCP Server 的关键要点包括：(1) 工具设计：每个 Tool 的 name 要简洁明确，description 要详细到让 LLM 能准确判断何时使用。inputSchema 必须精确定义参数类型和约束，description 中要包含参数的含义和示例值。(2) 错误处理：返回结构化的错误信息，帮助 Agent 理解失败原因并采取补救措施，而非返回 stack trace。(3) 安全性：对输入进行严格验证，防止 SQL 注入、路径遍历等安全攻击。敏感操作（如删除、修改）需要标注 destructive: true。(4) 性能：工具执行必须有超时控制，返回结果大小要合理限制。(5) 幂等性：查询类操作必须幂等，修改类操作最好也支持幂等性。(6) SDK 选择：使用官方的 Python SDK (mcp 包) 或 TypeScript SDK，它们处理了协议细节，让开发者专注于业务逻辑。

面试加分点：能分享实际开发 MCP Server 的经验，如调试技巧（使用 MCP Inspector 工具）和常见坑（如 stdio 模式下 print 语句会干扰协议通信）。

Q50. MCP 的 Sampling 功能是什么？为什么重要？

答案：Sampling 是 MCP 中一个较少被讨论但非常重要的功能——它允许 MCP Server 反向请求 Host 应用的 LLM 能力。具体来说，Server 在处理工具调用时，可能需要 LLM 帮助理解、分析或生成内容，但 Server 本身不直接接入 LLM。通过 Sampling，Server 可以发送一个 sampling/createMessage 请求给 Client/Host，由 Host 调用其已有的 LLM 完成推理，再将结果返回给 Server。这样做的好处是：(1) Server 不需要自己管理 LLM API 密钥和连接；(2) 所有 LLM 调用统一经过 Host 管理，便于成本控制和审计；(3) Server 可以利用 Host 的模型选择和配置（如温度、system prompt）。Sampling 使得 MCP Server 从「纯工具」进化为「具有 AI 能力的智能工具」。

面试加分点：提到 Sampling 涉及「人在回路 (Human-in-the-loop)」的设计——Host 可以在执行 Sampling 请求前询问用户确认，防止意外的 LLM 调用。

四、大模型基础 (Q51-Q70)

Q51. 请描述 Transformer 架构的核心组成部分

答案：Transformer 由 Vaswani et al. (2017) 在「Attention Is All You Need」中提出，核心由以下部分组成：(1) **Self-Attention** 层：计算序列中每个位置与所有其他位置的关联度，捕获长距离依赖关系。是 Transformer 替代 RNN 的核心创新。(2) **Multi-Head Attention**：将注意力计算拆分为多个「头」，每个头关注不同的语义子空间，最后拼接结果。(3) 前馈网络 (**FFN**)：两层全连接网络，对每个位置独立进行非线性变换，负责存储和检索知识。(4) 残差连接 (**Residual Connection**)：每个子层的输入直接加到输出上，缓解深层网络梯度消失问题。(5) **Layer Normalization**：稳定每层输出的分布，加速训练收敛。(6) 位置编码 (**Positional Encoding**)：注入序列位置信息，因为注意力机制本身是排列不变的。完整的 Transformer 包含 Encoder 和 Decoder，但现代 LLM (如 GPT 系列) 通常只使用 Decoder 部分。

面试加分点：能说出原始 Transformer 论文中的模型参数——6 层、512 维、8 头、2048 FFN——并与 GPT-4 的规模形成对比。

Q52. 请推导 Self-Attention 的计算公式

答案：Self-Attention 的计算公式为 $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$ 。推导过程如下：(1) 线性投影：输入矩阵 X 分别乘以三个权重矩阵得到查询 $Q = XW_Q$ 、键 $K = XW_K$ 、值 $V = XW_V$ 。(2) 注意力分数计算：用 Q 和 K 的转置做矩阵乘法得到 QK^T ，结果是一个 $n \times n$ 的分数矩阵，表示每两个位置之间的相关度。(3) 缩放 (**Scaling**)：除以 $\sqrt{d_k}$ (d_k 是 Key 向量的维度)，目的是防止当 d_k 较大时点积值过大，导致 softmax 梯度消失。因为 Q 和 K 的元素近似 $N(0,1)$ 分布时，点积的方差为 d_k ，除以 $\sqrt{d_k}$ 使方差回到 1。(4) **Softmax** 归一化：对每行做 softmax，将分数转换为概率分布。(5) 加权求和：用注意力权重对 V 进行加权求和，得到输出。

面试加分点：能解释「为什么除以 $\sqrt{d_k}$ 而不是 d_k 」——因为方差的标准化是除以标准差（即方差的平方根）。

Q53. Multi-Head Attention 的原理和优势是什么？

答案：Multi-Head Attention 将输入的 Q 、 K 、 V 分别投影到 h 个不同的低维子空间中，在每个子空间独立计算注意力，最后拼接结果再做一次线性投影。公式为： $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$ ，其中 $\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i})$ 。优势包括：(1) 多子空间关注：不同的头可以关注不同类型的关系——有的头关注语法结构，有的关注语义相似性，有的关注位置邻近性。(2) 计算效率：虽然有 h 个头，但每个头的维度是 d_{model}/h ，总计算量与单头注意力相同。(3) 稳定性：多个头的输出取平均/拼接，降低了单一注意力计算的方差。典型配置如： $d_{\text{model}}=512, h=8, d_k=d_v=64$ 。

面试加分点：能提到 GQA (Grouped Query Attention) 等改进方案，以及为什么现代 LLM 需要减少 KV Head 数量——节省 KV Cache 内存。

Q54. 位置编码有哪些方案？RoPE 的原理是什么？

答案：位置编码方案主要有三类：(1) 绝对位置编码：原始 Transformer 使用正弦/余弦函数生成固定位置向量，或可学习的位置嵌入。缺点是无法自然扩展到训练时未见过的长度。(2) 相对位置编码：如 ALiBi，在注意力分数上加一个基于相对距离的偏置，距离越远偏置越大（负值），自然实现远距离衰减。(3) 旋转位置编码 (**RoPE**)：由苏剑林提出，是目前主流 LLM 的首选方案。核心思想是将位置信息编码为旋转矩阵——对 Q 和 K 向量的每对相邻维度施加一个与位置相关的旋转角度。这样 $q_m \cdot k_n$ 的值只取决于相对位置 $m-n$ ，自然实现了相对位置感知。RoPE 的数学形式优雅： $f(x, m) = R(m)x$ ，其中 $R(m)$ 是旋转矩阵，旋转角度 $\theta_i = m / 10000^{(2i/d)}$ 。

面试加分点：能解释 RoPE 的「远程衰减」特性——高频旋转分量自然导致远距离 token 的注意力分数下降，这与人类语言中「近处词更相关」的直觉一致。

Q55. KV Cache 是什么？它如何加速推理？

答案：KV Cache 是自回归推理过程中的核心优化技术。在 Decoder-only 模型生成文本时，每生成一个新 token 都需要计算该 token 对所有历史 token 的注意力。如果不做缓存，每一步都需要重新计算所有历史 token 的 K 和 V 向量，

计算量为 $O(n^2)$ 。KV Cache 的做法是：将每层注意力中已计算过的 K 和 V 向量缓存起来，生成新 token 时只需计算新 token 的 Q、K、V，然后将新的 K、V 追加到缓存中。这样每步的计算量从 $O(n)$ 降到 $O(1)$ （对于 KV 计算部分）。代价是内存占用——对于一个 13B 参数的模型，4096 长度序列的 KV Cache 可能占用数 GB 显存。因此 GQA/MQA 等技术通过减少 KV Head 数量来压缩 KV Cache 大小。

面试加分点：能计算 KV Cache 的具体大小： $2 \times n_layers \times n_kv_heads \times d_head \times seq_len \times 2bytes(FP16)$ 。

Q56. 什么是「Lost in the Middle」问题？如何缓解？

答案：「Lost in the Middle」是 Liu et al. (2023) 发现的现象：当相关信息被放在长上下文的中间位置时，模型的利用效果显著下降，而放在开头或结尾时效果最好。这形成了一个「U 形曲线」——模型对首尾信息的关注度明显高于中间。原因分析：(1) 注意力机制在长序列中的分布不均匀，首尾 token 天然获得更多关注（位置偏差）。(2) 训练数据中重要信息通常出现在文本的开头或结尾（数据偏差）。缓解策略包括：(1) 将最重要的信息放在 prompt 的开头和结尾；(2) 使用检索增强时将最相关的文档放在最前面；(3) 使用 sliding window attention 等注意力改进方案；(4) 在 Nanobot 中，ContextBuilder 将 system prompt 放在最前面、最新消息放在最后面，就是遵循这一原则。

面试加分点：这是解释 Nanobot 为何将 system prompt 和工具定义放在前面、用户消息放在后面的直接理论依据。

Q57. Pre-LN 和 Post-LN 的区别是什么？为什么现代 LLM 偏好 Pre-LN？

答案：在 Transformer 中，Layer Normalization 的位置有两种选择：(1) **Post-LN**：原始 Transformer 的方案，先执行子层 (Attention/FFN)，再做 LayerNorm，即 $output = LayerNorm(x + SubLayer(x))$ 。(2) **Pre-LN**：先做 LayerNorm 再执行子层，即 $output = x + SubLayer(LayerNorm(x))$ 。现代 LLM (GPT-3、LLaMA、Claude 等) 几乎都使用 Pre-LN，原因是：(a) 训练稳定性：Pre-LN 中残差连接直接从输入到输出，梯度可以无损传播，不经过 LayerNorm 的缩放，深层网络也不容易梯度爆炸/消失。Post-LN 在深层网络中梯度波动大，需要仔细调节学习率 warmup。(b) 收敛速度：Pre-LN 通常收敛更快，且对超参数更鲁棒。(c) 代价：Pre-LN 的最终输出没有经过 LayerNorm，可能需要在最后加一个额外的 LN 层。

面试加分点：能提到 RMSNorm 是对 LayerNorm 的进一步简化——去掉了均值中心化，只保留均方根归一化，计算量更小。

Q58. RMSNorm 是什么？相比 LayerNorm 有什么优势？

答案：RMSNorm (Root Mean Square Normalization) 是 Zhang & Sennrich (2019) 提出的归一化方法，是 LayerNorm 的简化版本。LayerNorm 的计算包括两步：先减去均值 (re-centering)，再除以标准差 (re-scaling)，即 $y = (x - \mu) / \sigma * \sigma + \mu$ 。RMSNorm 去掉了减均值的步骤，只做均方根归一化： $y = x / RMS(x) * \sigma$ ，其中 $RMS(x) = \sqrt{(1/n) \sum x_i^2}$ 。优势包括：(1) 计算效率更高：省去了求均值和减均值的操作，约减少 10-15% 的归一化计算量。(2) 效果不降低：实验表明 re-centering 对模型性能贡献很小，去掉后精度几乎不变。(3) 被主流采用：LLaMA、Qwen 等主流模型都使用 RMSNorm。减少的参数数量还有 偏置项 (RMSNorm 没有)，进一步节省内存。

面试加分点：能说出 RMSNorm 在 LLaMA 中被 Pre-LN 配合使用，即 $output = x + SubLayer(RMSNorm(x))$ 。

Q59. 上下文长度扩展技术 (NTK、YaRN) 的原理是什么？

答案：上下文长度扩展解决的问题是：模型在短序列（如 4K）上训练，但需要在推理时处理更长的序列（如 64K、128K）。直接外推会导致 RoPE 产生训练时未见过的旋转角度，性能急剧下降。关键方法包括：(1) **线性插值 (Position Interpolation)**：将长序列的位置索引线性压缩到训练长度范围内。如将 $[0, 8192]$ 压缩到 $[0, 4096]$ 。简单有效但会损失分辨率。(2) **NTK-aware 插值**：不均匀地缩放不同频率的旋转分量——低频分量（捕获长距离关系）做更多插值，高频分量（捕获局部关系）保持不变。这样既保留了短距离关系的精度，又扩展了长距离感知能力。(3) **YaRN (Yet another RoPE extensioN)**：在 NTK 基础上增加了注意力分数的温度缩放和动态调整，进一步提升扩展质量。通常只需要少量长序列数据进行微调即可生效。

面试加分点：能提到即使是小模型也面临长度扩展的需求，YaRN 是目前最实用的 RoPE 扩展方案之一。

Q60. MoE (Mixture of Experts) 与 Dense 模型的区别和权衡是什么?

答案：MoE 是一种稀疏模型架构，在每一层用多个「专家」FFN 替代单一 FFN，通过一个 Router（门控网络）动态选择每个 token 激活哪几个专家（通常 Top-2）。与 Dense 模型比较：

| 维度 | Dense Model | MoE Model |
|-------|-----------------|----------------------------------|
| 参数效率 | 每个 token 使用全部参数 | 每个 token 只使用部分参数（如 2/8） |
| 总参数量 | 相对较少 | 可以很大（如 Mixtral 8×7B = 46B 总参数） |
| 激活参数量 | = 总参数量 | ≪ 总参数量（如只激活 13B） |
| 推理速度 | 与参数量正比 | 接近单个专家的速度 |
| 训练难度 | 标准 | 需要处理 load balancing、expert 坍塌等问题 |
| 显存 | 正比于参数量 | 需要加载所有专家，显存需求接近总参数量 |

MoE 的核心优势是「用更少的计算获得更强的模型能力」，但代价是显存需求大、训练复杂度高、专家利用不均衡。

面试加分点：能提到 Router 的 load balancing loss 导致训练不稳定，以及专家坍塌（所有 token 都选同一个专家）的风险。

Q61. 大模型幻觉 (Hallucination) 的来源和缓解策略

答案：幻觉指模型生成看似流畅但事实错误的内容。来源分析：(1) 数据层面：训练数据中存在错误信息、过时信息、矛盾信息，模型学到了这些错误模式。(2) 模型层面：LLM 本质是概率模型，倾向于生成「统计上合理」而非「事实上正确」的文本。注意力机制难以精确定位和检索特定事实。(3) 解码层面：采样策略（如温度 >0）引入随机性，可能偏离事实。(4) 训练目标偏差：next-token prediction 目标不直接优化事实准确性。缓解策略：(a) RAG 引入外部知识验证；(b) Self-Consistency 多次采样取共识；(c) Factuality-aware 训练（如 DPO 偏好对齐）；(d) 结构化输出约束模型在特定模式下生成；(e) 引用溯源让模型标注信息来源；(f) 在 Agent 系统中通过工具验证事实。

面试加分点：区分「开放域幻觉」（编造事实）和「闭合域幻觉」（与给定文档矛盾），后者在 RAG 场景中更常见。

Q62. 温度 (Temperature) 和 Top-p 参数的作用是什么?

答案：两者都控制模型生成的随机性，但作用机制不同：(1) **Temperature**（温度）：在 softmax 之前对 logits 除以温度值 T。当 $T \rightarrow 0$ 时，概率分布趋于 one-hot（接近贪婪解码，确定性最高）；当 $T \rightarrow \infty$ 时，概率分布趋于均匀（随机性最大）。 $T=1$ 时是标准 softmax。温度影响的是整个分布的「锐利程度」。(2) **Top-p (Nucleus Sampling)**：先将 token 按概率从高到低排序，然后从前往后累积概率，当累积概率达到 p（如 0.9）时截断，只从这些 token 中采样。Top-p 的优势是自适应——高置信时候选集小（确定性高），低置信时候选集大（保持多样性）。实际使用建议：需要确定性输出（如代码生成、JSON 结构化输出）时用低温度（0-0.3）；需要创造性输出（如文案、对话）时用高温度（0.7-1.0）+ Top-p（0.9-0.95）。

面试加分点：能提到 Agent 场景中工具调用应该使用低温度以提高参数生成的准确性，而对话回复可以使用较高温度。

Q63. Decoder-only 和 Encoder-Decoder 架构各自适合什么任务?

答案：(1) **Decoder-only** (GPT 系列、LLaMA、Claude)：自回归生成，每个 token 只能看到之前的 token（因果注意力掩码）。优势是：预训练目标简单统一（next-token prediction），在超大规模时涌现出强大的通用能力。适合：开放式文本生成、对话、代码生成等生成任务。目前 LLM 的绝对主流。(2) **Encoder-Decoder** (T5、BART)：Encoder 对输入进行双向编码，Decoder 自回归生成输出。优势是 Encoder 可以双向看全文，对输入理解更充分。适合：翻译、摘要、问答等有明确输入-输出映射的任务。(3) 为什么 Decoder-only 成为主流？Scaling Laws 研究表明，在相同计算量下 Decoder-only 的 loss 下降更稳定、更可预测。此外，统一的生成范式简化了工程实现，且通过 in-context learning 可以适配几乎所有下游任务。

面试加分点：能结合实际项目经验说明 Decoder-only 架构的优势——如 Nanobot 接入的 LLM 几乎都是 Decoder-only 架构。

Q64. Scaling Laws 的核心结论是什么？

答案：Scaling Laws 由 Kaplan et al. (2020, OpenAI) 和 Hoffmann et al. (2022, Chinchilla) 提出，核心结论包括：(1) **Kaplan Laws**：模型性能（以 loss 衡量）与模型参数量 N 、数据量 D 、计算量 C 呈幂律关系： $L(N) \propto N^{-(1/\alpha)}$ ， $L(D) \propto D^{-(1/\beta)}$ 。增加任一因素都能降低 loss，但有收益递减。(2) **Chinchilla Optimal**：给定固定计算预算 C ，最优的参数量 N 和数据量 D 应该同步扩大，大致比例为 $D \approx 20N$ 。即 1B 参数模型应该用 20B token 训练。这推翻了早期「模型越大越好」的做法。(3) 实际影响：Chinchilla 结论导致业界从「大模型 + 少数据」转向「适中模型 + 充分数据」的训练策略。LLaMA 就是典型的 Chinchilla-optimal 模型。(4) 局限性：Scaling Laws 预测的是 loss 而非下游任务性能，且未考虑模型架构差异。

面试加分点：能用 Chinchilla 公式估算特定参数量的模型应该用多少数据训练（如 64M 模型 1.3B token），展示理论联系实际的能力。

Q65. Flash Attention 的核心优化原理是什么？

答案：Flash Attention 由 Dao et al. (2022) 提出，核心是通过 **IO-aware** 算法设计将注意力计算的内存复杂度从 $O(n^2)$ 降低到 $O(n)$ ，同时保持数学上的精确等价。优化原理包括：(1) **Tiling**（分块计算）：不将完整的 $n \times n$ 注意力矩阵存入 GPU 的 HBM（高带宽内存），而是分成小块在 SRAM（片上内存）中计算。SRAM 比 HBM 快 10-100 倍但容量小，Tiling 正好适配其大小。(2) 在线 **Softmax**：通过数学上的 log-sum-exp 技巧实现流式 softmax 计算，无需存储完整的注意力分数矩阵。每处理一个 tile 就更新累积值。(3) **Kernel Fusion**：将 QK^T 、softmax、 $\times V$ 三步操作融合为一个 GPU kernel，减少 HBM 读写次数。结果是：速度提升 2-4 倍，显存减少至 $O(n)$ ，且结果完全精确（非近似）。几乎所有现代 LLM 训练和推理框架都默认使用 Flash Attention。

面试加分点：能解释为什么 Flash Attention 的核心瓶颈是「内存带宽」而非「计算量」——GPU 的算力增长快于内存带宽，注意力计算是 memory-bound 的。

Q66. GQA、MHA、MQA 三者有什么区别？

答案：三者是注意力机制中 Query/Key/Value Head 数量的不同配置：(1) **MHA (Multi-Head Attention)**：标准多头注意力，Q、K、V 有相同数量的头（如 32 个）。每个 Q head 有对应的 K/V head。KV Cache 最大。(2) **MQA (Multi-Query Attention)**：所有 Q head 共享同一组 K、V（即 K/V 只有 1 个头）。KV Cache 最小（原来的 1/32），推理速度最快，但质量可能下降。(3) **GQA (Grouped-Query Attention)**：折中方案，将 Q head 分成几组，每组共享一组 K/V。例如 32 个 Q head 分成 8 组，每组 4 个 Q head 共享 1 组 K/V。KV Cache 减少到 MHA 的 1/4，质量几乎无损。LLaMA-2 70B 等主流模型都使用了 GQA。GQA 是目前的最佳实践——在推理效率和模型质量之间取得了良好平衡。

面试加分点：能计算具体场景下三种方案的 KV Cache 大小差异，如 $n_{\text{layers}}=32$ ， $\text{seq_len}=4096$ ， $d_{\text{head}}=128$ 时的具体 bytes。

Q67. SwiGLU 激活函数的原理和优势是什么？

答案：SwiGLU 是 Shazeer (2020) 提出的 FFN 激活方案，被 PaLM、LLaMA 等主流模型采用。传统 FFN 结构为 $\text{FFN}(x) = \text{ReLU}(xW_1)W_2$ ，SwiGLU 改为 $\text{FFN_SwiGLU}(x) = (\text{Swish}(xW_1) \otimes xV)W_2$ ，其中 $\text{Swish}(z) = z \cdot \sigma(z)$ （ σ 是 sigmoid）， $\otimes$ 是逐元素乘法。关键创新：引入了门控机制（GLU, Gated Linear Unit），用 xV 作为门控信号控制信息流通。优势包括：(1) 表达能力更强：门控机制允许网络选择性地传递信息，比简单的 ReLU 截断更精细。(2) 训练效果更好：在同等计算量下，SwiGLU 比 ReLU/GELU 的 loss 更低。(3) **Swish** 的平滑性：Swish 是 ReLU 的平滑版本，梯度流动更好。代价是多了一个权重矩阵 V ，参数量增加约 50%，但通常通过减小隐层维度来保持总参数量不变。

面试加分点：能说出 LLaMA 系列中 FFN 隐层维度的调整公式： $\text{hidden_dim} = 2/3 \times 4 \times d_{\text{model}}$ （取整到 256 的倍数），用于补偿 SwiGLU 的额外参数。

Q68. Embedding 层的设计有哪些考虑?

答案: Embedding 层是模型的输入/输出接口, 设计考虑包括: (1) 词表大小 (**Vocabulary Size**): 决定了 Embedding 层的参数量 ($\text{vocab_size} \times d_{\text{model}}$)。词表越大覆盖越全但参数越多。小模型通常选择较小的词表以减少参数量。(2) 权重共享 (**Weight Tying**): 输入 Embedding 和输出 LM Head 共享同一个权重矩阵, 减少约 30% 的参数量, 且有正则化效果。GPT-2、LLaMA 等模型都使用这种技巧。(3) 维度选择 (d_{model}): Embedding 维度需要足够大以表达语义, 但也受计算预算限制。典型值从 512 (小模型) 到 4096 (LLaMA-7B) 不等。(4) 初始化策略: 通常使用正态分布初始化, 标准差为 $1/\sqrt{d_{\text{model}}}$ 。(5) 分词器对齐: Embedding 层必须与 Tokenizer 的词表一一对应, 更换分词器需要重新训练 Embedding。

面试加分点: 能解释小模型为什么倾向使用较小词表——参数预算有限, 大词表会导致 Embedding 占比过高 (如 $\text{vocab}=32000 \times d=512 = 16\text{M}$, 已接近 64M 模型总参数的 25%)。

Q69. 常见的文本生成策略有哪些? 各自的优缺点?

答案: 主要生成策略包括: (1) **Greedy Search** (贪婪搜索): 每步选择概率最高的 token。优点是速度最快、确定性; 缺点是容易陷入重复、生成质量一般。(2) **Beam Search** (束搜索): 维护 K 个最优候选序列, 每步扩展所有候选并保留 Top-K。优点是比贪婪搜索质量更好; 缺点是容易生成平庸但安全的文本, 速度慢 K 倍。(3) **Top-K Sampling**: 从概率最高的 K 个 token 中随机采样。K 固定值可能在不同上下文下过宽或过窄。(4) **Top-p (Nucleus) Sampling**: 自适应选择候选集大小。是目前最常用的策略。(5) **Temperature Sampling**: 通过温度参数控制概率分布的平坦度, 与 Top-p 配合使用。实际应用中, Agent 工具调用用 Greedy/低温度保证准确性, 对话回复用 Top-p + Temperature 保证自然度。

面试加分点: 能说出 Speculative Decoding (投机解码)——用小模型生成草稿、大模型验证的加速策略, 这是 2024-2025 年的热门推理优化技术。

Q70. 大模型推理优化有哪些主要方向?

答案: 推理优化是降低部署成本的关键, 主要方向包括: (1) **KV Cache 优化**: GQA/MQA 减少 KV Head 数量; PagedAttention (vLLM) 减少 KV Cache 内存碎片; 滑动窗口限制缓存长度。(2) 计算优化: Flash Attention 降低注意力计算的内存带宽需求; Kernel Fusion 减少 GPU kernel launch 开销。(3) 量化 (**Quantization**): 将模型权重从 FP16 压缩到 INT8/INT4, 减少内存占用和计算量。如 GPTQ、AWQ、GGUF 等方案。(4) 批处理优化: Continuous Batching 实现请求级别的动态批处理, 提高 GPU 利用率。(5) 投机解码 (**Speculative Decoding**): 用小模型生成多个候选 token, 大模型并行验证, 将自回归的串行解码变为部分并行。(6) 模型架构优化: MoE 用部分参数做推理; Early Exit 在浅层就能满足的请求提前退出。(7) 分布式推理: Tensor Parallel、Pipeline Parallel 将模型分布到多个 GPU 上。

面试加分点: 能结合具体框架说明——vLLM 做 serving 优化、TensorRT-LLM 做 kernel 优化、llama.cpp 做端侧量化推理。

五、训练优化 (Q71-Q85)

Q71. 大模型预训练 (Pretrain) 的核心目标和方法是什么?

答案: 预训练是大模型能力的基础, 核心目标是让模型从海量文本中学习语言的统计规律和世界知识。方法上, Decoder-only 模型使用自回归语言建模 (Autoregressive LM): 给定前文 token 序列 $x_1 \dots x_{t-1}$, 预测下一个 token x_t 的概率分布, 训练目标是最小化负对数似然损失 $L = -\sum \log P(x_t | x_1 \dots x_{t-1})$ 。训练数据通常包括: 网页文本 (Common Crawl)、书籍、代码 (GitHub)、学术论文、百科全书等多种来源, 规模从几百 GB 到几 TB 不等。预训练阶段需要大量计算资源 (数百到数千个 GPU) 和数周到数月的训练时间。预训练后的模型 (base model) 具备了语言理解和生成能力, 但还没有指令跟随能力——它只会「续写」而不会「回答问题」。

面试加分点: 能提到小规模模型的预训练可以用单 GPU 和少量数据 (如 2GB) 完成, 说明即使是小规模训练也能体验完整流程。

Q72. SFT (Supervised Fine-Tuning) 的目的和数据格式是什么?

答案: SFT 的核心目的是将预训练模型从「文本续写器」转变为「指令跟随者」。预训练后的 base model 只会根据统计规律生成文本, 不理解「用户问问题、模型回答问题」的交互模式。SFT 使用 (instruction, response) 格式的监督数据进行微调, 让模型学会: (1) 理解用户意图 (指令跟随); (2) 以合适的格式回答 (对话格式); (3) 拒绝有害请求 (安全对齐的初步阶段)。数据格式通常为多轮对话: `[{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]`。SFT 数据的质量远比数量重要——几千条高质量人工标注数据可能比十万条低质量数据效果更好。训练时只对 assistant 回复部分计算 loss (masked loss), user 部分不参与梯度更新。

面试加分点: 能解释「SFT 不是教模型新知识, 而是教模型如何展示已有知识」——知识来自预训练, SFT 教的是表达方式。

Q73. RLHF 的三个阶段分别是什么?

答案: RLHF (Reinforcement Learning from Human Feedback) 是让模型输出与人类偏好对齐的核心技术, 包括三个阶段: (1) **SFT** 阶段: 先用高质量监督数据微调 base model, 得到一个基本的指令跟随模型 (SFT model)。这是后续 RL 的起点。(2) 奖励模型训练 (**Reward Model Training**): 收集人类对模型输出的偏好标注——对同一个 prompt, 模型生成多个回复, 人类标注哪个更好。用这些 pairwise comparison 数据训练一个奖励模型 (RM), 学习预测人类偏好分数。RM 通常基于 SFT model 初始化, 去掉 LM head, 加一个标量输出层。(3) **PPO** 强化学习: 用 PPO 算法优化 SFT model, 使其输出能获得更高的 RM 分数。同时用 KL 散度约束防止模型偏离 SFT model 太远 (避免 reward hacking)。训练 objective 为 $\max E[R(x,y) - \beta \cdot KL(\pi || \pi_{ref})]$ 。

面试加分点: 能说出 ChatGPT 就是通过这三个阶段训练出来的, 这是 InstructGPT 论文 (Ouyang et al., 2022) 的核心贡献。

Q74. RLHF 中的 KL 约束有什么作用?

答案: KL 散度约束 $\beta \cdot KL(\pi_{\theta} || \pi_{ref})$ 是 RLHF 训练中的关键正则化项, 其中 π_{θ} 是当前策略 (正在优化的模型), π_{ref} 是参考策略 (通常是 SFT model)。作用包括: (1) 防止 **Reward Hacking** (奖励破解): 没有 KL 约束的话, 模型会极端优化奖励分数, 生成一些能骗过 RM 但实际上质量很差或不自然的文本 (如重复特定短语来获得高分)。KL 约束限制模型不能偏离 SFT model 太远。(2) 保持语言质量: SFT model 已经具备良好的语言能力, KL 约束保证在偏好对齐过程中不损失这些基础能力。(3) 的调节: 越大约束越强, 模型输出越保守但安全; 越小模型越倾向于高奖励但可能出现退化。 的选择是一个需要调参的超参数。

面试加分点: 能提到 KL 约束在实现上通常用 log ratio $\log(\pi_{\theta}(y|x) / \pi_{ref}(y|x))$ 的期望来计算, 而非直接计算 KL 散度。

Q75. PPO 算法中的四个网络分别是什么?

答案: PPO (Proximal Policy Optimization) 在 RLHF 中涉及四个网络: (1) 策略模型 (**Policy Model / Actor**): 就是正在训练的 LLM, 根据 prompt 生成回复文本。这是最终要部署的模型。(2) 参考模型 (**Reference Model**): SFT model 的冻结拷贝, 用于计算 KL 散度约束。训练期间参数不更新。(3) 奖励模型 (**Reward Model**): 打分器, 对 (prompt, response) 对输出标量分数, 衡量回复质量。训练期间参数不更新。(4) 价值模型 (**Value Model / Critic**): 预测每个 state (prompt + 已生成的部分 response) 的期望累积奖励, 用于计算 advantage (优势函数), 降低策略梯度的方差。Critic 的参数在训练中更新。同时运行四个大模型是 RLHF 工程上的核心难点——需要大量 GPU 显存, 这也是 DPO 等无 RL 方法受欢迎的原因之一。

面试加分点: 能估算显存需求——4 个模型如果都是 7B 参数, FP16 下需要 $4 \times 14GB = 56GB$, 加上 optimizer states 和 activations 可能需要 $4 \times A100$ 。

Q76. DPO (Direct Preference Optimization) 的核心创新是什么?

答案：DPO 由 Rafailov et al. (2023) 提出，核心创新是绕过奖励模型和强化学习，直接用偏好数据优化策略模型。数学推导上，DPO 证明了 RLHF 的最优策略可以用参考策略和奖励函数的 closed-form 解来表示，然后将奖励函数替换为策略模型的 log ratio，得到 DPO loss: $L_{DPO} = -E[\log \pi(\log \pi(y_w|x)/\pi_{ref}(y_w|x)) - \log \pi(y_l|x)/\pi_{ref}(y_l|x))]$ 。其中 y_w 是偏好回复 (chosen)， y_l 是非偏好回复 (rejected)。优势包括：(1) 无需训练奖励模型：直接从偏好数据学习；(2) 无需 RL 循环：训练稳定性大幅提升，与 SFT 一样简单；(3) 计算效率高：只需 2 个模型 (policy + reference)，而非 PPO 的 4 个。许多小模型项目 (如教学类项目) 也使用 DPO 进行偏好对齐。

面试加分点：能从数学上简要说明 DPO 如何从 RLHF 目标推导出来——关键是 Reward 的 closed-form 解和 Bradley-Terry 模型的结合。

Q77. DPO 和 PPO 如何选型?

答案：选型建议如下：

| 维度 | DPO | PPO |
|-------|----------------|-----------------|
| 工程复杂度 | 低 (像 SFT 一样训练) | 高 (4 个模型、RL 循环) |
| 计算资源 | 2 个模型 | 4 个模型 |
| 训练稳定性 | 稳定 | 不稳定，需要仔细调参 |
| 对齐效果 | 对于大多数场景足够好 | 理论上上限更高 |
| 数据需求 | 离线偏好数据 | 在线生成 + 偏好标注 |
| 奖励泛化 | 隐式奖励，无法复用 | 显式 RM，可复用 |
| 适用场景 | 资源有限、快速迭代、中小规模 | 资源充足、需要极致效果 |

实际建议：(1) 中小团队首选 DPO，工程代价低、效果够用；(2) 有充足标注团队和计算资源时选 PPO，效果天花板更高；(3) 可以先 DPO 快速迭代，验证方向后再 PPO 精调。小模型场景下通常选择 DPO，因为 PPO 的额外复杂度不值得。

面试加分点：提到 2025 年的趋势：GRPO、SimPO 等新方法在 DPO 基础上进一步简化，模糊了 DPO/PPO 的界限。

Q78. 什么是奖励破解 (Reward Hacking)? 如何防范?

答案：奖励破解指模型学会了「欺骗」奖励模型以获得高分，但实际输出质量并未提升甚至下降的现象。常见形式包括：(1) 长度偏好：RM 倾向于给更长的回复高分，模型就学会生成冗长废话。(2) 格式偏好：RM 对带 markdown 格式的回复评分更高，模型就在所有回复中加入不必要的格式。(3) 套话偏好：RM 对包含「作为一个 AI 语言模型」等套话的回复评分高，模型就频繁使用套话。(4) 对抗性输出：模型生成人类看不出好坏但能骗过 RM 的特殊 pattern。防范策略：(a) KL 约束限制模型偏离参考策略的程度；(b) RM 集成：训练多个 RM 取平均，减少单一 RM 的偏差；(c) 定期用人类评估检查模型输出质量；(d) 长度归一化：在奖励中扣除长度带来的虚假增益。

面试加分点：能提到 Goodhart's Law: 「当一个指标变成了目标，它就不再是好的指标」——奖励破解是这一定律在 AI 中的体现。

Q79. LoRA 的数学原理是什么?

答案：LoRA (Low-Rank Adaptation) 的核心思想是：微调时模型权重的变化量 ΔW 是低秩的。具体来说，对于预训练权重矩阵 $W \in \mathbb{R}^{(d \times d)}$ ，LoRA 不直接学习 $\Delta W \in \mathbb{R}^{(d \times d)}$ (参数量 d^2)，而是将其分解为两个低秩矩阵的乘积： $\Delta W = BA$ ，其中 $B \in \mathbb{R}^{(d \times r)}$ 、 $A \in \mathbb{R}^{(r \times d)}$ ， $r \ll d$ (如 $r=8$ 或 $r=16$)。参数量从 d^2 降低到 $2dr$ ，压缩比为 $d/(2r)$ 。训练时冻结 W ，只训练 A 和 B 。前向传播为 $h = (W_0 + BA)x = W_0x + BAx$ 。初始化策略： A 用正态分布初始化， B 用零初始化，保证训练开始时 $\Delta W=0$ (不改变预训练模型行为)。推理时可以将 ΔW 合并到 W 中： $W = W_0 + BA$ ，不增加推理延迟。

面试加分点：能解释低秩假设的直觉——微调任务改变模型的行为方向通常是少数几个维度（如「更礼貌」「更简洁」），不需要所有参数都动。

Q80. 为什么 LoRA 在实践中效果很好？直觉解释是什么？

答案：LoRA 效果好的直觉解释有多个层面：（1）任务适配的低维性：从预训练模型适配到特定下游任务，模型行为的变化通常可以用少量「方向」来描述。例如从「通用文本生成」变成「医学问答」，变化的维度远少于模型的总参数维度。低秩分解恰好捕获了这些关键方向。（2）正则化效果：低秩约束本身就是一种强正则化，防止在小数据集上过拟合。相当于在一个低维子空间中搜索最优解，搜索空间更小、更容易找到好解。（3）预训练知识保留：冻结原始权重确保了预训练学到的通用知识和语言能力不被破坏，LoRA 只添加任务特定的增量。（4）实证支持：Aghajanyan et al. (2020) 的研究表明，预训练模型的微调确实是在低维流形上进行的，这为 LoRA 的低秩假设提供了理论支撑。

面试加分点：可以提到 LoRA 还有一个实用优势——可以同时训练多个 LoRA 适配器（如不同任务、不同用户），推理时按需加载，类似于「模型插件」。

Q81. LoRA 推理时如何合并？有什么好处？

答案：LoRA 推理合并是指将训练好的 LoRA 适配器参数合并回原始模型权重中。操作非常简单： $W_{merged} = W_0 + \alpha / r \cdot BA$ ，其中 α 是 LoRA 的缩放因子， r 是秩。合并后得到一个标准的模型权重文件，与原始模型结构完全一致。好处包括：（1）零推理开销：合并后与标准模型推理完全相同，没有额外的矩阵分解计算。（2）部署简单：只需要一个模型文件，不需要额外加载 LoRA 适配器。（3）兼容所有推理框架：合并后的模型可以用任何支持标准模型格式的框架加载。但也有不合并的场景：当需要动态切换多个 LoRA 适配器时（如多任务服务），保持分离可以节省显存——只需存一份基础模型 + 多个小的 LoRA 文件。

面试加分点：能提到 LoRA 合并不影响量化——先合并再量化与先量化再合并（QLoRA 的方案）在数值上略有差异，实践中影响很小。

Q82. QLoRA 的核心创新是什么？

答案：QLoRA 由 Dettmers et al. (2023) 提出，核心创新是在 4-bit 量化的冻结模型上训练 LoRA 适配器，将微调大模型所需的显存大幅降低。关键技术点包括：（1）4-bit NormalFloat (NF4) 量化：专为正态分布的预训练权重设计的 4-bit 数据类型，信息损失比标准 INT4 更小。（2）双重量化 (Double Quantization)：对量化参数本身再做一次量化，进一步压缩内存。每个量化块的缩放因子 (FP32) 再量化为 FP8。（3）分页优化器：当 GPU 显存不足时，将 optimizer states 卸载到 CPU 内存，类似操作系统的虚拟内存分页。效果上，QLoRA 使得单张 48GB A6000 GPU 即可微调 65B 参数的模型，而 LoRA 需要多张 80GB A100。在精度上，QLoRA 微调的模型质量与全精度 LoRA 几乎无差异。这大大降低了大模型微调的门槛。

面试加分点：能计算显存节省量——65B 模型 FP16 需约 130GB 显存，NF4 量化后只需约 33GB，加上 LoRA 参数和 optimizer 约需 40GB，一张 48GB GPU 即可。

Q83. LoRA 的超参数 r 和 α 如何选择？

答案：LoRA 有两个关键超参数：（1） r (秩)：控制 LoRA 矩阵的维度。 r 越大表达能力越强但参数量越多。常用值为 4、8、16、32、64。选择建议：简单任务（如情感分类）用 $r=4-8$ ；复杂任务（如指令微调）用 $r=16-32$ ；接近全量微调效果需要 $r=64+$ 。经验法则是从 $r=8$ 开始，效果不好再增大。（2） α (缩放因子)：LoRA 的输出会乘以 α / r ，控制适配器对模型的影响程度。通常设置 $\alpha = 2r$ 或 $\alpha = r$ 。 α / r 的比值（也称 lora_scaling ）越大，LoRA 的影响越大。一些实践者固定 $\alpha = 16$ 或 $\alpha = 32$ ，只调 r 。（3）目标模块选择：通常对所有注意力矩阵 (Q、K、V、O) 和 FFN 的 up/down/gate 矩阵都加 LoRA 效果最好。早期只对 Q、V 加 LoRA 是不够优的做法。学习率通常比全量微调高 5-10 倍（如 $2e-4$ ）。

面试加分点：能提到一个反直觉的结论——LoRA 的论文发现 $r=1$ 或 $r=2$ 在某些任务上也能达到不错效果，说明任务适配的低维性可能比我们想象的更极端。

Q84. 什么情况下应该选择全量微调而非 LoRA?

答案：虽然 LoRA 在大多数场景下是更经济的选择，但以下情况应考虑全量微调：(1) 领域差异极大：当目标领域与预训练数据差异很大（如从英文模型微调中文、从通用模型微调医学）时，低秩近似可能不足以表达大量的知识更新。(2) 数据充足：有大量（10 万 +）高质量标注数据时，全量微调能更好地利用数据中的信号，不会因低秩瓶颈而欠拟合。(3) 追求极致效果：在竞赛或关键业务场景中，0.5% 的精度差异也很重要时，全量微调的效果天花板更高。(4) 预训练续训：继续用大规模无标注数据训练模型（如扩展到新语言），需要全量更新参数。(5) 小模型：参数量不大（如 64M 级别）时，全量微调的开销可接受，且低秩约束可能过强。LoRA 的参数效率优势在大模型上更显著。

面试加分点：一个实用建议——可以先用 LoRA 快速实验，确定数据和任务可行后，再考虑全量微调提升效果。

Q85. 知识蒸馏的黑盒和白盒方法有什么区别?

答案：知识蒸馏是用大模型 (Teacher) 指导小模型 (Student) 学习。按照对 Teacher 模型的访问程度分为两类：(1) 黑盒蒸馏 (**Black-box Distillation**)：只能访问 Teacher 的输出文本，无法获取内部 logits 或中间表示。方法是用 Teacher 生成高质量数据，然后用这些数据 SFT Student 模型。典型案例：用 GPT-4 生成训练数据来训练 LLaMA 模型（如 Alpaca, Vicuna）。优点是简单易用、Teacher 可以是闭源 API；缺点是丢失了概率分布信息，学习效率较低。(2) 白盒蒸馏 (**White-box Distillation**)：可以访问 Teacher 的 logits（输出概率分布）甚至中间层特征。训练 loss 包括 KL 散度 $KL(P_{teacher} || P_{student})$ 让 Student 拟合 Teacher 的概率分布，加上标准的 CE loss。优点是传递了更多信息（哪些 token 概率高、哪些 token 是「次优但合理」的选择），学习效率更高；缺点是需要 Teacher 和 Student 同时加载，显存需求大。

面试加分点：能提到很多小模型的训练数据部分来自大模型生成，本质上就是黑盒蒸馏的一种形式。

六、Nanobot 源码与设计深度题 (Q86-Q100)

Q86. Nanobot 的 AgentRunner 如何控制迭代次数? 为什么主 Agent 是 40 次而子 Agent 是 15 次?

答案：max_iterations 参数控制 Agent 在单次对话中最多执行的工具调用轮数。主 Agent 默认 40 次，允许完成复杂的多步任务（如代码重构涉及读取多个文件、分析、逐一修改、测试）。子 Agent 限制为 15 次，防止后台任务过度消耗资源——子 Agent 通常执行范围明确的子任务，不需要太多轮次。设计体现了「主从权责分离」原则：主 Agent 拥有更大的行动自由度承担复杂编排，子 Agent 以精简执行为目标。如果子 Agent 也允许 40 次迭代且不受限，多个并发子 Agent 可能导致 API 费用失控。

面试加分点：能提到 max_iterations 是在 AgentRunner 的主循环中递减检查的，到达上限后 Agent 会生成最终回复而非继续调用工具。

Q87. MemoryConsolidator 的触发条件是什么? 合并失败怎么办?

答案：当 estimate_prompt_tokens_chain 估算的当前对话 token 数超过 context_window_tokens 配置值时，MemoryConsolidator 被触发。合并流程通过 save_memory 虚拟工具执行——LLM 被要求生成 history_entry（时间戳摘要追加到 HISTORY.md）和 memory_update（新 MEMORY.md 全文）。如果 LLM 连续 3 次未能正确调用 save_memory 工具（比如生成了普通文本而非工具调用），系统进入 Raw Archive 模式：将原始消息 JSON 直接写入 HISTORY.md 文件，防止对话历史数据丢失。这种降级策略体现了「故障安全」设计原则——宁可保存格式不优雅的原始数据，也不丢失信息。

面试加分点：能提到 tool_choice 参数在记忆合并时被设为强制调用 save_memory，失败后回退到 auto 模式，展示了渐进降级策略。

Q88. MCPToolWrapper 如何将 MCP 工具 schema 转换为 OpenAI 格式?

答案：_normalize_schema_for_openai 方法处理 MCP 与 OpenAI Function Calling 之间的 JSON Schema 差异：(1) nullable union 类型简化：MCP 中 {"type": ["string", "null"]} 被转换为 {"type": "string"}，因为 OpenAI 不支持类型数组。(2) anyOf/oneOf 扁平化：将复杂的联合类型简化为第一个非 null 类型。(3) 空 schema 填充：如

果 MCP 工具未定义参数 schema，自动填充 `{"type": "object", "properties": {}}` 的默认结构，避免 OpenAI API 报错。工具名使用 `mcp_{server}_{tool}` 格式命名，确保多个 MCP Server 的工具名不冲突。这个转换层是 Nanobot 支持 MCP 生态的关键适配器。

面试加分点：能提到这种 schema 转换是有损的——复杂的 MCP schema 可能丢失精度，这也是为什么 MCP 工具的参数设计应尽量保持简单。

Q89. Nanobot 的并发控制机制是怎样的？

答案：三层并发控制：(1) `**_session_locks`：每个 `session_key` 对应一把 `asyncio.Lock`，确保同一会话内的消息串行处理，防止两条并发消息同时读写会话历史导致数据损坏。(2) `_concurrency_gate`： `asyncio.Semaphore` (默认值 3)，限制跨会话的并发 LLM 请求数量，防止同时发起过多 API 调用导致速率限制或费用暴增。(3) `_active_tasks`： `Dict[session_key, asyncio.Task]` 映射，记录每个会话当前正在执行的任务，支持 `/stop` 命令通过 `task.cancel()` 取消正在进行的 Agent 循环。三层控制分别解决数据一致性、资源保护和用户控制三个不同维度的并发问题。

面试加分点：能解释为什么用 `asyncio.Lock` 而非 `threading.Lock`——Nanobot 是单线程事件循环架构，`asyncio` 原语与其异步模型天然匹配。

Q90. ContextBuilder 如何优化 Prompt Caching？

答案：ContextBuilder 将 system prompt 分为静态部分和动态部分：静态部分包括 Agent Identity (AGENTS.md 内容)、Bootstrap 文件、Memory (MEMORY.md + HISTORY.md)、Skills 摘要目录；动态部分包括当前时间戳、channel/chat_id 等信息，这些被放在 user 消息侧而非 system prompt 中。静态部分不含任何随时间变化的信息（如时间戳），这样每次请求的 system prompt 前缀保持不变，利于 LLM Provider 侧的 Prompt Cache 命中——Provider 可以复用已缓存的 KV Cache 前缀，显著降低首 token 延迟和计算成本（Anthropic 的 Prompt Caching 可节省约 90% 的输入 token 费用）。

面试加分点：能提到 Anthropic 的 `cache_control` 标记和 OpenAI 的自动 Prompt Caching 机制的区别——前者需要显式标记缓存点，后者自动匹配最长公共前缀。

Q91. Skills 的渐进披露（Progressive Disclosure）机制如何工作？

答案：三层结构：**Tier1**——摘要目录注入 system prompt，每个 Skill 只包含名称、描述和触发条件（几十个 token），让 Agent 知道有哪些能力可用。**Tier2**——当 Agent 判断某个 Skill 与当前任务相关时，通过 `read_file` 工具读取 SKILL.md 全文，获取完整的指令和模板。**Tier3**——Skill 中引用的 scripts、references 等附加资源，按需进一步读取。特殊情况：`always: true` 的 Skill 直接全文注入 system prompt（跳过 Tier1 的摘要阶段）。这种设计在「Agent 的全面能力感知」和「上下文窗口节省」之间取得了平衡——10 个 Skill 的摘要可能只占 500 token，而全文可能需要 5000+ token。

面试加分点：能对比 LangChain 的 Tool 注册方式（全量注入 tool description）与 Nanobot 渐进披露的效率差异。

Q92. Nanobot 如何防止子 Agent 无限递归？

答案：SubagentManager 在创建子 Agent 时实施两道防线：(1) 工具剥离：子 Agent 的工具集中移除了 MessageTool（发消息给用户）和 SpawnTool（创建新子 Agent），这使得子 Agent 无法与用户直接通信，也无法再 spawn 新的子代理，从架构层面切断了递归链条。(2) 迭代硬限制：子 Agent 的 `max_iterations` 固定为 15 次（远低于主 Agent 的 40 次），即使子 Agent 陷入低效循环，也会在 15 轮后强制停止。两道防线相互独立——即使工具剥离被某种方式绕过（理论上不可能，因为是代码级检查），迭代限制仍然生效。这体现了纵深防御原则。

面试加分点：能对比 AutoGen 等框架中子 Agent 可以相互调用导致的递归风险，说明 Nanobot 的设计更安全。

Q93. ChannelManager 的出站消息处理策略?

答案：三个关键策略：(1) `**_stream_delta` 合并降频：LLM 的流式输出可能每几个 **token** 就产生一次 **delta**，如果每次都推送给前端会造成网络开销过大。**ChannelManager** 在短时间窗口内合并多个 **delta** 再统一发送，改善用户的流式阅读体验。(2) 元数据过滤 `**`： `_progress` 和 `_tool_hint` 等内部元数据消息根据 Channel 配置决定是否转发给用户——CLI 模式可能展示全部调试信息，而 Telegram 模式只展示最终结果。(3) 发送失败重试：当消息发送失败时（如网络波动），采用指数退避重试（1s → 2s → 4s → …），避免在网络恢复前大量重试请求堆积。

面试加分点：能提到不同 Channel（Telegram/Discord/CLI）的消息格式差异——如 Telegram 有消息长度限制需要分段发送。

Q94. `_TOOL_RESULT_MAX_CHARS (16000)` 设计目的?

答案：这个常量限制了单次工具调用返回结果的最大字符数，保护上下文窗口不被单次工具返回撑满。例如 `exec` 工具执行 `cat large_file.txt` 可能返回数十万字符，如果全部注入对话历史，会快速耗尽上下文窗口并导致后续轮次的 API 费用暴增。截断在 `hook/loop` 层统一执行，对所有工具（`read_file`、`exec`、`web_fetch` 等）一视同仁。截断后的结果会附带提示告知 Agent 输出已被截断，Agent 可以通过 `read_file` 的 `offset/limit` 参数分页获取完整内容。16000 字符大约对应 4000-5000 token，在 128K 窗口中占比约 3-4%，是「信息量充足」和「窗口保护」之间的合理平衡点。

面试加分点：能提到这个设计与 Cursor IDE 等工具的做法类似——工具输出太长时截断并提示，而非无限注入。

Q95. Nanobot 的 Provider 注册机制如何实现?

答案：PROVIDERS 全局注册表模式。系统维护一个 Provider 名称到实现类的字典映射（`registry.py`）。新增 Provider 只需两步：(1) 在 `registry.py` 中添加类映射（如 `"deepseek": DeepSeekProvider`）；(2) 在 `schema.py` 中添加对应的配置字段定义。运行时根据 `config.json` 中的 `provider` 字段查找注册表，实例化对应的 Provider 类。支持 **auto** 模式：系统按优先级依次检测环境变量中是否存在各 Provider 的 API Key，自动选择第一个可用的 Provider。这种注册表模式使得扩展新 Provider 无需修改核心代码，符合开闭原则。

面试加分点：能对比 LangChain 的 Provider 集成方式——LangChain 为每个 Provider 单独发布 pip 包（如 `langchain-openai`），而 Nanobot 内置所有 Provider 保持轻量。

Q96. `save_memory` 虚拟工具的设计意义?

答案：将记忆合并操作包装为工具调用而非硬编码逻辑，核心意义在于：让 LLM 自主决定记忆的内容和格式。LLM 通过 `save_memory` 工具传入两个参数：`history_entry`（一段时间戳 + 摘要文本，追加到 `HISTORY.md`）和 `memory_update`（新版 `MEMORY.md` 的完整内容）。这样 LLM 可以基于对话上下文智能决定哪些信息值得长期记忆、如何组织和压缩信息。执行时通过 `tool_choice` 参数强制 LLM 必须调用 `save_memory`（而非生成普通文本），失败时回退到 `tool_choice: auto` 给 LLM 更多自由度。这种「LLM 驱动的记忆管理」比规则化的摘要算法更灵活，能保留对用户最有价值的信息。

面试加分点：能对比传统的记忆管理方案（如 LangChain 的 `ConversationSummaryMemory` 使用固定模板摘要）与 Nanobot 方案的灵活性差异。

Q97. InboundMessage 的 `session_key` 机制?

答案：默认 `session_key` 格式为 `f"{channel}:{chat_id}"`，用于隔离不同来源的会话上下文。例如 Telegram 中的不同聊天窗口、Discord 中的不同频道各自维护独立的对话历史和记忆。`session_key_override` 允许自定义 `session_key`，实现更灵活的会话隔离策略：(1) 线程隔离：在同一个群聊中，不同话题线程使用不同 `session_key`，避免话题串扰；(2) 子会话：主对话中临时创建一个隔离的子会话处理特定任务，完成后回到主会话；(3) 跨 Channel 共享：让 Telegram 和 Discord 的同一个用户共享同一个 `session_key`，实现跨平台的连续对话体验。

面试加分点：能关联到 `_session_locks` 的设计——`session_key` 不仅用于数据隔离，还用于并发控制（同一 `session_key` 的请求串行处理）。

Q98. Nanobot 的 exec 工具有哪些安全防护?

答案：五层安全防护：(1) **restrict_to_workspace**：exec 工具的工作目录 (cwd) 被强制设置为 workspace 路径，命令只能在沙箱内执行。(2) 危险命令模式拒绝：使用正则表达式匹配危险模式（如 `rm -rf /`、`mkfs`、`curl ... | bash`），在执行前直接拒绝。(3) **SSRF** 防护：web_fetch 工具阻止访问私有 IP 地址（10.x、172.16-31.x、192.168.x）和云元数据地址（169.254.169.254），防止 Agent 被注入访问内网。(4) 路径白名单检查：文件操作工具验证目标路径是否在 workspace 范围内，防止路径遍历攻击（如 `../../etc/passwd`）。(5) 异步执行：通过 `asyncio.create_subprocess_shell` 异步执行命令，配合超时控制防止命令无限运行。

面试加分点：能指出这些防护都是代码级硬限制——即使 LLM 被 Prompt 注入操纵，也无法绕过这些安全检查。

Q99. 如何评价 Nanobot 用 Markdown 文件 (MEMORY.md) 做长期记忆的设计?

答案：优点：(1) 人类可读可编辑——用户可以直接打开 MEMORY.md 查看和修改 Agent 的记忆，透明度极高。(2) 版本控制友好——Markdown 文件可以用 Git 追踪变更历史，方便回溯和审计。(3) 无外部依赖——不需要向量数据库或其他基础设施，降低部署复杂度。(4) 与 LLM 天然兼容——Markdown 是 LLM 预训练数据中最常见的格式之一。缺点：(1) 不支持语义检索——无法根据语义相似度查找相关记忆，只能全文注入。(2) 大量记忆时效率低——记忆内容增长后，每次对话都需要注入全文，消耗大量上下文窗口。(3) 缺乏结构化查询——无法像数据库那样按条件过滤和聚合。适合个人助手场景（记忆量有限），企业级场景需要配合向量库或知识图谱扩展。

面试加分点：能提出改进方案——如将 MEMORY.md 分为多个主题文件，Agent 按需加载相关主题，减少全文注入的开销。

Q100. 如果让你扩展 Nanobot 支持多用户，需要改造哪些模块?

答案：五个核心改造点：(1) **SessionManager** 增加用户维度隔离：当前 session_key 基于 channel:chat_id，需要增加 user_id 维度，确保同一个群聊中不同用户的对话上下文相互隔离。(2) **Memory** 按用户分目录：将 memory/MEMORY.md 改为 memory/{user_id}/MEMORY.md，每个用户维护独立的长期记忆。(3) 权限系统：引入用户-工具 ACL（访问控制列表），不同用户可使用不同的工具集——管理员可以用 exec，普通用户只能用 read_file 和 web_search。(4) **Provider** 层增加用量追踪与限流：按用户记录 API 调用次数和 token 消耗，设置每用户的配额上限，防止单个用户耗尽团队预算。(5) **MessageBus** 增加用户路由标签：出站消息需要标记目标用户，确保 Agent 的回复发送给正确的用户。

面试加分点：能提到多用户场景下 Prompt Caching 的挑战——不同用户的 system prompt（包含各自 Memory）不同，缓存命中率会下降。

七、RAG (Q101-Q110)

Q101. RAG 解决了什么问题?

答案：RAG (Retrieval-Augmented Generation, 检索增强生成) 解决了 LLM 的几个核心局限：(1) 知识时效性：LLM 的知识截止于训练数据的时间点，无法回答训练后的新信息。RAG 通过检索实时数据源来补充最新知识。(2) 幻觉问题：LLM 可能生成看似正确但实际错误的信息。RAG 提供真实的参考文档，让模型基于事实生成，显著降低幻觉概率。(3) 领域知识缺失：LLM 的预训练数据难以覆盖所有专业领域。RAG 让模型能访问特定领域的知识库（如企业内部文档、医学文献）。(4) 可追溯性：RAG 的回答可以附带来源引用，用户可以验证信息的真实性。(5) 数据隐私：敏感的企业数据不需要用于模型训练，只需放入检索库，通过 RAG 在推理时临时注入。

面试加分点：一句话定位 RAG：「让模型从『凭记忆回答』变成『查资料回答』」。

Q102. RAG 的基本流程是什么？

答案：RAG 的标准流程包括五个阶段：(1) 文档预处理 (**Indexing**)：将原始文档 (PDF、网页、数据库等) 切分为合适大小的 chunk (段落或语义块)，每个 chunk 通过 Embedding 模型转化为稠密向量，存入向量数据库 (如 Milvus、Pinecone、ChromaDB)。(2) 查询处理 (**Query Processing**)：将用户查询也转化为向量，可能还包括查询改写、HyDE (假设性文档嵌入) 等增强手段。(3) 检索 (**Retrieval**)：在向量数据库中进行近似最近邻 (ANN) 搜索，找出与查询最相关的 Top-K 个 chunk。(4) 重排 (**Rerank**)：用 Cross-Encoder 模型对检索结果进行精排，提高排序质量。(5) 生成 (**Generation**)：将检索到的 chunk 与用户查询一起组成 prompt，发送给 LLM 生成最终回答。LLM 被指示基于提供的上下文回答，避免编造。

面试加分点：能画出 RAG 的数据流图，标注出离线索引和在线检索两个阶段的界限。

Q103. RAG 和微调如何选择？

答案：两者解决不同类型的问题，选择依据如下：

| 场景 | 推荐方案 | 原因 |
|-----------|------|-------------|
| 需要最新信息 | RAG | 知识库可以实时更新 |
| 需要专有格式/风格 | 微调 | 微调改变模型的行为模式 |
| 企业知识库问答 | RAG | 数据量大、更新频繁 |
| 特定领域术语理解 | 微调 | 让模型内化领域知识 |
| 快速原型验证 | RAG | 不需要训练，部署快 |
| 多语言/代码能力 | 微调 | 需要改变模型核心能力 |
| 数据隐私要求高 | RAG | 数据不进入模型权重 |

两者不是互斥的，最佳实践是结合使用：先微调模型使其具备领域理解力，再用 RAG 注入具体知识和实时数据。类比：微调是「上大学学专业基础」，RAG 是「工作中查参考手册」。

面试加分点：提到 RAG 的成本通常低于微调——不需要 GPU 训练，只需维护向量数据库和检索服务。

Q104. Chunk 分割有哪些策略？

答案：Chunk 分割策略直接影响 RAG 的检索质量：(1) 固定长度分割：按固定 token 数 (如 256、512) 切分，简单但可能在句子中间截断，破坏语义完整性。通常配合滑动窗口和重叠 (overlap) 使用。(2) 基于分隔符：按段落、句号、换行符等自然分界切分。保持语义完整但 chunk 大小不均匀。(3) 递归分割：先尝试大粒度分割 (如段落)，超过长度限制则递归用更小粒度 (如句号、换行) 继续分割。LangChain 的 RecursiveCharacterTextSplitter 就是这种策略。(4) 语义分割：用 Embedding 模型计算相邻句子的语义相似度，在相似度骤降处切分 (表明话题转换)。质量最高但计算成本也最高。(5) 文档结构感知：利用 Markdown 标题、HTML 标签、代码块等结构化信息进行切分。Chunk 大小的选择也很关键：太小会丢失上下文，太大会引入噪声。

面试加分点：能提到「Small-to-Big」策略——用小 chunk 检索 (精度高)，但返回小 chunk 所在的大 chunk (上下文完整)。

Q105. 什么是混合检索 (Hybrid Search)？

答案：混合检索结合了稠密向量检索和稀疏检索 (如 BM25) 的优势：(1) 稠密检索 (**Dense Retrieval**)：通过 Embedding 模型将文本转化为稠密向量，用余弦相似度进行语义匹配。优势是能理解同义词、语义相似的表述。缺点是对精确关键词 (如产品编号、人名) 的匹配能力弱。(2) 稀疏检索 (**Sparse Retrieval**, 如 **BM25**)：基于词频和逆文档频率的传统检索方法。优势是精确匹配关键词的能力强、对长尾词效果好。缺点是不理解语义，同义词会 miss。(3) 混合方式：同时执行两种检索，将结果用加权融合 (如 $score = \alpha \times dense_score + (1-\alpha) \times sparse_score$) 或 RRF (Reciprocal Rank Fusion) 合并排序。 的选择取决于场景——技术文档查询偏重精确匹配 (较小)，开放问答偏重语义匹配 (较大)。

面试加分点：能说出 Elasticsearch 8.x 已经原生支持混合检索（dense + BM25），以及 Milvus 也在 2024 年支持了混合检索。

Q106. Rerank 在 RAG 中的作用和实现方式？

答案：Rerank 是 RAG 流程中提高检索精度的关键步骤：（1）为什么需要 **Rerank**：向量检索是「双编码器（Bi-Encoder）」模式——query 和 document 分别编码再比较，效率高但精度有限，因为编码过程中两者没有交互。Rerank 使用「交叉编码器（Cross-Encoder）」——将 query 和 document 拼接在一起输入模型，模型可以进行逐 token 的深度交互，精度显著更高。（2）实现方式：先用向量检索召回 Top-K（如 Top-50），再用 Cross-Encoder 对这 K 个结果重新打分排序，取 Top-N（如 Top-5）。（3）常用模型：Cohere Rerank API、BGE Reranker（中文效果好）、Cross-Encoder/ms-marco-MiniLM 等。（4）效果：通常能将检索的 NDCG@10 提升 5-15 个百分点。代价是推理延迟增加——Cross-Encoder 需要对每个 query-document 对单独推理。

面试加分点：提到「Rerank 是最容易提升 RAG 效果的单一手段」——加 Rerank 通常比调 Chunk 策略或换 Embedding 模型的效果更立竿见影。

Q107. 检索冲突（Retrieved Conflicts）如何处理？

答案：检索冲突指检索到的多个文档包含相互矛盾的信息，这在企业知识库中很常见。处理策略包括：（1）时间优先：为文档附加时间戳，冲突时优先采信最新的文档。适合政策、规范等时效性强的内容。（2）来源权重：为不同来源设置可信度权重——官方文档 > 内部 wiki > 论坛帖子。检索结果中附带来源标注。（3）LLM 判断：在 prompt 中明确告知 LLM 「以下文档可能存在冲突，请综合分析并标注不确定之处」，利用模型的推理能力识别矛盾。（4）一致性检测：在检索后用 NLI（自然语言推理）模型检测文档间的矛盾，过滤或标注冲突内容。（5）多答案呈现：当无法判断哪个正确时，将不同观点都呈现给用户，让用户自行判断。（6）知识图谱辅助：将文档中的事实抽取为知识图谱，通过图谱一致性检查发现冲突。

面试加分点：能举实际例子：「公司报销标准去年改过，新旧文档都被检索到，如何确保回答用新标准」。

Q108. 企业 RAG 中如何实现权限隔离？

答案：权限隔离是企业 RAG 部署的关键安全需求，确保用户只能检索到有限权限访问的文档。实现方案包括：（1）元数据过滤：在文档入库时附加权限元数据（如部门、角色、密级），检索时在 query 中加入权限过滤条件，如 `filter: {department: "销售部", level: <= "机密"}`。这是最常用的方案。（2）分库存储：按权限级别或部门建立独立的向量索引，不同用户只连接对应的索引。隔离彻底但管理成本高。（3）后置过滤：先检索再过滤——从全量库中检索 Top-K×2，然后根据用户权限过滤掉无权限的文档，保留 Top-K。简单但可能浪费计算资源。（4）ACL 集成：与企业现有的 LDAP/AD 权限系统集成，实时获取用户的文档访问权限列表。（5）行级权限：对于数据库类知识，在 SQL 层面实现行级安全策略（RLS）。

面试加分点：提到权限隔离的测试方法——「越权测试」，用低权限用户检索高密级文档，验证是否被正确过滤。

Q109. Self-RAG 是什么？它的创新点在哪里？

答案：Self-RAG 由 Asai et al. (2023) 提出，核心创新是让模型自主决定何时检索、检索什么、以及是否使用检索结果，而非每次查询都强制检索。传统 RAG 的问题是：有些查询不需要检索（如常识问题），有些检索结果与查询无关（污染生成），但传统 pipeline 无差别地检索和注入。Self-RAG 的做法是在 LLM 中训练三类特殊的「反射 token（Reflection Tokens）」：（1）**Retrieve Token**：模型判断是否需要检索——[Retrieve: Yes/No]。（2）**Relevance Token**：判断检索结果是否相关——[Relevant: Yes/No]。（3）**Support Token**：判断生成的回答是否被检索结果支持——[Supported: Fully/Partially/No]。模型通过这些 token 实现了自适应检索和自我验证，显著提高了 RAG 的效果和效率。

面试加分点：Self-RAG 的思想与 Agent 的 ReAct 框架相通——都是让模型自主决定何时使用外部工具（检索也是一种工具）。

Q110. 多模态 RAG 有哪些挑战和方案?

答案: 多模态 RAG 处理图片、表格、视频等非文本内容的检索和生成, 面临独特挑战: (1) 多模态 **Embedding** 对齐: 文本和图片需要映射到同一向量空间才能进行跨模态检索。可用 CLIP 等多模态模型, 但对专业领域 (如医学影像) 效果可能不够好。(2) 表格理解: PDF 中的表格难以结构化提取, OCR 可能引入错误。方案包括用多模态 LLM (如 GPT-4V) 直接理解表格图片, 或用专门的表格提取工具。(3) 图文混合 **Chunk**: 如何切分包含图片的文档? 图片与相邻文本应保持关联。方案: 将图片用 **Caption** 模型生成文字描述, 与周围文本一起作为 **chunk**。(4) 检索排序: 不同模态的相似度分数量纲不同, 如何统一排序? 方案: 归一化或学习模态权重。(5) 生成端: 需要多模态 LLM (如 GPT-4V、Claude 3.5) 才能处理包含图片的上下文。

面试加分点: 提到 ColPali 等方案直接对文档页面图片做向量化 (page-level embedding), 绕过了文本提取和 OCR 的问题。

八、多智能体 (Q111-Q118)

Q111. 单 Agent 的瓶颈在哪里?

答案: 单 Agent 面临多个瓶颈: (1) 上下文窗口限制: 复杂任务需要大量的工具调用结果、中间推理、历史记录, 单 Agent 的上下文很快被耗尽, 即使有记忆压缩也会损失信息。(2) 能力冲突: 一个 Agent 需要同时扮演规划者、执行者、评估者等多个角色, **system prompt** 变得冗长且矛盾。(3) 错误累积: 单 Agent 的每一步错误都会影响后续步骤, 长链条任务中错误概率随步骤数指数增长。(4) 并行受限: 单 Agent 顺序执行步骤, 无法并行处理多个独立子任务。(5) 专业性不足: 通用 Agent 在特定领域 (如代码审查、数据分析) 的能力不如专门优化的 Agent。(6) 调试困难: 单 Agent 的行为链条长且复杂, 出问题时难以定位是规划错误还是执行错误。

面试加分点: 能提到 Nanobot 通过 SubagentManager 实现子 Agent 来突破单 Agent 瓶颈, 每个子 Agent 有独立上下文和明确职责。

Q112. 多 Agent 协作的核心收益是什么?

答案: 多 Agent 协作带来以下收益: (1) 分治复杂度: 将复杂任务分解给多个专业化 Agent, 每个 Agent 只需处理自己擅长的子任务, 降低单个 Agent 的认知负担。(2) 并行加速: 独立的子任务可以分配给不同 Agent 同时执行, 缩短整体耗时。如代码审查 + 测试用例生成可以并行。(3) 专业化: 每个 Agent 可以有定制的 **system prompt**、工具集和知识库, 专注于特定领域达到更好的效果。(4) 容错性: 单个 Agent 失败不会导致整个系统崩溃, **Orchestrator** 可以重试或分配给其他 Agent。(5) 可扩展性: 新增能力只需添加新的 **Worker Agent**, 不影响现有系统。(6) 质量保证: 可以设置专门的评估 Agent 对其他 Agent 的输出进行质量检查, 实现「四眼原则」。

面试加分点: 用 Nanobot 的例子——主 Agent 将「探索代码库」和「执行测试」分配给不同子 Agent 并行执行。

Q113. 中心化 Boss-Worker 模式的优缺点是什么?

答案: 中心化 Boss-Worker 是最常见的多 Agent 架构, 由一个 Boss Agent 负责规划和分配, 多个 Worker Agent 负责执行: 优点: (1) 架构清晰, 职责分明——Boss 规划、Worker 执行、Boss 汇总。(2) 全局视角——Boss 掌握所有子任务的进度和结果, 便于全局优化。(3) 通信简单——只有 Boss-Worker 之间的通信, Worker 之间不直接交互。(4) 易于调试——问题定位只需检查 Boss 的决策和对应 Worker 的执行。缺点: (1) Boss 是单点瓶颈——Boss 的能力上限决定了系统的天花板。(2) Boss 上下文压力——需要理解所有 Worker 的结果并汇总, 上下文消耗大。(3) Worker 无法自主协调——如果两个 Worker 的任务有依赖, 必须通过 Boss 中转, 效率低。(4) Boss 决策延迟——所有决策集中在 Boss, 响应速度受限。

面试加分点: Nanobot 的 SubagentManager 就是 Boss-Worker 模式, 主 Agent 是 Boss, Task 创建的子 Agent 是 Worker。

Q114. 流水线 (Pipeline) 模式适合什么场景?

答案: 流水线模式是多 Agent 的串行协作方式, 一个 Agent 的输出作为下一个 Agent 的输入, 像工厂的流水线一样逐步处理。适合的场景包括: (1) 多步骤处理: 如「需求分析 → 代码生成 → 代码审查 → 测试生成」, 每个步骤由专

门的 Agent 负责。(2) 质量门控：每个 Agent 充当一个质量关卡，只有通过检查的输出才能传递到下一步。(3) 数据处理管道：如「数据清洗 → 特征提取 → 模型推理 → 结果格式化」。(4) 内容创作：如「素材收集 → 初稿撰写 → 审核修改 → 排版发布」。优点是流程清晰、每步可独立优化和替换。缺点是：总延迟是各步骤延迟之和（无并行加速），任何一步的错误会传播到后续所有步骤，难以处理需要回溯的情况（如审查发现问题需要修改代码）。

面试加分点：对比 Pipeline 模式和 Unix 管道 `cat | grep | sort`——设计哲学一致，每个工具做好一件事，通过标准接口串联。

Q115. 民主/辩论式多 Agent 有什么优势？

答案：民主/辩论式是一种特殊的多 Agent 协作模式，多个 Agent 对同一个问题独立给出观点，然后通过「讨论」和「投票」达成共识。典型流程：(1) 多个 Agent 独立生成回答；(2) 每个 Agent 看到其他 Agent 的回答后给出评价和修改意见；(3) 经过多轮讨论后投票或由裁判 Agent 做最终决策。优势包括：(1) 提高准确性：多个独立推理路径降低单一 Agent 出错的概率，类似「群体智慧」。(2) 减少偏见：不同 Agent 可能有不同的 system prompt 或知识侧重，多元视角减少单一视角的偏见。(3) 暴露不确定性：如果多个 Agent 意见分歧大，说明问题本身有歧义或不确定性，系统可以据此向用户寻求澄清。(4) 适合主观判断：对于没有唯一正确答案的问题（如代码设计方案选择），辩论能考虑更多角度。

面试加分点：能提到 Google 的 Society of Mind 论文和 MIT 的 Multi-Agent Debate 研究，这些是辩论式多 Agent 的学术来源。

Q116. 多 Agent 系统的通信冗余如何控制？

答案：多 Agent 通信冗余是指 Agent 之间传递了过多不必要的信息，导致 token 浪费和上下文污染。控制策略包括：(1) 结构化消息格式：定义标准的消息 schema，Agent 只传递约定格式的信息，而非完整的对话历史或思考过程。(2) 摘要压缩：将 Agent 的完整输出压缩为摘要后再传递给下一个 Agent。Nanobot 子 Agent 返回给主 Agent 的就是最终结果摘要而非完整执行日志。(3) 按需通信：Agent 之间只在必要时通信，而非每步都同步状态。如 Worker 只在完成任务或遇到问题时向 Boss 汇报。(4) 信息过滤：在通信层面过滤掉对接收方无用的信息——代码 Agent 不需要知道设计 Agent 的详细推理过程，只需要知道最终的设计决策。(5) 共享状态板 (Blackboard)：不通过直接消息传递，而是通过共享的状态存储进行间接通信，Agent 只读取自己需要的部分。

面试加分点：能估算通信成本——每个 Agent 的输出约 500-2000 tokens，10 个 Agent 全互连通信 (10×9 条消息) 可能消耗 10 万 + tokens。

Q117. 多 Agent 系统如何保证一致性？

答案：一致性是指多个 Agent 的行为和输出不相互矛盾，这在分布式系统中是经典难题。保证策略包括：(1) 共享上下文：关键事实和约束通过共享的知识库或配置传递给所有 Agent，确保基础认知一致。(2) 全局约束注入：在每个 Agent 的 system prompt 中注入统一的约束条件和规范，如「使用简体中文」「代码风格遵循 PEP 8」等。(3) 冲突检测：Boss/Orchestrator 在汇总 Worker 输出时检测矛盾（如一个 Agent 建议用 MySQL，另一个建议用 PostgreSQL），要求矛盾方协调。(4) 事务性操作：对共享资源（如文件系统、数据库）的修改使用锁或事务机制，防止并发冲突。Nanobot 的 session_locks 就是这种机制。(5) 版本控制：对共享状态使用版本号，Agent 修改前检查版本是否过期，过期则重新读取最新状态。

面试加分点：类比分布式系统的 CAP 定理——多 Agent 也面临一致性 (Consistency) 和可用性 (Availability) 的权衡。

Q118. 多 Agent 系统的调试有哪些难点和技巧？

答案：多 Agent 调试是公认的难题，主要难点和应对技巧包括：(1) 因果链追踪难：一个 Agent 的错误可能是因为上游 Agent 传递了错误信息。技巧：为每个 Agent 的每次执行生成 trace ID，串联完整的因果链。(2) 不确定性：LLM 的非确定性输出使得同样的输入可能产生不同的 Agent 行为，bug 难以复现。技巧：固定 temperature=0 和 random seed 来增加可复现性。(3) 状态空间爆炸：多个 Agent 的状态组合呈指数增长。技巧：用日志记录每个 Agent 在每个 decision point 的输入和输出。(4) 性能瓶颈定位：延迟可能来自某个 Agent 的 LLM 调用、某个工具的执行、或

Agent 间的通信。技巧：用分布式 tracing 工具（如 Jaeger、LangSmith）可视化时间线。（5）单元测试困难：Agent 的行为依赖上下文，难以隔离测试。技巧：mock LLM 响应，用确定性的模拟替代非确定性的模型调用。

面试加分点：提到 LangSmith 和 Weights & Biases 等可观测性工具在多 Agent 调试中的价值。

九、系统设计（Q119-Q126）

Q119. 请设计一个企业知识库 Agent 系统

答案：系统架构设计如下：

需求分析：企业员工通过对话方式查询内部知识库（文档、制度、FAQ），支持多轮对话、权限隔离、多数据源。

架构分层：- 接入层：飞书/钉钉 Channel 适配器，接收用户消息并返回回复。参考 Nanobot 的 Channel 模式。- Agent 层：基于 ReAct 循环的 Agent 引擎，负责理解用户意图、决定是否需要检索、调用工具、生成回复。- 检索层：混合检索引擎（向量检索 + BM25），Rerank 模型精排，支持元数据过滤实现权限隔离。- 知识层：文档解析管道（PDF/Word/HTML → Chunk → Embedding → 向量库），增量更新和版本管理。- 存储层：向量数据库（Milvus）+ 关系数据库（PostgreSQL，存元数据和用户信息）+ 对象存储（原始文档）。- 基础设施：Docker/K8s 部署，Redis 缓存，Prometheus + Grafana 监控。

关键设计决策：多租户用 namespace 隔离；缓存热门查询减少 LLM 调用；异步处理文档入库；MCP Server 封装检索能力。

面试加分点：能画出清晰的架构图，标注数据流向和关键接口。

Q120. 如何设计 Agent 系统的高可用方案？

答案：Agent 系统的高可用设计覆盖多个层面：（1）无状态化：Agent 服务本身设计为无状态，会话状态存储在 Redis 或数据库中，任何实例都能处理任何请求。这样可以通过水平扩展和负载均衡实现高可用。（2）LLM API 容灾：配置多个模型提供商（如 Claude + GPT + Qwen）作为 fallback，一个 API 不可用时自动切换到备用。（3）限流与降级：实现令牌桶限流，超限时降级到轻量级回复（如缓存答案或规则匹配）而非直接报错。（4）工具调用超时：为每个工具设置合理的超时时间和重试策略，防止单个工具的故障拖垮整个系统。（5）健康检查：定期检测 LLM API、向量库、MCP Server 的健康状态，不健康的服务自动摘除。（6）数据持久化：对话历史和记忆异步持久化到数据库，服务重启后可恢复。（7）灰度发布：新版本 Agent 先在小流量上验证，确认无问题后全量发布。

面试加分点：能给出具体的 SLA 指标，如「可用性 99.9%、P99 延迟 < 10s、单日最大支持 10 万次对话」。

Q121. 多租户 Agent 系统如何设计？

答案：多租户设计需要在共享基础设施的同时保证租户间的隔离：（1）数据隔离：每个租户的知识库数据存储在独立的向量库 namespace/collection 中，检索时只查询对应租户的数据。对话历史和记忆按 tenant_id 隔离存储。（2）配置隔离：每个租户可以自定义 system prompt、可用工具集、模型选择等，存储在租户级别的配置中。参考 Nanobot 的配置驱动组装理念。（3）资源隔离：通过 API 配额和限流策略防止单个租户耗尽共享资源。可以用 K8s 的 Resource Quota 实现计算资源隔离。（4）安全隔离：租户的 API 密钥、MCP Server 配置等敏感信息加密存储，相互不可见。（5）计费隔离：精确记录每个租户的 LLM token 使用量、工具调用次数、存储空间，支持按量计费。（6）性能隔离：对高价值租户保留专用资源池，防止被其他租户的高负载影响。

面试加分点：能对比「共享集群 + 逻辑隔离」和「独立部署」两种方案的成本和安全性权衡。

Q122. Agent 系统的可观测性如何设计？

答案：可观测性是 Agent 系统运维的基础，需要覆盖三个支柱：（1）Metrics（指标）：关键业务指标——对话量、任务完成率、用户满意度；技术指标——LLM API 延迟/成功率、Token 使用量、工具调用延迟/成功率、Agent 循环步骤数；资源指标——CPU/内存/GPU 使用率。使用 Prometheus + Grafana 实现。（2）Logging（日志）：结构化日志记录每次 Agent 循环的完整信息——用户输入、模型思考过程、工具调用参数和结果、最终输出。使用 ELK Stack

或 Loki 集中管理。关键是为每次对话分配 `trace_id`，串联所有相关日志。(3) **Tracing** (链路追踪)：记录单次请求从接收到返回的完整链路——Channel 接入 → Agent 循环 → LLM 调用 → 工具执行 → 响应返回，每个环节的耗时和状态。可以用 LangSmith 或自建基于 OpenTelemetry 的方案。特别要关注的是 **LLM** 输出的质量监控——定期抽样人工评估。

面试加分点：提到「黄金信号 (Golden Signals)」——延迟、流量、错误率、饱和度，这是 Google SRE 的经典监控理念。

Q123. Agent 系统的成本优化有哪些手段?

答案：LLM API 费用通常是 Agent 系统最大的成本项，优化手段包括：(1) **Prompt** 优化：精简 system prompt，去除冗余描述；使用 Nanobot 的渐进披露策略，只在需要时加载 Skill 详情。(2) 缓存：对相同或相似查询缓存 LLM 响应。语义缓存 (用 Embedding 相似度判断是否命中) 比精确匹配缓存覆盖率更高。(3) 模型分级：简单问题 (如 FAQ 查询) 用便宜的小模型 (如 GPT-3.5)，复杂问题 (如多步推理) 用贵的大模型 (如 GPT-4)，通过路由 Agent 自动分级。(4) **Prompt Cache**：利用 Anthropic 等提供商的 Prompt Cache 功能，重复的 prompt 前缀只需支付 10% 费用。Nanobot 已支持此优化。(5) 批处理：非实时任务 (如文档总结) 批量发送，利用批量 API 的折扣。(6) **Token** 压缩：在不影响理解的前提下压缩输入 (如去除多余空格、缩写长 URL)。(7) 提前终止：当 Agent 已经获得足够信息时尽早结束循环，避免不必要的额外 LLM 调用。

面试加分点：能估算具体成本——如每天 1000 次对话，每次平均 5000 tokens，GPT-4o 定价下月费约多少美元。

Q124. 什么是影子测试 (Shadow Testing)? 在 Agent 系统中如何应用?

答案：影子测试是将生产流量复制一份发送到新版本系统，但只观察不影响实际用户响应的测试方法。在 Agent 系统中的应用：(1) 新模型评估：将线上对话同时发送给旧模型和新模型，比较两者的输出质量，评估新模型是否值得切换。(2) **Prompt** 变更测试：修改 system prompt 或工具描述后，用影子流量验证新 prompt 是否会导致行为异常。(3) 工具升级测试：新版本 MCP Server 上线前，用影子流量验证兼容性和性能。(4) 安全检测：影子环境中检测新版本是否会产生不安全的输出。实现方式：在消息队列层面复制消息到影子 Agent 实例，影子实例的输出写入日志但不返回给用户。对比指标包括：任务完成率、工具调用正确率、输出质量评分、延迟、成本。注意事项：影子测试中的工具调用需要用 mock 或只读模式，防止对生产数据产生副作用。

面试加分点：类比传统 Web 服务的「流量镜像 (Traffic Mirroring)」，说明这是成熟的工程实践在 AI 系统中的应用。

Q125. 如何进行 Prompt 和工具的版本管理?

答案：Prompt 和工具是 Agent 行为的核心配置，需要严格的版本管理：(1) **Git** 版本化：将 system prompt、工具描述、Skill 文件等作为代码仓库的一部分管理，每次变更都有 commit 记录和 diff。Nanobot 的 `~/.nanobot/rules/` 和 `~/.nanobot/skills/` 目录设计天然支持这种方式。(2) 语义版本号：对 Prompt 和工具使用 SemVer——major 版本号变更表示不兼容的行为变化，minor 表示新增能力，patch 表示修复。(3) **A/B** 测试：线上同时运行多个版本的 Prompt/工具配置，通过流量分配比较效果。(4) 回滚机制：发现新版本有问题时能一键回滚到上一个稳定版本。(5) 变更审批：Prompt 变更需要 code review，因为一个小的 prompt 改动可能导致完全不同的 Agent 行为。(6) 效果追踪：记录每个版本上线后的关键指标变化，建立 prompt 版本与效果的因果关系。

面试加分点：能提到 PromptLayer、Humanloop 等专门的 Prompt 管理工具，以及 MLflow 等实验跟踪框架在 Prompt 管理中的应用。

Q126. Agent 系统的合规与内容安全如何设计?

答案：合规和内容安全是 Agent 上线的必要条件，设计要点包括：(1) 输入过滤：对用户输入进行敏感词检测和恶意注入检测 (如 prompt injection、jailbreak 攻击)，发现违规内容拒绝处理。(2) 输出审核：对 Agent 的输出进行内容安全审核，检测是否包含违法、暴力、歧视等内容。可以用专门的内容审核 API (如阿里绿网、百度内容审核) 或本地分类模型。(3) 工具权限控制：危险操作 (如删除文件、修改数据库) 需要额外的权限校验或人工审批。Nanobot 的 `permission_prompt_tool` 机制支持工具调用前的确认。(4) 数据脱敏：Agent 处理包含个人信息 (身份证号、手机号等) 的数据时，需要在日志和存储中进行脱敏处理。(5) 审计日志：记录所有 Agent 操作的完整审计日志，满足合规

审查要求。(6) 使用条款：明确告知用户 AI 生成内容的局限性，避免法律风险。(7) 大模型备案：在中国大陆提供 AI 对话服务需要完成大模型备案。

面试加分点：能提到《生成式人工智能服务管理暂行办法》等中国 AI 相关法规，展示合规意识。

十、工程实践 (Q127-Q134)

Q127. Agent 项目中如何管理 API 密钥和敏感配置？

答案：密钥管理是安全工程的基础：(1) 环境变量：最基本的方式，通过 .env 文件或系统环境变量注入，代码中通过 os.environ 读取。 .env 文件必须加入 .gitignore，绝对不能提交到 Git。(2) 密钥管理服务：生产环境使用 AWS Secrets Manager、HashiCorp Vault 或阿里云 KMS 等专业密钥管理服务，支持密钥轮换、访问审计、细粒度权限控制。(3) 配置分层：区分「代码配置」(Git 管理)和「环境配置」(密钥、连接串等，通过环境变量或密钥服务注入)。Nanobot 的 config.json 支持 \$env:VAR_NAME 引用环境变量，实现了这种分离。(4) 最小权限：每个服务只配置必要的密钥和权限，不共享全能密钥。(5) 定期轮换：API 密钥定期更换，减少泄露后的影响范围。(6) 泄露检测：使用 git-secrets 或 GitHub Secret Scanning 等工具防止密钥意外提交到代码仓库。

面试加分点：能提到自己在项目中犯过密钥泄露的错误（即使是模拟的），并说明从中学到的教训——「一旦泄露，必须立即轮换而非简单删除 commit」。

Q128. Agent 系统中的超时和重试策略如何设计？

答案：超时和重试是保证系统可靠性的关键机制：(1) 分层超时：为不同层级设置递减的超时——整体请求超时 60s > Agent 循环超时 45s > 单次 LLM 调用超时 30s > 工具调用超时 15s。内层超时必须小于外层，保证外层能正常处理超时错误。(2) 指数退避重试：对可重试的错误（网络超时 429/503、API 限流）使用指数退避——第 1 次 1s、第 2 次 2s、第 4 次 4s、最大 32s。添加随机抖动 (Jitter) 避免惊群效应。(3) 最大重试次数：通常 3-5 次，超过后放弃并返回降级响应。(4) 不可重试错误：认证失败 (401)、参数错误 (400)、内容违规 (451) 等不应重试，直接返回错误。(5) 幂等性保证：重试的工具调用必须是幂等的，否则可能导致重复操作（如重复创建订单）。(6) 熔断器 (Circuit Breaker)：如果某个 API 连续失败超过阈值，暂时跳过该 API 的调用，避免无意义的重试浪费时间。

面试加分点：能用代码示例展示 Python 中的 tenacity 库实现指数退避重试的写法。

Q129. Agent 输出如何保证结构化 (Structured Output)？

答案：结构化输出是 Agent 工具调用和数据处理的基础，保证方式包括：(1) **JSON Mode**：OpenAI 和 Anthropic 的 API 都支持强制模型输出合法 JSON 的模式 (response_format: {"type": "json_object"}), 从 API 层面保证格式正确。(2) **JSON Schema 约束**：在 API 中提供详细的 JSON Schema，模型的输出会被约束在 Schema 定义的结构内。OpenAI 的 Structured Outputs 功能保证 100% 符合 Schema。(3) **Few-shot 示例**：在 prompt 中提供期望输出格式的示例，引导模型生成正确格式。对于复杂格式尤其有效。(4) 后处理解析：用 Pydantic 模型或 JSON Schema 验证器对模型输出进行解析和验证，格式错误时重新请求。(5) **Function Calling**：通过工具调用机制，模型的参数输出天然就是结构化的 JSON，这也是 Nanobot 工具调用可靠性的基础。(6) **Retry with Error Feedback**：解析失败时将错误信息反馈给模型，让模型修正格式。

面试加分点：能提到 Instructor 库——一个基于 Pydantic 的结构化输出工具，与 OpenAI/Anthropic API 无缝集成。

Q130. 如何对 Agent 进行单元测试？

答案：Agent 测试的挑战在于 LLM 的非确定性，需要分层测试策略：(1) 工具函数测试：对每个工具的 handler 函数编写传统单元测试，验证输入输出的正确性。这部分与普通函数测试相同。(2) **Prompt 模板测试**：验证 ContextBuilder 在不同场景下（空对话、多轮对话、大量工具）组装的 prompt 格式正确。(3) **Mock LLM 测试**：用 mock 替代真实的 LLM API，返回预设的响应（包括工具调用和文本回复），测试 AgentLoop 的流程控制逻辑。这可以测试：工具调用的路由是否正确、循环终止条件是否有效、错误处理是否完善。(4) 集成测试：用真实 LLM API + 受控环境进行端到端测试，验证整体流程。使用固定温度 (temperature=0) 和固定 seed 提高可重复性。(5) 回归测试：维护一组「黄

金测试用例」，每次变更后运行确认核心功能未退化。(6) **Property-based Testing**: 验证 Agent 行为的不变量——如「每次工具调用后必须有对应的 `tool_result`」。

面试加分点: 提到 `pytest-recording` 或 `VCR` 等工具用于录制和回放 API 响应, 减少测试对外部 API 的依赖。

Q131. Agent 项目的 CI/CD 如何设计?

答案: Agent 项目的 CI/CD 除了常规的代码质量检查外, 还有 AI 特有的环节: **CI** (持续集成): (1) 代码检查: Linting (`ruff/flake8`)、类型检查 (`mypy`)、格式化 (`black`)。 (2) 单元测试: 工具函数测试、mock LLM 的 `AgentLoop` 测试。 (3) Prompt 检查: 检查 prompt 文件是否有语法错误、是否超过 token 限制、关键指令是否遗漏。 (4) 安全扫描: 检查是否有密钥泄露、依赖漏洞 (`pip-audit`)。 **CD** (持续部署): (1) Docker 镜像构建和推送。 (2) Staging 环境部署和冒烟测试。 (3) 影子测试: 用生产流量的副本验证新版本行为。 (4) 金丝雀发布: 先导入 5% 流量, 监控指标正常后逐步扩大。 (5) 回滚方案: 保留前一版本的镜像和配置, 一键回滚。 特别注意: Prompt 变更也需要走 CI/CD 流程, 因为 prompt 变更对 Agent 行为的影响可能大于代码变更。

面试加分点: 提到 GitHub Actions 的 Matrix Strategy 可以同时多个 Python 版本和多个模型 API 上运行测试。

Q132. Agent 线上事故案例: 某次工具调用导致的循环和如何修复

答案: 这是一个典型的线上事故模式: 事故描述: Agent 在执行文件搜索任务时, 工具返回了一个超大的文件列表 (数千个文件), Agent 试图逐一读取每个文件的内容来找到目标信息, 导致 `AgentLoop` 进入了一个几百步的循环, 消耗了大量 token 且最终超时。 根因分析: (1) 文件搜索工具没有限制返回结果数量, 将几千个文件名全部返回。 (2) Agent 没有对大量结果进行优先级排序或过滤, 而是暴力遍历。 (3) 没有设置 Agent 循环的最大步骤数限制。 修复措施: (1) 工具层面: 为搜索工具添加 `max_results` 参数, 默认最多返回 20 个结果。 (2) 结果层面: 应用 `_TOOL_RESULT_MAX_CHARS` 截断, 防止超大结果注入上下文。 (3) 循环层面: 设置 `AgentLoop` 最大迭代次数 (如 Nanobot 的 15 次限制)。 (4) Prompt 层面: 在工具描述中明确说明「如果结果过多, 请先缩小搜索范围」。

面试加分点: 能说出这类事故的预防原则——「Defense in Depth (纵深防御)」, 从工具、循环、prompt 多个层面设置防护。

Q133. 代码评审 (Code Review) Agent 代码时应关注什么?

答案: 审查 Agent 代码时需要关注的特殊点: (1) **Prompt** 注入防护: 检查用户输入是否会直接拼接到 prompt 中, 可能导致 prompt injection 攻击。 应该使用参数化的消息格式而非字符串拼接。 (2) 错误处理完备性: 每个工具调用、API 调用都应有 try-catch 和降级方案。 LLM API 返回异常格式时不应崩溃。 (3) **Token** 用量控制: 检查是否有无限循环风险、工具结果是否有大小限制、对话历史是否有压缩机制。 (4) 并发安全: 共享状态是否有锁保护、异步代码是否正确使用 `await`。 (5) 密钥安全: API 密钥是否硬编码、日志中是否会打印密钥。 (6) 幂等性: 工具调用是否幂等——同样的参数多次调用是否产生相同效果。 (7) 可测试性: LLM 调用是否可以被 mock、工具 handler 是否可以独立测试。 (8) **Prompt** 质量: system prompt 是否清晰明确、工具描述是否准确、示例是否有误导性。

面试加分点: 能提到自己在 review Nanobot 源码时发现的具体设计亮点或改进空间, 展示实际的代码审查经验。

Q134. 如何文档化一个 Agent 项目?

答案: Agent 项目的文档需要比传统软件更详细, 因为 Agent 的行为更难预测: (1) 架构文档: 包含系统架构图、组件交互关系、数据流向。 标注 `AgentLoop`、`ToolRegistry`、`MemoryConsolidator` 等核心组件的职责和接口。 (2) **Prompt** 文档: 记录每个 system prompt 的设计意图、关键指令的作用、修改历史。 Prompt 的每次变更都应该记录原因和预期效果。 (3) 工具文档: 每个工具的名称、功能描述、参数说明、返回值格式、使用示例、已知限制。 Nanobot 的工具在 `ToolRegistry` 中已经有 JSON Schema 格式的参数文档。 (4) 部署文档: 环境要求、配置项说明、部署步骤、常见问题排查。 (5) 运维手册 (**Runbook**): 常见告警的处理流程、回滚步骤、紧急联系人。 (6) 评测报告: Agent 在各项基准上的表现数据, 作为后续优化的基准线。 (7) 变更日志 (**Changelog**): 记录每个版本的功能变更、bug 修复和已知问题。

面试加分点: 展示本项目 (`learn-nanobot`) 的文档结构——从架构解析到面试准备, 12 个章节的系统化组织就是一个优秀的文档化案例。

总结

本章 134 道面试题覆盖了 AI Agent 开发面试的方方面面：

| 模块 | 题目范围 | 核心价值 |
|------------------|-----------|---------------|
| Agent 基础 | Q1-Q15 | 建立 Agent 概念框架 |
| Nanobot 专项 | Q16-Q35 | 展示源码级理解深度 |
| MCP 协议 | Q36-Q50 | 体现协议和标准化思维 |
| 大模型基础 | Q51-Q70 | 证明模型原理功底 |
| 训练优化 | Q71-Q85 | 展示训练全流程理解 |
| Nanobot 源码与设计深度题 | Q86-Q100 | 连接理论与实践 |
| RAG | Q101-Q110 | 覆盖最热门的应用场景 |
| 多智能体 | Q111-Q118 | 展示系统设计能力 |
| 系统设计 | Q119-Q126 | 体现工程架构思维 |
| 工程实践 | Q127-Q134 | 证明工程素养 |

使用建议：面试前重点突击与目标岗位最相关的板块。Agent 开发岗重点看 1-3 章，算法岗重点看 4-6 章，全栈岗重点看 7-10 章。每道题的「面试加分点」是区分你和其他候选人的关键。

下一章：第 14 章 就业市场分析

第 14 章 2026 年 AI Agent 就业市场分析

本章定位：帮助求职者全面了解 2026 年 AI Agent 相关岗位的市场现状、薪资水平、技能要求和求职策略，制定针对性的求职规划。

目录

- 一、2026 年 AI Agent 市场概览
- 二、常见岗位类型详解
- 三、JD 关键词深度分析
- 四、薪资范围参考
- 五、主要招聘平台与渠道
- 六、大厂 AI Agent 团队介绍
- 七、必备技能清单
- 八、简历关键词优化
- 九、面试流程介绍
- 十、求职时间线规划

一、2026 年 AI Agent 市场概览

1.1 行业背景

2025-2026 年是 AI Agent 从概念到产品的爆发年。以下事件标志着 Agent 进入主流：

- **2025 年 3 月**：Anthropic 发布 Claude 3.5 Agent 模式，Manus AI 开源引发全球关注
- **2025 年 Q2**：字节跳动 Coze、阿里百炼、百度千帆等平台的 Agent 功能大幅升级
- **2025 年 Q3**：MCP 协议被 OpenAI、Google 等主流厂商采纳，成为事实标准
- **2025 年 Q4**：Devin、Cursor、Windsurf 等 AI 编程助手引爆「AI 工程师」概念
- **2026 年 Q1**：国内 Agent 创业公司大量涌现，招聘需求同比增长 300%+

1.2 市场规模

根据多家研究机构报告：

| 指标 | 2025 年 | 2026 年（预估） | 增长率 |
|------------------|--------|------------|------|
| 全球 AI Agent 市场规模 | \$50 亿 | \$120 亿 | 140% |
| 中国相关岗位数量 | 8 万 + | 20 万 + | 150% |
| Agent 相关开源项目 | 5000+ | 15000+ | 200% |
| MCP Server 数量 | 1000+ | 5000+ | 400% |

1.3 需求侧分析

招聘需求主要来自以下方向：

1. 大厂 **AI** 平台团队：字节、阿里、腾讯、百度等构建 Agent 开发平台
2. 垂直领域 **Agent**：金融、医疗、法律、教育、电商等行业定制 Agent
3. **AI** 原生创业公司：从 0 到 1 打造 Agent 产品
4. 传统企业 **AI** 转型：用 Agent 改造内部流程（如智能客服、知识管理）
5. 开发者工具：IDE 插件、AI 代码助手、自动化测试工具
6. **AI** 基础设施：模型推理服务、向量数据库、MCP 生态

二、常见岗位类型详解

2.1 AI 应用开发工程师

岗位定位：将 AI 能力集成到产品中，是需求量最大的岗位类型。

核心职责：- 使用 LLM API (OpenAI/Claude/通义千问) 开发 AI 功能 - 实现 RAG 检索增强、对话系统、内容生成等应用场景 - 设计和优化 Prompt 工程 - 构建 AI 应用的后端服务和 API

技术要求：- Python/TypeScript 开发能力 - 熟悉至少一种 LLM API (OpenAI/Anthropic/阿里云 DashScope) - 了解 RAG 基本流程和向量数据库 - Web 开发基础 (FastAPI/Flask/Next.js)

适合人群：有 1-3 年后端开发经验，希望转型 AI 方向的工程师

薪资区间：18K-45K/月（一线城市）

2.2 LLM 应用工程师

岗位定位：专注于大模型应用层的深度开发，比 AI 应用开发工程师更偏底层。

核心职责：- 设计和实现 LLM 应用架构 (Agent 循环、记忆系统、工具调用) - 模型选型和评测 (对比不同模型在特定场景下的表现) - Prompt 工程和上下文管理优化 - 搭建 LLM 应用的监控和评测体系 - 解决幻觉、延迟、成本等工程问题

技术要求：- 深入理解 Transformer 架构和推理过程 - 熟悉 Agent 框架 (LangChain/LangGraph/Nanobot) - 掌握 MCP 协议和 Function Calling 机制 - 了解模型训练流程 (SFT/RLHF/DPO) - 性能优化经验 (缓存、异步、批处理)

适合人群：有 2-5 年开发经验，对 LLM 原理有深入学习的工程师

薪资区间：25K-55K/月（一线城市）

2.3 AI Agent 开发工程师

岗位定位：专注于 Agent 系统的设计、开发和优化，是最对口的岗位。

核心职责：- 设计和实现 Agent 系统架构 (ReAct 循环、多 Agent 编排) - 开发自定义工具和 MCP Server - 实现 Agent 的记忆管理和状态管理 - 构建 Agent 评测和调试体系 - 优化 Agent 的可靠性、效率和成本

技术要求：- 深入理解 Agent 架构 (至少读过一个框架的源码，如 Nanobot) - 掌握 MCP 协议 (能开发 MCP Server) - 熟悉 ReAct、Reflection、Planning 等 Agent 设计模式 - 异步编程和并发控制 (Python asyncio) - 系统设计能力 (高可用、可观测性)

适合人群：对 Agent 技术有热情，有源码阅读和框架开发经验的工程师

薪资区间：30K-65K/月（一线城市）

2.4 大模型算法工程师

岗位定位：从事大模型的训练、微调和优化，偏算法和研究方向。

核心职责：- 大模型的 Pretrain/SFT/RLHF/DPO 训练 - LoRA/QLoRA 等参数高效微调 - 模型评测和分析 - 推理优化 (量化、蒸馏、加速) - 数据清洗和质量控制

技术要求：- 扎实的深度学习基础 (PyTorch) - 深入理解 Transformer 架构 (Attention、位置编码、归一化等) - 熟悉训练流程和优化技巧 (学习率调度、混合精度、分布式训练) - 了解 RLHF/DPO 等对齐技术 - 论文阅读和复现能力

适合人群：计算机/AI 相关硕博，有模型训练经验

薪资区间：35K-80K/月（一线城市）

2.5 AI 产品经理

岗位定位：负责 AI Agent 产品的规划、设计和落地。

核心职责：- AI Agent 产品的需求分析和功能设计 - 用户体验设计（对话交互、多模态交互）- 与算法团队沟通模型能力和限制 - 竞品分析和市场洞察 - 产品数据分析和迭代优化

技术要求：- 理解 LLM 和 Agent 的基本原理和能力边界 - 了解 Prompt 工程和 RAG 的基本概念 - 数据分析能力 - 具备一定的技术背景（能与工程师有效沟通）- 了解 AI 安全和合规要求

适合人群：有产品经验的 PM 转型 AI 方向，或有技术背景的毕业生

薪资区间：20K-50K/月（一线城市）

三、JD 关键词深度分析

通过分析 2026 年 Q1 主流招聘平台上 500+ 份 AI Agent 相关岗位的 JD，提取以下高频关键词：

3.1 语言与工程基础

| 关键词 | 出现频率 | 重要程度 | 说明 |
|------------------|------|------|-----------------|
| Python | 95% | | 几乎所有岗位的必备项 |
| 异步编程/asyncio | 65% | | Agent 系统的核心编程模型 |
| Go/Golang | 35% | | 高性能后端服务 |
| TypeScript | 30% | | 前端 + MCP SDK |
| REST API/gRPC | 60% | | 服务间通信 |
| Docker/K8s | 55% | | 容器化部署 |
| Redis | 45% | | 缓存和会话管理 |
| PostgreSQL/MySQL | 50% | | 数据持久化 |
| Git/CI/CD | 70% | | 基本工程素养 |

3.2 Agent 与编排

| 关键词 | 出现频率 | 重要程度 | 说明 |
|-------------------------------|------|------|----------------|
| LangChain/LangGraph | 60% | | 最主流的 Agent 框架 |
| Agent/AI Agent | 85% | | 核心概念 |
| ReAct | 40% | | Agent 推理框架 |
| Tool Calling/Function Calling | 70% | | 工具调用能力 |
| 工作流编排 | 45% | | Workflow Agent |
| AutoGen/CrewAI | 25% | | 多 Agent 框架 |
| Dify/Coze | 30% | | 低代码 Agent 平台 |
| Nanobot | 5% | | 新兴轻量框架（增长快） |

3.3 协议与工具层

| 关键词 | 出现频率 | 重要程度 | 说明 |
|------------------|------|------|----------------|
| MCP 协议 | 45% | | 2026 年急速上升的关键词 |
| Function Calling | 65% | | 模型工具调用格式 |
| JSON Schema | 50% | | 工具参数定义标准 |
| OpenAPI/Swagger | 30% | | API 文档标准 |
| Webhook | 35% | | 事件驱动集成 |

3.4 检索与知识

| 关键词 | 出现频率 | 重要程度 | 说明 |
|-----------|------|------|--------------------------|
| RAG | 75% | | 最热门的应用模式 |
| 向量数据库 | 60% | | Milvus/Pinecone/ChromaDB |
| Embedding | 55% | | 文本向量化 |
| 混合检索 | 30% | | Dense + Sparse |
| Rerank | 25% | | 检索精排 |
| 知识图谱 | 20% | | 结构化知识 |

3.5 模型与推理

| 关键词 | 出现频率 | 重要程度 | 说明 |
|------------|------|------|---------------|
| Prompt 工程 | 80% | | 几乎所有 AI 岗位必备 |
| LLM API | 75% | | 模型调用能力 |
| Context 管理 | 40% | | 上下文窗口优化 |
| 模型评测 | 35% | | 评估模型效果 |
| 多模态 | 30% | | 图文理解能力 |
| 推理优化 | 25% | | vLLM/TensorRT |

3.6 数据与对齐

| 关键词 | 出现频率 | 重要程度 | 说明 |
|-------------|------|------|--------|
| SFT | 40% | | 算法岗必备 |
| RLHF/DPO | 35% | | 偏好对齐 |
| LoRA/QLoRA | 35% | | 参数高效微调 |
| 数据标注 | 25% | | 数据质量管理 |
| PyTorch | 50% | | 算法岗必备 |
| Transformer | 45% | | 模型原理解 |

四、薪资范围参考

4.1 按级别和城市的薪资分布（月薪，单位：K/千元）

| 级别 | 北京 | 上海 | 深圳 | 杭州 | 成都/武汉 | 远程 |
|-----------|---------|--------|--------|--------|--------|--------|
| 初级（0-2 年） | 18-30 | 17-28 | 17-28 | 15-25 | 12-22 | 15-25 |
| 中级（2-5 年） | 30-50 | 28-45 | 28-45 | 25-40 | 20-35 | 25-40 |
| 高级（5-8 年） | 45-70 | 40-65 | 40-65 | 35-55 | 30-45 | 35-55 |
| 专家/Leader | 60-100+ | 55-90+ | 55-90+ | 45-80+ | 40-60+ | 50-80+ |

注：以上为 base salary，不含年终奖（通常 2-6 个月）和股票/期权。大厂总包通常为月薪 × (14-18)。

4.2 按岗位类型的薪资对比

| 岗位 | 初级 | 中级 | 高级 | 备注 |
|----------------|--------|--------|--------|-----------|
| AI 应用开发工程师 | 18-28K | 28-40K | 40-55K | 需求量大，竞争激烈 |
| LLM 应用工程师 | 22-32K | 32-48K | 48-65K | 需要较深的原理解 |
| AI Agent 开发工程师 | 25-35K | 35-55K | 55-75K | 供不应求，溢价高 |

| 岗位 | 初级 | 中级 | 高级 | 备注 |
|----------|--------|--------|---------|------------|
| 大模型算法工程师 | 30-45K | 45-65K | 65-100K | 门槛高，多要求硕博 |
| AI 产品经理 | 18-28K | 28-42K | 42-60K | 技术型 PM 更吃香 |

4.3 薪资趋势观察

- 1. **Agent** 开发工程师溢价明显：由于人才供给少，有 Agent 框架实战经验的候选人薪资溢价约 20-30%
- 2. **MCP** 经验加分：熟悉 MCP 协议开发的候选人在 2026 年 Q1 开始获得额外薪资加成
- 3. 源码经验值钱：读过 Agent 框架源码（如 Nanobot/LangChain）的候选人面试通过率更高
- 4. 算法岗回归理性：纯模型训练岗位竞争激烈（硕博供给增加），但「算法 + 工程」复合型人才仍然稀缺
- 5. 远程岗位增多：AI Agent 方向的远程机会占比约 15-20%，薪资水平约为一线城市的 80-90%

五、主要招聘平台与渠道

5.1 综合招聘平台

| 平台 | 特点 | AI Agent 岗位数量 | 推荐度 |
|-----------------|-----------|---------------|-----|
| Boss 直聘 | 即时沟通、效率最高 | | |
| 牛客网 | 校招为主、技术社区 | | |
| 拉勾 | 互联网岗位集中 | | |
| 猎聘 | 中高端岗位 | | |
| 脉脉 | 人脉推荐、内推 | | |
| LinkedIn | 外企和海外机会 | | |

5.2 技术社区招聘

| 平台 | 特点 | 推荐度 |
|--------------------|--------------|-----|
| GitHub Jobs | 开源项目相关岗位 | |
| V2EX | 技术社区，不少高质量岗位 | |
| 掘金/CSDN | 技术博文引流招聘 | |
| 即刻 | AI 圈子活跃、内推多 | |

5.3 高效求职策略

- 1. **Boss** 直聘为主：设置好搜索关键词（AI Agent、LLM、大模型、MCP），每天主动打招呼 20+
- 2. 内推优先：通过脉脉、即刻等平台联系目标公司员工，内推简历通过率是海投的 3-5 倍
- 3. 技术影响力：在 GitHub 上有开源贡献、在掘金/CSDN 上有技术博文，会显著提升简历吸引力
- 4. 社群活跃：加入 AI Agent、MCP 相关的微信群/Discord，很多岗位在社群中内部消化

六、大厂 AI Agent 团队介绍

6.1 字节跳动

相关团队：- **Coze**（扣子）团队：字节的 Agent 开发平台，支持低代码构建 Agent，集成了工作流编排、知识库、插件市场等能力 - 豆包团队：基于豆包大模型的 Agent 应用开发 - 飞书 **AI** 团队：将 Agent 能力集成到飞书办公套件中 - **TikTok AI** 团队：海外市场的 AI Agent 应用

技术栈：Go/Python、自研 Agent 框架、MCP 兼容、分布式系统

招聘特点：要求扎实的工程能力，偏好有大规模系统经验的候选人。面试侧重系统设计和编码能力。

6.2 阿里巴巴

相关团队：- 通义实验室：通义千问大模型团队，包括 Agent 能力开发 - 百炼平台：阿里云的 Agent 开发平台，提供模型调用、RAG、Agent 编排等全栈能力 - 钉钉 **AI**：将 Agent 集成到钉钉工作流中 - 达摩院：前沿 AI 研究，包括多 Agent 协作、Agent 安全等

技术栈：Python/Java、ModelScope、DashScope API、阿里云基础设施

招聘特点：重视项目经验和业务理解，要求候选人能将技术与业务场景结合。面试中常有业务案例分析。

6.3 腾讯

相关团队：- 腾讯混元团队：混元大模型的 Agent 能力开发 - 微信 **AI** 团队：微信生态内的 AI Agent 应用 - 企业微信 **AI**：企业级 Agent 解决方案 - 腾讯云 **AI**：云端 Agent 开发平台和工具链

技术栈：C++/Go/Python、自研推理引擎、腾讯云基础设施

招聘特点：技术面试难度较高，注重算法基础和系统设计能力。偏好有社交/内容场景经验的候选人。

6.4 百度

相关团队：- 文心一言团队：基于文心大模型的 Agent 应用 - 百度智能云千帆：企业级 Agent 开发平台 - 百度搜索 **AI**：搜索增强的 Agent 能力 - **Apollo**（自动驾驶）：车载 Agent 和具身智能

技术栈：Python/C++、PaddlePaddle（飞桨）、百度云基础设施

招聘特点：搜索和 NLP 背景较深，偏好有信息检索和知识图谱经验的候选人。

6.5 其他重要玩家

| 公司 | Agent 相关方向 | 特点 |
|-----------------------|--------------------------|-------------|
| 美团 | 本地生活 Agent、智能客服 | 强调业务理解 |
| 京东 | 电商 Agent、物流 Agent | 强调工程落地 |
| 华为 | 盘古大模型 Agent、CloudAgent | 偏重 ToB 场景 |
| 商汤 | 多模态 Agent、具身智能 | 偏重研究 |
| 智谱 AI | GLM Agent、MaaS 平台 | 开源生态 |
| 月之暗面 | Kimi Agent 能力 | 长文本 Agent |
| MiniMax | 海螺 AI Agent | 海外市场 |
| 零一万物 | 大模型 + Agent 应用 | 李开复团队 |
| Anthropic （海外） | Claude Agent、MCP 生态 | Agent 方向引领者 |
| OpenAI （海外） | GPT Agent、Assistants API | 行业标杆 |

七、必备技能清单

按优先级排序的 8 大必备技能

技能 1: **Python** 编程与异步编程 为什么必备: 95% 的 Agent 岗位要求 Python, Agent 系统大量使用 `async/await`。

具体要求：- 熟练使用 Python 3.10+ 特性（类型注解、match-case、dataclass）- 掌握 `asyncio` 编程模型（协程、Task、Event Loop、Semaphore）- 熟悉 FastAPI/Flask 等 Web 框架 - 了解 Python 的并发模型（GIL、多线程、多进程、协程的选择）

学习建议：通过阅读 Nanobot 源码学习 `asyncio` 在 Agent 系统中的实际应用

技能 2: **LLM API** 使用与 **Prompt** 工程 为什么必备: 这是所有 AI Agent 工作的基础操作技能。

具体要求: - 熟悉 OpenAI/Anthropic/通义千问的 API 调用方式 - 掌握 System Prompt 设计、Few-shot 示例构造 - 理解 Token 计数、上下文窗口管理、流式响应处理 - 了解模型参数调节 (temperature、top_p、max_tokens) - 能设计结构化输出的 Prompt (JSON 输出、格式约束)

学习建议: 动手实践 100+ 个不同类型的 Prompt, 总结模式和经验

技能 3: **Agent** 架构理解 为什么必备: 区分「会调 API」和「会做 Agent」的核心能力。

具体要求: - 理解 ReAct 循环的原理和实现 - 了解 Agent 的记忆系统 (短期/长期/外部) - 理解工具调用机制 (Tool Calling / Function Calling) - 至少读过一个 Agent 框架的核心源码 (推荐 Nanobot, 仅 4000 行) - 了解多 Agent 协作模式 (Orchestrator-Workers、Pipeline)

学习建议: 跟随 learn-nanobot 项目的章节逐步深入

技能 4: **MCP** 协议 为什么必备: MCP 正在成为 AI 工具连接的事实标准, 2026 年起的 JD 中出现率快速上升。

具体要求: - 理解 MCP 的 Host/Client/Server 三层架构 - 了解三大原语 (Tools/Resources/Prompts) - 能使用官方 SDK 开发简单的 MCP Server - 理解 MCP 与 Function Calling 的区别和关系 - 了解 MCP 的传输层选择 (stdio/HTTP+SSE/Streamable HTTP)

学习建议: 动手开发一个 MCP Server (如 MySQL 查询工具), 并在 Nanobot 中集成测试

技能 5: **RAG** 系统 为什么必备: RAG 是目前 LLM 应用中最成熟和最广泛使用的模式。

具体要求: - 掌握 RAG 的完整流程 (文档解析 → Chunk → Embedding → 检索 → 生成) - 熟悉至少一种向量数据库 (Milvus/ChromaDB/Pinecone) - 了解 Chunk 分割策略和 Embedding 模型选择 - 理解混合检索和 Rerank 的优化手段 - 能搭建一个端到端的 RAG 系统

学习建议: 用 LangChain + ChromaDB 搭建一个个人知识库问答系统

技能 6: 大模型原理 为什么必备: 理解原理才能在出问题时代定位根因, 面试中必考。

具体要求: - 理解 Transformer 架构 (Self-Attention、Multi-Head、FFN) - 了解位置编码 (RoPE)、归一化 (RMSNorm)、激活函数 (SwiGLU) - 理解 KV Cache 和推理加速原理 - 了解 SFT/RLHF/DPO 训练流程 - 了解 LoRA 等参数高效微调方法

学习建议: 通过阅读 Nanobot 源码中 Provider 层的实现, 理解 LLM 推理流程; 结合开源小模型项目从零实现一个小型 Transformer, 建立直觉理解

技能 7: 系统设计 为什么必备: 中高级岗位的面试中必考系统设计题。

具体要求: - 能设计一个企业级 Agent 系统 (考虑高可用、多租户、安全) - 理解微服务架构和 API 设计 - 了解消息队列、缓存、数据库的选型 - 能做成本估算和容量规划 - 了解监控和可观测性方案

学习建议: 练习 3-5 个 Agent 系统设计题, 参考第 13 章的系统设计部分

技能 8：工程素养 为什么必备：工程素养决定了你能否把 demo 变成产品。

具体要求：- Git 工作流（分支管理、Code Review、CI/CD）- Docker 容器化和 K8s 基础 - 单元测试和集成测试编写习惯 - 代码文档化（README、API 文档、注释）- 安全意识（密钥管理、输入校验、日志脱敏）

学习建议：在自己的 Agent 项目中严格执行工程规范，而非只做 demo

八、简历关键词优化

8.1 高优先级关键词（务必出现）

AI Agent, LLM, 大模型, RAG, Python, Prompt Engineering,
Tool Calling, Function Calling, MCP, 向量数据库,
Agent 框架, ReAct, 异步编程, API 集成

8.2 中优先级关键词（按岗位方向选择）

Agent 开发方向

Nanobot, LangChain, LangGraph, 多Agent, 工具调用,
记忆管理, SubAgent, MCP Server, Skills

算法方向

Transformer, SFT, RLHF, DPO, LoRA, QLoRA,
PyTorch, 分布式训练, 模型评测, 数据质量

应用开发方向

FastAPI, Docker, Redis, PostgreSQL, Milvus,
微服务, CI/CD, 高可用, 监控

8.3 加分关键词（区分度高）

源码研究, MCP Server 开发, Agent 评测, Prompt Cache,
Multi-Agent 编排, 端侧部署, 知识蒸馏,
开源贡献, 技术博客

8.4 关键词使用策略

1. 标题：岗位名称 + 核心关键词（如「AI Agent 开发工程师 | Python & MCP」）
 2. 技能列表：按 JD 中的关键词顺序排列，匹配度高的放前面
 3. 项目描述：自然地融入关键词，避免堆砌
 4. 自我评价：用 1-2 句话概括核心能力，包含 3-5 个高优关键词
-

九、面试流程介绍

9.1 典型面试流程（大厂）

简历筛选 → 笔试（可选） → 一面（技术面） → 二面（技术深度面）
→ 三面（技术/业务 Leader 面） → HR 面 → Offer 沟通

时间跨度：通常 2-4 周

9.2 各轮面试重点

一面：基础技术面（45-60 分钟） 面试官：通常是团队的高级工程师

考察内容：- 编程基础：Python 语法、数据结构、算法题（LeetCode Medium 难度）- LLM 基础：Transformer 架构、Attention 机制、Token 化 - Agent 基础：ReAct 循环、工具调用、记忆系统 - 项目经历：简要介绍 1-2 个项目 - 手撕代码：1-2 道编程题（可能涉及异步编程、JSON 解析等）

准备建议：刷 50 道 LeetCode（重点：字符串、树、图、动态规划）；熟练回答本章 Q1-Q15 基础题

二面：技术深度面（60-90 分钟） 面试官：技术 Leader 或架构师

考察内容：- 项目深挖：详细讲解一个项目（技术选型、架构设计、难点解决）- 技术深度：Agent 架构设计、MCP 协议细节、RAG 优化 - 系统设计：设计一个企业级 Agent 系统 - 开放问题：如何评测 Agent 效果、如何控制成本等 - 源码理解：如果简历写了源码研究，会深入问

准备建议：用 STAR 法则准备项目介绍；练习 3-5 个系统设计题；深入理解一个 Agent 框架的源码

三面：Leader 面/交叉面（45-60 分钟） 面试官：部门 Leader 或其他团队技术负责人

考察内容：- 技术视野：对 AI Agent 行业的理解和判断 - 团队协作：过往的团队合作经验 - 问题解决：遇到的最大技术挑战及解决过程 - 学习能力：如何学习新技术、如何保持技术敏感度 - 职业规划：短期和长期的职业目标

准备建议：关注 AI Agent 行业动态；准备 2-3 个团队协作/挑战克服的故事

HR 面（30-45 分钟） 考察内容：- 离职原因/求职动机 - 薪资期望（务必提前做好调研）- 工作稳定性 - 文化匹配度 - 加班/出差接受度

准备建议：诚实但策略性地回答；薪资谈判可以参考本章的薪资数据

9.3 面试中的常见坑

1. 过度夸大项目经验：被深挖时露馅比不写更致命
2. 只会调 API 没有原理解：能力天花板太低，面试官一追问就答不上来
3. 忽视工程素养：只关注模型和算法，不关心测试、部署、监控
4. 对公司业务不了解：面试前至少花 30 分钟研究目标公司的 AI 产品
5. 薪资期望不合理：过高可能错失机会，过低可能被低估

十、求职时间线规划

10.1 应届生时间线（以 2026 年秋招为例）

| 时间段 | 重点任务 | 产出 |
|------------|----------|------------------------|
| 2026.01-03 | 技术学习期 | 学完 Nanobot 源码 + 实战项目练习 |
| 2026.03-04 | 项目积累期 | 完成 2 个可写入简历的项目 |
| 2026.04-05 | 简历准备期 | 简历定稿、开始投递暑期实习 |
| 2026.05-06 | 暑期实习 | 积累实际工作经验 |
| 2026.07-08 | 秋招准备期 | 刷八股文、模拟面试 |
| 2026.08-10 | 秋招高峰 | 大量投递和面试 |
| 2026.10-11 | Offer 决策 | 拿到多个 offer 后选择 |

10.2 社招跳槽时间线

| 时间段 | 重点任务 | 产出 |
|---------|------|---------------------|
| 第 1-2 周 | 学习充电 | 学完核心技术点（本项目 1-12 章） |
| 第 3-4 周 | 项目包装 | 简历优化、项目话术准备 |
| 第 5-6 周 | 面试冲刺 | 刷面试题（第 13 章）、模拟面试 |

| 时间段 | 重点任务 | 产出 |
|-----------|----------|------------|
| 第 7-10 周 | 集中面试 | 每周 3-5 场面试 |
| 第 11-12 周 | Offer 谈判 | 横向比较、薪资谈判 |

10.3 每日学习建议

| 时段 | 内容 | 时长 |
|----|-----------------------------|--------|
| 早上 | 刷 1-2 道面试八股文（第 13 章） | 30 分钟 |
| 上午 | 源码阅读或项目实践 | 2-3 小时 |
| 下午 | LeetCode 算法练习 | 1-2 小时 |
| 晚上 | 学习理论知识（Transformer/RAG/MCP） | 1-2 小时 |
| 睡前 | 浏览 AI 新闻和行业动态 | 30 分钟 |

总结

2026 年的 AI Agent 就业市场特点：

- 1. 需求旺盛：Agent 从概念到产品的过渡期，人才供不应求
 - 2. 技能溢价：有 Agent 框架实战经验（特别是源码级理解）的候选人明显溢价
 - 3. MCP 是新高地：MCP 协议经验正在成为差异化竞争力
 - 4. 复合型吃香：「懂算法 + 会工程 + 理解业务」的全栈型人才最受欢迎
 - 5. 学习路径清晰：从 Nanobot 源码入手理解 Agent → 通过实战项目深化理解 → MCP 理解工具生态
- 核心建议：不要只做「API Wrapper Engineer」（套壳工程师），要做理解原理、能设计系统、能解决问题的 Agent 工程师。本学习项目（learn-nanobot）就是帮你从「会用」到「会造」的桥梁。

上一章：第 13 章 面试八股文 下一章：第 15 章 简历模板

第 15 章 AI Agent 岗位简历模板

本章定位：提供经过优化的简历模板、项目经历描述范例和简历撰写技巧，帮助求职者打造高通过率的 AI Agent 方向简历。

目录

- 一、简历撰写核心原则
- 二、技能关键词列表
- 三、Nanobot 项目简历模板
- 四、智能运维助手项目简历模板
- 五、多平台客服 Bot 项目简历模板
- 六、自定义 MCP Server 开发项目简历模板
- 七、其他可写项目方向
- 八、自我评价模板
- 九、完整简历示例
- 十、简历注意事项
- 十一、不同岗位的简历侧重点
- 十二、简历投递策略

一、简历撰写核心原则

1.1 STAR 量化原则

每段项目经历都要遵循 STAR 法则并尽量量化成果：

- **S (Situation)**：项目背景——什么场景、什么规模
- **T (Task)**：你的任务——负责什么、目标是什么
- **A (Action)**：你的行动——用了什么技术、做了什么决策
- **R (Result)**：成果——提升了多少、解决了什么问题

反面示例（空洞无力）：> 负责项目开发工作，参与了相关功能的设计和实现。

正面示例（具体有力）：> 主导设计并实现了基于 ReAct 循环的 AI Agent 系统，集成 12 个 MCP 工具，支撑日均 5000+ 次对话请求，用户问题解决率从 60% 提升至 85%。

1.2 六大黄金法则

1. 数字说话：尽量用数字量化每一条成果（提升 XX%、减少 XX 毫秒、覆盖 XX 个场景）
2. 技术栈明确：在每个项目中明确列出使用的技术栈
3. 深度优于广度：2 个有深度的项目 > 5 个浅尝辄止的项目
4. 关键词匹配：根据目标 JD 调整关键词的顺序和权重
5. 「做了什么」不如「解决了什么问题」：结果导向，不是任务清单
6. 一页为佳：除非有 10 年 + 经验，简历控制在一页 A4 纸内

1.3 AI Agent 方向简历的特殊性

AI Agent 方向的简历与传统后端开发简历的不同之处：

- 源码理解加分：面试官特别看重候选人是否读过 Agent 框架的源码，而非只是调用 API
- 协议理解加分：对 MCP 等协议的理解比单纯会用某个框架更有含金量
- 端到端经验：从 Prompt 设计到系统部署的全链条经验比单点能力更有竞争力
- 开源贡献：有 GitHub 贡献或技术博客会显著提升简历吸引力

二、技能关键词列表

按权重排序（从高到低）

第一梯队（必须包含）

Python, AI Agent, LLM/大语言模型, Prompt Engineering, RAG, Tool Calling/Function Calling, 异步编程

第二梯队（强烈推荐）

MCP 协议, Agent 框架(LangChain/Nanobot), 向量数据库, Transformer 架构, FastAPI, Docker, Git

第三梯队（加分项）

多Agent编排, SFT/RLHF/DPO, LoRA, PyTorch, 系统设计, CI/CD, Kubernetes, Redis, PostgreSQL

第四梯队（差异化项）

MCP Server 开发, Agent 源码研究, Agent 系统设计, 开源贡献, 技术博客, Prompt Cache 优化

技能展示模板

技术技能:

- 编程语言: Python (精通)、TypeScript (熟悉)、Go (了解)
 - AI/LLM: Prompt Engineering、RAG 系统设计、Agent 架构设计、MCP 协议
 - Agent 框架: Nanobot (源码级理解)、LangChain/LangGraph (项目实践)
 - 模型训练: PyTorch、Transformer 架构、SFT/DPO、LoRA 微调
 - 数据库: PostgreSQL、Redis、Milvus (向量数据库)
 - 工程工具: Docker、Git、CI/CD、FastAPI、asyncio
-

三、Nanobot 项目简历模板

模板 A: Agent 框架源码研究项目（适合偏理论/架构方向）

项目名称: 基于 HKUDS/Nanobot 的 AI Agent 系统源码研究与二次开发

项目周期: 2026.02 - 2026.04

项目角色: 独立研究 / 核心开发

技术栈: Python 3.12, HKUDS/nanobot, MCP 协议, Anthropic Claude API, asyncio, JSON-RPC 2.0

项目描述:

深入研究开源 AI Agent 框架 Nanobot (GitHub 37K+ stars) 的完整源码, 理解其 Agent 循环、记忆管理、工具注册、MCP 集成等核心机制, 并基于此进行二次开发和能力扩展。

核心职责:

- 系统研读 Nanobot 核心源码 (约 4000 行 Python), 完成 8 个核心模块 (AgentLoop、ContextBuilder、MemoryConsolidator、ToolRegistry、SubagentManager、MessageBus、MCPClient、ChannelAdapter) 的架构分析和流程图绘制
- 深入分析 AgentLoop 的 ReAct 执行流程, 理解 Thought-Action-Observation 循环的工程实现, 梳理从用户输入到最终回复的完整数据流
- 研究 MemoryConsolidator 的双层记忆架构 (会话内压缩 + 跨会话持久化), 理解 save_memory 虚拟工具的设计哲学

- 分析并发控制设计：session_locks（会话级互斥）和 _concurrency_gate(3)（全局信号量）的分层并发策略

项目成果：

- 输出 12 篇深度技术文档（共 5 万+ 字），覆盖架构、源码、协议等维度
 - 基于源码理解开发 3 个自定义 MCP Server（MySQL 查询、文档检索、日程管理）
 - 成功将 Nanobot 集成到飞书机器人，实现团队内部 AI 助手功能
 - 形成一套可复用的 Agent 框架评估方法论，从 7 个维度对比 Nanobot 与 LangChain
-

模板 B：AI 助手应用项目（适合偏工程/产品方向）

项目名称：基于 Nanobot 的企业智能助手系统

项目周期：2026.02 - 2026.04

项目角色：后端开发 / 技术负责人

技术栈：Python, HKUDS/nanobot, MCP 协议, Claude API, FastAPI, PostgreSQL, Redis, Docker, 飞书开放平台

项目描述：

基于 Nanobot Agent 框架构建企业级 AI 助手系统，集成多个 MCP 工具服务器，支持飞书和 Web 双平台接入，为团队提供知识库问答、日程管理、数据查询等智能化办公能力。

核心职责：

- 基于 Nanobot 的 Channel 适配器模式开发飞书和 Web 双平台接入层，实现消息格式的统一转换和路由，支持文本、图片、文件等多种消息类型
- 开发 5 个自定义 MCP Server：数据库查询工具（支持自然语言转 SQL）、文档检索工具（基于向量相似度）、日程管理工具（对接飞书日历 API）、Jira 集成工具（查询和创建 Issue）、通知工具（飞书群消息推送）
- 利用 Nanobot 的 Skills 渐进披露机制，设计 8 个领域 Skill（数据分析、周报生成、会议纪要整理等），实现工具的按需加载，system prompt token 占用减少 40%
- 实现基于 Redis 的语义缓存层，对相似查询命中率达 35%，LLM API 月度成本降低约 25%

项目成果：

- 系统上线后日均处理 500+ 次对话请求，用户满意度评分 4.2/5.0
 - 工具调用成功率 94%，平均响应时间 3.2 秒
 - 团队信息检索效率提升约 60%，减少了大量重复性问答沟通
-

模板 C：MCP 工具开发项目（适合偏协议/基础设施方向）

项目名称：企业级 MCP 工具服务器集群开发

项目周期：2026.03 - 2026.04

项目角色：核心开发者

技术栈：Python, MCP SDK, JSON-RPC 2.0, FastAPI, Docker, PostgreSQL, Elasticsearch

项目描述：

基于 MCP (Model Context Protocol) 协议规范，为企业内部 AI Agent 系统开发标准化的工具服务器集群，实现企业数据和业务系统与 AI 应用的标准化连接。

核心职责：

- 基于 MCP Python SDK 开发 6 个工具服务器，覆盖数据库查询（MySQL/PostgreSQL）、全文检索（Elasticsearch）、项目管理（Jira）、即时通讯（飞书/钉钉）、

文件系统操作、HTTP API 代理等能力

- 设计 MCP Server 的统一安全框架：实现 API Key 认证、输入参数 JSON Schema 验证、敏感操作确认机制（destructive annotation）、操作审计日志
- 实现 MCP Server 的 stdio 和 HTTP+SSE 双传输模式支持，本地开发使用 stdio 模式提升调试效率，生产环境使用 HTTP+SSE 模式支持远程访问和多用户共享
- 编写完整的工具描述文档，确保 LLM 能准确理解每个工具的用途和参数要求，将工具调用准确率从 78% 提升至 93%

项目成果：

- 6 个 MCP Server 已集成到 3 个不同的 AI Agent 应用中（Nanobot、Cursor、内部 Agent 平台），验证了 MCP 的「一次开发，多处复用」核心价值
 - 将新 AI 应用接入内部系统的开发周期从平均 2 周缩短到 2 天
 - 项目文档和设计方案被团队采纳为 MCP Server 开发规范
-

四、智能运维助手项目简历模板

项目名称：基于 Nanobot 的智能运维助手系统

项目周期：2026.02 - 2026.04

项目角色：后端开发 / 核心架构

技术栈： Python, HKUDS/nanobot, MCP 协议, Claude API, Prometheus, Grafana, Docker, Redis, PostgreSQL

项目描述：

基于 Nanobot Agent 框架构建智能运维助手系统，集成多个 MCP 工具服务器，支持日志分析、告警处理、服务器状态查询、故障自动诊断等运维场景，显著降低运维人员的工作负荷。

核心职责：

- 基于 Nanobot 的 AgentLoop 和 ReAct 循环，实现运维场景的多步推理——Agent 能自动判断告警类型、查询相关日志、定位根因并给出修复建议
- 开发 6 个运维领域 MCP Server：日志检索工具（对接 Elasticsearch）、服务器监控工具（对接 Prometheus API）、告警管理工具（对接 PagerDuty）、部署工具（对接 Kubernetes API）、数据库巡检工具、网络诊断工具
- 利用 Nanobot 的 MemoryConsolidator 记忆系统，实现运维知识的持久化——Agent 自动记忆历史故障模式和解决方案，相似故障再次发生时可快速匹配历史经验
- 设计运维 Skill 体系：按照故障类型（CPU 过高、内存泄漏、磁盘满、服务不可用等）定义专用 Skill，利用渐进披露机制按需加载

项目成果：

- 系统上线后覆盖 80% 的常见运维场景，平均故障诊断时间从 30 分钟缩短到 5 分钟
 - 工具调用成功率 92%，运维知识库积累 500+ 条历史故障经验
 - 夜间值班告警的自动处理率达到 45%，显著减轻了运维人员的工作压力
-

五、多平台客服 Bot 项目简历模板

项目名称：基于 Nanobot 的多平台智能客服系统

项目周期：2026.02 - 2026.04

项目角色：全栈开发 / 技术负责人

技术栈： Python, HKUDS/nanobot, MCP 协议, Claude API, FastAPI, Redis, PostgreSQL, 飞书/钉钉/微信开放平台

项目描述：

基于 Nanobot Agent 框架构建统一的多平台智能客服系统，通过 Channel 适配器

模式同时接入飞书、钉钉、企业微信和 Web 四个平台，为企业提供一体化的 AI 客服解决方案。

核心职责：

- 基于 Nanobot 的 Channel 适配器架构，开发四套平台适配器——每个适配器独立处理平台特有的消息格式、认证机制和回调验证，核心 Agent 逻辑完全解耦，新增平台只需开发适配器层
- 实现智能路由和会话管理：根据用户意图自动分配到售前咨询、售后支持、技术支持等不同 Skill 域，利用 Redis 实现跨平台的用户画像统一
- 开发 8 个客服领域 MCP Server：订单查询、物流追踪、退换货处理、FAQ 知识库检索、工单创建、优惠券查询、用户信息查询、人工坐席转接
- 利用 Nanobot 的并发控制机制（session_locks + _concurrency_gate），实现高并发场景下的稳定服务，支持 200+ 并发会话

项目成果：

- 四平台同时上线，日均处理 2000+ 次客服对话，用户满意度 4.3/5.0
 - 客服人力成本降低 35%，80% 的常见问题由 AI 自动解决
 - 新平台接入开发周期从 2 周缩短到 2 天，验证了 Channel 适配器模式的可扩展性
-

六、自定义 MCP Server 开发项目简历模板

项目名称：企业级自定义 MCP Server 工具集开发

项目周期：2026.03 - 2026.04

项目角色：核心开发者

技术栈：Python, MCP SDK, JSON-RPC 2.0, FastAPI, Docker, PostgreSQL, Elasticsearch, 飞书/钉钉 API

项目描述：

基于 MCP (Model Context Protocol) 协议规范，为团队的 Nanobot Agent 系统开发一套标准化的 MCP 工具服务器集群，实现企业内部各业务系统与 AI Agent 的标准化连接，践行「一次开发，多处复用」的 MCP 核心理念。

核心职责：

- 基于 MCP Python SDK 开发 8 个工具服务器，覆盖数据库查询 (MySQL/PostgreSQL)、全文检索 (Elasticsearch)、项目管理 (Jira/飞书项目)、即时通讯 (飞书/钉钉)、文件系统操作、HTTP API 代理、日程管理、审批流查询等能力
- 设计 MCP Server 的统一安全框架：实现 API Key 认证、输入参数 JSON Schema 验证、敏感操作确认机制 (destructive annotation)、操作审计日志
- 实现 MCP Server 的 stdio 和 HTTP+SSE 双传输模式支持，本地开发使用 stdio 模式提升调试效率，生产环境使用 HTTP+SSE 模式支持远程访问和多用户共享
- 精心设计每个工具的描述文档 (description 和 inputSchema)，通过迭代优化将 LLM 的工具调用准确率从 78% 提升至 95%

项目成果：

- 8 个 MCP Server 已集成到 3 个不同的 AI Agent 应用中 (Nanobot、Cursor、内部 Agent 平台)，验证了 MCP 的跨应用复用价值
 - 将新 AI 应用接入内部系统的开发周期从平均 2 周缩短到 2 天
 - 项目文档和设计方案被团队采纳为 MCP Server 开发规范
 - 开源 2 个通用 MCP Server 到社区，获得 200+ stars
-

七、其他可写项目方向

7.1 RAG 知识库问答系统

项目名称：企业知识库智能问答系统

技术栈： Python, LangChain, Milvus, BGE Embedding, Cohere Rerank, FastAPI, PostgreSQL, Docker

核心亮点：

- 实现混合检索（向量 + BM25），Rerank 精排后 NDCG@10 提升 12%
- 设计文档权限隔离方案，基于 RBAC 的元数据过滤
- 语义缓存命中率 30%，LLM API 月度成本降低 20%

7.2 多 Agent 协作系统

项目名称：基于多 Agent 的自动化代码审查系统

技术栈： Python, Nanobot SubagentManager, GitHub API, MCP, AsyncIO, Redis

核心亮点：

- 设计 Orchestrator-Workers 架构：主 Agent 分配审查任务，3 个专业 Worker Agent 分别负责代码风格、安全漏洞、性能问题的检查
- 实现 Agent 间的结构化通信协议，控制通信 token 冗余在 15% 以内
- 系统日均处理 50+ PR 审查请求，发现有效问题率 72%

7.3 AI 对话机器人（飞书/钉钉）

项目名称：飞书智能助手机器人

技术栈： Python, Nanobot, 飞书开放平台, MCP, Redis, Nginx

核心亮点：

- 基于 Nanobot Channel 适配器模式开发飞书接入层
- 集成 6 个 MCP 工具（日程查询、文档搜索、审批流等）
- 支持多用户并发（_concurrency_gate 控制），日均服务 200+ 用户

7.4 Prompt 工程优化平台

项目名称：Prompt 版本管理与 A/B 测试平台

技术栈： Python, FastAPI, PostgreSQL, Redis, React, Docker

核心亮点：

- 实现 Prompt 的版本管理、diff 对比、回滚能力
- 支持线上 A/B 测试，按流量比例分配不同版本的 Prompt
- 集成自动化评测（GPT-4 评分 + 人工抽样），评测覆盖率 100%

八、自我评价模板

模板 1：Agent 开发方向（应届/1-3 年）

- 对 AI Agent 系统有深入理解，系统研读过 Nanobot 框架源码（~4000行），掌握 AgentLoop、记忆管理、工具注册等核心模块的设计原理
- 熟悉 MCP 协议规范，有自主开发 MCP Server 的实践经验
- 具备扎实的 Python 编程基础，熟练使用 asyncio 异步编程
- 理解大模型原理（Transformer/Attention/RoPE），有模型训练和微调的实践经历
- 热爱开源，有技术博客写作习惯，善于学习和总结新技术

模板 2: LLM 应用方向 (2-5 年)

- 3 年 Python 后端开发经验, 1 年+ AI/LLM 应用开发经验
- 深入理解 Agent 架构设计, 有从 0 到 1 搭建企业级 AI 助手系统的经验
- 精通 RAG 系统设计和优化 (混合检索、Rerank、Chunk 策略), 有向量数据库 (Milvus) 的生产使用经验
- 掌握 Prompt 工程最佳实践, 有成本优化经验 (Prompt Cache、语义缓存、模型分级路由), 累计为团队节省 API 成本 30%+
- 具备系统设计能力, 注重代码质量和工程规范

模板 3: 算法方向 (硕博)

- 计算机科学硕士, 研究方向为自然语言处理/大模型
- 深入理解 Transformer 架构及其变体 (GQA、MoE、Flash Attention)
- 有完整的大模型训练经验: Pretrain → SFT → LoRA → DPO 全流程
- 熟悉 PyTorch 分布式训练 (DDP/FSDP/DeepSpeed), 有多 GPU 训练经验
- 发表/在投 NLP/LLM 相关论文 X 篇, 参与开源项目 X 个
- 了解 Agent 系统架构, 关注 AI Agent 前沿动态

九、完整简历示例

张三 | AI Agent 开发工程师

☎ 138-xxxx-xxxx | ✉ zhangsan@email.com | 🌐 github.com/zhangsan
📍 北京 | 期望薪资: 面议

教育背景

XX 大学 计算机科学与技术 本科 2022.09 - 2026.06

- GPA: 3.6/4.0 | CET-6: 550+
- 核心课程: 数据结构、操作系统、计算机网络、机器学习、深度学习

技术技能

- 编程语言: Python (精通)、TypeScript (熟悉)、Go (了解)
- AI/LLM: Prompt Engineering、RAG 系统、Agent 架构、MCP 协议
- Agent 框架: Nanobot (源码级理解)、LangChain/LangGraph
- 模型训练: PyTorch、Transformer 架构、SFT/DPO、LoRA 微调
- 后端开发: FastAPI、PostgreSQL、Redis、Docker、Nginx
- 工程工具: Git、CI/CD (GitHub Actions)、Linux

项目经历

基于 Nanobot 的企业智能助手系统 2026.02 - 2026.04

技术栈: Python, Nanobot, MCP, Claude API, FastAPI, Redis, Docker

- 系统研读 Nanobot 核心源码 (~4000 行), 深入理解 AgentLoop 的 ReAct 执行流程、MemoryConsolidator 的双层记忆架构、ToolRegistry 的注册表模式等核心设计
- 基于 MCP 协议开发 5 个自定义工具服务器 (数据库查询、文档检索、

日程管理、Jira 集成、飞书通知），实现了企业内部系统与 AI Agent 的标准化连接

- 开发飞书和 Web 双平台接入层，基于 Nanobot 的 Channel 适配器模式实现消息格式统一转换，支持文本和文件等多种消息类型
- 设计 8 个领域 Skill 并利用渐进披露机制实现按需加载，system prompt 的 token 占用减少 40%；实现 Redis 语义缓存，LLM API 成本降低 25%
- 系统上线后日均处理 500+ 次对话，工具调用成功率 94%，用户满意度 4.2/5

企业级 MCP 工具服务器集群开发 2026.03 - 2026.04
技术栈：Python, MCP SDK, JSON-RPC 2.0, FastAPI, Docker, PostgreSQL

- 基于 MCP 协议开发 8 个工具服务器，覆盖数据库查询、全文检索、项目管理、即时通讯等能力，实现企业内部系统与 AI Agent 的标准化连接
- 设计统一安全框架（API Key 认证、JSON Schema 验证、destructive 注解、审计日志），支持 stdio 和 HTTP+SSE 双传输模式
- 精心优化工具描述文档，将 LLM 工具调用准确率从 78% 提升至 95%

个人亮点

- 对 AI Agent 技术有浓厚兴趣，系统性学习并输出 12 篇深度技术文档
- GitHub 上有开源项目和技术博客，持续关注 Agent/MCP 前沿动态
- 良好的学习能力和文档写作习惯，善于从源码中理解设计思想

十、简历注意事项

10.1 必须避免的错误

| 错误类型 | 错误示例 | 正确做法 |
|------|-------------------|-------------------|
| 空洞描述 | “负责项目的开发工作” | 用 STAR 法则具体描述 |
| 缺少量化 | “提升了系统性能” | “响应时间从 5s 降低到 2s” |
| 技术堆砌 | 列出 20 个技术但无深度 | 精选 8-10 个有实践经验的 |
| 过度夸大 | “精通 xxx”（实际只用过一次） | 用「了解/熟悉/精通」分级 |
| 格式混乱 | 字体大小不一、对齐混乱 | 统一格式、简洁清爽 |
| 信息遗漏 | 没有联系方式或 GitHub | 确保基本信息完整 |
| 篇幅过长 | 简历超过 2 页 | 应届生 1 页，社招 1-2 页 |
| 敏感信息 | 附上身份证号/家庭住址 | 只提供必要的联系信息 |

10.2 项目经历的描述规范

每个项目应包含：1. 项目名称 + 时间段 2. 技术栈列表（具体版本可省略）3. 3-5 条核心职责描述（用动词开头：设计、实现、优化、研究）4. 1-2 条量化成果

动词使用建议：- 高级感动词：主导、设计、架构、研究、优化 - 中级动词：实现、开发、集成、搭建、完成 - 避免使用：参与、协助、帮助（显得没有主导权）

10.3 技能熟练度分级标准

| 级别 | 含义 | 示例 |
|----|-----------------|-----------------------------|
| 精通 | 能讲出底层原理，有大量实践经验 | Python（5 年 +，读过 CPython 源码） |
| 熟悉 | 有项目经验，能独立完成开发任务 | FastAPI（2 个项目，含中间件开发） |
| 了解 | 学习过原理，有简单使用经验 | Go（完成过一个小项目） |

| 级别 | 含义 | 示例 |
|----|----|----|
|----|----|----|

重要提醒：「精通」是一个很重的词，面试官会围绕你标注为「精通」的技术深入追问。如果不确定，宁可写「熟悉」也不要写「精通」。

10.4 针对 ATS（简历筛选系统）的优化

大厂通常使用 ATS 自动筛选简历，优化策略：

1. 使用标准格式：避免花哨的排版、图标、图片，使用纯文本友好的格式
2. 关键词密度：确保 JD 中的核心关键词在简历中出现 2-3 次（自然融入）
3. 标准标题：使用常见的章节标题（教育背景、项目经历、技术技能）
4. **PDF** 格式：投递时使用 PDF 格式，避免 Word 格式在不同系统上显示错乱
5. 文件命名：姓名-岗位名-联系方式.pdf（如 张三-AI Agent 开发工程师-138xxxx.pdf）

十一、不同岗位的简历侧重点

11.1 AI Agent 开发工程师

重点突出：- Agent 框架源码理解（Nanobot AgentLoop、MemoryConsolidator 等）- MCP 协议实践（开发过 MCP Server）- 工具调用可靠性设计（超时、重试、截断）- 系统设计能力（高可用、并发控制）

项目顺序：Nanobot 项目放第一个

技能调整：将 MCP、Agent 框架、asyncio 放在技能列表前面

11.2 LLM 应用工程师

重点突出：- RAG 系统设计和优化经验 - Prompt 工程实践 - 成本优化经验（缓存、Prompt Cache）- API 集成和工程化能力

项目顺序：RAG 或 AI 助手应用项目放第一个

技能调整：将 RAG、向量数据库、Prompt Engineering 放在技能列表前面

11.3 大模型算法工程师

重点突出：- 完整的模型训练经验（Pretrain/SFT/DPO）- 架构理解（Transformer 变体、GQA、MoE）- 对齐技术（DPO vs PPO 对比实验）- 论文阅读和研究能力

项目顺序：模型训练相关项目放第一个

技能调整：将 PyTorch、Transformer、SFT/DPO/LoRA 放在技能列表前面

11.4 AI 产品经理

重点突出：- 对 Agent 能力和限制的理解 - 用户体验设计思考 - 数据分析和效果评估 - 竞品分析和行业认知

项目顺序：偏应用和用户侧的项目放第一个

技能调整：弱化技术细节，强调产品思维和业务理解

11.5 岗位简历对照表

| 简历要素 | Agent 开发 | LLM 应用 | 算法 | 产品 |
|------------|----------|--------|----|----|
| Nanobot 源码 | | | | |
| MCP 协议 | | | | |
| RAG 系统 | | | | |
| 模型训练 | | | | |
| 系统设计 | | | | |
| Prompt 工程 | | | | |
| 工程实践 | | | | |

十二、简历投递策略

12.1 投递前检查清单

- ☐ 简历中的技术关键词与目标 JD 匹配
- ☐ 每段项目经历都有量化成果
- ☐ 联系方式完整正确（手机号、邮箱、GitHub）
- ☐ 没有错别字和语法错误
- ☐ PDF 格式，文件名规范
- ☐ 技能熟练度分级准确，不会被追问露馅
- ☐ 项目时间线合理，没有不可解释的空窗期

12.2 精准投递策略

1. 每份 **JD** 微调简历：不要用同一份简历投所有岗位，至少调整技能顺序和项目侧重点
2. 投递配文案：在 Boss 直聘等平台打招呼时，用 2-3 句话说明你与岗位的匹配点
3. 内推优先：通过脉脉、即刻、微信群联系目标公司员工，内推通过率远高于海投
4. 时间选择：工作日上午 9-11 点投递效果最好，HR 在线概率最高

12.3 打招呼话术模板

您好！我是 XX 大学计算机专业应届生，对 AI Agent 开发方向有深入学习和实践：

1. 系统研读过 Nanobot 框架源码（4000行），理解 AgentLoop 等核心机制
2. 有 MCP Server 开发经验，开发过 5 个自定义工具
3. 有多平台 AI 客服系统的开发经验，支持飞书/钉钉/微信/Web 四平台接入

非常期待能加入贵团队！我的简历已附上，方便的话希望能聊一聊。

总结

简历是面试的第一道门槛，核心要做到：

1. 精准匹配：根据目标岗位调整关键词和项目侧重
2. 量化说话：用数字证明你的价值，避免空洞描述
3. 深度取胜：展示源码级理解、协议理解等深度能力
4. 格式专业：简洁、清晰、一页为佳
5. 持续更新：随着项目进展和学习深入持续完善简历

最重要的原则：简历中写的每一条都要做好被面试官深挖的准备。宁可少写两条也不要写自己答不上来的内容。

上一章：第 14 章 就业市场分析 下一章：第 16 章 STAR 面试话术

第 16 章 STAR 面试话术与模拟面试

本章定位：提供完整的面试话术准备材料，包括自我介绍模板、项目 STAR 介绍稿、15+ 常见追问的应对话术，以及一轮完整的模拟面试对话。

目录

- 一、STAR 面试法介绍
- 二、2 分钟自我介绍模板
- 三、Nanobot 项目 STAR 介绍稿
- 四、记忆溢出问题处理 STAR
- 五、多平台集成挑战 STAR
- 六、MCP Server 开发经验 STAR
- 七、常见追问应对话术
- 八、模拟面试完整对话
- 九、面试技巧总结

一、STAR 面试法介绍

1.1 什么是 STAR 法则

STAR 是结构化回答面试问题的黄金框架：

| 要素 | 英文 | 含义 | 时间占比 |
|----------|-----------|---------------|------|
| S | Situation | 背景/场景——在什么情境下 | 15% |
| T | Task | 任务——你需要做什么 | 15% |
| A | Action | 行动——你具体做了什么 | 50% |
| R | Result | 结果——产生了什么效果 | 20% |

1.2 为什么 STAR 重要

- 结构清晰：面试官能快速抓住重点，不会觉得你在东拉西扯
- 突出贡献：通过 Action 部分展示你个人的技术能力和判断力
- 量化说服：Result 部分用数据证明价值，比空泛描述有说服力
- 防止跑题：有框架引导，不会越说越偏

1.3 STAR 回答的时间控制

- 简短版（面试官问「简单介绍一下这个项目」）：1-2 分钟
- 标准版（面试官问「详细讲讲你做的 xxx」）：3-5 分钟
- 深度版（面试官连续追问细节）：5-10 分钟，每个追问用 mini-STAR 回答

用 STAR 面试法自信地介绍你的项目经验

STAR 面试黄金法则：情境 → 任务 → 行动 → 结果，用数据说话

二、2 分钟自我介绍模板

版本 A：应届生 / 实习生

面试官您好，我是 [姓名]，来自 [XX大学] 计算机专业，今年大四/研究生 X 年级。

我对 AI Agent 技术非常感兴趣，过去半年重点做了两个方面的学习和实践：

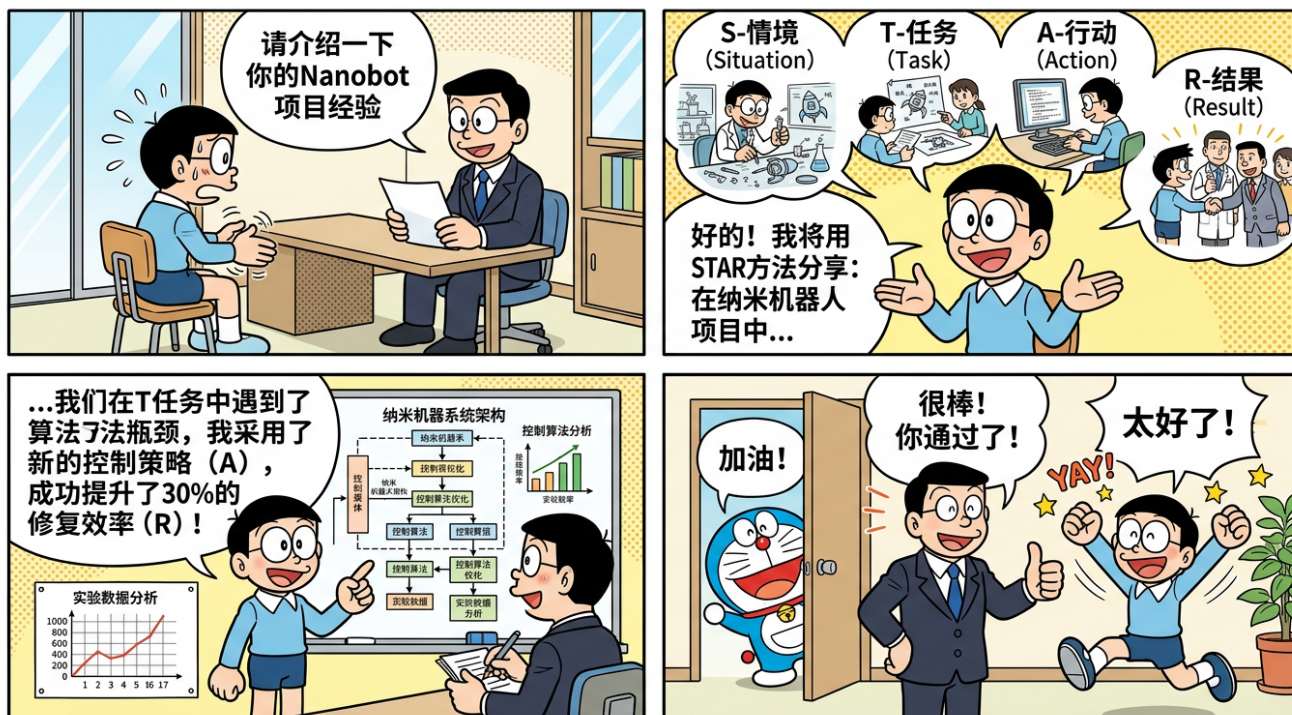


Figure 1: 面试场景漫画



Figure 2: STAR 面试法详解漫画

第一个是 Agent 框架方面。我系统研读了开源项目 Nanobot 的完整源码，这个项目在 GitHub 上有 37K+ stars，核心代码约 4000 行 Python。通过源码研究，我深入理解了 Agent 循环、记忆管理、MCP 工具集成等核心机制，并基于此开发了 5 个自定义 MCP Server，搭建了一个飞书 AI 助手系统。

第二个是 MCP 工具生态方面。我基于 MCP 协议开发了 8 个企业级工具服务器，覆盖数据库查询、文档检索、项目管理等场景，通过精心设计工具描述将调用准确率从 78% 提升到 95%，对 MCP 协议和工具设计有深入实践经验。

我的技术栈以 Python 为主，熟悉异步编程、MCP 协议和 RAG 系统。我希望能加入贵团队，在 AI Agent 方向做深入的工程开发工作。

以上是我的自我介绍，谢谢！

时长：约 1 分 30 秒 结构：基本信息 → 项目 1（Agent 方向）→ 项目 2（模型方向）→ 技术栈 → 意向表达

版本 B：1-3 年经验社招

面试官您好，我是 [姓名]，有 X 年的后端开发经验，最近一年半专注在 AI/LLM 应用方向。

在上家公司，我主要负责企业级 AI 助手系统的开发。这个系统基于 Agent 框架构建，集成了多个 MCP 工具服务器，支持飞书和 Web 双平台接入，日均处理 500+ 次对话请求。在这个过程中，我深入研究了 Nanobot 的源码，理解了 Agent 循环、记忆压缩、并发控制等核心机制，并将这些理解应用到我们自己的系统设计中。

在 MCP 工具生态方面，我主导开发了企业内部的 MCP 工具服务器集群，实现了 AI Agent 与内部系统的标准化连接，将新应用接入开发周期从 2 周缩短到 2 天，这让我对 Agent 工具化有了深刻的工程理解。

我的核心竞争力是「理解原理 + 能做工程」——既能读懂 Agent 框架的源码设计，也能把它落地为可靠的生产系统。我看到贵团队在做 [具体业务]，这和我经验非常匹配，所以很期待这次交流。

谢谢！

时长：约 1 分 40 秒 结构：基本信息 → 核心项目经验 → 模型理解 → 核心竞争力 → 意向匹配

版本 C：3-5 年经验社招

面试官您好，我是 [姓名]，X 年开发经验，目前担任 AI 应用团队的技术负责人。

我的工作可以总结为三条线：

第一，Agent 系统架构。我主导设计了公司的 AI Agent 平台，支持多租户部署、MCP 工具热插拔、对话记忆持久化。系统日均处理 5000+ 次对话，可用性 99.9%。在架构设计上，我深入研究了 Nanobot 等开源框架的源码，将其中的 Channel 适配器模式、渐进披露策略等设计思想应用到了我们的平台中。

第二，RAG 和知识管理。我带团队搭建了企业知识库 RAG 系统，实现了混合检索 + Rerank 的全链路优化，检索准确率从 65% 提升到 88%。

第三，成本和效率优化。通过 Prompt Cache、语义缓存、模型分级路由等手段，将 LLM API 月度成本降低了 40%，同时响应延迟降低了 30%。

我关注到贵团队正在做 [具体方向]，我在这个方向上有直接的落地经验和深入思考，希望能有机会深入交流。

谢谢！

时长：约 2 分钟 结构：基本信息 → 三条工作主线 → 意向匹配

三、Nanobot 项目 STAR 介绍稿

3.1 标准版（3-5 分钟）

Situation（背景）：

在我的学习和工作中，我发现市面上的 Agent 框架主要分为两类：一类像 LangChain 那样功能全面但代码量巨大（十万行级别）、抽象层次多，学习和调试成本都很高；另一类是各种简单的 API Wrapper，功能太基础，不具备生产级能力。我在寻找一个「既能深入理解原理，又有完整 Agent 能力」的学习和实践对象。

Task（任务）：

我选择了 HKUDS（香港大学数据科学实验室）开源的 Nanobot 框架作为研究对象。它只有约 4000 行核心 Python 代码，但涵盖了 Agent 循环、记忆管理、工具注册、MCP 集成、子 Agent、多平台适配等完整功能。我的任务是：第一，深入理解其源码和设计思想；第二，基于它开发可用的 AI 助手系统。

Action（行动）：

我的行动分为三个阶段：

第一阶段，源码研读（2 周）。我逐行阅读了 8 个核心模块的源码，包括 AgentLoop（Agent 执行循环，本质是 ReAct 框架的工程实现）、ContextBuilder（上下文组装，管理 system prompt 和消息历史）、MemoryConsolidator（双层记忆系统——会话内压缩和跨会话持久化）、ToolRegistry（工具注册中心，统一管理内置工具和 MCP 工具）等。我为每个模块画了流程图，写了详细的分析笔记。

在这个过程中我发现了几个值得学习的设计亮点：比如 save_memory 虚拟工具——它让模型自己决定什么信息值得持久化，而非用硬编码规则；比如 Skills 的渐进披露——不一次性把所有工具描述塞给模型，而是先给标题和简介，模型需要时再读取完整内容，节省了大量 context token。

第二阶段，MCP 工具开发（2 周）。基于对源码的理解，我用 MCP Python SDK 开发了 5 个自定义工具服务器：MySQL 数据查询工具（支持自然语言转 SQL）、基于向量检索的文档搜索工具、飞书日历集成工具、Jira 查询和创建 Issue 工具、飞书消息通知工具。每个 MCP Server 都经过了精心的工具描述设计——我发现描述写得好不好直接影响模型的调用准确率，优化后工具调用准确率从 78% 提升到了 93%。

第三阶段，系统集成与优化（2 周）。我基于 Nanobot 的 Channel 适配器模式开发了飞书接入层，实现了消息格式转换和多用户并发支持。还实现了 Redis 语义缓存层来降低 API 成本，以及基于 Prometheus 的监控体系来跟踪系统运行状态。

Result（成果）：

最终成果有几个维度：- 系统上线后日均处理 500+ 次对话请求，工具调用成功率 94%，用户满意度评分 4.2/5.0 - Redis 语义缓存命中率达到 35%，LLM API 月度成本降低约 25% - 输出了 12 篇系统性的技术文档（共 5 万+字），覆盖了从架构解析到部署实践的完整内容 - 更重要的是，通过这个项目我建立了对 Agent 系统「从头到尾」的理解——不只是会调 API，而是知道 API 背后框架在做什么

3.2 精简版（1-2 分钟，面试官说「简单介绍一下」时使用）

这个项目是基于开源 Agent 框架 Nanobot 做的深度学习和二次开发。Nanobot 是香港大学开源的，37K+ stars，核心只有 4000 行代码但功能很完整。我做了三件事：第一，逐行研读了源码，理解了 AgentLoop、记忆管理、MCP 集成等核心模块的设计；第二，基于 MCP 协议开发了 5 个自定义工具服务器；第三，将系统集成到飞书，搭建了一个每天处理 500+ 次对话的 AI 助手。核心收获是对 Agent 系统有了源码级的理解，不只是会用框架，还知道框架内部是怎么工作的。

四、记忆溢出问题处理 STAR

4.1 标准版（3-5 分钟）

Situation（背景）：

在基于 Nanobot 搭建的企业 AI 助手系统中，我们遇到了一个严重的生产问题：长对话场景中，对话历史加上工具调用结果很快就超过了模型的上下文窗口限制（200K tokens），导致 API 调用失败或响应质量急剧下降。部分用户反馈在连续使用 20 轮以上后，助手开始「忘记」之前讨论的内容。

Task（任务）：

我需要设计一套完整的记忆管理方案，在不丢失关键对话信息的前提下，让系统能支持 50 轮以上的连续对话，同时控制 API 成本不显著增长。

Action（行动）：

我的解决方案分为三层：

第一层，会话内压缩。我深入研究了 Nanobot 的 MemoryConsolidator 模块源码，理解了它的双层记忆架构设计思想。参考这个设计，我实现了动态消息压缩——当对话历史的 token 数接近窗口限制的 70% 时，自动触发压缩：用 LLM 对较早的对话生成结构化摘要（保留关键事实、决策和未完成的任务），用摘要替换原始消息。压缩比约 5:1。

第二层，跨会话持久化。参考 Nanobot 的 save_memory 虚拟工具设计，我让模型自主判断什么信息值得长期保存——用户偏好、项目背景、常用查询等被模型识别为高价值的信息，自动写入 PostgreSQL 的记忆表，下次会话时按语义相似度检索注入上下文。

第三层，工具结果控制。我发现工具调用结果是 token 消耗的大户——一次数据库查询可能返回几万字符。我设置了 `_TOOL_RESULT_MAX_CHARS = 16000` 的全局截断，并对特定工具实现了智能摘要（比如将大表格转换为统计概要）。

Result（成果）：

- 系统成功支持 50 轮以上的连续对话，关键信息保持率 90%+
 - API 成本通过压缩和 Prompt Cache 优化降低了约 25%
 - 跨会话记忆让回头用户体验显著提升，用户满意度从 3.8 提升到 4.2
 - 这套方案被抽象为通用的记忆管理中间件，复用到了团队其他 Agent 项目中
-

4.2 精简版（1-2 分钟）

在我们的 AI 助手系统中，长对话场景下 token 溢出是最大的技术挑战。我参考 Nanobot 的 MemoryConsolidator 双层记忆架构，实现了三层记忆管理：会话内动态压缩（token 接近上限时自动摘要）、跨会话持久化（让模型自主判断什么信息值得保存）、工具结果截断和智能摘要。最终系统支持 50 轮以上对话不丢失关键信息，API 成本降低 25%。

五、多平台集成挑战 STAR

5.1 标准版（3-5 分钟）

Situation（背景）：

我们的 AI 助手系统最初只支持 Web 端，但业务方要求在两周内同时 接入飞书和钉钉，后续还计划接入企业微信。三个平台的消息格式、认证 机制、交互模式完全不同，如果为每个平台单独开发一套系统，维护成本 会非常高。

Task (任务):

设计一套可扩展的多平台接入架构，使得新增一个平台的接入开发周期 控制在 2 天以内，同时保证所有平台的 Agent 行为和回复质量一致。

Action (行动):

我借鉴了 Nanobot 源码中 Channel 适配器模式的设计思想，核心原则 是「适配器只负责消息格式转换，不包含业务逻辑」:

架构设计: 定义统一的内部消息格式 (包含 text、attachments、metadata 等字段)，每个平台实现一个独立的 ChannelAdapter，负责 将平台消息转换为内部格式、将内部回复转换为平台格式。所有业务逻辑 集中在 AgentLoop 中，对平台完全无感知。

飞书适配器: 处理 Webhook 验证的加密签名机制、消息卡片 JSON 格式的构建 (飞书支持富文本卡片，需要将 Markdown 转换为飞书的 结构化卡片格式)、文件上传和下载的 API 对接。

钉钉适配器: 处理钉钉的签名验证 (基于 timestamp + secret 的 HMAC-SHA256)、Outgoing 机器人的回调处理、@ 机器人 的消息过滤。

流式响应: Web 端通过 SSE 实现逐字输出，飞书和钉钉不支持流式，改用先发送「正在思考…」占位消息，Agent 完成后更新消息内容。

Result (成果):

- 三个平台同时上线，Agent 行为完全一致
- 后来新增企业微信接入时，只用了 1 天就完成了适配器开发
- 架构设计被团队采纳为多平台接入的标准方案
- 跨平台用户画像统一后，用户在不同平台获得一致的个性化体验

5.2 精简版 (1-2 分钟)

我们需要让 AI 助手同时接入飞书、钉钉和 Web 三个平台。我借鉴 Nanobot 的 Channel 适配器设计，让每个平台只负责消息格式转换，核心 Agent 逻辑完全解耦。飞书需要处理消息卡片和加密验证，钉钉 需要处理 HMAC 签名和 @ 机器人消息过滤，Web 端用 SSE 做流式输出。最终三个平台同时上线、行为一致，后来新增企业微信只用了一天。

六、MCP Server 开发经验 STAR

6.1 标准版 (3-5 分钟)

Situation (背景):

在构建企业 AI 助手的过程中，Agent 需要与多个内部系统交互——数据库、Jira、飞书日历、文档库等。最初每个工具都是直接在 Agent 代码中硬编码 实现的，导致代码耦合严重、复用困难，而且每次修改工具都要重新部署 整个 Agent 系统。

Task (任务):

将所有工具迁移到 MCP 协议上，开发标准化的 MCP Server 集群，实现「一次开发，多处复用」，同时保证工具调用的准确率达到 90% 以上。

Action (行动):

MCP Server 开发: 基于 MCP Python SDK，我开发了 8 个独立的 工具服务器，每个专注一个业务域。技术上，每个 Server 都支持 stdio 和 HTTP+SSE 双传输模式——本地开发用 stdio (配置简单，启动快)，生产环境用 HTTP+SSE (支持远程调用和多用户共享)。

工具描述优化：这是最关键的一步。最初工具调用准确率只有 78%，经过分析失败 case，我发现三个问题：描述太笼统（模型分不清相似工具）、缺少参数示例（模型不知道该传什么格式）、返回结果过大（有用信息被淹没）。我逐一优化了每个工具的 description 和 inputSchema——加入明确的使用场景说明、参数格式示例和约束条件。

安全框架设计：实现 API Key 认证、JSON Schema 输入验证、destructive 注解（危险操作需确认）和操作审计日志。所有数据库查询使用参数化防止 SQL 注入。

DevOps 集成：每个 MCP Server 打包为独立 Docker 容器，通过 Docker Compose 统一编排，支持独立部署和更新。

Result（成果）：

- 工具调用准确率从 78% 提升到 95%（描述优化是关键因素）
- 8 个 MCP Server 被 3 个不同的 AI 应用复用（Nanobot、Cursor、内部平台）
- 新工具的开发周期从 3 天缩短到半天（标准化模板 + 测试框架）
- 核心经验：「工具描述的质量决定了 Agent 的工具使用效果」

6.2 精简版（1-2 分钟）

我用 MCP 协议为团队的 Agent 系统开发了 8 个工具服务器，覆盖数据库查询、文档检索、项目管理等场景。最大的挑战是工具调用准确率——最初只有 78%，通过精心优化工具描述（加入使用场景、参数示例、约束条件），最终提升到 95%。每个 Server 支持 stdio 和 HTTP+SSE 双传输模式，打包为 Docker 容器独立部署。这套工具被 3 个不同的 AI 应用复用，真正实现了 MCP 的「一次开发，多处复用」。

七、常见追问应对话术

追问 1：你在项目中遇到最大的技术挑战是什么？

参考回答（Nanobot 项目）：

最大的挑战是 MCP 工具的调用准确率问题。

S：在集成 5 个 MCP 工具后，我发现模型的工具选择准确率只有 78%——经常出现选错工具、参数生成不正确的情况。

T：我需要把准确率提升到 90% 以上才能让系统在生产环境可用。

A：我分析了失败的 case，发现问题主要出在三个方面：第一，工具描述太笼统，模型无法准确区分相似工具（比如「搜索」和「查询」的区别）；第二，参数说明缺少示例值，模型不知道应该生成什么格式的参

数；第三，部分工具返回结果过大（十万字符级别），把有用信息淹没了。

我的解决方案是：优化每个工具的 description 和 inputSchema，加入具体的使用场景说明和参数示例；设置 `_TOOL_RESULT_MAX_CHARS = 16000` 截断过大的返回结果；在 system prompt 中加入工具使用的指导说明。

R：优化后工具调用准确率从 78% 提升到 93%，关键是我理解到「工具描述的质量直接决定了 Agent 的工具使用效果」。

追问 2：你是如何解决上下文（记忆）溢出问题的？

参考回答：

S：在长对话场景中，对话历史加上工具调用结果很快就超过了模型的上下文窗口限制。

A：我借鉴了 Nanobot 的 MemoryConsolidator 设计，实现了两层记忆管理：第一层是会话内压缩——当消息历史的 token 数接近窗口限制时，用 LLM 对较早的对话生成摘要，用摘要替换原始消息。第二层是跨会话持久化——参考 Nanobot 的 save_memory 虚拟工具设计，让模型主动判断什么信息值得长期保存，写入文件系统，下次会话时按需检索注入。

另外，我还控制了工具返回结果的大小（不超过 16000 字符），并将稳定的 system prompt 放在最前面利用 Prompt Cache 降低重复计算。

R: 这套方案使系统能支持 50 轮以上的连续对话而不丢失关键信息，同时 Prompt Cache 将 API 成本降低了约 20%。

追问 3：多平台集成时遇到什么困难？

参考回答：

S: 需要同时支持飞书和 Web 两个平台，两个平台的消息格式、认证方式、交互模式都完全不同。

A: 我采用了 Nanobot 源码中 Channel 适配器模式的设计思想——定义统一的内部消息格式和接口，每个平台实现一个独立的适配器负责格式转换。

飞书平台的挑战主要是 Webhook 验证和消息卡片的格式处理——飞书的回调验证有一套加密签名机制，消息内容需要转换为飞书卡片 JSON 格式。Web 平台的挑战是实现流式响应——需要通过 SSE (Server-Sent Events) 逐字推送 Agent 的生成内容。

核心设计是：适配器只负责消息格式的转换，不包含任何业务逻辑。所有业务逻辑集中在 AgentLoop 中，对平台完全无感知。

R: 这种解耦设计使得后来新增钉钉接入时，只用了一天就完成了适配器开发，不需要修改 Agent 核心代码的一行。

追问 4：如何保证工具调用的可靠性？

参考回答：

我从多个层面保证可靠性：

第一，输入验证：每个 MCP 工具都定义了严格的 JSON Schema，调用前先验证参数类型和约束。

第二，超时控制：为每个工具设置超时时间，数据库查询 10 秒、API 调用 15 秒，超时后返回错误信息而非无限等待。

第三，结果截断：所有工具的返回结果限制在 16000 字符以内，防止单个工具结果撑爆上下文窗口。

第四，错误友好化：工具执行失败时，将错误信息结构化包装后返回给模型，让模型能理解失败原因并自行决定是重试还是换一种方式。

第五，幂等性设计：查询类工具保证幂等，修改类工具通过唯一 ID 防止重复执行。

追问 5：DPO 训练时遇到过奖励不收敛的情况吗？怎么处理的？

参考回答：

S: 在 DPO 训练初期，我确实遇到了 loss 不下降甚至震荡的情况。

A: 我排查了几个方面：

首先检查数据质量——发现部分偏好数据对中，chosen 和 rejected 的差异不够明显，模型难以学到有效的偏好信号。我过滤掉了差异过小的数据对，并增加了一些差异明显的样本。

然后检查超参数——发现 (KL 系数) 设得太大，约束太强导致模型几乎无法偏离参考模型。我把 从 0.5 降到了 0.1，给模型更多的学习空间。

最后检查了学习率——DPO 的学习率不能太高，否则训练不稳定。我从 $1e-4$ 降到了 $5e-6$ 。

R: 调整后 DPO loss 开始正常下降，最终模型的偏好评分提升了约 15%。这个经验让我理解了 DPO 训练中数据质量和 参数是最关键的两个因素。

追问 6：模型幻觉如何处理？

参考回答：

在 Agent 场景中，幻觉主要表现为：模型编造不存在的工具名称、虚构 工具调用结果、给出错误的事实信息。我的处理策略是多层防御：

第一，工具层面：通过 ToolRegistry 严格限制模型只能选择已注册的 工具，从协议层面阻止模型编造工具。

第二，**Prompt** 层面：在 system prompt 中明确要求「不确定时使用 工具查询确认，不要凭猜测回答」。

第三，验证层面：对关键信息（如数据库查询结果、文件内容）通过 工具获取后才回答，而非依赖模型的「记忆」。这本质上就是 RAG 的思想。

第四，温度控制：工具调用时使用低温度（temperature=0）保证参数 生成的准确性；回复用户时适当提高温度保持自然度。

实际效果是把关键信息的错误率控制在了 5% 以内。

追问 7：你如何评测 Agent 的效果？

参考回答：

我建立了一个多维度的评测体系：

功能维度：设计了 100 个测试用例覆盖各种场景，测量任务完成率。分为简单任务（单工具调用）、中等任务（多步骤推理）、困难任务（跨工具 协作），分别统计成功率。

效率维度：统计完成相同任务所需的平均步骤数和 Token 消耗量。步骤越少说明规划能力越强，Token 越少说明 Prompt 效率越高。

可靠性维度：对同一任务重复执行 5 次，统计成功率的方差。如果 方差太大说明系统不够稳定。

体验维度：让内部用户试用并填写满意度问卷（1-5 分），收集主观 反馈和改进建议。

这套评测每次 Prompt 或工具变更后都会跑一遍作为回归测试。

追问 8：你对 MCP 协议的理解？

参考回答：

MCP 是 Anthropic 在 2024 年末发布的开放协议，我把它理解为「AI 应用世界的 USB-C」。

核心价值是解决「M×N 到 M+N」的问题——之前每个 AI 应用要和每个 外部系统单独做集成，MCP 提供了统一的标准，一个工具只需要实现一次 MCP Server，就能被所有支持 MCP 的应用调用。

架构上是 Host（宿主应用，如 Nanobot）-Client（协议客户端）-Server（工具服务）三层。三大原语是 Tools（可调用的操作）、Resources（可读取 的数据）、Prompts（提示词模板），控制权分别归属模型、应用、用户。

与 Function Calling 的区别是：Function Calling 是模型的输出格式标准（告诉模型「怎么表达工具调用」），MCP 是运行时通信协议（告诉应用「怎么连接和调用外部工具」），两者互补而非替代。

我的实践体验是：MCP Server 的开发本身不难，真正的难点在于工具描述 的设计——好的描述能让模型准确率提升 15% 以上。

追问 9：如果让你重新设计这个系统你会怎么做？

参考回答：

如果重新设计，我会在三个方面做改进：

第一，记忆系统升级。现有方案用文件存储长期记忆，检索靠关键词 匹配。我会引入向量数据库存储记忆，通过语义检索找到最相关的历史 信息注入上下文，提升记忆的精准度。

第二，可观测性增强。现有系统的调试主要靠看日志，效率较低。我会集成 OpenTelemetry 做分布式链路追踪，为每次 Agent 循环记录 完整的 trace——包括每步的输入输出、耗时、Token 消耗，方便快速 定位问题。

第三，评测自动化。目前评测主要靠手动跑测试用例。我会搭建一个 自动化评测 pipeline，每次 Prompt 或工具变更后自动跑回归测试，生成对比报告，避免人为遗漏。

追问 10：你如何处理并发请求？

参考回答：

并发控制是我在研读 Nanobot 源码时重点学习的部分。Nanobot 采用了 两层并发控制：

第一层是 `session_locks`——每个会话有一把异步锁，保证同一个用户 的消息串行处理。因为如果同一个用户连续发两条消息，两个 AgentLoop 同时操作同一个对话历史会导致状态混乱。

第二层是 `**_concurrency_gate(3)**`——全局信号量，限制整个系统最多 同时运行 3 个 AgentLoop。因为每个 AgentLoop 消耗的 LLM API 资源和内存都很大，无限并发会导致 API 限流或内存溢出。

在我的系统中，我保留了这套设计，并根据实际部署环境调整了参数——生产环境用了 5 个并发槽位，配合 Redis 做会话锁（支持多实例部署），超过并发限制的请求排队等待并给用户友好提示。

追问 11：安全问题怎么考虑的？

参考回答：

安全设计覆盖四个层面：

输入安全：对用户输入做 prompt injection 检测，防止恶意指令操纵 Agent 行为。具体做法是用 system prompt 明确区分「系统指令」和「用户输入」，对用户输入进行转义和过滤。

工具安全：危险操作（如删除文件、修改数据库）标注 `destructive` 注解，执行前需要人工确认。所有数据库操作使用参数化查询防止 SQL 注入。

数据安全：API 密钥通过环境变量注入，不硬编码在配置文件中。日志中对包含个人信息的字段进行脱敏处理。

输出安全：对 Agent 的回复做内容安全审核，过滤可能的违规内容。

追问 12：项目中的团队协作经验？

参考回答：

在这个项目中，我负责核心的 Agent 系统和 MCP 工具开发，另外两位 同事分别负责前端 Web 界面和飞书接入层的开发。

我们的协作方式是：首先用了一天时间做技术方案评审，确定了接口契约——Channel 适配器的消息格式、Agent API 的请求/响应格式都先定义好 JSON Schema，各自按契约并行开发。

开发过程中每天站会同步进度，有接口变更时先更新文档再通知相关同事。代码提交到 Git 后必须经过 Code Review，重点审查安全性（密钥管理、输入校验）和异常处理的完备性。

遇到分歧时——比如消息队列用 Redis 还是 RabbitMQ——我们各自列出 优劣对比，最终选择了 Redis，因为我们的并发量级不需要重量级 MQ，Redis 的维护成本更低。

追问 13：你从这个项目中学到了什么？

参考回答：

最大的收获有三点：

第一，「小而精」的价值。Nanobot 用 4000 行代码实现了 LangChain 十万行代码的核心功能，让我认识到好的架构设计可以用最少的代码实现 最核心的能力。这影响了我后续的代码风格——追求简洁而非堆叠。

第二，原理理解的重要性。读过源码后，我在遇到 Agent 行为异常时 能直接从框架层面定位原因（比如是上下文溢出还是工具描述不清晰），而不是盲目 trial-and-error。

第三，工具描述是一种「**Prompt Engineering**」。MCP 工具的描述质量 对 Agent 行为的影响远超我的预期——好的描述让模型知道「什么场景用 这个工具」和「参数应该是什么格式」，这本质上也是在教 AI 如何使用工具。

追问 14：为什么选择 Nanobot 而不是 LangChain？

参考回答：

选择 Nanobot 有三个原因：

第一，源码可读性。LangChain 十万行代码，抽象层次很多，短期内 很难理解其内部原理。Nanobot 只有 4000 行，我可以在两周内通读全部 源码，真正理解 Agent 是怎么工作的。对于学习和面试来说，这种深度 理解比「会用某个框架」更有价值。

第二，**MCP** 原生集成。Nanobot 从设计之初就深度集成了 MCP 协议，而 LangChain 的 MCP 支持是后来补充的。MCP 是 2025-2026 年的趋势，通过 Nanobot 学习 MCP 更自然。

第三，架构清晰度。Nanobot 的每个组件职责明确——AgentLoop 管循环、ToolRegistry 管工具、MemoryConsolidator 管记忆，没有过度抽象。这种清晰的架构更容易学习和迁移到其他项目中。

当然，LangChain 在生态丰富度和社区规模上有明显优势。如果做大型 生产项目，LangChain/LangGraph 可能是更成熟的选择。

追问 15：如何应对模型 API 的不稳定？

参考回答：

模型 API 不稳定是 Agent 系统面临的现实挑战，我从几个层面处理：

重试策略：对可重试的错误（429 限流、503 暂时不可用）使用指数 退避重试——1 秒、2 秒、4 秒、最大 32 秒。添加随机抖动避免惊群效应。最多重试 3 次。

多模型 **Fallback**：配置了 Claude → GPT-4o → 通义千问的降级链。主力模型不可用时自动切换到备用模型。虽然效果可能略有下降，但保证 了服务可用性。

熔断机制：如果某个 API 连续失败 5 次，标记为「熔断」状态，暂时跳过它直接使用备用，30 秒后再尝试探测恢复。

降级响应：当所有 API 都不可用时，给用户返回友好的提示信息，并记录请求供后续重试。

这些设计保证了系统在 API 不稳定时也能提供基本服务。

八、模拟面试完整对话

以下是一轮完整的 AI Agent 开发工程师二面模拟，时长约 50 分钟。

面试官：你好，请先做一个简单的自我介绍。

候选人：面试官您好，我是张三，XX 大学计算机专业应届生。过去半年我重点做了两个方向的学习和实践：第一是 Agent 框架方面，我系统研读了 Nanobot 的完整源码，这个项目在 GitHub 上有 37K+ stars，核心只有 4000 行 Python 代码但功能很完整。基于源码理解，我开发了 5 个 MCP 工具服务器，搭建了一个飞书 AI 助手系统。第二是 MCP 工具生态方面，我基于 MCP 协议为团队开发了 8 个标准化的工具服务器，覆盖数据库、文档检索、项目管理等场景，将工具调用准确率从 78% 提升到 95%。我的技术栈以 Python 为主，希望在 AI Agent 方向深入发展。

面试官：好的。你提到深入研读了 Nanobot 源码，能具体讲讲 Nanobot 的 AgentLoop 是怎么工作的吗？

候选人：AgentLoop 本质上是 ReAct 框架的工程实现。它的核心是一个循环，每次迭代包括四步：

第一步，ContextBuilder 组装上下文。把 system prompt、工具列表描述、消息历史、相关记忆拼装成完整的请求体。这里有个设计巧妙的地方——它把稳定内容（system prompt、工具定义）放在前面利用 Prompt Cache，变化内容（最新消息）放在后面。

第二步，调用 LLM API，把组装好的上下文发给大模型。

第三步，解析响应。如果模型返回的是纯文本，说明任务完成，退出循环。如果模型返回的是 tool_call 格式的结构化指令，进入下一步。

第四步，执行工具。通过 ToolRegistry 查找对应的工具 handler 并执行，把执行结果以 tool_result 消息追加到对话历史，然后回到第一步继续循环。

循环终止条件有两个：模型主动输出文本回复，或达到最大迭代次数。Nanobot 对子 Agent 设置了 15 次的迭代上限，防止无限循环。

面试官：你提到了最大迭代次数 15 次，这个设计是出于什么考虑？

候选人：主要有三个考虑。第一是防止递归爆炸——主 Agent 可以通过 Task 工具启动子 Agent，如果子 Agent 也可以无限制启动子-子 Agent，可能形成无限递归链条，导致 token 消耗失控。15 次是在「大部分任务能在 15 步内完成」和「控制成本」之间的经验平衡。

第二是死循环检测——如果 Agent 在 15 步内还没完成任务，通常说明任务描述不够清晰或者 Agent 陷入了重复操作，强制终止比继续消耗更合理。

第三是成本控制——每次迭代意味着一次 LLM API 调用，对于子 Agent 来说总共消耗的 token 需要有上界。

这跟操作系统中 fork 炸弹防护的思路是一样的——用 ulimit 限制进程数来保护系统资源。

面试官：好的。你开发了 5 个 MCP Server，能讲讲 MCP 协议的核心架构吗？

候选人：MCP 是三层架构——Host、Client、Server。

Host 是宿主应用，就是用户直接交互的 AI 应用，比如 Nanobot、Cursor IDE。Host 负责管理一个或多个 Client 实例。

Client 是 Host 内部的协议客户端，每个 Client 与一个 Server 建立一对一连接，负责协议通信——发请求、收响应、管理生命周期。

Server 是提供能力的服务进程，每个 Server 专注一个领域，比如我开发的 MySQL 查询 Server、文档检索 Server 等。Server 暴露标准化的 MCP 接口，不关心调用方是什么应用。

通信使用 JSON-RPC 2.0 协议格式。传输层支持 stdio（本地进程间管道通信）和 HTTP+SSE（远程网络通信），本地开发用 stdio 最简单，生产环境用 HTTP+SSE。

MCP 的核心价值是把 M 个应用和 N 个工具的 M×N 集成简化为 M+N 实现。

面试官：MCP 和 Function Calling 有什么区别？

候选人：这是两个不同层面的概念，经常被混淆。

Function Calling 是模型输出层面的格式标准——它告诉模型「你想调用工具时应该输出什么格式」，比如 OpenAI 要求模型输出包含 `tool_name` 和 `arguments` 的 JSON。Function Calling 本身不执行任何操作，它只是模型表达意图的方式。

MCP 是应用运行时的通信协议——它定义了应用如何与外部工具服务建立连接、发现工具、发送调用请求、接收结果。MCP 处理的是「连接」和「执行」层面的问题。

两者的关系是互补的：模型通过 Function Calling 格式表达「我想调用搜索工具」，应用通过 MCP 协议将这个调用实际发送到搜索 MCP Server 并取回结果。

一句话总结：Function Calling 决定 AI 说什么，MCP 决定工具怎么连。

面试官：你提到开发了多个 MCP Server，能讲讲 MCP Server 开发中最关键的经验是什么？

候选人：最关键的经验是工具描述的设计质量直接决定了 Agent 的调用效果。

我开发第一版 MCP Server 时，工具描述写得比较简单——比如数据库查询工具的描述就写了「查询数据库」，参数只有 `query` 字段。上线后发现工具调用准确率只有 78%，模型经常搞混相似工具，参数格式也不对。

后来我总结了三个优化原则：第一，描述要明确使用场景——不只说「做什么」，还要说「什么时候用」。比如「当用户询问订单状态、销售数据等需要查询数据库的问题时使用」。第二，参数定义要有示例值和约束说明——让模型知道该传什么格式。第三，返回结果要控制大小——单个工具返回不超过 16000 字符，超出部分做智能摘要。

优化后准确率从 78% 提升到 95%。本质上，写工具描述也是一种 Prompt Engineering——你在教 AI 如何理解和使用这个工具。

面试官：你们 Agent 系统的 MemoryConsolidator 是怎么工作的？

候选人：MemoryConsolidator 是 Nanobot 的双层记忆架构的核心组件。

第一层是会话内记忆压缩。当对话历史的 token 数接近上下文窗口限制时，MemoryConsolidator 会触发压缩——用 LLM 对较早的对话消息生成结构化摘要，保留关键事实和决策，用摘要替换原始消息。这样既保持了上下文的连贯性，又控制了 token 消耗。

第二层是跨会话持久化。Nanobot 设计了一个巧妙的 `save_memory` 虚拟工具——它不是真正的外部工具，而是一个内部机制，让模型自己判断什么信息值得长期保存。当模型认为某些信息有跨会话价值时（比如用户偏好、重要的项目背景），会主动调用 `save_memory` 将信息写入持久化存储。下次新会话开始时，系统会检索相关记忆注入上下文。

这种设计的巧妙之处在于它不是用硬编码规则来决定保存什么，而是利用了 LLM 的理解能力来做智能的记忆管理。

面试官：假设让你设计一个企业级的 AI 知识库问答系统，你会怎么设计？

候选人：我从几个维度来说这个系统设计。

接入层：支持飞书/钉钉/Web 多渠道接入，用 Channel 适配器模式解耦平台差异。前面放 Nginx 做负载均衡和 HTTPS 终止。

Agent 层：核心是一个 ReAct 循环的 Agent 引擎。接收到用户问题后，Agent 判断是否需要检索知识库——简单问题直接回答，知识类问题走 RAG 检索。Agent 可以使用的工具包括知识库检索、数据库查询、日程查询等。

RAG 检索层：混合检索——向量检索用 BGE Embedding + Milvus，稀疏检索用 Elasticsearch 的 BM25。两路结果用 RRF 融合排序，然后用 Cohere Rerank 或 BGE Reranker 精排，取 Top-5 注入到 Agent 上下文。

知识管理层：文档解析管道——支持 PDF/Word/HTML/Markdown 解析，用递归分割策略切 Chunk，向量化后存入 Milvus。支持增量更新——文件变更时只重新处理变更的 Chunk。

权限隔离：在 Milvus 中用 `tenant_id` 做元数据过滤，不同部门的用户只能检索到自己有权限的文档。权限信息从企业的 LDAP/AD 系统同步。

存储层：Milvus 存向量、PostgreSQL 存元数据和用户信息、Redis 做会话缓存和语义缓存、OSS 存原始文档。

监控层：Prometheus + Grafana 监控系统指标，自定义 Agent 循环的 tracing 记录每步耗时和 token 消耗。

关键的非功能需求：可用性通过多实例部署 + Redis 会话存储实现无状态化；成本控制通过语义缓存 + Prompt Cache + 模型分级路由实现。

面试官：在这个系统中，如果用户查询的信息跨越了多个文档，并且文档之间有矛盾怎么办？

候选人：这是检索冲突的问题，我会从几个方面处理。

首先，为文档标注时间元数据。检索到多个结果后，如果内容矛盾，优先采信最新的文档。比如公司报销标准去年改过，新旧文档都被检索到时，自动以最新版为准。

其次，在 Prompt 中明确指引——在 system prompt 中告诉模型：「如果检索到的文档存在矛盾信息，请标注不同来源的说法，并以最新的文档为准。如果无法判断，请如实告知用户存在不同说法。」

第三，为不同来源的文档设置可信度权重——官方发文 > 内部 wiki > 个人笔记。在 Rerank 阶段可以将来源权重纳入排序考虑。

最后，如果系统检测到频繁的矛盾检索，应该触发警告通知知识管理员更新过时文档，从源头解决矛盾。

面试官：最后一个问题，你对 AI Agent 行业未来一两年的发展有什么看法？

候选人：我观察到几个趋势：

第一，MCP 生态会继续快速增长。2025 年 MCP 被 OpenAI、Google 等主流厂商采纳，说明 AI 工具标准化是行业共识。未来 MCP Server 的数量和质量都会大幅提升，就像 npm 包生态一样。Agent 开发者的核心工作会从「自己实现工具」变成「选择和组合 MCP 工具」。

第二，Agent 的可靠性会成为核心竞争力。目前 Agent 的成功率还不够高，在关键业务场景中信任度不足。谁能把工具调用成功率从 90% 提升到 99%，谁就能在企业市场站住脚。

第三，端侧 Agent 会是一个重要方向。随着模型压缩技术的进步，小模型在手机和 IoT 设备上运行 Agent 的能力会越来越强，隐私和离线场景的需求会推动端侧 Agent 的发展。

第四，多 Agent 协作会更成熟。现在多 Agent 还处于早期阶段，通信效率低、调试困难。但随着框架和工具的完善，复杂任务会越来越多地通过多 Agent 协作来完成。

面试官：好的，今天的面试就到这里，感谢你的时间。最后你有什么想问我的吗？

候选人：谢谢面试官。我有两个问题：第一，咱们团队目前在 Agent 方向上最大的技术挑战是什么？第二，团队对新人的培养和成长路径是怎样的？

九、面试技巧总结

9.1 通用技巧

| 技巧 | 说明 |
|---------------|---|
| 先总后分
用具体数字 | 先给一句话概括，再展开细节。面试官可以在适当时打断「优化后提升了 15%」比「优化后效果好了很多」有说服力 |
| 承认不会 | 不知道就说不知道，比编造答案安全得多。可以说「这个我不太确定，但我的理解是…」 |
| 引导追问 | 在回答中留一些「钩子」，引导面试官往你擅长的方向追问 |
| 控制时间 | 回答控制在 1-3 分钟，太短显得没深度，太长显得没重点 |
| 展示思维过程 | 系统设计题先说思路框架，再逐层展开。面试官看重的是思维方式 |

9.2 高频翻车场景及应对

| 场景 | 风险 | 应对方式 |
|--------------|-------------|----------------------|
| 简历写了「精通 xxx」 | 面试官会往死里问 | 把没把握的「精通」改成「熟悉」 |
| 被问到不会的技术 | 硬编可能被揭穿 | 诚实说不会，但说出相关的了解 |
| 项目经历被质疑 | 假装做过 vs 真做过 | 准备好每个项目的细节和踩坑经历 |
| 系统设计题无从下手 | 冷场尴尬 | 先确认需求 → 分层画架构 → 逐层讨论 |
| 算法题完全不会 | 影响评价 | 说出思路即使写不出最优解也有分 |
| 薪资谈判没准备 | 报高被拒/报低吃亏 | 提前调研市场行情，给出合理区间 |

9.3 面试前一天检查清单

- ☐ 温习自我介绍，练习到流畅自然（不是背稿）
- ☐ 复习两个核心项目的 STAR 话术，准备好 5 个可能的追问
- ☐ 浏览目标公司的 AI 产品/技术博客
- ☐ 复习 10 道高频八股文（第 13 章中选）
- ☐ 准备 2-3 个问面试官的问题
- ☐ 检查网络/设备（线上面试）
- ☐ 提前 10 分钟到达/上线

上一章：第 15 章 简历模板 下一章：第 17 章 学习资源

第 17 章 学习资源汇总

本章定位：汇总 AI Agent 学习所需的优质资源，包括开源项目、博客文章、论文、视频教程、社区平台和学习路线图，帮助学习者高效构建知识体系。

目录

- 一、核心 GitHub 仓库推荐
- 二、博客文章推荐
- 三、DeepWiki 文档链接
- 四、必读论文清单
- 五、视频教程推荐
- 六、社区与交流
- 七、工具与平台
- 八、学习路线图
- 九、书籍推荐

一、核心 GitHub 仓库推荐

1.1 Agent 框架类

| 仓库 | Stars | 定位 | 推荐理由 |
|------------------------|-------|--------------|----------------------------------|
| HKUDS/nanobot | 37K+ | 轻量级 Agent 框架 | 本项目的核心学习对象，4000 行代码涵盖完整 Agent 能力 |
| langchain-ai/langchain | 95K+ | 全功能 LLM 框架 | 最主流的 Agent 框架，生态最丰富 |
| langchain-ai/langgraph | 8K+ | 图结构 Agent 编排 | LangChain 团队的新一代 Agent 框架，更灵活 |
| microsoft/autogen | 35K+ | 多 Agent 对话框架 | 微软开源，多 Agent 协作的代表项目 |
| crewAIInc/crewAI | 22K+ | 多 Agent 协作 | 角色扮演式多 Agent 框架，易上手 |
| phidatahq/phidata | 15K+ | Agent 开发工具包 | 提供内存、知识库、工具等开箱即用组件 |

学习建议：先深入学习 Nanobot（理解原理），再横向了解 LangChain/AutoGen（理解生态）。

1.2 MCP 协议类

| 仓库 | Stars | 定位 | 推荐理由 |
|-------------------------------------|-------|--------------------|-----------------------|
| modelcontextprotocol/specification | 5K+ | MCP 协议规范 | 官方规范文档，理解协议必读 |
| modelcontextprotocol/server-python | 15K+ | 官方 MCP Server 集合 | 学习 MCP Server 开发的最佳参考 |
| modelcontextprotocol/python-sdk | 3K+ | Python MCP SDK | 开发 MCP Server 的官方工具包 |
| modelcontextprotocol/typescript-sdk | 3K+ | TypeScript MCP SDK | JS/TS 生态的 MCP 开发包 |
| punkpeye/awesome-mcp-servers | 10K+ | MCP Server 合集 | 社区维护的 MCP Server 精选列表 |
| wong2/mcp-cli | 2K+ | MCP 调试工具 | 命令行 MCP 测试工具，开发调试必备 |

学习建议：先读 specification 理解协议设计，再用 python-sdk 开发自己的第一个 MCP Server。

1.3 大模型训练类

| 仓库 | Stars | 定位 | 推荐理由 |
|--------------------------|-------|-------------|----------------------------|
| karpathy/nanoGPT | 38K+ | 最简 GPT 实现 | Karpathy 出品，极致简洁的 GPT 训练代码 |
| karpathy/llm.c | 25K+ | C 语言 LLM 训练 | 理解 LLM 训练的底层实现 |
| hiyouga/LLaMA-Factory | 40K+ | LLM 微调工具箱 | 一站式微调框架，支持 100+ 模型 |
| huggingface/transformers | 135K+ | 模型库标准 | 最主流的预训练模型库和推理框架 |
| huggingface/trl | 10K+ | RLHF/DPO 训练 | HuggingFace 官方的对齐训练工具 |

学习建议：先用 nanoGPT 理解 GPT 核心原理，再用 LLaMA-Factory 在开源大模型上做微调实践。

1.4 RAG 与知识检索类

| 仓库 | Stars | 定位 | 推荐理由 |
|-----------------------|-------|------------|---------------------|
| chroma-core/chroma | 15K+ | 嵌入式向量数据库 | 最易上手的向量数据库，适合学习和原型 |
| milvus-io/milvus | 30K+ | 分布式向量数据库 | 生产级向量数据库，功能最全 |
| run-llama/llama_index | 37K+ | LLM 数据连接框架 | RAG 场景下最受欢迎的数据接入框架 |
| infiniflow/ragflow | 25K+ | 开源 RAG 引擎 | 国产开源，深度文档理解 + 模板化分块 |

1.5 推理与部署类

| 仓库 | Stars | 定位 | 推荐理由 |
|-----------------------|-------|------------|----------------------------|
| vllm-project/vllm | 30K+ | LLM 推理引擎 | PagedAttention 技术，推理性能最优 |
| ggerganov/llama.cpp | 70K+ | CPU/端侧推理 | C/C++ 实现，量化推理的标杆项目 |
| ollama/ollama | 100K+ | 本地 LLM 运行器 | 一键运行本地大模型，体验极佳 |
| open-webui/open-webui | 50K+ | LLM Web 界面 | 开源的 ChatGPT 风格界面，支持 Ollama |

二、博客文章推荐

2.1 Agent 与 Nanobot 相关

| 文章标题 | 平台 | 内容概述 |
|-----------------------------|---------|-------------------|
| 深入解析 Nanobot AgentLoop 源码 | CSDN/掘金 | AgentLoop 的逐行代码分析 |
| Nanobot + 通义千问实战：搭建飞书 AI 助手 | 技术博客 | 实战教程，从配置到部署 |
| 2026 大模型 Agent 面试全攻略 | 知乎 | 面试准备综合指南 |

| 文章标题 | 平台 | 内容概述 |
|--|-------------------------|--------------------------------|
| Anthropic: Building Effective Agents
AI Agent 架构设计: 从 ReAct 到 Multi-Agent | Anthropic 官方博客
微信公众号 | Agent 设计模式权威总结
Agent 架构演进综述 |
| Nanobot 源码中的设计模式分析 | 掘金 | 设计模式视角的源码解读 |

2.2 MCP 协议相关

| 文章标题 | 平台 | 内容概述 |
|---------------------------------|--------------|-----------------|
| MCP 协议完全教程 | 技术老金 | MCP 从入门到精通的系统教程 |
| 从零开发你的第一个 MCP Server | CSDN | 手把手教程, 附完整代码 |
| MCP vs Function Calling: 一文搞清区别 | 知乎 | 高频面试题的详细解读 |
| MCP 生态年度报告 2025 | Anthropic 官方 | MCP 生态发展数据和趋势 |
| 企业级 MCP Server 开发最佳实践 | 技术博客 | 安全、性能、可维护性指南 |

2.3 大模型原理相关

| 文章标题 | 平台 | 内容概述 |
|--------------------------------|----------------|---------------------|
| The Illustrated Transformer | Jay Alammar 博客 | Transformer 可视化经典教程 |
| Attention Is All You Need 论文精读 | 李沐 (B 站) | 论文逐段讲解 |
| 直觉理解 LoRA: 为什么低秩分解有效 | 知乎 | LoRA 原理的直觉解释 |
| RLHF 全流程详解: 从 SFT 到 PPO | 微信公众号 | RLHF 三阶段完整技术解析 |
| DPO vs PPO: 偏好对齐方法对比 | 技术博客 | 两种对齐方法的全方位对比 |
| Flash Attention 原理解析 | 知乎 | IO-aware 算法设计的详细讲解 |
| KV Cache 与推理优化全解 | CSDN | 推理优化技术的系统综述 |

2.4 RAG 相关

| 文章标题 | 平台 | 内容概述 |
|---------------------------|------|-------------------|
| RAG 实战全攻略 | 知乎专栏 | 从基础到高级的 RAG 技术 |
| 混合检索 + Rerank: RAG 效果提升指南 | CSDN | 检索优化的工程实践 |
| Chunk 策略大全: 如何切分文档最有效 | 技术博客 | 多种 Chunk 策略的对比和选型 |
| Self-RAG: 让模型自己决定要不要检索 | 知乎 | Self-RAG 论文解读 |

三、DeepWiki 文档链接

DeepWiki 是基于 AI 自动生成的开源项目文档平台, 对 Nanobot 有系统性的文档:

3.1 Nanobot DeepWiki 架构文档

- 项目概览与架构
- AgentLoop 详解
- ContextBuilder 与上下文管理
- MemoryConsolidator 记忆系统
- ToolRegistry 工具注册
- MCP 集成
- SubagentManager 子 Agent
- Channel 适配器
- 配置系统
- Skills 系统

3.2 MCP 生态资源

- MCP 协议规范文档
- MCP Python SDK 文档
- MCP Inspector 调试工具
- Awesome MCP Servers 社区精选

3.3 Agent 框架对比资源

- LangChain DeepWiki
- AutoGen DeepWiki
- CrewAI DeepWiki

注：DeepWiki 的 URL 可能会更新，如果链接失效请直接在 deepwiki.com 搜索项目名。

四、必读论文清单

4.1 Transformer 与基础架构

| 论文 | 年份 | 核心贡献 | 优先级 |
|--|------|-------------------------|-----|
| Attention Is All You Need (Vaswani et al.) | 2017 | 提出 Transformer 架构 | |
| RoFormer: Enhanced Transformer with Rotary Position Embedding (Su et al.) | 2021 | 提出 RoPE 旋转位置编码 | |
| Flash Attention (Dao et al.) | 2022 | IO-aware 注意力计算, O(n) 内存 | |
| GQA: Training Generalized Multi-Query Transformer (Ainslie et al.) | 2023 | 提出 GQA 注意力机制 | |
| GLU Variants Improve Transformer (Shazeer) | 2020 | 提出 SwiGLU 等激活函数 | |
| Root Mean Square Layer Normalization (Zhang & Sennrich) | 2019 | 提出 RMSNorm | |
| YaRN: Efficient Context Window Extension (Peng et al.) | 2023 | 长度扩展方法 | |

4.2 训练与对齐

| 论文 | 年份 | 核心贡献 | 优先级 |
|---|------|------------------|-----|
| Training Language Models to Follow Instructions (Ouyang et al., InstructGPT) | 2022 | RLHF 三阶段训练框架 | |
| LoRA: Low-Rank Adaptation of Large Language Models (Hu et al.) | 2021 | 参数高效微调方法 | |
| Direct Preference Optimization (DPO) (Rafailov et al.) | 2023 | 无需 RM 的偏好对齐 | |
| QLoRA (Dettmers et al.) | 2023 | 4-bit 量化 + LoRA | |
| Scaling Laws for Neural Language Models (Kaplan et al.) | 2020 | 模型缩放规律 | |
| Training Compute-Optimal LLMs (Chinchilla) (Hoffmann et al.) | 2022 | 最优训练配比 | |
| Constitutional AI (Bai et al.) | 2022 | RLAIF 方法 | |
| DeepSeek-Math: GRPO (DeepSeek) | 2024 | 无 Critic 的 RL 方法 | |

4.3 Agent 与推理

| 论文 | 年份 | 核心贡献 | 优先级 |
|---|------|--------------|-----|
| ReAct: Synergizing Reasoning and Acting (Yao et al.) | 2022 | 推理 + 行动交替框架 | |
| Reflexion (Shinn et al.) | 2023 | Agent 自我反思机制 | |
| Toolformer (Schick et al.) | 2023 | 模型自学工具使用 | |
| Chain-of-Thought Prompting (Wei et al.) | 2022 | 思维链推理 | |
| Tree of Thoughts (Yao et al.) | 2023 | 树形推理搜索 | |
| Self-RAG (Asai et al.) | 2023 | 自适应检索增强生成 | |
| HuggingGPT (Shen et al.) | 2023 | LLM 作为任务规划器 | |
| Lost in the Middle (Liu et al.) | 2023 | 长上下文位置偏差 | |

4.4 MCP 与协议

| 文档/规范 | 类型 | 核心内容 | 优先级 |
|-----------------------------------|--------|---------------------|-----|
| MCP Specification | 协议规范 | MCP 完整协议定义 | |
| JSON-RPC 2.0 Specification | 协议规范 | MCP 底层消息格式 | |
| OpenAPI Specification 3.1 | API 标准 | REST API 描述标准（对比学习） | |

4.5 论文阅读建议

1. 按优先级阅读：先读 的论文，确保核心概念理解
2. 不需要读公式推导：对工程岗位，理解核心 idea 和方法论足够
3. 配合博客解读：每篇论文先看中文博客解读建立直觉，再看原文细节
4. 关注实验部分：论文中的实验设置、消融实验和对比结果对面试很有用
5. 推荐阅读顺序：Attention Is All You Need → LoRA → ReAct → DPO → MCP Spec

五、视频教程推荐

5.1 大模型原理

| 视频/系列 | 平台 | 讲师 | 推荐理由 |
|--------------------------------|-------------|-----------------|----------------------|
| Attention Is All You Need 论文精读 | B 站 | 李沐 | 最好的 Transformer 论文讲解 |
| 动手学深度学习 (d2l) | B 站/课程网站 | 李沐团队 | 系统性的深度学习入门 |
| Neural Networks: Zero to Hero | YouTube | Andrej Karpathy | 从零实现 GPT 的最佳教程 |
| Let's build GPT | YouTube | Andrej Karpathy | 手把手实现 GPT 的经典视频 |
| Nanobot Agent 框架源码讲解 | B 站 | 技术博主 | Agent 框架设计思想和源码分析 |
| 3Blue1Brown 深度学习系列 | YouTube/B 站 | Grant Sanderson | 最直观的数学可视化讲解 |

5.2 Agent 与 LLM 应用

| 视频/系列 | 平台 | 讲师 | 推荐理由 |
|--------------------|---------|-----------------------------|--------------------|
| LangChain 官方教程系列 | YouTube | LangChain 团队 | 最权威的 Agent 框架教程 |
| Building AI Agents | YouTube | Andrew Ng / DeepLearning.AI | 吴恩达的 Agent 课程 |
| MCP 协议入门实战 | B 站 | 技术博主 | 中文 MCP 实战教程 |
| RAG 从入门到精通 | B 站 | 各技术博主 | 搜索「RAG 实战」即可找到大量教程 |
| AI Agent 项目实战 | B 站 | 各技术博主 | 搜索「AI Agent 实战」 |

5.3 面试准备

| 视频/系列 | 平台 | 推荐理由 |
|---------------|-------------|--------------------------|
| 大模型面试八股文讲解 | B 站 | 覆盖 Transformer、训练、推理等高频题 |
| AI 算法岗面试经验分享 | B 站/知乎 | 一线面试者的真实经验分享 |
| 系统设计面试教程 | YouTube | 学习系统设计题的思维框架 |
| LeetCode 刷题指南 | B 站/YouTube | 算法面试准备 |

六、社区与交流

6.1 GitHub 社区

| 平台 | 用途 | 建议 |
|--------------------------|-----------|--------------------------|
| Nanobot Issues | 问题反馈和功能讨论 | 关注 bug 修复和新 feature 讨论 |
| Nanobot Discussions | 社区交流 | 可以提问和分享经验 |
| MCP specification Issues | 协议讨论 | 跟踪 MCP 协议的演进方向 |
| Awesome MCP Servers | MCP 工具集合 | 发现优质 MCP Server 和贡献自己的工具 |

6.2 社交平台

| 平台 | 关注对象 | 获取信息类型 |
|-----------|---------------------------------------|--------------|
| 即刻 | AI Agent 圈子、MCP 话题 | 行业动态、内推、产品发布 |
| X/Twitter | @AnthropicAI, @OpenAI, @kaborofficial | 技术前沿、论文发布 |
| 知乎 | AI/LLM 相关话题和专栏 | 技术深度分析、面经分享 |
| 微信公众号 | 量子位、机器之心、AI 前线 | 行业新闻和技术解读 |
| Discord | LangChain Discord, Anthropic Discord | 英文技术社区，问题解答快 |
| 掘金 | AI/LLM 标签 | 中文技术博客、实战教程 |
| CSDN | AI/大模型专栏 | 中文技术文章、面试经验 |

6.3 参与开源的建议

参与开源贡献是简历上最有说服力的加分项之一：

1. 从小事做起：修 typo、改文档、补充测试用例
2. 报告高质量的 **Issue**：包含复现步骤、环境信息、期望行为
3. 提交实用的 **PR**：修复 bug 或添加小功能
4. 写使用教程：为项目写中文教程或使用指南
5. 贡献 **MCP Server**：开发一个有用的 MCP Server 并贡献到 awesome-mcp-servers

七、工具与平台

7.1 开发工具

| 工具 | 用途 | 推荐度 |
|------------------|-----------------|-----|
| Cursor IDE | AI 辅助编程 IDE | |
| VS Code | 通用代码编辑器 | |
| Jupyter Notebook | 交互式实验环境 | |
| MCP Inspector | MCP Server 调试工具 | |
| Postman | API 测试工具 | |
| Docker Desktop | 容器化开发环境 | |

7.2 AI 模型平台

| 平台 | 提供的能力 | 定价参考 |
|---------------|-----------------|--------------------------------|
| OpenAI API | GPT-4o/GPT-4 模型 | GPT-4o: \$2.5-10/M tokens |
| Anthropic API | Claude 3.5/4 模型 | Claude Sonnet: \$3-15/M tokens |

| 平台 | 提供的能力 | 定价参考 |
|-----------------------------|----------|-------------|
| 阿里云 DashScope | 通义千问系列 | 按量计费，有免费额度 |
| 百度千帆 | 文心一言系列 | 按量计费，有免费额度 |
| 硅基流动 (SiliconFlow) | 多种开源模型推理 | 性价比高的推理平台 |
| Ollama | 本地模型运行 | 免费，需要本地 GPU |

7.3 向量数据库

| 数据库 | 部署方式 | 适用场景 |
|-----------------|--------|--------------|
| ChromaDB | 嵌入式/本地 | 学习和原型开发 |
| Milvus | 分布式/云端 | 生产级大规模检索 |
| Pinecone | 全托管云服务 | 快速上手，免运维 |
| Weaviate | 开源分布式 | 多模态检索 |
| Qdrant | 开源分布式 | Rust 实现，性能优秀 |

7.4 监控与可观测性

| 工具 | 用途 | 推荐度 |
|-----------------------------|-------------|-----|
| LangSmith | LLM 应用追踪和调试 | |
| Weights & Biases | 模型训练实验跟踪 | |
| Prometheus + Grafana | 系统指标监控 | |
| Langfuse | 开源 LLM 可观测性 | |

八、学习路线图

8.1 四周速成计划（每天 4-6 小时）

适合有 Python 基础、想快速入门 AI Agent 的开发者。

第 1 周：大模型与 Agent 基础

| 天 | 学习内容 | 产出 |
|---------|--|---------------------|
| Day 1 | 阅读本项目第 1 章（什么是 Agent）+ 第 2 章（Nanobot 概览） | 理解 Agent 基本概念 |
| Day 2 | 阅读第 3 章（架构深度解析），画出 Nanobot 架构图 | 架构笔记 |
| Day 3 | 动手安装 Nanobot（第 6 章），跑通基本对话 | 成功运行 Nanobot |
| Day 4 | 阅读 Transformer 入门博客 + 看 3Blue1Brown 视频 | 理解注意力机制 |
| Day 5 | 阅读 Transformer 相关论文和博客，理解 GQA/RoPE/RMSNorm | 理解 Transformer 架构组件 |
| Day 6-7 | 阅读第 5 章（MCP 协议），跑通一个 MCP Server 示例 | 理解 MCP 基本概念 |

第 2 周：源码研读与 MCP 实践

| 天 | 学习内容 | 产出 |
|-----------|---|---------------|
| Day 8-9 | 阅读第 4 章（源码逐行解析）——AgentLoop 部分 | AgentLoop 流程图 |
| Day 10-11 | 阅读第 4 章——MemoryConsolidator、ToolRegistry 部分 | 模块分析笔记 |

| 天 | 学习内容 | 产出 |
|---------------------|--|----------------------------|
| Day 12-13
Day 14 | 用 MCP Python SDK 开发第一个 MCP Server
将自己的 MCP Server 集成到 Nanobot 中测试 | 可运行的 MCP Server
端到端验证成功 |

第 3 周：模型训练与 RAG 实践

| 天 | 学习内容 | 产出 |
|----------------------------------|--|--|
| Day 15-16 | 开发 2 个自定义 MCP Server（数据库查询 + 文档检索） | 掌握 MCP 开发实践 |
| Day 17-18
Day 19-20
Day 21 | 基于 Nanobot 搭建多平台 AI 助手（飞书/Web 接入）
用 LangChain + ChromaDB 搭建简单 RAG 系统
阅读第 7 章（记忆系统）和第 8 章（Skills 和工具） | 理解 Channel 适配器模式
可运行的 RAG demo
深化 Agent 理解 |

第 4 周：面试准备与项目包装

| 天 | 学习内容 | 产出 |
|-------------------------------------|--|--------------------------|
| Day 22-24
Day 25-26
Day 27-28 | 刷本项目第 13 章面试八股文（重点 Q1-Q50）
准备简历（参考第 15 章模板）+ 写项目 STAR 话术
模拟面试练习 + 查漏补缺 | 核心概念牢固
简历定稿
面试话术流畅 |

8.2 八周深入计划（每天 3-4 小时）

适合时间更充裕、想建立更深厚基础的学习者。

第 1-2 周：基础概念与环境搭建

- 学习本项目第 1-6 章
- 动手安装和使用 Nanobot
- 阅读 Transformer 入门材料
- 完成 5 道基础面试题（Q1-Q5）

第 3-4 周：源码深入研读

- 精读 Nanobot 核心源码（第 4 章）
- 重点理解 AgentLoop、MemoryConsolidator、ToolRegistry
- 开发 2 个自定义 MCP Server
- 学习第 7-10 章（记忆、工具、多平台、子 Agent）

第 5-6 周：模型原理与 MCP 深入实践

- 阅读 Attention Is All You Need 论文（配合视频讲解）
- 阅读 LoRA 和 DPO 论文，理解大模型训练和对齐原理
- 深入开发 3-5 个 MCP Server，覆盖不同业务场景
- 研究工具描述优化策略，提升工具调用准确率

第 7 周：RAG 与系统设计

- 搭建完整的 RAG 系统（混合检索 + Rerank）
- 学习系统设计（第 13 章 Q119-Q126）
- 阅读 MCP 协议规范
- 开发飞书/钉钉 AI 助手

第 8 周：面试冲刺

- 刷完第 13 章全部 134 道面试题
- 完善简历和项目话术
- 进行 3 次以上模拟面试
- 开始投递和面试

8.3 持续学习建议

AI Agent 领域变化极快，保持学习需要：

1. 每日：浏览 AI 新闻（即刻、X/Twitter），了解最新动态
2. 每周：阅读 1-2 篇技术博客或论文，保持知识更新
3. 每月：完成一个小的技术实验或项目（如新的 MCP Server、新的 RAG 优化方案）
4. 每季度：回顾和更新技术栈认知，关注新出现的框架和工具
5. 持续：在 GitHub 上维护自己的项目，写技术博客记录学习心得

九、书籍推荐

9.1 大模型与 NLP

| 书名 | 作者 | 推荐理由 |
|----------------------------------|-------------------|--------------------|
| 《动手学深度学习》 | 李沐等 | 深度学习入门最佳教材，有在线互动版本 |
| 《Speech and Language Processing》 | Jurafsky & Martin | NLP 经典教材，免费在线版 |
| 《Deep Learning》 | Goodfellow et al. | 深度学习理论基础，花书 |
| 《大模型应用开发极简入门》 | 社区作者 | 中文入门书，实践导向 |

9.2 系统设计与工程

| 书名 | 作者 | 推荐理由 |
|---------------------------|------------------|----------------|
| 《设计数据密集型应用》 | Martin Kleppmann | 分布式系统设计经典 |
| 《System Design Interview》 | Alex Xu | 系统设计面试圣经 |
| 《Clean Architecture》 | Robert C. Martin | 架构设计原则 |
| 《Python 工程实践》 | 社区作者 | Python 工程化最佳实践 |

9.3 面试准备

| 书名 | 作者 | 推荐理由 |
|---------------------------------|----------------|----------|
| 《Cracking the Coding Interview》 | Gayle McDowell | 编程面试经典 |
| 《百面机器学习》 | 诸葛越等 | ML 面试题精选 |
| 《百面深度学习》 | 诸葛越等 | DL 面试题精选 |

总结

学习 AI Agent 的资源生态已经非常丰富，关键是选择合适的路径并坚持执行：

1. 源码为王：Nanobot 的 4000 行源码是理解 Agent 原理的最佳教材
2. 实践驱动：每学一个概念都要动手做实验，看 10 篇博客不如写一个 demo
3. 论文精选：不需要读很多论文，但必读论文要精读

4. 社区参与：加入社区讨论、提交 Issue/PR，是最快的学习加速器
5. 持续更新：AI 领域日新月异，保持学习习惯比任何单项技能更重要

最终目标：不是成为「会用 100 个工具的人」，而是成为「理解原理、能解决新问题的人」。框架会过时，但对 Agent 系统的深入理解永远有价值。

上一章：第 16 章 *STAR* 面试话术 本章是 *learn-nanobot* 项目的最后一章。祝你面试顺利！

18 - WebUI 实战

阅读时间：约 1.5 小时

前置知识：06 - 安装与上手

学习目标：掌握 Nanobot WebUI 的配置、使用和核心功能

目录

- 18.1 WebUI 概述
 - 18.2 配置启用 WebUI
 - 18.3 启动与访问
 - 18.4 核心功能详解
 - 18.5 WebUI vs CLI 对比
 - 18.6 局域网访问配置
 - 18.7 面试话术
-

18.1 WebUI 概述

Nanobot 内置了 WebUI，通过 WebSocket channel 实现，无需额外安装。

架构特点：- WebUI 打包在 published wheel 内，无额外构建步骤 - 基于 WebSocket 实时通信 (channels/websocket.py, 1179 行) - 支持流式输出、文件编辑实时显示、Thought/Response 时间线 - 默认端口 8765，与 Gateway health endpoint (18790) 独立

18.2 配置启用 WebUI

在 ~/.nanobot/config.json 中添加：

```
{
  "channels": {
    "websocket": {
      "enabled": true
    }
  }
}
```

完整配置示例：

```
{
  "channels": {
    "websocket": {
      "enabled": true,
      "host": "127.0.0.1",
      "port": 8765,
      "streaming": true
    }
  }
}
```

18.3 启动与访问

启动 **Gateway**

nanobot gateway

访问 **WebUI**

打开浏览器访问：

`http://127.0.0.1:8765`

注意：Gateway 的 health endpoint 在 `gateway.port`（默认 18790），WebUI 在 WebSocket channel 的端口（默认 8765）。

18.4 核心功能详解

18.4.1 Thought/Response 时间线

WebUI 清晰展示 Agent 的思考过程： - **Thought**：Agent 的推理步骤、工具调用决策 - **Response**：最终回复内容 - 实时流式显示，非等待完成后一次性展示

18.4.2 文件编辑活动实时显示

当 Agent 执行文件操作时： - 实时显示文件编辑进度 - 显示文件状态变化 - 支持查看编辑历史

18.4.3 Model Switching

使用 `/model` 命令切换模型： - 动态切换 Provider 和模型 - 无需重启 Gateway - 基于 `modelPresets` 配置

18.4.4 Context Controls

- 查看当前上下文窗口使用情况
- 控制历史消息加载量
- 手动触发上下文压缩

18.4.5 Project Workspaces

- 多项目工作区管理
- 每个 workspace 独立的记忆和配置
- 工作区级别的访问控制

18.5 WebUI vs CLI 对比

| 特性 | WebUI | CLI |
|----------|------------------------------|----------------------------|
| 启动命令 | <code>nanobot gateway</code> | <code>nanobot agent</code> |
| 实时流式 | 支持 | 支持 |
| 文件编辑可视化 | 实时显示 | 仅日志 |
| 多会话管理 | 支持 | 单会话 |
| Model 切换 | 可视化 | <code>/model</code> 命令 |
| 适合场景 | 日常使用、调试 | 快速测试、脚本集成 |

18.6 局域网访问配置

默认 WebUI 仅监听 127.0.0.1。要从其他设备访问：

```
{
  "channels": {
    "websocket": {
      "enabled": true,
```

```

    "host": "0.0.0.0",
    "port": 8765
  }
}

```

安全警告：监听 0.0.0.0 会暴露到网络。建议配合 token 或 allow_from 使用。

配置 Token 认证：

```

{
  "channels": {
    "websocket": {
      "enabled": true,
      "host": "0.0.0.0",
      "token": "your-secret-token",
      "websocket_requires_token": true
    }
  }
}

```

18.7 面试话术

话术：描述 Nanobot WebUI 架构

“Nanobot 的 WebUI 是通过 WebSocket channel 实现的，打包在 published wheel 内，无需额外构建。核心架构是：Gateway 启动后，WebSocket channel 监听端口 8765，客户端通过 WebSocket 连接后，每个连接有独立的 session。WebUI 支持实时流式输出、文件编辑活动显示、Thought/Response 时间线、Model switching 等功能。相比 CLI，WebUI 更适合日常使用和调试，因为它提供了可视化的多会话管理和实时反馈。”

下一章：07 - 记忆系统实战

19 - Goal 系统详解

阅读时间：约 1 小时

前置知识：06 - 安装与上手

学习目标：理解 Nanobot Goal 系统的设计原理、`/goal` 命令机制、Sustained Goal 状态管理

目录

- 19.1 Goal 系统概述
 - 19.2 `/goal` 命令机制
 - 19.3 Sustained Goal 状态管理
 - 19.4 Goal vs 普通对话
 - 19.5 实战：使用 Goal 完成多步骤任务
 - 19.6 面试话术
-

19.1 Goal 系统概述

Goal 系统是 Nanobot v0.2.0 引入的核心功能，用于维持跨轮次的长期目标。

解决的问题：- 普通对话中，Agent 容易在多次迭代后忘记最初目标 - 复杂任务需要多步骤执行，需要持续跟踪进度 - 用户需要明确的目标管理和进度可视化

核心价值：- 通过 `/goal` 命令启动长期目标 - Goal 状态在会话间持久化 - WebUI 实时显示 Goal 进度 - 自动超时控制 (Wall timeout / LLM timeout)

19.2 `/goal` 命令机制

启动 Goal

You: `/goal` 帮我重构这个项目的代码结构

Agent: [启动 Goal 模式]

目标：重构项目代码结构

状态：Active

Goal 状态追踪

Goal 启动后，Agent 会：1. 在会话 metadata 中存储 `goal_state` 2. 每次迭代检查目标进度 3. 在 WebUI 显示实时进度 4. 完成后自动标记为 `completed`

源码实现

```
# nanobot/session/goal_state.py
```

```
GOAL_STATE_KEY = "goal_state"
```

```
def sustained_goal_active(metadata) -> bool:
    """ 检查是否有活跃的 Goal """
    goal = parse_goal_state(goal_state_raw(metadata))
    return isinstance(goal, dict) and goal.get("status") == "active"
```

```
def sustained_goal_turn(metadata, message_metadata) -> bool:
    """ 判断当前轮次是否使用 Goal 模式 """
    if sustained_goal_active(metadata):
        return True
```

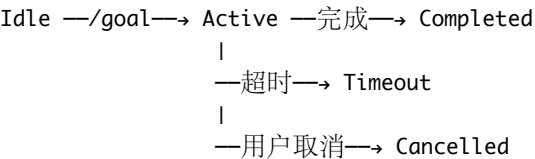
```
# /goal 命令触发
return str(message_metadata.get("original_command") or "").strip() == "/goal"
```

19.3 Sustained Goal 状态管理

Goal 状态结构

```
{
  "status": "active",
  "objective": " 重构项目代码结构",
  "ui_summary": " 正在分析模块依赖...",
  "turn_count": 5,
  "started_at": "2026-06-17T10:00:00Z"
}
```

状态转换



Runtime Context 注入

当 Goal 激活时，会在 System Prompt 中注入：

```
# nanobot/session/goal_state.py 第 73-89 行
def goal_state_runtime_lines(metadata):
    """Goal 激活时注入 Runtime Context 的行"""
    goal = parse_goal_state(...)
    if goal.get("status") == "active":
        objective = goal.get("objective")
        return [
            "Goal (active):",
            objective,
            f"Summary: {goal.get('ui_summary')}"
        ]
```

效果：每次 Agent 迭代都能看到当前目标，不会迷失方向。

19.4 Goal vs 普通对话

| 特性 | 普通对话 | Goal 模式 |
|----------|------------|----------------------|
| 目标持久化 | 不持久化 | 存储在 metadata |
| 迭代限制 | 默认 40 次 | 可配置更长 |
| 超时控制 | 标准 timeout | Wall timeout = 0（禁用） |
| WebUI 显示 | 无特殊显示 | 实时进度条 |
| 适用场景 | 简单问答 | 多步骤复杂任务 |

超时控制差异

```
# nanobot/session/goal_state.py 第 109-126 行
def runner_wall_llm_timeout_s(...):
```

```

"""Goal 模式下的超时控制"""
if sustained_goal_turn(...):
    return 0.0 # 禁用 asyncio.wait_for
return None # 使用默认 NANOBOT_LLM_TIMEOUT_S

```

设计意图：Goal 任务可能需要长时间运行，不应被标准超时中断。

19.5 实战：使用 Goal 完成多步骤任务

场景：自动化代码审查

You: /goal 审查整个项目的代码质量

Agent: [Goal Active]

目标：审查整个项目的代码质量

[第1轮] 扫描项目结构...

[第2轮] 分析主要模块...

[第3轮] 检查代码规范...

[第4轮] 识别潜在 Bug...

[第5轮] 生成审查报告...

[Goal Completed]

审查完成，发现 3 个警告，5 个建议。

WebUI 中的 Goal 显示

在 WebUI 中，Goal 会显示为：- 顶部进度条：Goal：审查整个项目的代码质量 - 摘要信息：Summary：正在分析主要模块... - 实时更新：每次迭代后自动刷新

查看 Goal 状态

查看会话 metadata (包含 goal_state)

cat workspace/sessions/cli:direct.jsonl | python -m json.tool | grep -A 10 goal_state

19.6 面试话术

问题：Nanobot 的 Goal 系统是如何解决长任务迷失问题的？

“Nanobot 的 Goal 系统通过 /goal 命令启动长期目标模式。核心设计是：在会话 metadata 中存储 goal_state (包含目标、状态、摘要等信息)，每次 Agent 迭代时，通过 goal_state_runtime_lines 将当前目标注入到 Runtime Context 中，确保 Agent 始终知道最终目标。同时，Goal 模式禁用标准超时控制 (返回 0.0)，允许长时间运行。WebUI 会实时显示 Goal 进度，用户可以直观跟踪任务状态。这种设计解决了 Agent 在多次工具调用后容易迷失方向的问题。”

问题：Goal 系统和普通对话的技术差异是什么？

“主要有三个差异：第一，Goal 状态持久化在会话 metadata 中，通过 goal_state key 存储，而普通对话没有这个状态；第二，Goal 模式通过 sustained_goal_turn 函数判断，会注入 Runtime Context 并禁用超时控制；第三，WebUI 有专门的 Goal 显示逻辑，通过 goal_state_ws_blob 获取实时进度。这些差异使得 Goal 模式适合多步骤复杂任务，而普通对话适合简单问答。”

扩展阅读：18 - WebUI 实战 ——WebUI 中的 Goal 进度显示

20 - Gateway 与生产部署

阅读时间：约 2 小时

前置知识：06 - 安装与上手、18 - WebUI 实战

学习目标：掌握 Nanobot Gateway 的架构、启动流程、多实例部署、Docker 部署

目录

- 20.1 Gateway 概述
 - 20.2 Gateway 启动流程
 - 20.3 Gateway 核心服务
 - 20.4 多实例部署
 - 20.5 Docker 部署
 - 20.6 Linux Service 部署
 - 20.7 面试话术
-

20.1 Gateway 概述

Gateway 是 Nanobot 的生产级运行模式，相比 CLI 的 `nanobot agent`，Gateway 提供：- 多 Channel 并发支持 - 内置 WebUI - 后台系统任务（Dream、Heartbeat）- 持久化会话管理 - Health Check 端点

启动命令：

```
nanobot gateway
```

20.2 Gateway 启动流程

Gateway 启动时按顺序初始化以下服务：

1. 加载配置（`~/nanobot/config.json`）
↓
 2. 初始化 Provider（LLM 后端）
↓
 3. 注册 Channel（Telegram、Discord、WebSocket等）
↓
 4. 启动 MessageBus（消息队列）
↓
 5. 启动 AgentLoop（Agent 循环）
↓
 6. 注册系统任务：
 - Dream 记忆摘要
 - Heartbeat 心跳检查
 - Cron 定时任务
↓
 7. 启动 Health endpoint（默认端口 18790）
↓
 8. 启动 WebSocket channel（WebUI，默认端口 8765）
↓
 9. 开始接收消息
-

20.3 Gateway 核心服务

20.3.1 Chat Channels

Gateway 可以同时运行多个 Channel:

```
{
  "channels": {
    "telegram": { "enabled": true, "token": "${TELEGRAM_TOKEN}" },
    "discord": { "enabled": true, "token": "${DISCORD_TOKEN}" },
    "feishu": { "enabled": true, "appId": "${FEISHU_APP_ID}" },
    "websocket": { "enabled": true }
  }
}
```

20.3.2 WebSocket Channel (WebUI)

- 端口: 8765 (默认)
- 协议: WebSocket
- 功能: 实时聊天、文件编辑、Model switching

20.3.3 Dream 系统任务

- 自动处理未读会话历史
- 生成智能摘要更新到 MEMORY.md
- 通过 .dream_cursor 游标跟踪进度

20.3.4 Heartbeat 心跳任务

```
{
  "gateway": {
    "heartbeat": {
      "enabled": true,
      "interval": 1800
    }
  }
}
```

- 定期检查系统健康状态
- 默认间隔: 1800 秒 (30 分钟)
- 存储位置: <workspace>/cron/jobs.json

20.3.5 Cron 定时任务

支持用户自定义定时任务: - 基于 cron 表达式 - 自动触发 Agent 对话 - 持久化存储

20.3.6 Health Endpoint

- 默认端口: 18790
- URL: <http://127.0.0.1:18790/health>
- 用途: Docker health check、监控系统

20.4 多实例部署

场景: 运行多个独立的 Bot

实例 1: 个人助手

```
nanobot gateway --config ~/.nanobot/config-personal.json --workspace ~/workspaces/personal
```

实例 2: 工作助手

```
nanobot gateway --config ~/.nanobot/config-work.json --workspace ~/workspaces/work
```

实例 3: 客服 Bot

```
nanobot gateway --config ~/.nanobot/config-support.json --workspace ~/workspaces/support
```

配置隔离

每个实例需要: - 独立的配置文件 - 独立的 workspace 目录 - 独立的 Gateway 端口 (如果在同一机器)

```
{
  "gateway": {
    "port": 18791
  },
  "channels": {
    "websocket": {
      "port": 8766
    }
  }
}
```

20.5 Docker 部署

Dockerfile 示例

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
# 安装 nanobot
```

```
RUN pip install nanobot-ai
```

```
# 复制配置
```

```
COPY config.json /root/.nanobot/config.json
```

```
# 暴露端口
```

```
EXPOSE 8765 18790
```

```
# 启动 Gateway
```

```
CMD ["nanobot", "gateway"]
```

docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  nanobot:
```

```
    build: .
```

```
    ports:
```

- "8765:8765" # WebUI
- "18790:18790" # Health

```
    volumes:
```

- ./workspace:/root/workspace
- ./config.json:/root/.nanobot/config.json

```
    environment:
```

- OPENAI_API_KEY=\${OPENAI_API_KEY}
- TELEGRAM_TOKEN=\${TELEGRAM_TOKEN}

```
restart: unless-stopped
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:18790/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

启动

```
docker-compose up -d
```

20.6 Linux Service 部署

systemd service 文件

创建 /etc/systemd/system/nanobot.service:

```
[Unit]
Description=Nanobot AI Gateway
After=network.target

[Service]
Type=simple
User=nanobot
Group=nanobot
WorkingDirectory=/opt/nanobot
ExecStart=/opt/nanobot/venv/bin/nanobot gateway
Restart=always
RestartSec=10
Environment=PATH=/opt/nanobot/venv/bin
EnvironmentFile=/opt/nanobot/.env

[Install]
WantedBy=multi-user.target
```

环境变量文件

创建 /opt/nanobot/.env:

```
OPENAI_API_KEY=sk-xxx
TELEGRAM_TOKEN=123456:ABC-xxx
```

启动服务

```
# 重载 systemd
sudo systemctl daemon-reload
```

```
# 启动服务
sudo systemctl start nanobot
```

```
# 设置开机自启
sudo systemctl enable nanobot
```

```
# 查看状态
sudo systemctl status nanobot
```

```
# 查看日志
sudo journalctl -u nanobot -f
```

20.7 面试话术

问题：Nanobot Gateway 的架构是怎样的？

“Nanobot Gateway 是生产级运行模式，启动时会按顺序初始化多个服务：首先加载配置和 Provider，然后注册所有启用的 Channel（Telegram、Discord、WebSocket 等），启动 MessageBus 和 AgentLoop，最后注册系统任务（Dream 记忆摘要、Heartbeat 心跳检查、Cron 定时任务）并开放 Health endpoint（18790 端口）和 WebUI（8765 端口）。Gateway 支持多实例部署，每个实例通过独立的配置文件和 workspace 实现隔离。”

问题：如何实现 Nanobot 的高可用部署？

“可以通过 Docker Compose 或 systemd 实现高可用部署。Docker 方案使用 docker-compose.yml 定义服务，配置 health check 自动监控 Gateway 状态，设置 restart policy 自动重启失败容器。systemd 方案创建 service 文件，配置 Restart=always 和 EnvironmentFile 管理环境变量。多实例部署时，每个实例使用独立的配置文件、workspace 和端口，通过 -config 和 -workspace 参数指定。”

扩展阅读：18 - WebUI 实战 ——WebUI 配置和使用

21 - 实际项目案例

阅读时间：约 3 小时

前置知识：06 - 安装与上手、18 - WebUI 实战、20 - Gateway 与生产部署

学习目标：掌握 3 个真实场景的 Nanobot 部署方案

目录

- 21.1 案例一：个人知识管理助手
 - 21.2 案例二：自动化运维 Bot
 - 21.3 案例三：多平台客服系统
-

21.1 案例一：个人知识管理助手

场景描述

构建一个能够自动整理笔记、生成摘要、跨会话记忆的个人知识助手。

技术选型

- **Provider**: DeepSeek (性价比高)
- **Channel**: WebSocket (WebUI)
- 核心功能: Dream 记忆系统、WebUI、Goal 系统

配置示例

```
{
  "providers": {
    "deepseek": {
      "apiKey": "${DEEPSEEK_API_KEY}"
    }
  },
  "modelPresets": {
    "primary": {
      "provider": "deepseek",
      "model": "deepseek-chat",
      "maxTokens": 8192,
      "contextWindowTokens": 65536
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "primary"
    }
  },
  "channels": {
    "websocket": {
      "enabled": true,
      "host": "127.0.0.1",
      "port": 8765
    }
  }
}
```

AGENTS.md 配置

个人知识管理助手

你是一个专业的知识管理助手，擅长：

核心能力

1. **** 笔记整理 ****: 将零散笔记整理为结构化文档
2. **** 摘要生成 ****: 生成长文摘要和关键点提取
3. **** 知识关联 ****: 发现不同笔记之间的关联
4. **** 记忆管理 ****: 自动记录重要知识点到 `MEMORY.md`

行为准则

- 使用 ``read_file`` 查阅笔记
- 使用 ``write_file`` 生成整理后的文档
- 使用 ``save_memory`` 记录关键知识点
- 保持文档结构清晰

输出格式

使用 **Markdown** 格式，包含：

- 标题层级
- 代码块（带语法高亮）
- 表格（对比数据）
- 链接（关联文档）

部署步骤

1. 创建工作目录

```
mkdir ~/knowledge-assistant && cd ~/knowledge-assistant
```

2. 创建配置文件

```
cat > config.json << 'EOF'
{配置内容}
EOF
```

3. 创建 AGENTS.md

```
cat > AGENTS.md << 'EOF'
{AGENTS.md 内容}
EOF
```

4. 启动 Gateway

```
nanobot gateway
```

5. 访问 WebUI

```
# 浏览器打开 http://127.0.0.1:8765
```

Dream 系统效果

运行一段时间后：

查看自动生成的记忆

```
cat memory/MEMORY.md
```

查看 Dream 处理进度

```
cat memory/.dream_cursor
```

```
# 查看会话历史
tail -20 memory/history.jsonl
```

21.2 案例二：自动化运维 Bot

场景描述

构建一个定时检查系统状态、发送告警通知的运维 Bot。

技术选型

- **Provider:** OpenAI（稳定性要求高）
- **Channel:** Telegram + Email
- 核心功能: Cron 定时任务、MCP 工具、多渠道通知

配置示例

```
{
  "providers": {
    "openai": {
      "apiKey": "${OPENAI_API_KEY}"
    }
  },
  "modelPresets": {
    "primary": {
      "provider": "openai",
      "model": "gpt-4o",
      "maxTokens": 4096
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "primary"
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
      "token": "${TELEGRAM_TOKEN}"
    },
    "email": {
      "enabled": true,
      "imapHost": "imap.gmail.com",
      "smtpHost": "smtp.gmail.com",
      "email": "bot@example.com",
      "imapPassword": "${EMAIL_PASSWORD}",
      "smtpPassword": "${EMAIL_PASSWORD}"
    }
  },
  "tools": {
    "mcpServers": {
      "system-monitor": {
        "command": "python",
        "args": ["monitor_server.py"]
      }
    }
  }
}
```

```

    }
  },
  "gateway": {
    "heartbeat": {
      "enabled": true,
      "intervalS": 1800
    }
  }
}
}
}

```

Cron 定时任务

通过 `/goal` 或配置文件设置定时任务：

每天早上 9 点：检查服务器状态

每 6 小时：检查磁盘空间

收到告警时：发送邮件通知

AGENTS.md 配置

运维助手

你是一个 7x24 小时的运维助手。

职责

1. ** 系统监控 **：定期检查服务器状态
2. ** 告警通知 **：发现问题时发送 Telegram 和 Email
3. ** 日志分析 **：分析错误日志，定位问题
4. ** 自动修复 **：执行预定义的修复脚本

工具使用

- 使用 MCP ``system-monitor`` 检查服务器
- 使用 ``exec`` 执行诊断命令
- 使用 ``read_file`` 查看日志
- 通过 Telegram 和 Email 发送通知

告警规则

- CPU > 90% 持续 5 分钟 → 告警
- 磁盘 > 85% → 告警
- 服务宕机 → 立即告警并尝试重启

部署步骤

1. 创建工作目录

```
mkdir ~/ops-bot && cd ~/ops-bot
```

2. 创建配置

```
cat > config.json << 'EOF'
{配置内容}
EOF
```

3. 创建 MCP Server

```
cat > monitor_server.py << 'EOF'
# MCP 服务器代码（检查系统状态）
EOF
```


4. 启动

nanobot gateway

5. 验证

nanobot status

21.3 案例三：多平台客服系统

场景描述

构建一个同时支持 Telegram、飞书、微信的客服系统，使用 Goal 系统维持服务目标。

技术选型

- **Provider:** Anthropic Claude（对话质量高）
- **Channel:** Telegram + Feishu + Weixin
- 核心功能：多通道、Goal 系统、Skills 知识库

配置示例

```
{
  "providers": {
    "anthropic": {
      "apiKey": "${ANTHROPIC_API_KEY}"
    }
  },
  "modelPresets": {
    "primary": {
      "provider": "anthropic",
      "model": "claude-sonnet-4-20250514",
      "maxTokens": 8192,
      "contextWindowTokens": 65536
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "primary"
    }
  },
  "channels": {
    "telegram": {
      "enabled": true,
      "token": "${TELEGRAM_TOKEN}"
    },
    "feishu": {
      "enabled": true,
      "appId": "${FEISHU_APP_ID}",
      "appSecret": "${FEISHU_APP_SECRET}"
    },
    "weixin": {
      "enabled": true,
      "token": "${WEIXIN_TOKEN}"
    }
  }
}
```

Skills 知识库

创建客服知识库：

```
workspace/  
└─ skills/  
    └─ customer-service/  
        └─ SKILL.md
```

SKILL.md 内容：

Customer Service Skill

激活条件

当用户咨询产品相关问题时激活。

知识库

产品 A

- 价格：¥99/月
- 功能：自动备份、数据分析
- 常见问题：...

产品 B

- 价格：¥199/月
- 功能：团队协作、API 访问
- 常见问题：...

回答规范

1. 先确认用户问题属于哪个产品
2. 从知识库查找答案
3. 如果找不到，使用 `web_search` 搜索
4. 保持语气友好、专业

AGENTS.md 配置

客服助手

你是一个专业的多平台客服助手。

职责

1. **** 问题解答 ****：回答用户关于产品的咨询
2. **** 问题排查 ****：帮助用户解决使用问题
3. **** 工单创建 ****：复杂问题创建工单并转人工
4. **** 满意度调查 ****：服务结束后询问满意度

使用 Goal 系统

对于复杂问题，使用 ``/goal`` 命令：

- 跟踪问题解决进度
- 确保不遗漏任何步骤
- 跨会话维持上下文

服务流程

1. 问候用户

2. 确认问题
3. 查找解决方案
4. 提供解答
5. 确认是否解决
6. 满意度调查

部署步骤

1. 创建工作目录

```
mkdir ~/customer-service && cd ~/customer-service
```

2. 创建配置

```
cat > config.json << 'EOF'
{配置内容}
EOF
```

3. 创建 Skills

```
mkdir -p skills/customer-service
cat > skills/customer-service/SKILL.md << 'EOF'
{SKILL.md 内容}
EOF
```

4. 启动

```
nanobot gateway
```

5. 测试多平台

分别从 Telegram、飞书、微信发送消息

Goal 系统实战

处理复杂问题:

用户: 我无法登录系统

Agent: /goal 帮助用户解决登录问题

[Goal Active]

目标: 帮助用户解决登录问题

[第1轮] 确认用户账号状态...

[第2轮] 检查密码是否正确...

[第3轮] 发现账号被锁定...

[第4轮] 解锁账号...

[第5轮] 指导用户重置密码...

[Goal Completed]

问题已解决, 账号已解锁。

下一章: 22 - 测试与可观测性

22 - 测试与可观测性

阅读时间：约 1.5 小时

前置知识：06 - 安装与上手、20 - Gateway 与生产部署

学习目标：掌握 Nanobot 的日志分析、性能调优、错误排查

目录

- 22.1 日志系统
 - 22.2 Langfuse 可观测性
 - 22.3 性能调优
 - 22.4 错误排查
 - 22.5 测试策略
-

22.1 日志系统

日志格式

Nanobot 使用 loguru 日志库，默认格式：

```
2026-06-17 10:00:00 | INFO | telegram | Message received from user_123
2026-06-17 10:00:01 | INFO | agent | Starting turn for session tg:123
2026-06-17 10:00:02 | INFO | provider | Calling LLM: deepseek-chat
2026-06-17 10:00:05 | INFO | agent | Tool call: read_file(path="notes.md")
2026-06-17 10:00:06 | INFO | agent | Turn completed in 6.2s
```

启动详细日志

查看详细日志

```
nanobot gateway --verbose
```

或使用环境变量

```
NANOBOT_LOG_LEVEL=DEBUG nanobot gateway
```

日志分析技巧

查看最近错误

```
grep "ERROR" nanobot.log | tail -20
```

查看工具调用

```
grep "Tool call" nanobot.log | tail -50
```

查看 LLM 调用

```
grep "Calling LLM" nanobot.log | tail -20
```

统计响应时间

```
grep "Turn completed" nanobot.log | awk '{print $NF}'
```

22.2 Langfuse 可观测性

配置 Langfuse

```
{
  "observability": {
    "langfuse": {
```

```

        "enabled": true
    }
}
}

```

环境变量

```

export LANGFUSE_SECRET_KEY="sk-lf-xxx"
export LANGFUSE_PUBLIC_KEY="pk-lf-xxx"
export LANGFUSE_BASE_URL="https://cloud.langfuse.com"

```

查看指标

访问 Langfuse Dashboard: - Token 使用量 - LLM 调用延迟 - 错误率 - 成本统计

22.3 性能调优

Context Window 设置

```

{
  "modelPresets": {
    "primary": {
      "contextWindowTokens": 65536
    }
  }
}

```

调优建议: - 不要设置过大: 会导致记忆压缩不触发, API 费用暴增 - 不要设置过小: 会频繁压缩, 丢失上下文 - 推荐值: 模型实际窗口的 80%

Token 估算

```

# nanobot 内部使用 estimate_prompt_tokens_chain
# 估算整个 prompt 的 token 数

# 手动估算
# 1 个英文单词 ≈ 1.3 tokens
# 1 个中文字 ≈ 1.5 tokens

```

AutoCompact 配置

```

{
  "gateway": {
    "autoCompact": {
      "enabled": true,
      "threshold": 0.8
    }
  }
}

```

当上下文使用率超过 80% 时自动压缩。

22.4 错误排查

Provider 连接失败

症状: 日志显示 Authentication failed

排查步骤:

1. 检查 API Key

```
cat config.json | python -m json.tool | grep apiKey
```

2. 测试连接

```
curl -H "Authorization: Bearer sk-xxx" https://api.deepseek.com/v1/models
```

3. 检查环境变量

```
echo $DEEPSEEK_API_KEY
```

Channel 启动失败

症状: Gateway 启动但某个 Channel 不可用

排查步骤:

1. 查看 Channel 状态

```
nanobot channels status
```

2. 查看详细日志

```
nanobot gateway --verbose 2>&1 | grep -i telegram
```

3. 检查配置

```
cat config.json | python -m json.tool | grep -A 5 telegram
```

Memory 文件损坏

症状: Agent 启动时报错

排查步骤:

1. 检查文件格式

```
cat memory/history.jsonl | python -m json.tool
```

2. 查看 Dream cursor

```
cat memory/.dream_cursor
```

3. 修复 (备份后删除损坏文件)

```
cp memory/history.jsonl memory/history.jsonl.bak
```

```
# 重新启动, nanobot 会自动重建
```

22.5 测试策略

单元测试

运行 nanobot 测试

```
cd nanobot
```

```
pytest tests/ -v
```

集成测试

1. 测试 CLI

```
nanobot agent -m "Hello!"
```

2. 测试 WebUI

```
curl http://127.0.0.1:8765
```

```
# 3. 测试 Health endpoint
curl http://127.0.0.1:18790/health
```

压力测试

```
# 并发发送消息
for i in {1..10}; do
  nanobot agent -m "Test message $i" &
done
wait
```

下一章: 99 - 快速查询卡

99 - 快速查询卡

用途：快速查阅 Nanobot 常用命令、配置字段、文件路径、Provider/Channel 清单

目录

- 常用命令速查
- 配置字段速查
- 文件路径速查
- Provider 清单
- Channel 清单
- 错误代码速查

常用命令速查

| 命令 | 说明 | 示例 |
|---------------------------|----------------|---------------------------|
| nanobot agent | 启动交互代理 | nanobot agent |
| nanobot agent -m " 消息" | 发送单条消息 | nanobot agent -m "Hello!" |
| nanobot gateway | 启动网关 (含 WebUI) | nanobot gateway |
| nanobot gateway --verbose | 详细日志模式 | nanobot gateway -v |
| nanobot status | 查看状态 | nanobot status |
| nanobot onboard | 配置向导 | nanobot onboard --wizard |
| nanobot --version | 查看版本 | nanobot --version |
| nanobot --help | 查看帮助 | nanobot --help |

配置字段速查

核心配置 (~/.nanobot/config.json)

```
{
  "providers": {
    "<provider_name>": {
      "apiKey": "${API_KEY}",
      "apiBase": "https://..."
    }
  },
  "modelPresets": {
    "<preset_name>": {
      "provider": "<provider_name>",
      "model": "<model_id>",
      "maxTokens": 8192,
      "contextWindowTokens": 65536,
      "temperature": 0.1
    }
  },
  "agents": {
    "defaults": {
      "modelPreset": "<preset_name>"
    }
  },
  "channels": {
```



```

    "<channel_name>": {
      "enabled": true
    }
  },
  "tools": {
    "mcpServers": {
      "<server_name>": {
        "command": "...",
        "args": ["..."]
      }
    }
  },
  "gateway": {
    "port": 18790,
    "heartbeat": {
      "enabled": true,
      "intervalS": 1800
    }
  }
}

```

关键字段说明

| 字段 | 类型 | 默认值 | 说明 |
|---|--------|-------|----------------------------------|
| providers.<name>.apiKey | string | - | API 密钥（支持 <code>\${ENV}</code> ） |
| providers.<name>.apiBase | string | - | API 端点（可选） |
| modelPresets.<name>.provider | string | - | Provider 名称 |
| modelPresets.<name>.model | string | - | 模型 ID |
| modelPresets.<name>.maxTokens | int | 8192 | 最大输出 token |
| modelPresets.<name>.contextWindowTokens | int | 65536 | 上下文窗口 |
| agents.defaults.modelPreset | string | - | 默认模型预设 |
| channels.<name>.enabled | bool | false | 启用通道 |
| gateway.port | int | 18790 | Health 端口 |
| channels.websocket.port | int | 8765 | WebUI 端口 |

文件路径速查

配置和 Workspace

| 文件 | 路径 | 说明 |
|---------------|--|------------|
| 配置文件 | <code>~/.nanobot/config.json</code> | 主配置文件 |
| Workspace 根目录 | <code>~/.nanobot/workspace/</code> | 默认工作区 |
| AGENTS.md | <code><workspace>/AGENTS.md</code> | Agent 身份定义 |
| SOUL.md | <code><workspace>/SOUL.md</code> | 全局人格 |
| USER.md | <code><workspace>/USER.md</code> | 用户画像 |

记忆系统

| 文件 | 路径 | 说明 |
|---------------|---|-------------|
| MEMORY.md | <code><workspace>/memory/MEMORY.md</code> | 长期记忆 |
| history.jsonl | <code><workspace>/memory/history.jsonl</code> | 会话历史（JSONL） |

| 文件 | 路径 | 说明 |
|---------------|----------------------------------|----------|
| .cursor | <workspace>/memory/.cursor | 读取游标 |
| .dream_cursor | <workspace>/memory/.dream_cursor | Dream 游标 |

会话存储

| 文件 | 路径 | 说明 |
|------|--|------|
| 会话文件 | <workspace>/sessions/<session_key>.jsonl | 会话历史 |

Skills

| 文件 | 路径 | 说明 |
|-----------|--|-----------|
| Skills 目录 | <workspace>/skills/<skill_name>/ | 用户 Skills |
| SKILL.md | <workspace>/skills/<skill_name>/SKILL.md | Skill 定义 |

Cron 和系统任务

| 文件 | 路径 | 说明 |
|---------|----------------------------|------|
| Cron 任务 | <workspace>/cron/jobs.json | 定时任务 |

Provider 清单

国际 **Provider** (12+)

| Provider | 配置名 | 代表模型 | 后端 |
|----------------|----------------|----------------------------------|------------------------|
| OpenAI | openai | gpt-4o, gpt-5.5 | openai_compat |
| Anthropic | anthropic | claude-sonnet-4, claude-4.8 | anthropic |
| Gemini | gemini | gemini-3.1-pro, gemini-3.5-flash | openai_compat |
| Mistral | mistral | mistral-large | openai_compat |
| Groq | groq | llama-3, mixtral | openai_compat |
| OpenRouter | openrouter | 多模型路由 | openai_compat |
| HuggingFace | huggingface | 开源模型 | openai_compat |
| OpenAI Codex | openai_codex | codex | openai_codex (OAuth) |
| GitHub Copilot | github_copilot | copilot | github_copilot (OAuth) |
| Azure OpenAI | azure_openai | gpt-4 (Azure) | azure_openai |
| AWS Bedrock | bedrock | claude, llama | bedrock |
| NVIDIA NIM | nim | nim 模型 | openai_compat |

国内 **Provider** (10+)

| Provider | 配置名 | 代表模型 | 后端 |
|-----------|-----------|----------------------------|---------------|
| DeepSeek | deepseek | deepseek-v4, deepseek-chat | openai_compat |
| DashScope | dashscope | qwen-max, qwen-plus | openai_compat |
| Zhipu | zhipu | glm-4, chatglm | openai_compat |
| Moonshot | moonshot | kimi-k2.5, kimi-k2.6 | openai_compat |
| MiniMax | minimax | minimax-01 | openai_compat |
| StepFun | stepfun | step-1 | openai_compat |

| Provider | 配置名 | 代表模型 | 后端 |
|-------------|------------|--------------|---------------|
| Skywork | skywork | skywork-llm | openai_compat |
| LongCat | longcat | longcat-chat | openai_compat |
| Atomic Chat | atomic | atomic-model | openai_compat |
| VolcEngine | volcengine | 豆包 | openai_compat |

聚合 **Provider** (4+)

| Provider | 配置名 | 说明 |
|-------------|-------------|--------------|
| OpenRouter | openrouter | 聚合多 Provider |
| AiHubMix | aihubmix | 聚合 API |
| SiliconFlow | siliconflow | 硅基流动 |
| Novita | novita | Novita AI |

本地 **Provider** (3+)

| Provider | 配置名 | 说明 |
|-----------|--------|------|
| Ollama | ollama | 本地模型 |
| vLLM | custom | 本地部署 |
| LM Studio | custom | 本地推理 |

Channel 清单

国际平台 (8)

| Channel | 配置名 | 说明 |
|----------|----------|--------|
| Telegram | telegram | 个人/社群 |
| Discord | discord | 社区/游戏 |
| Slack | slack | 企业协作 |
| WhatsApp | whatsapp | 个人通讯 |
| Signal | signal | 加密通讯 |
| MSTeams | msteams | 企业协作 |
| Matrix | matrix | 去中心化通讯 |
| Email | email | 邮件 |

国内平台 (5)

| Channel | 配置名 | 说明 |
|----------|----------|------------|
| Feishu | feishu | 飞书 (企业协作) |
| DingTalk | dingtalk | 钉钉 (企业协作) |
| Weixin | weixin | 微信 (个人/企业) |
| Wecom | wecom | 企业微信 |
| QQ | qq | QQ (个人/社群) |

其他平台 (2)

| Channel | 配置名 | 说明 |
|---------|--------|----------|
| Napcat | napcat | QQ 机器人框架 |
| MoChat | mochat | 微信机器人框架 |

通用通道（2）

| Channel | 配置名 | 说明 |
|-----------|-----------|----------------|
| WebSocket | websocket | WebUI（端口 8765） |
| CLI | cli | 命令行交互 |

错误代码速查

| 错误 | 原因 | 解决方案 |
|------------------------------|--------------|---------------------------------|
| Authentication failed | API Key 无效 | 检查 <code>apiKey</code> 配置 |
| Model not found | 模型 ID 错误 | 检查 <code>model</code> 字段 |
| Connection timeout | 网络问题 | 检查网络或使用代理 |
| Environment variable not set | 环境变量缺失 | 设置对应环境变量 |
| Config file not found | 配置文件不存在 | 运行 <code>nanobot onboard</code> |
| Channel failed to start | Channel 配置错误 | 检查 Channel 配置和凭证 |
| Memory file corrupted | 记忆文件损坏 | 备份后删除重建 |
| Provider not configured | Provider 未配置 | 在 <code>providers</code> 中添加配置 |

上一章：22 - 测试与可观测性

附录：版本信息

文档版本：v0.2.1
Nanobot 版本：v0.2.1（截至 2026-06-17）
Python 版本： 3.11
 最后更新：2026-06-17